

基于连续覆盖证书与观测共享调度的无线电干扰源自动定位与清除

摘 要

针对干扰源个数、位置、接收半径及定向方向未知，且测角误差有界、同地点误差固定的自动清除任务，建立以相容位置集合为信息状态、以实际任务时间为目标的搜索定位模型。将“如何证明不遗漏”与“如何减少移动和检测费用”分别处理，再通过证据账本与预算检查连接，形成可在线执行的完整策略。

针对问题一，将示向度转化为前向楔形的半平面约束，先判定交会区域的空集、无界及有界情形，再通过凸包和旋转卡壳计算直径。构造可由三次测向产生的正三角形反例，证明以区域直径为直径的圆不一定覆盖区域，并给出最远点对中点圆的充要检验条件。

针对问题二，利用“首次已接收”建立源距离与未知接收半径的耦合约束，推导三个圆盘与前向半平面的交作为保证接收候选域。对两条误差带交集的解析直径上界进行有界优化：当目标上界为 163 米时，得到移动距离 886.3434 米、计算下界 886.2614 米，间隙 0.08194 米；该最优性仅针对所用解析判据。

针对问题三，采用原点与半径 1125 米六点环构成的七站布局。联合维护未知、已发现、已清除及已排除频道，把剩余搜索站和已知源放入同一服务任务排序。在局部定位中复用完整观测历史，在可清除时直接投影到最近的保证清除位置，并通过有限试探额度与下降秩保证备用过程可接续执行。

针对问题四，采用 21 站布局，并以“合法源位置位于 1000 米内测点凸包中”为定向发现的充分条件。以连续区域证书核查覆盖；用真实正、负观测估计补测位置的接收概率，选择共享观察点与原地补测。对狭窄可行域构造完整光学覆盖链，通过 Voronoi 分区逐块证明覆盖后，按预计首次成功成本安排清除次序。

当前 30 场离线合成验证中，390 个源全部清除，62 条实际选用的光学链通过独立核查。20 场同输入历史对照中，问题三、四相对旧版平均分别节省 517.16 秒、952.62 秒；19 场变快，1 场变慢。文中区分离线结果、截图报告的演练成绩与尚未完成的正式测试，正式测试及未测敏感性数据保留空栏。

关键词：有界误差；交会定位；连续覆盖；观测共享；联合调度；光学清除

1 问题重述

1.1 背景与任务边界

无线电干扰源的初步监测只能确定目标分布区域，精准定位仍需要近距离观测和现场清除。题目给定半径 1800 米的圆形目标区域，机器狗从圆心出发，携带测向机、光学探测仪和激光清除设备，在不知道实际源数和位置的条件下自主决策 [1]。任务目标不仅是找到若干个源，还要给出足以结束搜索的证据，并在全部清除的前提下缩短总任务时间。

20 个频道中每个频道至多对应一个源，实际源总数为 10 至 16 个。全向源在全部方向发射，定向源仅在未知的闭 180 度角域内发射；有效接收半径分别固定于 1000 至 1500 米之间。不同源互不干扰，因此频道提供了可识别的目标身份，但不能提供“该频道是否有源”的先验答案。

机器狗速度为 5 米/秒，每次测向耗时 5 秒，改变测向频道另耗时 1 秒。光学探测耗时 3 秒，距离源不超过 20 米时再耗时 2 秒清除。移动与检测不能同时进行。这些条件使距离最短、观测次数最少与总时间最短成为不同的目标，须在同一个代价模型中权衡。

1.2 四个问题的输入与输出

表 1 四问任务分解

问题	已知信息	主要输出	验证重点
一	检测点和示向度	交会区域直径算法、同直径圆覆盖结论	退化状态、最远点对及反例
二	全向源的一次有效示向	第二测点候选域及选择策略	保证接收、定位精度与移动量
三	全部源为全向，具体数目未知	搜索、定位、清除及停止策略	全清比例、平均定位清除时间
四	全向与定向混合，类型与方向未知	对不可见性稳健的完整策略	定向不漏检、成本波动与时间

前两问研究单源几何信息如何转化为决策，后两问将其嵌入多频道在线任务。问题一回答可行域如何表示；问题二回答下一次观测怎样选择；问题三建立完整搜索与调度；问题四在不可见性增加后修改证据规则和执行方式。四问共享物理参数，但各问的可行域定义不能混用。

1.3 研究对象与评价指标

本文以题设协议和当前快速版程序为实现对象 [3]。主要指标为全清比例和每个实际清除源的平均任务时间。虚拟任务时间反映移动及设备操作，现实运行时间反映程序计算和通信；两者分别记录。后文的合成模拟、用户提供的演练汇总和正式测试表互相独立，避免用不同来源的数据拼成一次完整成绩。

2 问题分析与整体思路

2.1 三个相互耦合的困难

第一，误差无法通过同点重复采样消除。题设说明同一地点的电磁误差固定。将反复读数当作独立样本并求平均，会人为缩小置信区间。本文保留每次测角对应的整个角区间，通过不同位置的约束相交压缩候选源域。

第二，无信号不是无源。对全向源，无信号还可能表示超出半径；对定向源，还可能落在发射背面。一个频道只有在全部合法源状态都与实际观测矛盾时，才能从搜索任务中删除。尤其不能因为已经清除 10 个源，就停止检查其他频道。

第三，定位精度与任务费用不等价。机器狗可能已经处于能覆盖整个可行域的清除点，却继续为缩小几何直径而移动测量；也可能逐源采用看似最优的定位方案，却让源间往返距离增加。因此，单源动作与多源访问顺序需要同时考虑预计服务出口。

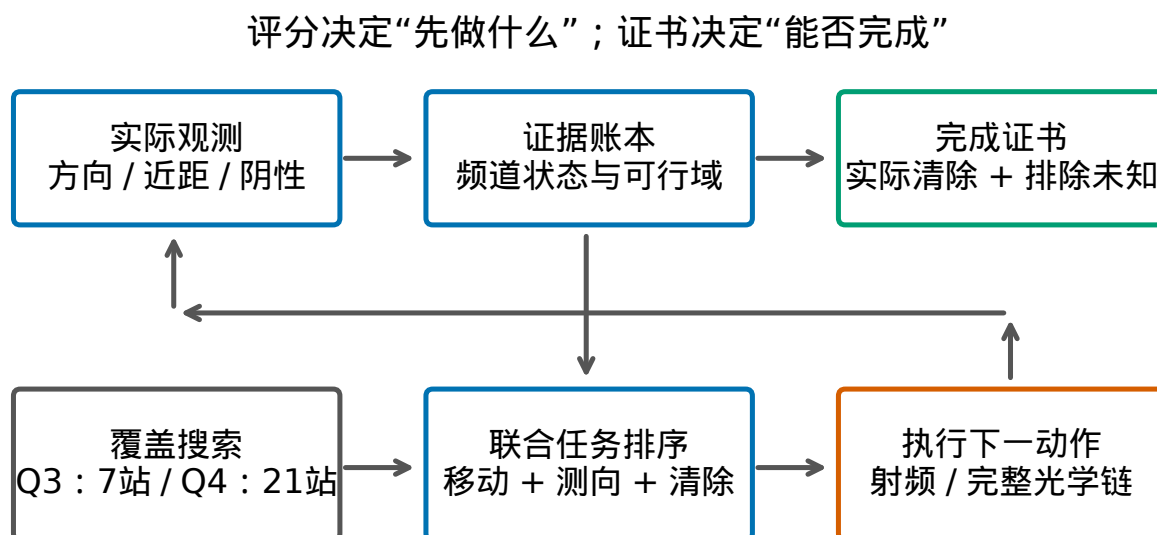


图 1 统一决策流程。实际响应进入证据账本；几何证书和数量上界约束完成判断；费用评分用于安排可执行动作。箭头表示数据或执行结果的传递。

2.2 模型之间的连接

整体流程分为四步。首先，将接收、方向和清除结果转化为集合约束，维护每个频道的状态。其次，用搜索布局或历史证书保证未知源仍有被发现的途径。再次，在剩余搜索与已知源之间选择下一项任务，并在任务内部比较射频测向和光学清除。最后，对动作的预算、实际返回及完成条件做检查，再进入下一轮。

本方法的核心结果是：启发式评分可以影响访问次序，但不直接消除一个可能存在的源。即使概率估计不准，几何责任区域、实际清除响应和完整备用策略仍决定能否宣告完成。因而可以优化平均费用，同时保留独立核查的完备性依据。

3 题设条件、建模约定与符号

3.1 采用的条件与补充约定

1. 一次案例中，源的位置、频道、有效半径与定向方向固定；不同频道的证据分别维护。该约定对应题目物理环境，不假设源在运行中迁移。
2. 采用平面欧氏距离，机器狗可沿直线移动至目标圆域外。圆域限制源的位置，并不限制机器狗位置，故问题四外环站可位于 1800 米圆外 [3]。
3. 测角误差按任意有界扰动处理，不假定不同检测位置的误差独立、正态或均匀。理论原始误差为 1° ，程序考虑两位小数舍入后采用 $\varepsilon = 1.005^\circ$ 。
4. 证明与执行分开：几何计算采用有理数和向外包络，保证清除判据采用 19.8 米半径；实际接口的成功阈值仍是 20 米。
5. 接收概率评分中使用均匀半径和朝向、全向与定向各半的工作假设。这是行动排序的近似模型，题设未给出这些分布，不将其作为排除证据。

3.2 主要符号

表 2 跨问题共用的符号

符号	含义	单位
$\mathcal{D}, x_c$	源分布圆域、频道 c 的源位置	m
N, R_c, n_c	源总数、接收半径、定向单位法向量	个、m、无量纲
p, h	机器狗当前位置与当前测向频道	m、频道编号
$u(\theta), v(\theta)$	方位单位向量及其垂直单位向量	无量纲
$P_c, \hat{P}_c$	精确相容位置集合及其保守外包	m 坐标集合
U, F, C, A	未知、已发现未清除、已清除、已排除频道集	集合
Q_3, Q_4	全向及混合场景的预设搜索站集	m 坐标集合
M, S, K_s, K_f	测向、切频、光学成功及失败次数	次
L, T_v, T_r	总行进距离、虚拟时间、现实运行时间	m、s、s
E, Φ	剩余试探额度与任务下降秩	非负整数
$\bar{T}, \eta$	平均定位清除时间与实际全清比例	s/源、无量纲

所有角度在文字中用度表示，在三角函数内部换算为弧度。局部二维叉积记作 $[a, b] = a_x b_y - a_y b_x$ 。不同问题的辅助变量在引入处定义，避免把可行域直径、接收半径和清除半径都记成同一个字母。

4 统一观测模型与时间目标

4.1 可接收、方向与近距观测

令 $\mathcal{D} = \{x \in \mathbb{R}^2 : \|x\| \leq 1800\}$ 。全向源在位置 q 可被接收, 当且仅当 $\|q - x_c\| \leq R_c$ 。定向源还须满足

$$n_c^\top(q - x_c) \geq 0, \quad \|n_c\| = 1. \quad (1)$$

这里 n_c 是从源指向有效发射半平面的法向量, 不能与检测点指向源的测向向量混淆。

若可接收且距离大于 5 米, 响应为方向值 θ , 真实源落在以 q 为顶点、方向区间 $[\theta - \varepsilon, \theta + \varepsilon]$ 为边界的前向楔形内。若可接收且距离不超过 5 米, 返回近距响应。若不可接收, 返回无信号。近距响应直接给出 5 米圆盘, 不应人为补出一个方向值。

4.2 动作费用与频道状态

设前后动作位置为 p, q , 当前测向频道为 h 。一次测向与一次光学动作的费用分别为

$$t_m(p, q, c, h) = \frac{\|q - p\|}{5} + 5 + \mathbf{1}_{c \neq h}, \quad (2)$$

$$t_o(p, q, c) = \frac{\|q - p\|}{5} + 3 + 2\mathbf{1}_{\text{清除成功}}. \quad (3)$$

光学动作指定目标频道, 但不改变测向机的当前频道, 也不产生切频费用 [3]。进入和主动退出不推进虚拟时钟。于是整场费用严格分解为

$$T_v = \frac{L}{5} + 5M + S + 5K_s + 3K_f. \quad (4)$$

该公式用于逐条响应回放, 是结果表和费用图的共同计算依据。

4.3 优化与完成约束

令策略 π 根据截至当前的真实观测历史 H_t 选择下一动作。理想目标是对所有合法源状态保证 $K_s = N$, 并使 T_v 尽量小。本文实际采用有界计算预算的在线近似决策, 而不求整个连续信念空间的精确最优策略。

$$\eta = \frac{K_s}{N}, \quad \bar{T} = \frac{T_v}{K_s} \quad (K_s > 0). \quad (5)$$

由于接口不直接返回 N , η 只能在演练结束获得真值后核验。在线停止条件由频道证据定义, 不能以预测的 $\eta = 1$ 替代。

现实运行时间还包括求解和 HTTP 通信。算法始终服从进入时返回的剩余窗口; 本文的几何完备性结论以有效响应、正确数值包络和足够执行时间为条件, 不据虚拟时间较小推断现实窗口必然满足。

5 问题一：交会定位区域与直径

5.1 前向楔形的半平面表达

令第 i 个检测点为 p_i ，示向度为 θ_i ， $u(\theta) = (\cos \theta, \sin \theta)$ 。对应的前向楔形为

$$W_i = \{x : [u(\theta_i - \varepsilon), x - p_i] \geq 0, [u(\theta_i + \varepsilon), x - p_i] \leq 0\}. \quad (6)$$

因为 $2\varepsilon < 180^\circ$ ，上述两式确定的是朝向测量方向的凸楔形。若只保留“靠近示向直线”的条件，会把检测点背后的对称区域错误地纳入定位区域。

将每个楔形写成两条线性不等式，得到

$$P = \bigcap_{i=1}^n W_i = \{x : a_j^\top x \leq b_j, j = 1, \dots, 2n\}. \quad (7)$$

问题一计算的对象是这个纯交会区域。用于后续行动时可以再与源圆域、已接收半径上限等先验相交，但不能在回答问题一时悄悄加入人工边界盒，使原本无界的交集变成一个有限多边形。

5.2 空、无界与有界状态的分类

枚举所有非平行边界线对。若 a_j, a_k 线性独立，其交点通过二元线性方程求出；只有满足全部 $2n$ 条不等式的点才是可行候选。存在至少一个尖楔形时，非空交集不含整条直线，因此存在极点。精确算术下，找不到可行候选可以判空；浮点实现对近平行数值情况保留未决标记。

对非空交集，再考察衰退锥

$$\mathcal{R} = \{d : a_j^\top d \leq 0, j = 1, \dots, 2n\}. \quad (8)$$

若存在非零 $d \in \mathcal{R}$ ，则对可行点 x_0 有 $x_0 + td \in P$ ， $t \geq 0$ ，故直径无穷。二维尖锥的极射线与某约束边界平行，可枚举约束法向量的两个垂直方向并检验。没有观测时直接返回全平面无界状态。

5.3 实现中的数值判据

纯交会模块采用浮点边界枚举，对可行性使用相对容差；近平行交点、无穷值及空集按状态区分返回。后两问的证书模块另外使用有理数裁剪。本文不把前者的浮点直径输出解释为严格区间证明；这一区别也决定了问题一结果的保留小数位和后续行动的安全余量。

5.4 直径计算与复杂度

命题 5.1. 若 P 是非空有界凸多边形, 顶点为 $z_1, \dots, z_m$, 则

$$\text{diam}(P) = \max_{i,j} \|z_i - z_j\|. \quad (9)$$

证明. 任意 $x \in P$ 可写成顶点的凸组合 $x = \sum_i \lambda_i z_i$. 对固定 y , 三角不等式给出 $\|x - y\| \leq \sum_i \lambda_i \|z_i - y\| \leq \max_i \|z_i - y\|$. 再对 y 作凸组合展开, 即可把最大值同时推到两个顶点上. $\square$

对候选点求逆时针凸包后, 采用旋转卡壳枚举对踵点对 [4]. 边界交点枚举有 $O(n^2)$ 对, 每点检验 $O(n)$ 个条件, 因此整体为 $O(n^3)$; 顶点凸包为 $O(m \log m)$, 卡壳扫描为 $O(m)$. 在本题小规模观测下, 直接枚举易于检查, 也能保留清楚的退化状态。

表 3 交会区域直径算法

步骤	运算与输出
1	为每次示向生成前向楔形的两条半平面约束。
2	枚举非平行边界交点, 检验全部约束, 保留可行点。
3	若无可行点, 返回空或数值未决; 若存在非零衰退方向, 返回无界。
4	对剩余点求凸包; 单点直径为 0, 线段直径为端点距离。
5	对多边形执行旋转卡壳, 返回最大距离和对应最远点对。
6	用最远点对中点及所有顶点检验同直径圆的覆盖性。

5.5 同直径圆的充要检验

设最远点对为 a, b , $D = \|a - b\|$. 任何半径 $D/2$ 且覆盖 a, b 的圆, 其圆心必须是 $c = (a + b)/2$. 因此存在同直径覆盖圆的充要条件是

$$\max_j \|z_j - c\| \leq D/2. \quad (10)$$

只检验这两个最远点不够; 所有其他顶点也必须满足条件。若条件不成立, 可另求最小包围圆或使用更保守的半径, 但不能把 $D/2$ 直接作为保证覆盖半径。该结论为后续“整个可行域均在 20 米内”的清除判据提供了必要修正。

5.6 可实现的反例与算例

考虑边长为 d 的正三角形。其区域直径为 d ，最小包围圆半径却为 $d/\sqrt{3}$ ，大于 $d/2$ 。故问题一的第二问答案为**不一定能覆盖**，且失败并非浮点误差造成。

这一反例可以由符合测向结构的楔形实际产生。沿单位正三角形各条逆时针边向后延长 100 米设置检测点，将示向度设为该边方向再加 1° ，误差半宽取 1° 。每个楔形的一条边界与三角形边重合，另一条边界不切入三角形；三个前向楔形之交恰为该三角形。这样，反例既满足凸几何性质，也对应题目允许的观测形式。

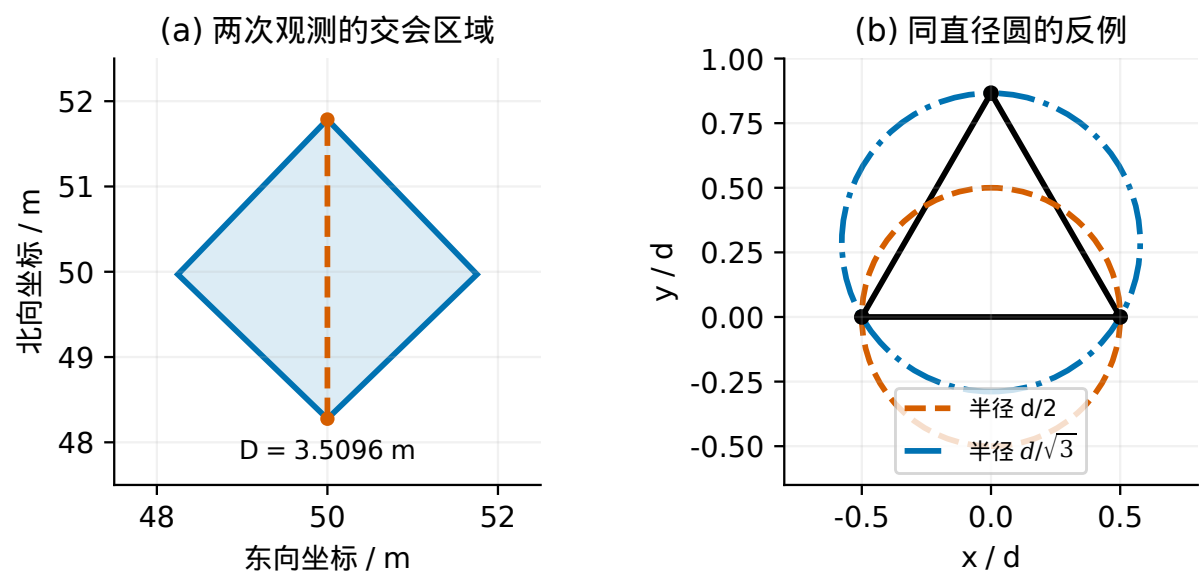


图 2 问题一的数值交会与解析反例。左图取检测点 $(0,0)$ 、 $(100,0)$ 及示向度 45° 、 135° ，误差半宽 1.005° ；虚线连接最远点。右图以边长 d 归一化，虚线半径为 $d/2$ ，点划线为最小包围圆。

表 4 问题一的确定性检验结果

输入情形	输出或理论结果	核验内容
单个正常示向	无界楔形	不引入人工有限边界
前向条件相互矛盾	空或数值未决	不输出虚构有限直径
图中两次示向	$D = 3.5095516$ 米	顶点与最远点对一致
单位正三角形	$D = 1, r_{\min} = 1/\sqrt{3}$	同直径圆覆盖判据不成立

在实际多次测向中，新增有效约束使精确交集满足 $P_{k+1} \subseteq P_k$ ，故直径不会增大。这一集合嵌套性比假设平均误差按样本量缩小更适合本题。若计算结果反而扩张，首先应检查角度环绕、前向方向、单位换算与数值容差，而不能把扩张当作真实源运动。

6 问题二：第二检测点的候选域

6.1 首次接收给出的半径耦合

设首次正常测向位置为 p_1 、示向度为 θ_1 。在局部正交基 $u = u(\theta_1)$ 、 $v = v(\theta_1)$ 下，真实源可写为

$$x = p_1 + r(\cos \delta u + \sin \delta v), \quad 5 < r \leq 1500, \quad |\delta| \leq \varepsilon. \quad (11)$$

因为首次已经接收到信号，未知半径并非独立自由参数，而满足 $R \geq \max\{1000, r\}$ 。设第二点为 $q = p_1 + au + bv$ ，选择前向半平面 $a \geq 0$ 。把 r 放宽到 $[0, 1500]$ 后，保证接收要求

$$\|(a, b) - r(\cos \delta, \sin \delta)\| \leq \max\{1000, r\}, \quad \forall r, \delta. \quad (12)$$

6.2 消去连续距离变量

当 $0 \leq r \leq 1000$ 时，距离平方关于 r 为凸函数，最大值出现在 $r = 0$ 或 $r = 1000$ 。当 $r \geq 1000$ 时，平方后抵消 r^2 ，式(12)等价于

$$a^2 + b^2 \leq 2r(a \cos \delta + b \sin \delta). \quad (13)$$

若 $r = 1000$ 时成立，右侧系数必非负，增大 r 不会使约束更紧。对 $a \geq 0$ 及小角区间，最不利角度是两个区间端点。于是只需检查三个圆盘：

$$\begin{aligned} \mathcal{C} = & B((0, 0), 1000) \cap \{a \geq 0\} \\ & \cap B((1000 \cos \varepsilon, 1000 \sin \varepsilon), 1000) \\ & \cap B((1000 \cos \varepsilon, -1000 \sin \varepsilon), 1000). \end{aligned} \quad (14)$$

该集合是放宽先验和前向条件下的完整保证接收域；对真实 $r > 5$ 及源圆域约束，它是保守的充分候选域。若首次响应为近距，不需要使用此构造，可直接在原位置清除。

6.3 相对于独立最坏半径处理的改进

若忽略首次接收事实，要求全部 1500 米端点都落在第二点的 1000 米接收圆内，会排除许多合法候选。例如 $(a, b) = (500, 840)$ 满足式(14)，却不满足上述独立处理。其改进来自保留观测造成的变量耦合，而不是放松真实接收约束。

6.4 定位精度的解析上界

候选域保证能够接收，但不自动保证交会角度良好。令 $\rho = \sqrt{a^2 + b^2}$ 、 $\alpha = \text{atan2}(b, a)$ ，先考虑 $b > 0$ 的一侧。真实源到第二点的距离上界为

$$d_2(\rho, \alpha) = \max_{r \in \{5, 1500\}} \sqrt{r^2 + \rho^2 - 2r\rho \cos(\alpha + \varepsilon)}. \quad (15)$$

第二点相对于首次方向的最小横向高度为 $h = b \cos \varepsilon - a \sin \varepsilon$ 。若 $h > 0$ ，两条名义测向方向的最小交角可保守估计为

$$\gamma = \arcsin(\min\{1, h/d_2\}) - 2\varepsilon. \quad (16)$$

当 $\gamma \leq 0$ 时，无法由此得到有限的条带交直径上界。令两条包络条带的半宽为 $w_1 = 1500 \sin \varepsilon$ 、 $w_2 = d_2 \sin \varepsilon$ ，则最宽对角线给出

$$D_{\text{ub}} = \frac{2\sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \gamma}}{\sin \gamma}. \quad (17)$$

这是对物理相容区域的外包估计。缩小该上界有助于获得稳健交会，但它与真实区域直径的最小化不是同一问题。

6.5 两种第二点选择方式

给定移动预算 ρ 时，在可行角区间

$$\varepsilon < \alpha < \arccos(\rho/2000) - \varepsilon, \quad 5 < \rho < 1000 \quad (18)$$

内最小化式(17)。给定期望直径阈值 D_0 时，则求

$$\min_{\rho, \alpha} \rho \quad \text{s.t.} \quad D_{\text{ub}}(\rho, \alpha) \leq D_0, (a, b) \in \mathcal{C}. \quad (19)$$

实现按区间维护目标下界和当前可行上界：不能改进当前解的区间剪除，其余区间二分。角度方向利用 d_2 与 h 的单调界；二维移动优化对距离和角度同时建立保守区间界。该思想与有界优化中的可行上界及松弛下界相一致 [5]。

获得 (a, b) 后，另一个对称解为 $(a, -b)$ 。若给定当前机器狗位置，选移动费用较小的一侧；否则使用固定侧作为可复现的规则。比较双方时不改变接收和精度约束。

6.6 计算结果及定位代价的解释

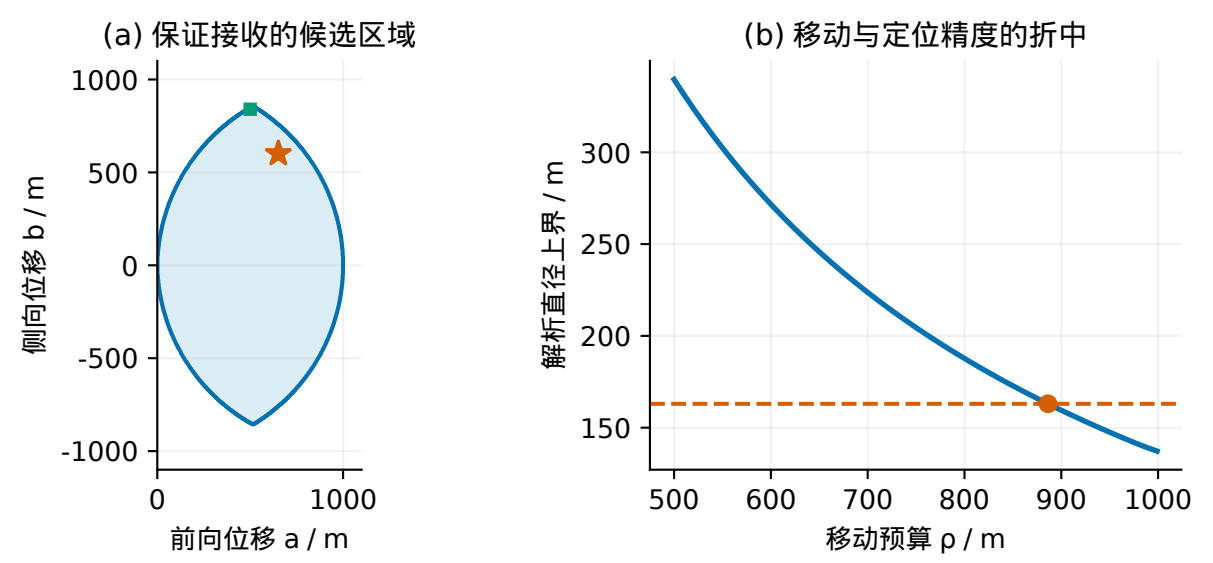


图 3 第二测点的候选域和移动精度折中。左图浅色区域为三个圆盘与前向半平面的交，星号为 163 米解析目标下的可行解，方点为 (500, 840)；右图曲线由当前解析优化程序逐点计算，虚线表示 163 米目标，不是演练实测曲线。

表 5 问题二的解析优化结果

量	计算值	单位
目标直径上界 D_0	163.0000	m
可行移动距离 ρ_U	886.3434	m
移动距离计算下界 ρ_L	886.2614	m
上下界间隙 $\rho_U - \rho_L$	0.08194	m
前向、侧向位移 (a, b)	(651.5213, 600.9365)	m
实际解析上界 D_{ub}	162.9998	m
保证接收的距离余量	113.6566	m

本问的主要数值结果是：在解析直径上界不超过 163 米这一判据下，最小移动距离已被限制在一个宽约 0.082 米的区间内。这只证明该解析设计问题的计算间隙很小，不证明连续源状态下所有自适应定位策略都已达到理论最优。

固定 900 米移动预算的另一种方案给出解析直径上界 159.4987 米。与 886.3434 米方案相比，它进一步压缩定位区，却增加了首段移动。后两问还包含多源移动和 20 米光学清除阈值，因此不能机械地把单源第二点固定为某一“精度最优点”。若当前观测已经足以产生保证清除位置，就应直接比较清除费用。

完整的实际定位次数、最终平均清除用时以及该设计相对固定第二点的演练增益尚需同输入试验，分别预留为_____、_____和_____。这些未测值不影响候选域的几何推导，但会影响第二点策略在整场任务中的经济性评价。

7 问题三：全向源的完整搜索与联合清除

7.1 七站布局的构造

取原点以及半径 1125 米圆周上等角分布的六个点：

$$Q_3 = \{(0, 0)\} \cup \{1125(\cos(k\pi/3), \sin(k\pi/3)) : k = 0, \dots, 5\}. \quad (20)$$

选择 1000 米最小有效半径作为保证依据，而不使用 1500 米最大半径。离原点不超过 1000 米的源由中心站覆盖；其余源选择方位差不超过 30° 的外环站。

命题 7.1. $\mathcal{D} \subseteq \bigcup_{q \in Q_3} B(q, 1000)$ ，故对任意合法全向源，只要在七站逐一检查其频道，必有一次可接收观测。

证明. 设源的极径为 $r \in [1000, 1800]$ ，最近环站与其方位差为 $\delta \leq \pi/6$ 。距离平方不超过 $r^2 + 1125^2 - 2250r \cos(\pi/6)$ 。该式关于 r 为凸函数，最大值在两个端点取得；较大者对应 $r = 1800$ ，距离约 999.1107 米，小于 1000 米。中心站覆盖其余区域。 $\square$

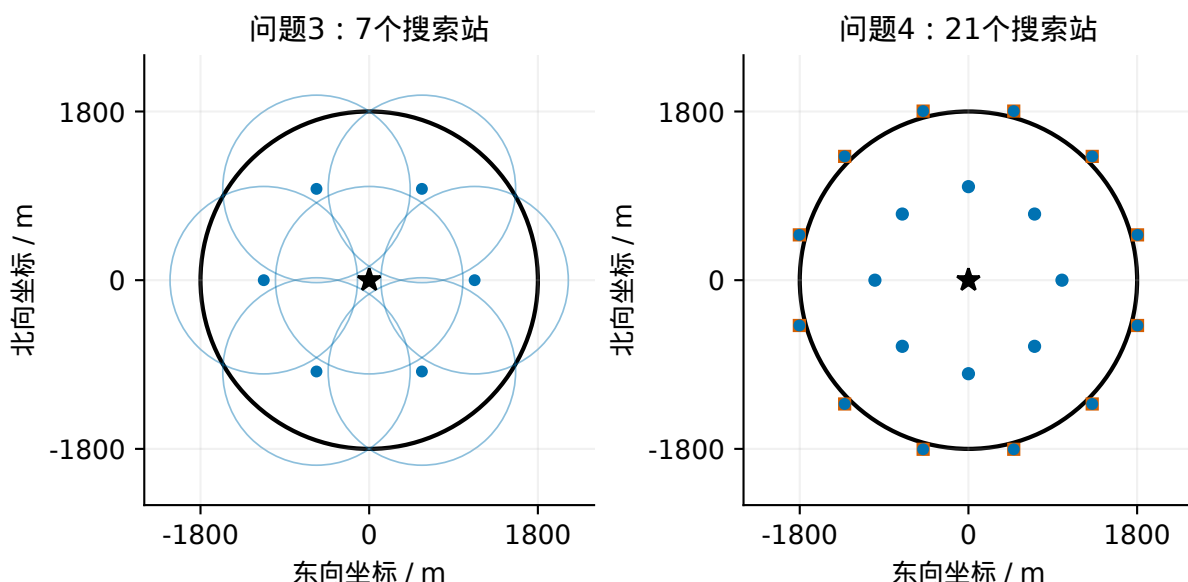


图 4 搜索站布局。左图的浅色圆为 1000 米接收圆，覆盖 1800 米源域；右图为问题四的 21 站，其中外环半径 1868 米。点集只提供初始覆盖结构，实际访问顺序随任务状态调整。

7.2 覆盖与访问顺序的区别

上述命题证明“到这些站逐频道搜索足以发现”，不要求必须把七站走完。已经发现 16 个不同源时，数量上界允许排除其余频道；历史实际观测或清除点若能替代某个剩余搜索站，也可以缩短路线。

程序对删站或替换站后的历史证据点与未来站点之并集重新做连续覆盖核查。通过只表示新的搜索计划具备完整发现能力，不会把尚未走到的未来站点记成实际阴性证据。因此路线可以提前优化，账本中的“无源”结论仍必须等待对应真实观测。

7.3 频道状态与证据账本

每个频道具有四种互斥状态：未知 U 、已发现未清除 F 、已清除 C 、已排除 A 。每次真实观测只更新被请求的频道。几何上接近的两个源也不能交换频道证据，因为不同频道对应不同物理对象。

表 6 频道状态转移规则

当前状态	实际返回或证明	更新方式
U	方向或近距	转为 F ，建立保守可行域和清除责任块
U	无信号	保存该点阴性；覆盖证明成立时转为 A
F	新方向、近距或有效阴性	缩小相容域，更新责任块，不直接标为已清除
F 或 U	光学清除成功	转为 C ，保存实际成功证据
F	光学未命中	删除已被该光学圆完整覆盖的责任块
U	已发现不同源达 16 个	依据题设数量上界转为 A

令 $D_t = F_t \cup C_t$ 。合法停止条件为

$$U_t = \emptyset, \quad F_t = \emptyset, \quad 10 \leq |C_t| \leq 16. \quad (21)$$

若 $|D_t| = 16$ ，其余未知频道可依据唯一频道和数量上界排除；若只发现 10 至 15 个源，则仍存在额外合法源，不能仅据已发现数量退出。

7.4 全向阴性如何压缩已知源域

设 p 为实际接收点， q 为同频道无信号点。全向源具有固定半径 R ，所以 $\|x - p\| \leq R < \|x - q\|$ 。平方后得到

$$2(q - p)^T x < \|q\|^2 - \|p\|^2. \quad (22)$$

实现保留非严格外包半平面，避免边界舍入误删真实源；同时可删除被 $B(q, 1000)$ 完整包含的责任块。式(22)使用了已接收和未接收的相对信息，比单纯记录“在此点没有方向”更有价值。

与此相对，问题四的无信号不能直接使用式(22)，因为无信号也可能来自定向背面。两种场景采用不同的阴性更新规则，是保证算法正确性的关键。

7.5 有限责任块与备用完成过程

正常方向观测给出长度不超过 1500 米的细长可行域。程序沿测向轴布置 61 个纵向位置，横向取 $-25, 0, 25$ 米三层，最多生成 183 个责任块。每个块只保留与当前可行多边形的交，且在建立时核查整个块均落在其对应清除点的 20 米圆内。近距观测只需要一个责任块。

未发现频道另外保留覆盖 $[-1800, 1800]^2$ 的 49 个 600 米网格点。备用策略沿固定蛇形顺序检查未知频道，一旦发现源就执行其有限光学责任序列。网格每个 600 米方格的四角距离均不超过 $600\sqrt{2} < 1000$ 米，且源位于四角凸包内，所以该备用点集也适用于定向情况。

7.6 下降秩与有限性

令 G_c 为未知频道尚未排查的备用网格点， B_c 为已知源尚未消除的责任块， E 为剩余可选试探额度。定义

$$\Phi = \sum_{c \in U} |G_c| + \sum_{c \in F} |B_c| + 183(16 - |D_t|) + E. \quad (23)$$

新发现源最多增加 183 个责任块，同时使第三项减少 183；网格排查、责任块清除或实际源清除会减少相应任务量。若动作没有使上述几何任务量严格下降，必须消耗一个不可重置的试探额度。因此，每次被接受的实际行动均使非负整数 Φ 严格下降。

初始 49 个网格点、20 个频道和 $E = 640$ 给出 $\Phi_0 = 4548$ 。这提供有限动作数的粗上界，不能解读为所有场景都需要这么多动作。任务结束时 Φ 也不必为零，因为实际源数可能小于 16，数量潜力项仍有剩余。

7.7 预算检查与保证的适用范围

每次可选动作前，计算当前状态的保守备用完成费用 $U(H_t, p)$ ，把额外移动、操作和潜在新源清除费用计入检查。若预算不足、规划到期或可选试探不足，则切入备用策略。实际执行还预留请求延迟和退出窗口。

由相容域包含真实源、责任块覆盖及下降秩，可以建立条件性的完整发现和清除结论：在题设物理规则、有效通信及足够执行窗口下，备用过程不会因有限次失败而丢失源。通信中断或现实时间耗尽不属于这个几何命题的保证范围，需在结果中单独报告。

7.8 搜索与已知源的统一服务任务

把剩余搜索站和已知未清除源视为一组任务 $\mathcal{J}$ 。每个任务 j 可具有若干服务模式 m ，分别描述入口 e_{jm} 、预计出口 x_{jm} 以及服务费用 s_{jm} 。搜索任务的入口和出口均为站点，服务费用保守取 6 乘以需要检查的频道数；已知源任务则提供直接清除或不同测向入口方案。

对任务序列 σ 及模式序列 m_i ，用下式评价完整计划：

$$\hat{T}(\sigma, m) = \frac{\|p - e_{\sigma_1 m_1}\|}{5} + s_{\sigma_1 m_1} + \sum_{i=2}^{|\mathcal{J}|} \left[\frac{\|x_{\sigma_{i-1} m_{i-1}} - e_{\sigma_i m_i}\|}{5} + s_{\sigma_i m_i} \right]. \quad (24)$$

入口是计划首先到达的位置，出口则是结束该项服务后的估计位置。源域中心仅用于估计出口，不作为真实源坐标输入证据账本。因此式(24)是排序代理目标，不是整场费用的严格上下界。

7.9 多个初始顺序与模式动态规划

程序比较五种顺序：上一轮仍有效的顺序、贪心顺序、先处理源、先处理搜索站，以及指派松弛经子环连接获得的顺序。指派问题用原始对偶势方法求解，其基础是匈牙利方法 [6]；取消子环约束得到的指派值不能直接充当真实定位任务的最优值。

对固定任务顺序，定义 $f_i(m)$ 为到第 i 项任务并使用模式 m 的最小累计代理费用，则

$$f_i(m) = s_{\sigma_i m} + \min_k \left\{ f_{i-1}(k) + \frac{\|x_{\sigma_{i-1} k} - e_{\sigma_i m}\|}{5} \right\}. \quad (25)$$

这一递推只优化给定顺序的模式组合。随后进行至多两轮单任务移位，将一个任务插入另一位置，并对较有希望的候选重新评价。计算量受到固定轮数和候选数约束，适合实时更新。

7.10 为何一次只承诺当前任务

新观测会改变源域、接收可能性和剩余未知频道，过早固定整场路线容易过时。算法选定当前任务后执行有效前缀；对源任务通常持续处理到清除，再重排剩余任务，同时将下一任务入口作为局部清除出口的参考。这样利用全局顺序信息，又保留新证据到来后的调整能力。

7.11 源内射频测点与连续清除位置

设当前可见锚点为 o ，示向单位向量为 u ，垂向单位向量为 v ，锚点至源距离上界为 L_c 。只有一次方向观测时，候选点取

$$q_{\pm} = o + [5 + (L_c - 5)\lambda]u \pm (L_c/50)v, \quad \lambda \in \{0.38, 0.50, 0.62\}. \quad (26)$$

已有至少两次阳性时，先将当前多边形投影到锚点轴，得到区间 $[l, h]$ ，再改取轴向中点 $(l + h)/2$ 和侧距 $\max\{5, (h - l)/20\}$ 。这样不会重新把历史已经排除的近距离范围纳入下一步设计。

对候选测点，前瞻枚举方向区间、近距及阴性分支，以分支后的完整备用清除费用评价。未能在规划预算内完成的候选不继续搜索。前瞻估计只帮助选动作；实际账本只能根据真实响应更新。

若整个当前源域都能从某位置清除，则可直接求

$$q^* = \arg \min_q \|q - p\| \quad \text{s.t.} \quad \|q - z\| \leq r_s, \quad \forall z \in \text{vert}(\hat{P}_c), \quad r_s = 19.8. \quad (27)$$

约束为圆盘交，是凸集 [5]。最近点可能是当前位置、一个活动圆上的投影，或两个活动圆的交点，故可枚举检查；若交为空，继续定位或构造多点光学序列。程序对候选点再次用整个有理多边形验证，不只检查代表点。

7.12 顺路观测和清除出口优化

已到达一个搜索点或清除点后，若测量其他已发现频道能产生足够视差，可在原地补测以节省再次绕行。评价利用当前沿轴宽度 w 、观测距离 d 及视差指标 κ ，估计补测后宽度

$$w' \approx \min\{w, 2\varepsilon_{\text{rad}}d / \max(0.01, \kappa)\}. \quad (28)$$

此式是费用启发量，不构成几何收缩证书。若选定下一目标站 t ，清除点还可在保证可行域内尝试减小 $\|p - q\| + \|q - t\|$ ，避免“到最近清除点后又绕回”造成额外移动。

同站同时发现多个源时，程序还比较 350 米环上的共享第二观察点。问题三按代理收益选较好方向；问题四另加接收概率和净收益门槛，具体见后文。该差异提示：共享动作在问题三中的实际增益仍需消融验证，不能由局部直觉直接保证。

7.13 问题三的离线结果

表7列出当前快速版的 15 场固定离线输入，合计 192 个源，均获得实际清除成功响应并正常退出。匹配、验证与新输入各类场景分开保留；边界、密集和备用场景的分布及执行方式不同，不把它们混合为一个随机总体的性能估计。

表 7 问题 3 的 15 场离线固定输入结果

案例	源数	总时间/s	平均时间/(s/源)	M	K_f
M1	16	2940.55	183.78	90	0
M2	10	3072.89	307.29	130	0
V10	10	2785.99	278.60	113	0
V12	12	3084.86	257.07	111	0
V14	14	3243.22	231.66	115	0
V16	16	3318.80	207.42	115	3
F10	10	2948.40	294.84	127	2
F12	12	3160.66	263.39	111	1
F14	14	2921.20	208.66	114	4
F16	16	3124.71	195.29	96	1
B10	10	4402.57	440.26	144	2
B16	16	4587.68	286.73	125	5
C10	10	2085.60	208.56	91	0
C16	16	437.67	27.35	33	0
R10	10	15348.44	1534.84	660	249

表中 M 为两场固定匹配输入， V 为验证输入， F 为另行保留的新输入， B 为边界压力输入， C 为密集输入， R 为强制备用过程；M1、M2 为匹配案例编号，其余数字后缀为源个数。完整频道、位置、半径和噪声输入保留在结果记录中。 M 表示测向次数， K_f 表示光学失败次数，均由协议记录统计。

十场 M 、 V 、 F 常规输入的平均总时间为 3060.13 秒。其中移动时间平均占比约 76.01%，说明单纯减少一次 5 秒测向通常不足以解释主要提速；任务次序、共享位置和清除出口对总费用更敏感。16 源密集案例耗时 437.67 秒，比其他案例明显小，是因为源集中且达到数量上界后可以结束未知频道搜索，不表示源越多一定越容易。

强制备用案例耗时 15348.44 秒，显著高于主动调度。其作用是验证在前瞻和主动优化停止后仍能接续完成，不是估计正常快速版的典型平均值。真实演练、正式测试以及现实运行窗口的结果另列，不用本表代替官方测试成绩。

8 问题四：未知定向下的完整策略

8.1 覆盖必须同时排除位置和方向的不确定性

问题三的七个 1000 米圆盘覆盖源域，只说明每个位置附近有测点。对定向源，唯一附近测点可能处于辐射背面，故圆盘覆盖不能直接推出一定发现。问题四需要对每个源位置考虑所有可能的发射半平面。

命题 8.1. 对任意 $x \in \mathcal{D}$ ，若存在 $Q_x \subseteq Q$ 满足

$$\|q - x\| \leq 1000 \quad (q \in Q_x), \quad x \in \text{conv}(Q_x), \quad (29)$$

则任意合法定向方向至少在一个 $q \in Q_x$ 上可被接收。

证明. 由凸包条件可写 $x = \sum_q \lambda_q q$, $\lambda_q \geq 0$, $\sum_q \lambda_q = 1$ 。对任意定向单位法向量 n ，有 $\sum_q \lambda_q n^\top (q - x) = 0$ ，所以不可能所有项都严格小于零。至少一个测点满足 $n^\top (q - x) \geq 0$ ，且其距离不超过最小接收半径 1000 米。 $\square$

8.2 二十一站布局与连续核查

当前快速版采用

$$\begin{aligned} Q_4 = \{(0, 0)\} \cup \{998u(k\pi/4) : k = 0, \dots, 7\} \\ \cup \{1868u(k\pi/6 + \pi/12) : k = 0, \dots, 11\}. \end{aligned} \quad (30)$$

外环允许超过源域边界，作用是从边缘外侧观察朝外发射的源。该点集不沿用旧稿中的 25 站；本文的代码、图形和结果均对应式(30)。

连续证书将源域外包方盒递归细分。对每个尚与源圆相交的小盒，选出能用 1000 米圆覆盖其全部顶点的测点，再检查小盒全部顶点是否位于这些测点的凸包内。圆盘和凸包都是凸集，因此顶点判据可推广到整个小盒。不能通过的盒继续分裂；仅采样中心点通过不算证明。

在当前 20000 节点、14 层深度的核查预算下，21 站证书访问 5013 个节点并留下 2448 个有效叶盒，完整通过。预算耗尽时只返回“未证实”，不能把它转化为无源结论。这种保守判定保留了几何完备性，同时允许对实际历史点做删站替换。

8.3 定向阴性的正确使用

定向情况下，一个实际阳性点 o 与阴性点 a 并不能推出 $\|x - o\| < \|x - a\|$ ： a 可能更近但在背面。因此程序不会将单个定向阴性直接转换为问题三的距离半平面。阴性首先进入接收模型和历史记录，只有满足额外几何条件时才收缩位置域。

8.4 两个阴性点的可验证收缩

设 o 曾实际返回方向值（故 $x \neq o$ ）， a, b 曾实际无信号。对候选源位置 x ，若 a, b 均在其 1000 米内，且 $o - x$ 属于 $a - x, b - x$ 生成的非负锥，则 x 不可能是真实源。

证明. 若源为全向， a, b 在 1000 米内必可接收，已与阴性矛盾。若源为定向，两个阴性在距离已满足时意味着 $n^T(a - x) < 0, n^T(b - x) < 0$ 。而非负锥关系推出 $n^T(o - x) < 0$ ，与 o 曾接收矛盾。边界退化时用闭半平面条件分别检验。□

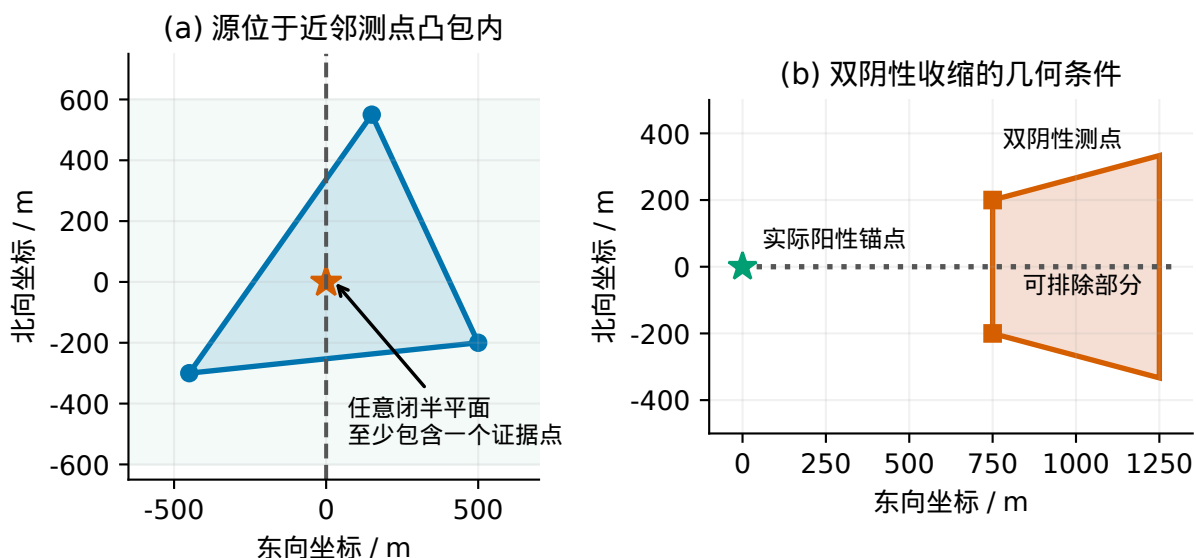


图 5 定向发现与阴性收缩的解析示意。左图源位于近邻测点凸包内，任意发射半平面均有测点可见。右图阴影为双阴性测点连线远离阳性锚点的一侧、且位于相应锥内的示意部分；只有同时通过两个 1000 米圆的包含检验，才可排除。阴影不表示一次实际案例轨迹。

实现用 a, b 连线切出拟排除的多边形，逐顶点核查锥条件及两个 1000 米圆条件。全部通过后才用相应半平面更新源域。位于 o, a, b 三角形内部的候选不能按此理由排除；它可能朝向 o 而背向 a, b ，这是容易混淆的几何边界。

8.5 避免无效重复

同地点电磁环境固定，已在同一源、同一对位置得到双阴性后，无信息变化的重复测量不能改善结论。程序识别这种点对并改用光学责任序列。该处理减少确定性的重复支出；它不允许跳过新的有效搜索点，也不把“重复无信号”累计成更强的统计置信度。

8.6 接收概率只用于动作排序

保证所有源被清除并不要求每一步都取最坏费用。对不会直接删除真实可能性的补测动作，可使用历史观测估计在哪些位置更可能收到信号。当前快速版将责任多边形的中心作为位置代表，并以面积赋权；对每个代表位置，再枚举与真实观测一致的半径区间与朝向区间。

对位置代表 x_i 及半径 r ，所有已接收点必须满足距离约束。定向允许朝向集合记为

$$\Theta_i(r) = \bigcap_{p \in H^+} \{\theta : u(\theta)^\top (p - x_i) \geq 0\} \setminus \bigcup_{q \in H^- : \|q - x_i\| \leq r} \{\theta : u(\theta)^\top (q - x_i) \geq 0\}. \quad (31)$$

程序把跨越 0° 的区间拆成普通区间，按实际负观测距离分割半径轴。代表位置与半径段的权重同时考虑面积、段长、剩余定向角度以及全向可能性，归一化后得到候选点 q 的接收概率估计 $\hat{p}_c(q)$ 。

8.7 概率模型的含义与边界

全向/定向各半、半径在区间内均匀、朝向均匀均为工作假设。真实赛题并未给出这些概率分布；位置代表也不能精确积分连续相容域。因此 $\hat{p}_c(q)$ 不是经过校准的客观概率，更不是“未检出源不存在”的概率。

设补测预计把沿轴宽度从 w 压缩至 w' ，代码使用

$$\hat{G}_c(q) = \hat{p}_c(q) \frac{w - w'}{10} - 6 \quad (32)$$

估计净收益，单位为秒。分母 10 把宽度变化粗略换算成剩余移动费用；6 秒包含一次测向及可能切频。它仅为经验代理量，没有被解释为严格的条件期望值。对有足够视差且 $\hat{G}_c(q) > 0$ 的频道，才考虑原地补测。

8.8 多源共享观察点

当同一搜索站至少新发现三个源时，在当前位置 350 米环上每 30 度设置一个候选。比较多个频道的正净收益之和与相对下一目标的额外绕行费用

$$\Delta T(q) = \frac{350 + \|q - t\| - \|p - t\|}{5}. \quad (33)$$

问题四仅在总收益超过绕行代价时执行，并逐频道检查额度和预算。实际返回阴性仍按阴性处理，不用概率预测补造方位；低概率事件也不会跳过完整备用责任。这保证了评分失准主要影响效率，而不改变证据逻辑。

8.9 完整光学链替代继续测向

光学清除与发射方向无关。当可行区域已经足够窄时，为获得另一条示向而绕到可见侧，可能比直接排查整个区域更昂贵。算法因此生成由 2 至 8 个清除点组成的候选链 $Q = (q_1, \dots, q_k)$ ，并要求

$$\hat{P}_c \subseteq \bigcup_{i=1}^k B(q_i, 19.8). \quad (34)$$

候选点沿测向轴及其垂直方向构造。若无法证明式(34)，这条链不进入比较；不能用“所有代表位置都被覆盖”替代连续区域覆盖。

8.10 Voronoi 责任分区的证书

定义离 q_i 最近的责任区域

$$V_i = \hat{P}_c \cap \bigcap_{j \neq i} \left\{ x : (q_j - q_i)^\top x \leq \frac{\|q_j\|^2 - \|q_i\|^2}{2} \right\}. \quad (35)$$

这些区域覆盖整个 $\hat{P}_c$ 。对每个非空 V_i ，若其所有顶点距 q_i 均不超过 19.8 米，则凸性保证整个 V_i 满足同一距离条件。因此逐个有理多边形裁剪和顶点距离比较，可以得到完整光学链的覆盖证书。

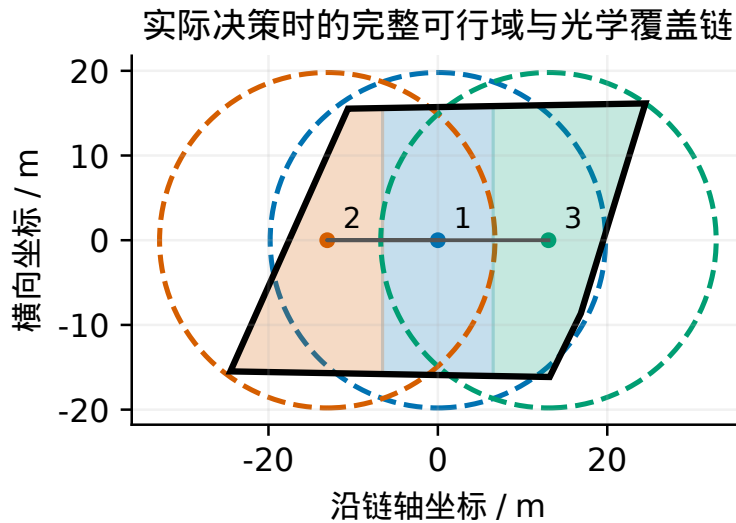


图 6 来自固定离线 Q4 案例的一次实际光学链决策。黑色边界为当时保守源域，虚线圆半径为 19.8 米，填色区为 Voronoi 责任块；数字为候选执行次序。以首点为原点、首末点连线为横轴作刚体变换，不改变覆盖关系。图中不使用源真值。

8.11 为什么执行成功前缀即可

实际源位于至少一个责任区域内，对应清除点必在 20 米以内。算法按序尝试，收到清除成功立即结束该源；此前失败响应用于更新责任区域。覆盖证书约束的是完整链，实际代价却通常只包含成功之前的前缀。这使链的最坏完成保证与平均首次成功费用可以分别评价。

8.12 光学序列的费用比较

给定顺序 Q 及第一次命中第 i 点的估计概率 $\hat{\pi}_i$ ，忽略后续任务时，首次成功的估计费用为

$$\hat{T}_o(Q) = \sum_{i=1}^k \hat{\pi}_i \left[\frac{\|p - q_1\| + \sum_{j=2}^i \|q_{j-1} - q_j\|}{5} + 3(i-1) + 5 \right]. \quad (36)$$

若已确定下一任务入口，还加入从首次成功点到下一入口的预计移动费用。概率由责任块代表和面积权重生成，只用于排序。程序比较轴向顺序、反向顺序和按单位费用命中质量构造的贪心顺序。

当前版本把较好的光学估计费用与一个射频乐观代理费用比较。该代理假定下一次测向顺利产生定位收益，并估计后续移动与清除费用。如果光学方案更便宜，则在完整覆盖及预算成立时选择它。这不是对真实连续问题的严格支配证明；没有把概率评分包装成“必然优于继续测向”的结论。

8.13 实际执行和保守更新

一次光学失败不能把整个已知源删除。程序只移除被该失败点 20 米圆完整覆盖的旧责任块，其余块保留；随后用剩余块的凸包形成保守外包。该操作可能保留部分已经不可能的位置，代价是多做少量工作，而不会因过度相减丢失真实源。

每个完整光学事件在执行前检查所需试探额度及最坏事件代价。执行过程中每个实际响应都要符合协议与计费规则，并生成下降秩记录。只有清除成功响应能改变“已清除”状态，计划中预测的命中不能直接改变状态。当前 30 场回归中实际选中的 62 条完整光学链均通过独立连续覆盖核查。

表 8 问题四的决策层与证据层

环节	用于效率的近似量	必须满足的实际或几何依据
原地补测	接收概率、视差和宽度收益	实际测向返回、额度与窗口
共享第二观察点	多源收益与绕行费用	每频道独立响应，不预判收缩
光学序列排序	代表质量和首次成功估计	完整连续覆盖、实际清除成功
删站与替换	剩余路线和频道检测费用	历史加未来点集覆盖证书
结束运行	不采用期望完成数	式(21)的全清条件

定向场景的时间波动由多个环节共同造成：进入可见侧的绕行、阴性但无法收缩的测量、不同源间的服务出口以及未知频道的最终排除。本文不根据五个不同随机案例的相关性判断其中某一环节的因果贡献，而在后文保留同输入消融表。

8.14 问题四的离线结果与波动解释

当前 15 场 Q4 离线输入合计 198 个源，均实际清除。表9报告总时间、平均每源时间及光学失败次数，含全定向、混合、边界、密集和强制备用输入。所有输入由独立模拟器持有，求解器只访问协议响应。

表 9 问题 4 的 15 场离线固定输入结果

案例	全向/定向	总时间/s	平均时间/(s/源)	K_f
M1	5/11	4551.45	284.47	1
M2	2/8	6229.88	622.99	2
V10	0/10	6365.14	636.51	0
V12	0/12	6534.05	544.50	18
V14	0/14	6399.30	457.09	1
V16	0/16	6680.12	417.51	15
F10	4/6	6381.56	638.16	2
F12	4/8	5939.68	494.97	1
F14	5/9	6359.10	454.22	3
F16	6/10	7176.66	448.54	20
B10	0/10	8060.39	806.04	16
B16	0/16	9206.85	575.43	28
C10	0/10	7148.16	714.82	8
C16	0/16	4290.26	268.14	37
R16	0/16	17850.63	1115.66	893

十场 M、V、F 常规输入平均总时间为 6261.69 秒，移动时间平均占比约 74.01%。与问题三相比，定向约束使“几何上接近”不再保证“可以测向”，进入新视角和排除未知频道的成本明显增加。相比仅根据源个数解释用时，接收结构和最终停止证据更有解释力。

边界 16 源案例耗时 9206.85 秒，说明外向源与边界移动可显著抬高费用；达到 16 源允许停止剩余频道检查，但这一好处未必抵消定位过程的困难。密集 16 源案例仍需 4290.26 秒，也说明位置集中不足以消除方向不可见性。

问题四的一项实际限制是光学链与射频的比较依赖代表点权重和服务出口近似。在真实源位于低权重责任块时，首次成功可能较晚。当前实现通过完整链保证后续仍有清除途径，但无法保证费用相对于旧策略逐场下降。正式性能评价需同时给出均值、最大值、较差案例和全清比例，而不只挑选最短案例。

9 成本下界与小规模精确校准

9.1 未知频道的必要排查代价

考虑源域边界上的潜在源,令边界半径 $R_0 = 1800$ 米,接收半径取最小值 1000 米。一个全向阴性测点最多排除的边界角度为 $\alpha_3 = 2 \arcsin(1000/R_0)$ 。对朝外定向的边界源族,最大角度为 $\alpha_4 = 2 \arctan(1000/R_0)$ 。一次失败光学动作最多排除 $\beta = 2 \arcsin(20/R_0)$ 的边界角度。

若某个实际无源频道用了 m 次测向和 k 次失败光学操作,必要条件为 $m\alpha_p + k\beta \geq 2\pi$ 。角区可以重叠,故此条件仅是下界。对非负整数 m, k 最小化 $5m + 3k$, 得到每个无源频道的必要设备操作代价分别至少 30 秒、35 秒。因此在 $N < 16$ 时,整场有

$$T_v \geq 5N + (20 - N)b_p, \quad b_3 = 30, b_4 = 35. \quad (37)$$

$N = 16$ 时可用数量上界停止,无源频道项改取 0。该界未包含完整移动和定位成本,数值较弱,不能据此宣布当前解逼近理论最优。

9.2 有限世界上的精确前瞻校准

用已知源数的一维有限位置集建立小问题,状态包括相容世界、已清除集合、当前位置和当前频道。对固定动作集求解

$$V(H) = \min_a \max_{y \in \mathcal{Y}(H,a)} \{t(H, a, y) + V(H_{a,y})\}. \quad (38)$$

该值只对枚举的有限世界和动作精确。三个算例的状态数为 16、42、99, 最优费用为 34、37、49 秒; 三层前瞻的费用分别为 34、37、67 秒。第三例仍有 18 秒间隙, 表明增加有限深度并不自动消除全部决策误差。

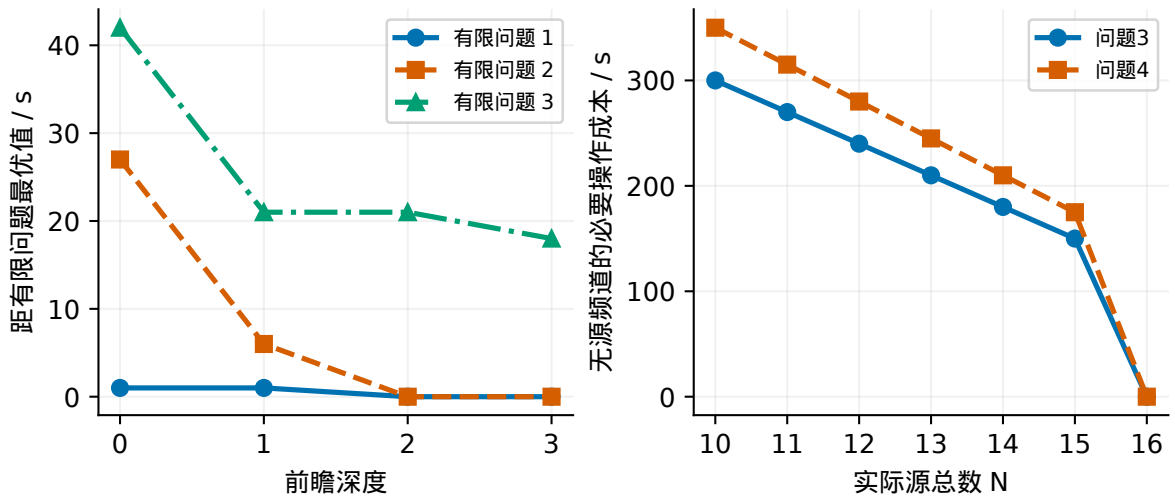


图 7 左图为三个确定性有限问题的前瞻费用与各自最优值之差; 右图为无源频道必要操作代价随实际源数变化的下界, 16 源处的下降来自题设数量上界。二者分别校准局部决策和停止信息成本, 均不替代连续整场最优值。

10 整体检验与同输入比较

10.1 结果与程序的一致性

30 场验证覆盖固定输入、验证输入、新输入、边界、密集和强制备用过程。每场同时检查：模拟器中的实际源全部清除；程序主动退出；每个接受动作的时间增量符合式(4)；证据日志可独立重放；下降秩严格递减；完整光学链的连续覆盖、额度与成功前缀费用一致。合计 390 个源全部清除，实际选用的 62 条光学链通过审计。

此前代码风格整理的 49 项测试全部通过，并与原快速版核对 30 场完整请求指纹一致。该证据说明文件和代码风格调整没有改变这些固定输入上的算法行为，不属于一次新的性能提升，也不能证明全部未知输入上的绝对无差异。

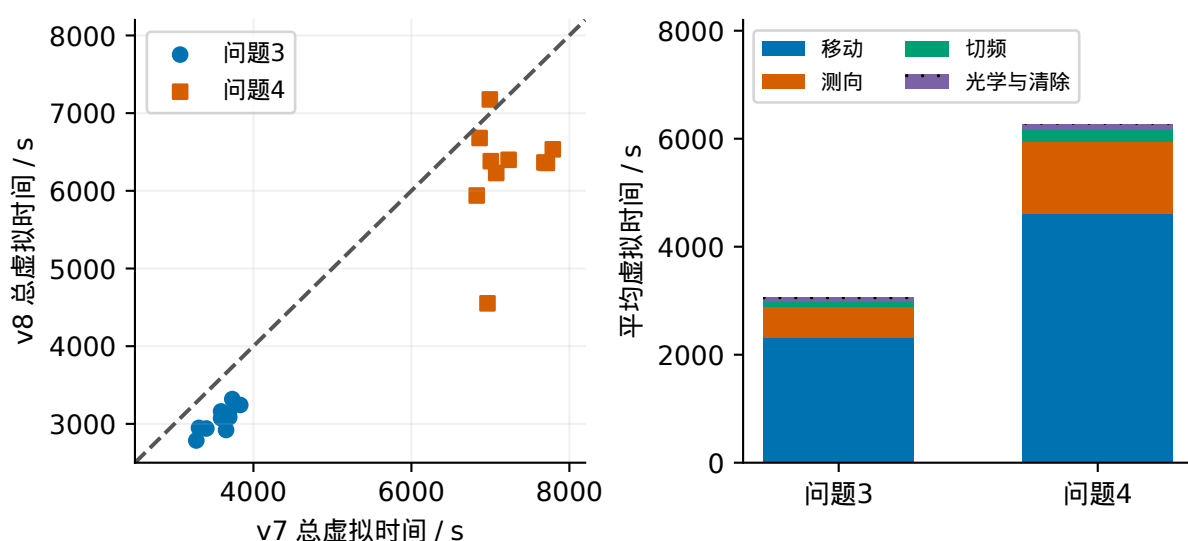


图 8 同输入性能与费用构成。左图每个点为同一源布局 and 噪声下的 v7、v8 总时间，虚线为两者相等；右图为两题各 10 场常规输入的平均费用，按移动、测向、切频、光学与清除分解。未加入误差条，因为这些点是固定案例，不是同一随机量的重复测量。

表 10 十场同输入比较的分题汇总

问题	v8 平均时间/s	平均节省/s	汇总时间降幅	变快场数
三	3060.13	517.16	14.46%	10/10
四	6261.69	952.62	13.20%	9/10

20 场中有 13 场节省至少 500 秒，但不能据此提出逐场“至少 500 秒”的保证。Q4 新输入 16 源案例由 6992.55 秒变为 7176.66 秒，反而增加 184.11 秒。该反例说明代理服务费用和接收估计可能误排访问顺序。后续应以相同输入重跑消融定位原因，不能从平均改进中删除不利案例。

10.2 几何与边界压力检验

目前已有的固定测试包括：测角区间的环绕、有理数距离上下取整、源域边界保留、两阴性收缩的证据前提、七站不能证明定向无源、未知频道不得共享证据、16 源数量停止、光学失败后仍保留真实位置，以及日志被篡改时拒绝完成结论。边界与密集压力输入都采用固定的极端正负一度误差，不通过多次同点平均降低误差。

这些测试检验离散实现中的关键条件。要评价参数选择对总时间的影响，还需要控制源位置、半径、方向和噪声，仅改变一个参数的同输入实验。下表保留尚未完成的结果，空栏不表示零变化。

表 11 待完成的参数敏感性实验

参数	比较水平	Q3 平均时间/s	Q4 平均时间/s
共享观察半径	250、350、450 m	_____	_____
射频前瞻预算	0.05、0.20、0.50 s	_____	_____
初始可选额度	320、640、960 次	_____	_____
安全清除半径	19.5、19.8、19.9 m	_____	_____

每组都须同时报告全清比例、最差案例时间、现实运行时间和备用触发次数。若缩小安全余量带来时间下降，还必须重新检查连续覆盖与数值边界，不能只比较平均时间。若前瞻预算增加而虚拟时间不变，应检查是否原候选已足够、是否决策主要由备用条件控制，或额外计算没有改变所选动作。

表 12 待完成的同输入消融实验

对照设置	全清比例	相对完整策略 $\Delta T/s$	现实时间/s
关闭原地补测	_____	_____	_____
关闭共享第二测点	_____	_____	_____
关闭任务重排，仅固定站序	_____	_____	_____
关闭 2-8 点光学链，保留完整备用	_____	_____	_____
Q4 接收评分不使用历史阴性	_____	_____	_____

预先约定 $\Delta T = T_{\text{对照}} - T_{\text{完整}}$ ，正值表示该模块在对应输入中有收益。两种方法必须使用相同案例，并保留失败和超时；模块间有相互作用，单项收益不能简单相加。上述分析方法已经确定，具体效应大小留待实际运行后填写。

11 官方演练汇总与正式测试安排

11.1 已经提供的演练记录

参赛队保存了两题各五份演练日志及按运行顺序排列的截图。当前可核对日志的案例编码、题号及时间等公开封装信息，正文为加密数据；尚未从中解析逐步动作与真实源布局。因此表13仅记载截图报告的源个数和整数总时间，不声称已独立解密验证全清比例。

表 13 十次演练截图的成绩汇总

题号/次序	案例编码	报告全向/定向	报告个数	总时间（约）/s
3/1	4GC8-BUAN-JTNR-732X	16/0	16	3233
3/2	ZAJ4-Y9H4-X94F-84RF	11/0	11	2595
3/3	PXX5-MVVF-4U5N-RJ29	15/0	15	3123
3/4	YVXK-H49D-HS89-HBGH	12/0	12	2826
3/5	VD4K-3FT8-EMN3-KHMN	11/0	11	3189
4/1	WZM3-U3MR-YW22-QHNC	0/14	14	7570
4/2	ZG4N-CQJP-AZ7F-6M34	14/2	16	4559
4/3	2AA5-BWCU-M8MH-M9A2	3/8	11	6439
4/4	C2MT-ZV9Z-3G7M-YE4G	2/9	11	7191
4/5	6QJH-B56T-UGBD-NJZ3	9/2	11	6249

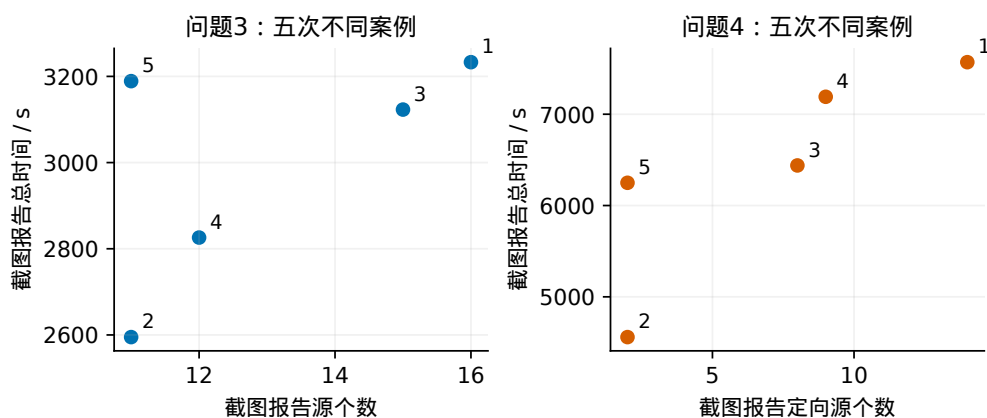


图 9 十次演练截图的描述性比较，点旁数字为运行顺序。总时间为截图取整值；Q4 横轴为报告的定向源数。不同点来自不同案例，未控制位置、半径、方向和噪声，因此未拟合因果趋势线。

问题四时间为 4559 至 7570 秒，跨度 3011 秒，大于问题三的 638 秒。定向较多可能增加绕行与无效测向，但源位置及数量也不同。全向 9 个、定向 2 个的案例仍耗时 6249 秒，说明仅用定向比例无法解释全部波动。需用同次运行的明文动作日志分解移动、测向、失败光学与末尾排除成本，再做同输入消融。

每次演练的实际清除比例和现实运行时间目前分别预留为_____、_____。只有补齐对应原始结果后，才能按题设定义计算平均定位清除时间，不将截图报告源数未经核对地当作清除成功数。

11.2 问题三正式测试结果

题目要求在充分演练后进行三次正式测试，并保留模拟器导出的原始日志 [1, 2]。已取得的正式成绩列于下表，尚未取得的字段保留空栏。测试完成后据实填写相应字段，不用其他演练或离线案例的数值占位。

表 14 问题三正式测试结果（空栏待实测）

测试案例编码	清除源个数	平均定位清除时间/s	程序运行时间/s
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

11.3 问题四正式测试结果

表 15 问题四正式测试结果（空栏待实测）

测试案例编码	清除源个数	平均定位清除时间/s	程序运行时间/s
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

11.4 取得数据后的计算与分析

对每次正式测试，先匹配案例编码、题号、策略标识和原始日志，核对实际清除成功数 K_s 及总虚拟时间 T_v ；再按 $\bar{T} = T_v/K_s$ 计算平均定位清除时间，现实运行时间按模拟器给出的记录填写。只有存在实际源总数 N 的可信结果时，才另报 $\eta = K_s/N$ ，不可用程序自身的完成声明替代外部真值。

三次结果的分析同时报告均值、最大值和取值范围。样本数仅为 3，不能据此稳定估计极端尾部概率；若一场明显较慢，应保留该场，检查是否由移动、定向阴性、光学失败或未知频道排除构成主要费用。若发生超时或未清完，则报告实际完成数、结束原因和已消耗时间，不只对成功场计算平均表现。

六份正式日志的原始文件名分别待填入支撑材料清单，不改名、不补造文件。材料整理和论文编译过程均不启动正式模块。本节保留完整的数据取得、指标计算和结果分析框架，待实测数据到位后即可补写具体比较结论。

12 模型评价与结论

12.1 主要优点

证据与费用分层。覆盖证书、实际成功响应及下降秩决定完成性，概率与路径代理决定动作次序。该结构允许近似评分存在误差，同时使关键完成结论能够独立回放核查。

统一考虑定位与移动。第二点设计利用首次接收约束，多源排序考虑服务入口与出口，狭窄源域可直接转为完整光学链。这些处理把几何精度转化为实际任务费用，而不是无限追求更精确的坐标估计。

对定向不可见作显式处理。21 站布局通过连续凸包覆盖核查，单个阴性不被误当成距离证据，双阴性收缩要求真实观测前提。完整备用过程使主动策略在计算预算不足时仍有可接续路径。

12.2 局限与可改进方向

全清结论以题设物理条件、正确数值包络及足够执行窗口为前提；当前入口对通信异常直接停止，没有自动重试。接收评分、源出口和光学首次成功质量均为近似，尚缺独立校准。30 场固定输入检验了实现行为，不能穷尽所有连续位置、半径和朝向；用于开发的输入也不能代替未见场景上的统计保证。

当前下界弱于实际总时间，有限问题最优值只适用于离散动作族。故本文不宣称已逼近连续原问题的全局理论最优。后续可在保持证据账本不变的前提下，先做同输入模块消融，再提高服务费用与接收模型的准确度，并研究包含移动和信息获取的更强下界。

12.3 结论

问题一建立了前向楔形交直径算法，并否定同直径圆必然覆盖的命题。问题二得到保证接收的三圆盘候选域，以及 163 米解析目标下约 886.34 米的可行移动设计。问题三、四分别采用七站与 21 站的连续发现依据，将频道证据、联合任务排序、共享观测和光学清除连接为在线策略。

30 场离线验证中 390 个源全部清除，20 场历史同输入比较有 19 场变快；Q4 的一个反例仍变慢，说明平均收益不等于逐场优势。正式测试与尚未完成的敏感性、消融结果均保留待填字段。当前结论由已验证的计算和观测范围支持，新增实测将用于补充效率与可靠性评价。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于算法讨论、代码编写与调试、资料整理、数值核查、图表生成和论文初稿撰写，详细使用情况见支撑材料。本稿的人工修改与逐项核验情况尚需参赛队据实补充，不将自动化测试通过等同于人工审查完成 [7]。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题: 无线电干扰源的快速自动定位与清除 [Z]. 2026.
- [2] 全国大学生数学建模竞赛组委会. 2026 年 B 题附件 1: 模拟器测试说明 [Z]. 2026.
- [3] 全国大学生数学建模竞赛组委会. 2026 年 B 题附件 2: 模拟器通信接口说明及编程指南 [Z]. 2026.
- [4] TOUSSAINT G T. Solving geometric problems with the rotating calipers[C]//IEEE MELECON'83. Athens, 1983: A10.02/1-4. <https://www-cgri.cs.mcgill.ca/~godfried/research/calipers.html>.
- [5] BOYD S, VANDENBERGHE L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004. <https://stanford.edu/~boyd/cvxbook/>.
- [6] KUHN H W. The Hungarian method for the assignment problem[J]. Naval Research Logistics Quarterly, 1955, 2(1-2): 83-97. DOI:10.1002/nav.3800020109.
- [7] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定 (2026 年试行) [EB/OL]. (2026-08-03)[2026-09-13]. https://www.mcm.edu.cn/html_cn/node/fe94648f2836ab6cc81586f4c38512b.html.

附录：支撑材料索引

文件或目录	对应内容
run_robot.py、start_robot.ps1	第三、四题运行入口
src/jammer_solver/	当前算法、精确几何、协议及前两问计算
solve_questions.py	前两问 JSON 输入输出入口
check.py、tests/	几何、协议、全清条件和集成检查
tools/	独立模拟、证据回放、光学链审计与下界计算
validation/	固定输入、离线记录与论文对照复算
paper/results.json、paper/figures/	论文数值汇总和可重生成图形
AI 工具使用详情.pdf	已知 AI 使用环节及人工核验待填记录
正式测试原始.jlog	问题三、四各三份，待实测生成

正文及参考文献、材料索引共 30 页，之后为完整程序清单。正式原始日志放入 submission_inputs/formal/ 下的六个对应目录；未测结果及人工核验事项据实补全。所有空栏表示尚未取得对应数据，不表示零值或测试失败。

附录：完整程序清单

下列程序按提交时的实际文件逐字列出，未省略函数或实现。主运行策略为 v8 快速版 r4；v7 仅用于复算正文保留的历史对照，不能从比赛入口切换。支撑压缩包保留相同目录，文件散列见程序清单索引。共 49 个源文件、9184 行。

run_robot.py

文件行数：86。

```
1 import argparse
2 from datetime import datetime
3 import json
4 from pathlib import Path as lj
5 import sys
6
7 sys.path.insert(0, str(lj(__file__).resolve().parent / "src"))
8 from jammer_solver.protocol import xjtk
9 from jammer_solver import solver
10
11
12 def main():
13     parser = argparse.ArgumentParser(description="2026 B 题: v8 快速版")
14     parser.add_argument("--problem", type=int, choices=(3, 4), required=True)
15     parser.add_argument(
16         "--case-code", required=True, help=" 模拟器界面显示的案例编码", dest="al"
17     )
18     parser.add_argument(
19         "--robot-id", help=" 运行时输入当前登录的队号，不写入提交源码", dest="dh"
20     )
21     parser.add_argument("--base-url", default="http://127.0.0.1:2026", dest="url")
22     parser.add_argument(
23         "--run-kind", choices=("rehearsal", "formal"), default="rehearsal", dest="lx"
24     )
25     parser.add_argument(
26         "--output", type=lj, default=lj(__file__).resolve().parent / "runs"
27     )
28     parser.add_argument("--connect", action="store_true", help=" 连接已就绪的接口")
29     parser.add_argument("--v8", action="store_true", help=argparse.SUPPRESS)
30     parser.add_argument(
31         "--planning-seconds",
32         "--v8-planning-seconds",
33         type=float,
34         default=0.2,
35         dest="ghsj",
36     )
37     parser.add_argument(
38         "--extra-actions", "--v8-extra-actions", type=int, default=640, dest="dzys"
39     )
40     cs = parser.parse_args()
41     if not cs.connect:
42         print(" 未连接。确认模拟器处于对应演练模式且接口就绪后，加 --connect 运行。")
43         return 0
44     if not cs.dh or not cs.dh.strip():
45         parser.error(" 连接模拟器时必须通过 --robot-id 输入当前登录的队号")
46     mulu = cs.output / f"{cs.lx}_q{cs.problem}_{datetime.now():%Y%m%d_%H%M%S%f}"
47     mulu.mkdir(parents=True)
48     xx = dict(
49         problem=cs.problem,
50         case_code=cs.al,
51         run_kind=cs.lx,
52         strategy=solver.celue,
53         policy_revision=solver.banben,
54         robot_id=cs.dh,
55         base_url=cs.url,
56         note=" 题号、测试类型和案例编码由操作者提供；接口不返回这些信息。",
57     )
```

```

58     (mulu / "metadata.json").write_text(
59         json.dumps(xx, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
60     )
61
62     def jd(sj):
63         if sj["event"] == "cleared":
64             print(
65                 f"已清除频道 {sj['channel']}, 累计 {sj['cleared_count']} 个, 虚拟时间
66                 ↪ {sj['virtual_time_s']:.3f} 秒",
67                 flush=True,
68             )
69         elif sj["event"] == "search_point_done":
70             print(f" 搜索点已完成 {sj['visited']}, 剩余 {sj['remaining']}", flush=True)
71
72     with (mulu / "actions.jsonl").open("x", encoding="utf-8") as rizhi:
73         jk = xjzk(cs.dh, rizhi, cs.url)
74         zt = solver.xjzt(jk, cs.problem, jd, ghsj=cs.ghsj, dzys=cs.dzys)
75         jieguo = solver.yunxing(zt)
76         jieguo.update(xx)
77         jieguo.update(error=None, unresolved_request=jk["pending"])
78         (mulu / "summary.json").write_text(
79             json.dumps(jieguo, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
80         )
81     print(json.dumps(jieguo, ensure_ascii=False, indent=2))
82     print(f" 本地记录: {mulu}")
83     return 0 if zt["certificate"] and zt["exit_confirmed"] else 2
84
85 if __name__ == "__main__":
86     sys.exit(main())

```

solve_questions.py

文件行数: 8。

```

1 from pathlib import Path as lj
2 import sys
3
4 sys.path.insert(0, str(lj(__file__).resolve().parent / "src"))
5 from jammer_solver.questions import main
6
7 if __name__ == "__main__":
8     main()

```

start_robot.ps1

文件行数: 21。

```

1 param(
2     [ValidateSet(3,4)][int]$problem = 4,
3     [ValidateSet('rehearsal','formal')][string]$runkind = 'rehearsal',
4     [string]$cascode,
5     [string]$robotid,
6     [switch]$v8,
7     [string]$python = 'python'
8 )
9 $ErrorActionPreference = 'Stop'
10 $runtime = (Get-Command $python -CommandType Application -ErrorAction Stop).Source
11 if ([string]::IsNullOrEmpty($cascode)) { $cascode = Read-Host 'Case code shown in
12     ↪ simulator' }
13 if ([string]::IsNullOrEmpty($cascode)) { throw 'Case code required' }
14 if ([string]::IsNullOrEmpty($robotid)) { $robotid = Read-Host 'Team ID currently logged into
15     ↪ simulator' }
16 if ([string]::IsNullOrEmpty($robotid)) { throw 'Team ID required at runtime' }
17 Write-Host "Question=$problem Run=$runkind Team=$robotid Case=$cascode"
18 Write-Host 'Policy=20260912-shared-service-r4'
19 Write-Host 'Start the matching simulator module yourself. This script does not start it.'

```

```

18 $confirmation = Read-Host 'Type CONNECT to send robot commands to the ready local interface'
19 if ($confirmation -cne 'CONNECT') { Write-Host 'No connection made.'; exit 0 }
20 & $runtime (Join-Path $PSScriptRoot 'run_robot.py') --problem $problem --run-kind $runkind
   ↳ --case-code $casecode --robot-id $robotid --connect
21 exit $LASTEXITCODE

```

check.py

文件行数: 15。

```

1 from pathlib import Path
2 import sys
3 import unittest
4
5
6 def main():
7     root = Path(__file__).resolve().parent
8     sys.path[:0] = [str(root / "src"), str(root / "tools")]
9     suite = unittest.defaultTestLoader.discover(str(root / "tests"))
10    result = unittest.TextTestRunner(verbosity=2).run(suite)
11    return 0 if result.wasSuccessful() else 1
12
13
14 if __name__ == "__main__":
15     raise SystemExit(main())

```

prepare_submission.py

文件行数: 225。

```

1 """Build the submission from an explicit file list; never connect to a simulator."""
2
3 import argparse
4 import csv
5 import hashlib
6 import json
7 import math
8 from pathlib import Path
9 import re
10 import sys
11 import zipfile
12
13 ROOT = Path(__file__).resolve().parent
14 sys.path.insert(0, str(ROOT / 'paper'))
15 from generate_appendix import manifest
16 from build import input_hashes
17
18 LIMIT = 20 * 1024 * 1024
19 REVIEW_KEYS = ('model_and_proofs_reviewed', 'code_reviewed', 'data_and_figures_reviewed',
20               'references_reviewed', 'ai_disclosure_completed', 'final_paper_checked')
21
22
23 def sha(data):
24     return hashlib.sha256(data).hexdigest()
25
26
27 def formal_metadata(path):
28     # Read the public envelope only. The encrypted and signed file is never edited.
29     with path.open('rb') as f:
30         prefix = f.read(262144)
31         if not prefix.startswith(b'JMBPLOG1'):
32             raise ValueError('不是可识别的模拟器日志')
33         start = prefix.find(b'{', 8, 64)
34         if start < 0:
35             raise ValueError('日志缺少公共元数据')
36         return json.JSONDecoder().raw_decode(prefix[start:].decode('utf-8', errors='replace'))[0]
37

```

```

38
39 def check_inputs(root=ROOT):
40     pending, errors, logs = [], [], []
41     with (root / 'submission_inputs/formal_results.csv').open(encoding='utf-8-sig', newline='')
42         ↪ as f:
43             rows = list(csv.DictReader(f))
44             slots = {(str(q), str(i)) for q in (3, 4) for i in (1, 2, 3)}
45             if len(rows) != 6 or {(r.get('problem'), r.get('test')) for r in rows} != slots:
46                 return ['正式结果必须包含 Q3、Q4 各三行'], ['正式结果行号不完整或重复'], []
47             codes = set()
48             for row in rows:
49                 label = f"Q{row['problem']} 第 {row['test']} 次"
50                 directory = root / f"submission_inputs/formal/q{row['problem']}/test_{row['test']}"
51                 found = sorted(directory.glob('*.jlog'))
52                 if not found:
53                     pending.append(f'{label}: 缺少原始正式日志')
54                 elif len(found) != 1:
55                     errors.append(f'{label}: 每个目录只能放同一次测试的一份日志')
56                 else:
57                     log = found[0]
58                     if log.is_symlink():
59                         errors.append(f'{label}: 不接受符号链接')
60                         continue
61                     try:
62                         meta = formal_metadata(log)
63                         if meta.get('package_type') != 'formal_behavior_log':
64                             raise ValueError('必须是正式日志, 演练日志不能代替')
65                         if int(meta.get('problem_no', -1)) != int(row['problem']):
66                             raise ValueError('日志题号与目录不一致')
67                         if int(meta.get('formal_index', -1)) != int(row['test']):
68                             raise ValueError('日志正式次数与目录不一致')
69                         if str(meta.get('case_code', '')) != row['case_code'].strip():
70                             raise ValueError('日志案例编码与表格不一致')
71                         if row['log_filename'].strip() != log.name:
72                             raise ValueError('表格中的原始文件名与日志不一致')
73                         logs.append(log)
74                     except (ValueError, TypeError, KeyError) as exc:
75                         errors.append(f'{label}: {exc}')
76             for key in ('case_code', 'cleared_count', 'virtual_time_s', 'real_time_s',
77                 ↪ 'log_filename'):
78                 if not row.get(key, '').strip():
79                     pending.append(f'{label}: {key} 待填')
80             code = row['case_code'].strip()
81             if code:
82                 if code in codes:
83                     errors.append(f'{label}: 案例编码重复')
84                 codes.add(code)
85             try:
86                 numbers = {k: float(row[k]) for k in ('cleared_count', 'virtual_time_s',
87                 ↪ 'average_time_s', 'real_time_s') if row.get(k, '').strip()}
88                 if any(not math.isfinite(v) or v < 0 for v in numbers.values()):
89                     raise ValueError('数值必须为有限非负数')
90                 count = numbers.get('cleared_count')
91                 if count is not None and (not count.is_integer() or count > 16):
92                     raise ValueError('实际清除数必须为 0 至 16 的整数')
93                 if count and 'virtual_time_s' in numbers:
94                     if 'average_time_s' not in numbers:
95                         pending.append(f'{label}: 平均时间待填 (总虚拟时间/实际清除数)')
96                     elif abs(numbers['average_time_s'] - numbers['virtual_time_s'] / count) > 0.02:
97                         raise ValueError('平均时间与总时间/清除数不一致')
98                 if count == 0 and row['average_time_s'].strip():
99                     raise ValueError('清除数为 0 时平均时间应留空')
100             except ValueError as exc:
101                 errors.append(f'{label}: {exc}')
102             if row.get('upload_confirmed', '').strip().lower() not in ('true', 'yes', '1'):
103                 pending.append(f'{label}: 尚未确认模拟器日志上传成功')
104     review = json.loads((root /
105         ↪ 'submission_inputs/human_review.json').read_text(encoding='utf-8'))

```

```

102     for key in REVIEW_KEYS:
103         if review.get(key) is not True:
104             pending.append(f'人工核验待完成: {key}')
105     ai_source = root / 'paper/ai-usage.tex'
106     if ai_source.exists() and any(r'\blank' in line and r'\newcommand' not in line
107                                  for line in ai_source.read_text(encoding='utf-8').splitlines()):
108         pending.append('AI 使用详情仍有待填项: 请据实补充工具信息和人工核验记录后重新编译')
109     for key, description in (
110         ('q2_comparison', '问题二的实测对照'), ('sensitivity', '参数敏感性实验'),
111         ('ablation', '同输入消融实验'), ('practice_details', '演练清除数和现实时间核对')):
112         if not any(p.is_file() and p.name not in ('.gitkeep', 'README.md') for p in (root /
113         ↪ 'submission_inputs' / key).rglob('*')):
114             pending.append(f'{description}: 当前论文有空栏, 需补测填写或据实改为未开展的后续工作')
115     return pending, errors, logs
116
117 def support_files(root=ROOT):
118     names = ['README.md', 'run_robot.py', 'solve_questions.py', 'start_robot.ps1',
119             'check.py', 'prepare_submission.py', 'AI 工具使用详情.pdf']
120     paths = [root / name for name in names]
121     for directory in ('src', 'tools', 'tests', 'docs', 'validation', 'paper',
122     ↪ 'submission_inputs'):
123         for p in sorted((root / directory).rglob('*')):
124             if not p.is_file() or any(part.startswith('.') or part == '__pycache__' for part in
125             ↪ p.relative_to(root).parts):
126                 continue
127             if p.suffix not in ('.py', '.ps1', '.md', '.json', '.jsonl', '.csv', '.tex', '.pdf'):
128                 continue
129             paths.append(p)
130     return paths
131
132 def check_anonymity(files):
133     # Pattern checks assist review; they cannot infer every name or school affiliation.
134     patterns = [r'(?![A-Za-z0-9.])\d{12}(?![A-Za-z0-9.])', '/' + r'home/[^\s]/+',
135                r'[A-Za-z0-9_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}',
136                r'https?://github\.com/[^\s]/+/'
137     ]
138     flagged = []
139     for path in files:
140         if path.suffix in ('.pdf', '.jlog'):
141             continue
142             value = path.read_text(encoding='utf-8-sig')
143             if any(re.search(pattern, value) for pattern in patterns):
144                 flagged.append(path.name)
145     return sorted(set(flagged))
146
147 def write_zip(destination, entries):
148     # Fixed timestamps make identical inputs produce identical ZIP bytes and MD5.
149     with zipfile.ZipFile(destination, 'w', compression=zipfile.ZIP_DEFLATED, compresslevel=9) as
150     ↪ archive:
151         for name, data in sorted(entries.items()):
152             info = zipfile.ZipInfo(name, date_time=(2026, 1, 1, 0, 0, 0))
153             info.compress_type = zipfile.ZIP_DEFLATED
154             info.external_attr = 0o100644 << 16
155             archive.writestr(info, data, compress_type=zipfile.ZIP_DEFLATED, compresslevel=9)
156
157 def prepare(root=ROOT, check_only=False):
158     pending, errors, logs = check_inputs(root)
159     files = support_files(root)
160     if any(p.is_symlink() or not p.resolve().is_relative_to(root.resolve()) for p in files):
161         errors.append('支撑材料不接受外部路径或符号链接')
162     flagged = check_anonymity(files)
163     if flagged:
164         errors.append('匿名检查发现待人工核对文件: ' + ', '.join(flagged))
165     try:
166         report = json.loads((root / 'paper/build-report.json').read_text(encoding='utf-8'))

```

```

166     if report.get('input_sha256') != input_hashes(root):
167         errors.append('论文输入已变更, 请先运行 python paper/build.py')
168     if json.loads((root / 'paper/source_manifest.json').read_text(encoding='utf-8')) !=
    ↪ manifest(root):
169         errors.append('完整程序附录已过期, 请重新编译论文')
170     for record in report['documents']:
171         path = root / record['file']
172         if sha(path.read_bytes()) != record['sha256']:
173             errors.append(f"PDF 与构建记录不一致: {record['file']}")
174         if path.stat().st_size > LIMIT:
175             errors.append(f"PDF 超过 20MB: {record['file']}")
176 except (OSError, KeyError, ValueError):
177     errors.append('缺少有效论文或构建记录, 请先编译论文')
178 status = dict(ready=not pending and not errors, pending=pending, errors=errors,
179             formal_logs=len(logs), official_simulator_used=False,
180             anonymity='Custom text checked; original signed official logs are preserved
    ↪ verbatim and may contain system identity fields.')
181 if check_only:
182     return status
183 output = root / 'submission'
184 output.mkdir(exist_ok=True)
185 if not errors:
186     entries = {p.relative_to(root).as_posix(): p.read_bytes() for p in files}
187     # The official AI disclosure is at the archive root.
188     for p in logs:
189         entries[p.relative_to(root).as_posix()] = p.read_bytes()
190     for p in (root / 'submission_inputs').rglob('.gitkeep'):
191         entries[p.relative_to(root).as_posix()] = b''
192     entries['FILE_MANIFEST.json'] = (json.dumps(
193         {name: dict(bytes=len(data), sha256=sha(data)) for name, data in
    ↪ sorted(entries.items())},
194         ensure_ascii=False, indent=2) + '\n').encode('utf-8')
195     write_zip(output / '支撑材料.zip', entries)
196     if (output / '支撑材料.zip').stat().st_size > LIMIT:
197         status['errors'].append('支撑材料 ZIP 超过 20MB, 不能按当前文件提交')
198         status['ready'] = False
199     lines = ['# 提交检查', '', '**' + ('文件检查通过, 提交前仍须人工核对.' if status['ready'] else
    ↪ '尚有待补项目, 当前为待完成材料.') + '**', '',
200             '## 缺项', '']
201     lines += [f'- {item}' for item in pending] or ['无。']
202     lines += ['', '## 文件检查错误', ''] + ([f'- {item}' for item in status['errors']] or ['无。'])
203     lines += ['', '正式日志必须使用模拟器导出的原始文件名和原始字节; 不要修改其中系统写入的身份字段。
    ↪ ',
204             '源码、论文和普通结果采用匿名材料; 原始日志按题目要求作为指定系统产物提交。',
205             '生成MD5之前先备份最终PDF和ZIP; 生成后不要改动文件, 任何改动均需重新生成并提交MD5。',
    ↪ '']
206     (output / '待补清单.md').write_text('\n'.join(lines), encoding='utf-8')
207     (output / 'status.json').write_text(json.dumps(status, ensure_ascii=False, indent=2) + '\n',
    ↪ encoding='utf-8')
208     return status
209
210
211 def main():
212     parser = argparse.ArgumentParser(description=__doc__)
213     parser.add_argument('--check', action='store_true', help='只检查; 缺项或错误时退出码为 2')
214     args = parser.parse_args()
215     try:
216         result = prepare(check_only=args.check)
217     except (OSError, ValueError, KeyError) as exc:
218         print(f'材料检查失败: {exc}', file=sys.stderr)
219         return 2
220     print(json.dumps(result, ensure_ascii=False, indent=2))
221     return 2 if result['errors'] or (args.check and not result['ready']) else 0
222
223
224 if __name__ == '__main__':
225     raise SystemExit(main())

```

src/jammer_solver/__init__.py

文件行数: 0。该文件为空, 用于标记 Python 包。

src/jammer_solver/exact_geometry.py

文件行数: 302。

```
1 from fractions import Fraction as fs
2 from functools import lru_cache as cache
3 import math
4
5 pilo = fs("3.14159265358979323846264338327950288419716939937510")
6 pihi = pilo + fs(1, 10**50)
7 eps = fs(201, 200)
8 domain = fs(1800)
9
10
11 def point(p):
12     if len(p) != 2 or not all((math.isfinite(float(x)) for x in p)):
13         raise ValueError("Non-finite point")
14     return tuple((fs(x) for x in p))
15
16
17 def jlxs(a, b, scale=1000):
18     v = ((a[0] - b[0]) * (a[0] - b[0]) + (a[1] - b[1]) * (a[1] - b[1])) * scale * scale
19     k = math.isqrt(v.numerator // v.denominator)
20     if fs(k * k) < v:
21         k += 1
22     return fs(k, scale)
23
24
25 def juxing(x0, y0, x1, y1):
26     return [point(p) for p in ((x0, y0), (x1, y0), (x1, y1), (x0, y1))]
27
28
29 def hull(points):
30     pts = sorted(set(points))
31     if len(pts) <= 1:
32         return pts
33     lo, hi = ([], [])
34     for p in pts:
35         while (
36             len(lo) > 1
37             and (lo[-1][0] - lo[-2][0]) * (p[1] - lo[-1][1])
38             - (lo[-1][1] - lo[-2][1]) * (p[0] - lo[-1][0])
39             <= 0
40         ):
41             lo.pop()
42         lo.append(p)
43     for p in reversed(pts):
44         while (
45             len(hi) > 1
46             and (hi[-1][0] - hi[-2][0]) * (p[1] - hi[-1][1])
47             - (hi[-1][1] - hi[-2][1]) * (p[0] - hi[-1][0])
48             <= 0
49         ):
50             hi.pop()
51         hi.append(p)
52     return lo[:-1] + hi[:-1]
53
54
55 def clip(poly, normal, bound):
56     if not poly:
57         return []
58     out = []
59     a = poly[-1]
60     fa = normal[0] * a[0] + normal[1] * a[1] - bound
```

```

61     for b in poly:
62         fb = normal[0] * b[0] + normal[1] * b[1] - bound
63         if (fa > 0) != (fb > 0):
64             t = fa / (fa - fb)
65             out.append((a[0] + t * (b[0] - a[0]), a[1] + t * (b[1] - a[1])))
66         if fb <= 0:
67             out.append(b)
68         a, fa = (b, fb)
69     result = []
70     for p in out:
71         if not result or result[-1] != p:
72             result.append(p)
73     if len(result) > 1 and result[0] == result[-1]:
74         result.pop()
75     return result
76
77
78 def contains(poly, p):
79     if not poly:
80         return False
81     if len(poly) == 1:
82         return p == poly[0]
83     if len(poly) == 2:
84         return (poly[1][0] - poly[0][0]) * (p[1] - poly[0][1]) - (
85             poly[1][1] - poly[0][1]
86         ) * (p[0] - poly[0][0]) == 0 and (p[0] - poly[0][0]) * (p[0] - poly[1][0]) + (
87             p[1] - poly[0][1]
88         ) * (p[1] - poly[1][1]) <= 0
89     return all(
90         (
91             (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0]) >= 0
92             for a, b in zip(poly, poly[1:] + poly[:1])
93         )
94     )
95
96
97 @cache(maxsize=4096)
98 def trig(degrees):
99     degrees = fs(degrees)
100     xq = int((degrees + 45) // 90)
101     d = degrees - 90 * xq
102     x = d * (pilo + pihi) / 360
103     err = abs(d) * (pihi - pilo) / 360
104     sin = sum(
105         ((-1) ** j * x ** (2 * j + 1) / math.factorial(2 * j + 1) for j in range(10)),
106         fs(0),
107     )
108     cos = sum(
109         ((-1) ** j * x ** (2 * j) / math.factorial(2 * j) for j in range(10)), fs(0)
110     )
111     se = fs(1, math.factorial(20)) + err
112     ce = fs(1, math.factorial(19)) + err
113     c, s = ((cos - ce, cos + ce), (sin - se, sin + se))
114     for _ in range(xq % 4):
115         c, s = ((-s[1], -s[0]), c)
116     scale = 10**12
117
118     def outward(pair):
119         a, b = pair
120         return (fs(a * scale // 1, scale), fs(-(-b * scale // 1), scale))
121
122     return (*outward(c), *outward(s))
123
124
125 def fangxiang(degrees):
126     cl, ch, sl, sh = trig(fs(degrees))
127     return ((cl + ch) / 2, (sl + sh) / 2), ((ch - cl) / 2, (sh - sl) / 2)
128

```

```

129
130 def bqdpmpoly(poly, origin, normal, error, rhs=fs(0)):
131     slack = error[0] * (domain + abs(origin[0])) + error[1] * (domain + abs(origin[1]))
132     return clip(
133         poly, normal, normal[0] * origin[0] + normal[1] * origin[1] + rhs + slack
134     )
135
136
137 @cache(maxsize=1)
138 def yfw():
139     poly = juxing(-1800, -1800, 1800, 1800)
140     for angle in range(0, 360, 15):
141         u, e = fangxiang(fs(angle))
142         poly = bqdpmpoly(poly, point((0, 0)), u, e, fs(1800))
143     return tuple(poly)
144
145
146 def fwys(lo, hi=None):
147     hi = lo if hi is None else hi
148     if fs(hi) - fs(lo) + 2 * eps >= 180:
149         return []
150     ulo, elo = fangxiang(fs(lo) - eps)
151     uhi, ehi = fangxiang(fs(hi) + eps)
152     return [
153         ((ulo[1], -ulo[0]), (elo[1], elo[0]), fs(0)),
154         ((-uhi[1], uhi[0]), (ehi[1], ehi[0]), fs(0)),
155     ]
156
157
158 def jcfw(poly, origin, bearing, radius=fs(1500), hi=None):
159     ys = fwys(bearing, hi)
160     if hi is None:
161         u, e = fangxiang(fs(bearing))
162         ce = trig(eps)[0]
163         ys += [(u, e, radius), ((-u[0], -u[1]), e, -5 * ce)]
164     for n, e, b in ys:
165         poly = bqdpmpoly(poly, origin, n, e, b)
166     for n, b in (
167         ((1, 0), origin[0] + 1500),
168         ((-1, 0), 1500 - origin[0]),
169         ((0, 1), origin[1] + 1500),
170         ((0, -1), 1500 - origin[1]),
171     ):
172         poly = clip(poly, n, b)
173     return poly
174
175
176 def xzqj(origin, bearing, bounds):
177     u, e = fangxiang(fs(bearing))
178     corners = juxing(*bounds)
179     out = []
180     for x, y in corners:
181         center = (origin[0] + u[0] * x - u[1] * y, origin[1] + u[1] * x + u[0] * y)
182         dx, dy = (e[0] * abs(x) + e[1] * abs(y), e[1] * abs(x) + e[0] * abs(y))
183         out += juxing(center[0] - dx, center[1] - dy, center[0] + dx, center[1] + dy)
184     return hull(out)
185
186
187 def xzd(origin, bearing, x, y):
188     u, e = fangxiang(fs(bearing))
189     mid = (origin[0] + u[0] * x - u[1] * y, origin[1] + u[1] * x + u[0] * y)
190     p = point(tuple((float(x) for x in mid)))
191     dx = abs(p[0] - mid[0]) + e[0] * abs(x) + e[1] * abs(y)
192     dy = abs(p[1] - mid[1]) + e[1] * abs(x) + e[0] * abs(y)
193     if dx * dx + dy * dy > 1:
194         raise ArithmeticError("Rotation exceeds the certified 1 m allowance")
195     return p
196

```

```

197
198 def ypbh(poly, center, radius):
199     return bool(poly) and all(
200         (
201             (v[0] - center[0]) * (v[0] - center[0])
202             + (v[1] - center[1]) * (v[1] - center[1])
203             <= radius * radius
204             for v in poly
205         )
206     )
207
208
209 def syxc(poly, pos, a, b):
210     va, vb = ((a[0] - pos[0], a[1] - pos[1]), (b[0] - pos[0], b[1] - pos[1]))
211     if va[0] * vb[1] - va[1] * vb[0] < 0:
212         a, b, va, vb = (b, a, vb, va)
213     if va[0] * vb[1] - va[1] * vb[0] == 0:
214         return (poly, False)
215     edge = (b[0] - a[0], b[1] - a[1])
216     normal = (edge[1], -edge[0])
217     bound = normal[0] * a[0] + normal[1] * a[1]
218     if normal[0] * pos[0] + normal[1] * pos[1] > bound:
219         normal = (-normal[0], -normal[1])
220         bound = -bound
221     outside = clip(poly, (-normal[0], -normal[1]), -bound)
222     if not outside:
223         return (poly, False)
224     if not all(
225         (
226             va[0] * (s[1] - pos[1]) - va[1] * (s[0] - pos[0]) >= 0
227             and (s[0] - pos[0]) * vb[1] - (s[1] - pos[1]) * vb[0] >= 0
228             for s in outside
229         )
230     ):
231         return (poly, False)
232     if not ypbh(outside, a, fs(1000)) or not ypbh(outside, b, fs(1000)):
233         return (poly, False)
234     result = clip(poly, normal, bound)
235     return (result, result != poly)
236
237
238 def sswd():
239     return [
240         point((600 * i, 600 * j))
241         for j in range(-3, 4)
242         for i in (range(-3, 4) if j % 2 else range(3, -4, -1))
243     ]
244
245
246 @cache(maxsize=256)
247 def fgzm(radius, optical, problem, jdmax=2048, sdmax=12):
248     radio, optical = (tuple(map(point, radio)), tuple(map(point, optical)))
249     stack = [(juxing(-1800, -1800, 1800, 1800), 0)]
250     nodes = 0
251     leaves = []
252     rf = [(q, tuple((float(x) for x in q))) for q in radio]
253     while stack and nodes < jdmax:
254         box, depth = stack.pop()
255         nodes += 1
256         x0, y0 = box[0]
257         x1, y1 = box[2]
258         dx, dy = (max(fs(0), x0, -x1), max(fs(0), y0, -y1))
259         if dx * dx + dy * dy > domain**2:
260             continue
261         hit = next((q for q in optical if ypbh(box, q, fs(20))), None)
262         if hit is not None:
263             leaves.append(("optical", box, [hit]))
264             continue

```

```

265     bf = [tuple((float(x) for x in v)) for v in box]
266     nearby = [
267         q
268         for q, qf in rf
269         if all((math.dist(qf, v) < 1000.000001 for v in bf))
270         and ypbh(box, q, fs(1000))
271     ]
272     if problem == 3 and nearby:
273         leaves.append(("omni", box, nearby[:1]))
274         continue
275     if problem == 4 and len(nearby) >= 3:
276         h = hull(nearby)
277         if all((contains(h, v) for v in box)):
278             leaves.append(("directional_hull", box, h))
279             continue
280     if depth >= sdmax:
281         return (False, nodes, tuple(map(float, (x0, y0, x1, y1))), ())
282     mx, my = ((x0 + x1) / 2, (y0 + y1) / 2)
283     stack.extend(
284         (
285             (juxing(*v), depth + 1)
286             for v in (
287                 (x0, y0, mx, my),
288                 (mx, y0, x1, my),
289                 (x0, my, mx, y1),
290                 (mx, my, x1, y1),
291             )
292         )
293     )
294     if stack:
295         box, _ = stack[-1]
296         return (
297             False,
298             nodes,
299             (*tuple((float(x) for x in box[0])), *tuple((float(x) for x in box[2]))),
300             (),
301         )
302     return (True, nodes, None, tuple(leaves))

```

src/jammer_solver/protocol.py

文件行数: 110。

```

1 import json
2 import math
3 import os
4 import time
5 import uuid
6 from urllib.request import Request as qqcls, ProxyHandler as dlcls, build_opener as ljg
7
8
9 def jyh(path, reply):
10     if not isinstance(reply, dict) or type(reply.get("accepted")) is not bool:
11         raise RuntimeError("Malformed acceptance status")
12     if not reply["accepted"]:
13         raise RuntimeError("accepted=false: action did not execute")
14     if (
15         not (
16             type(reply.get("virtual_time_s")) in (int, float)
17             and math.isfinite(reply.get("virtual_time_s"))
18         )
19         or reply["virtual_time_s"] < 0
20     ):
21         raise RuntimeError("Malformed virtual time in accepted response")
22     if path == "/enter":
23         seconds = reply.get("remaining_real_duration_s")
24         if (
25             not (type(seconds) in (int, float) and math.isfinite(seconds))

```

```

26         or not 0 <= seconds <= 1200
27         or int(seconds) != seconds
28     ):
29         raise RuntimeError("Malformed remaining real duration")
30     if (
31         not (
32             type(reply.get("max_virtual_duration_s")) in (int, float)
33             and math.isfinite(reply.get("max_virtual_duration_s"))
34         )
35         or reply["max_virtual_duration_s"] <= 0
36     ):
37         raise RuntimeError("Malformed maximum virtual duration")
38     elif path == "/measure":
39         result = reply.get("measure_result")
40         if result not in ("direction", "near", "no_signal"):
41             raise RuntimeError("Unknown measure result")
42         if result == "direction":
43             angle = reply.get("svd_deg")
44             if (
45                 not (type(angle) in (int, float) and math.isfinite(angle))
46                 or not 0 <= angle < 360
47             ):
48                 raise RuntimeError("Malformed bearing")
49     elif path == "/clear" and reply.get("clear_result") not in (
50         "success",
51         "no_target_in_range",
52     ):
53         raise RuntimeError("Unknown clear result")
54     elif path == "/exit" and reply.get("exit_reason") != "user_exit":
55         raise RuntimeError("Unknown exit result")
56
57
58 def xjkk(dh, rizhi, url="http://127.0.0.1:2026"):
59     jieku = dict(pending=None, deadline=math.inf)
60     bianhao = uuid.uuid4().hex[:16]
61     xulie = 0
62     opener = ljq(dlccls({}))
63
64     def baocun(shijian):
65         rizhi.write(json.dumps(shijian, ensure_ascii=False, allow_nan=False) + "\n")
66         rizhi.flush()
67         os.fsync(rizhi.fileno())
68
69     def qingqiu(path, weizhi=None, pindao=None):
70         nonlocal xulie
71         if jieku["pending"] is not None:
72             raise RuntimeError("Previous request has no confirmed response")
73         shengyu = jieku["deadline"] - time.monotonic()
74         if shengyu <= 0:
75             raise TimeoutError("Request deadline reached")
76         xulie += 1
77         neirong = dict(arena_id="default", robot_id=dh, request_id=f"{bianhao}-{xulie}")
78         if path in ("/measure", "/clear"):
79             neirong.update(position=dict(x=weizhi[0], y=weizhi[1]), channel=pindao)
80         zhengwen = json.dumps(
81             neirong, ensure_ascii=False, allow_nan=False, separators=(",", ":")
82         ).encode("utf-8")
83         baocun(dict(event="intent", wall_time=time.time(), path=path, payload=neirong))
84         jieku["pending"] = dict(path=path, payload=neirong)
85         request = qqcls(
86             url + path,
87             data=zhengwen,
88             headers={"Content-Type": "application/json"},
89             method="POST",
90         )
91         with opener.open(request, timeout=min(5.0, shengyu)) as response:
92             huifu = json.loads(response.read().decode("utf-8"))
93             baocun(

```

```

94         dict(
95             event="response",
96             wall_time=time.time(),
97             request_id=neirong["request_id"],
98             attempt=0,
99             http_status=response.status,
100            response=huifu,
101        )
102    )
103    if response.status != 200:
104        raise RuntimeError(f"HTTP {response.status}")
105    jyhf(path, huifu)
106    jiekou["pending"] = None
107    return huifu
108
109    jiekou.update(call=qingqiu, record=baocun)
110    return jiekou

```

src/jammer_solver/questions.py

文件行数: 370。

```

1 import argparse
2 import heapq
3 import json
4 import math
5 from itertools import combinations
6 from pathlib import Path as lj
7
8 eps = math.radians(1.005)
9
10
11 def hull(points):
12     points = sorted(set(points))
13     if len(points) <= 1:
14         return points
15
16     def chain(seq):
17         out = []
18         for p in seq:
19             while (
20                 len(out) >= 2
21                 and (out[-1][0] - out[-2][0]) * (p[1] - out[-1][1])
22                 - (out[-1][1] - out[-2][1]) * (p[0] - out[-1][0])
23                 <= 0
24             ):
25                 out.pop()
26                 out.append(p)
27         return out
28
29     return chain(points)[-1] + chain(points[::-1])[-1]
30
31
32 def zydd(vertices):
33     n = len(vertices)
34     if n == 1:
35         return (vertices[0], vertices[0])
36     if n == 2:
37         return tuple(vertices)
38     best = (vertices[0], vertices[1])
39     bestd = math.dist(*best)
40     j = 1
41     for i in range(n):
42         nxt = (i + 1) % n
43         edge = (vertices[nxt][0] - vertices[i][0], vertices[nxt][1] - vertices[i][1])
44
45         def area(k):
46             return abs(

```

```

47         edge[0] * (vertices[k][1] - vertices[i][1])
48         - edge[1] * (vertices[k][0] - vertices[i][0])
49     )
50
51     steps = 0
52     while area((j + 1) % n) > area(j) and steps < n:
53         j = (j + 1) % n
54         steps += 1
55     for a in (i, nxt):
56         for b in (j, (j + 1) % n):
57             d = math.dist(vertices[a], vertices[b])
58             if d > bestd:
59                 best, bestd = ((vertices[a], vertices[b]), d)
60     return best
61
62
63 def jhqy(gclb, wcjd=1.005):
64     planes = []
65     for obs in gclb:
66         x, y, angle = (float(obs[k]) for k in ("x", "y", "bearing_deg"))
67         if not all((math.isfinite(v) for v in (x, y, angle))):
68             raise ValueError("Observations must be finite")
69         lo, hi = (math.radians(angle - wcjd), math.radians(angle + wcjd))
70         for nx, ny in ((math.sin(lo), -math.cos(lo)), (-math.sin(hi), math.cos(hi))):
71             planes.append((nx, ny, nx * x + ny * y))
72     base = dict(
73         epsilon_deg=wcjd,
74         diameter=None,
75         vertices=[],
76         numerical_method="float64; relative feasibility tolerance 1e-10",
77     )
78     if not planes:
79         return dict(base, state="UNBOUNDED")
80
81     def feasible(x, y):
82         return all(
83             (
84                 a * x + b * y - c <= 1e-10 * (1 + abs(a * x) + abs(b * y) + abs(c))
85                 for a, b, c in planes
86             )
87         )
88
89     bx = []
90     for (a, b, c), (d, e, f) in combinations(planes, 2):
91         det = a * e - b * d
92         if abs(det) <= 1e-14:
93             continue
94         x, y = ((c * e - b * f) / det, (a * f - c * d) / det)
95         if not math.isfinite(x) or not math.isfinite(y):
96             return dict(base, state="NUMERICALLY_UNRESOLVED")
97         if feasible(x, y):
98             bx.append((x, y))
99     if not bx:
100         return dict(base, state="EMPTY_OR_NUMERICALLY_UNRESOLVED")
101     for a, b, _ in planes:
102         for dx, dy in ((b, -a), (-b, a)):
103             if all((u * dx + v * dy <= 1e-14 for u, v, _ in planes)):
104                 return dict(base, state="UNBOUNDED", feasible_point=bx[0])
105     vertices = hull(bx)
106     p, q = zydd(vertices)
107     diameter = math.dist(p, q)
108     center = ((p[0] + q[0]) / 2, (p[1] + q[1]) / 2)
109     radius = diameter / 2
110     return dict(
111         base,
112         state={1: "POINT", 2: "SEGMENT"}.get(len(vertices), "POLYGON"),
113         vertices=vertices,
114         diameter=diameter,

```

```

115     farthest_pair=(p, q),
116     diameter_circle_center=center,
117     diameter_circle_radius=radius,
118     diameter_circle_covers=all(
119         (math.dist(v, center) <= radius + 1e-09 * (1 + radius) for v in vertices)
120     ),
121 )
122
123
124 def ecdqy(first, bearing, query=None):
125     centers = [tuple(first)]
126     for sign in (1, -1):
127         angle = math.radians(bearing) + sign * eps
128         centers.append(
129             (first[0] + 1000 * math.cos(angle), first[1] + 1000 * math.sin(angle))
130         )
131     normal = (math.cos(math.radians(bearing)), math.sin(math.radians(bearing)))
132     result = dict(
133         disk_centers=centers,
134         disk_radii=[1000] * 3,
135         forward_halfplane=dict(
136             normal=normal, origin=first, relation="dot(normal, q-origin) >= 0"
137         ),
138         epsilon_deg=1.005,
139         scope="Exact for relaxed radial prior [0,1500] within forward halfplane; conservative for
140             ↪ actual r>5 and source domain",
141     )
142     if query is not None:
143         if len(query) != 2 or not all((math.isfinite(v) for v in query)):
144             raise ValueError("Query must contain two finite coordinates")
145         result["query_in_region"] = (
146             all((math.hypot(query[0] - c[0], query[1] - c[1]) <= 1000 for c in centers))
147             and sum((normal[k] * (query[k] - first[k]) for k in (0, 1))) >= 0
148         )
149     return result
150
151 def zjsj(rho, theta):
152     a, b = (rho * math.cos(theta), rho * math.sin(theta))
153     dist = max(
154         (
155             math.sqrt(r * r + rho * rho - 2 * r * rho * math.cos(theta + eps))
156             for r in (5.0, 1500.0)
157         )
158     )
159     height = b * math.cos(eps) - a * math.sin(eps)
160     return (djsj(dist, height), dist)
161
162
163 def djsj(dist, height):
164     sine = max(0.0, min(1.0, height / max(dist, 1e-12)))
165     gamma = math.asin(sine) - 2 * eps
166     if gamma <= 0:
167         return math.inf
168     w1, w2 = (1500 * math.sin(eps), dist * math.sin(eps))
169     return (
170         2
171         * math.sqrt(w1 * w1 + w2 * w2 + 2 * w1 * w2 * math.cos(gamma))
172         / math.sin(gamma)
173     )
174
175
176 def jdyh(rho, tol=0.02, jdmax=1024):
177     if not 5 < rho < 1000:
178         raise ValueError("A positive reception margin requires 5 < movement < 1000")
179     lo = eps + 1e-08
180     hi = math.acos(rho / 2000) - eps - 1e-08
181

```

```

182 def lower(a, b):
183     d = zjsj(rho, a)[1]
184     h = rho * math.sin(b - eps)
185     return max(0.0, djsj(d, h) - 1e-06)
186
187 best = min(((zjsj(rho, t)[0], t) for t in (lo, (lo + hi) / 2, hi)))
188 heap = [(lower(lo, hi), lo, hi)]
189 nodes = 0
190 prunelb = math.inf
191 while heap and nodes < jdmax:
192     lb, a, b = heapq.heappop(heap)
193     if best[0] - lb <= tol:
194         prunelb = min(prunelb, lb)
195         break
196     mid = (a + b) / 2
197     for x, y in ((a, mid), (mid, b)):
198         t = (x + y) / 2
199         value = zjsj(rho, t)[0]
200         nodes += 1
201         if value < best[0]:
202             best = (value, t)
203             bound = lower(x, y)
204             if bound < best[0]:
205                 heapq.heappush(heap, (bound, x, y))
206             else:
207                 prunelb = min(prunelb, bound)
208 boundlo = min(best[0], prunelb, min((entry[0] for entry in heap), default=math.inf))
209 bound, theta = best
210 a, b = (rho * math.cos(theta), rho * math.sin(theta))
211 endpos = max(
212     (
213         math.dist((a, b), (1000 * math.cos(eps), s * 1000 * math.sin(eps)))
214         for s in (-1, 1)
215     )
216 )
217 return dict(
218     movement_m=rho,
219     a_m=a,
220     b_m=b,
221     diameter_upper_m=bound,
222     angle_family_lower_m=boundlo,
223     optimization_gap_m=bound - boundlo,
224     reception_margin_m=1000 - max(rho, endpos),
225     nodes=nodes,
226     scope="entire_feasible_angle_interval_at_fixed_movement_radius",
227 )
228
229
230 def zxdj(target=163.0, jdmax=6000, tol=0.1):
231     seed = jdyh(900.0)
232     if seed["diameter_upper_m"] > target:
233         seed = jdyh(999.0)
234     if seed["diameter_upper_m"] > target:
235         raise ValueError(
236             "No initial feasible design for this target; choose a larger diameter target"
237         )
238     best = (
239         seed["movement_m"],
240         math.atan2(seed["b_m"], seed["a_m"]),
241         seed["diameter_upper_m"],
242     )
243     heap = []
244     serial = 0
245     nodes = 0
246
247     def add(r0, r1, t0, t1):
248         nonlocal serial
249         r1 = min(r1, 2000 * math.cos(t0 + eps))

```

```

250     t1 = min(t1, math.acos(r0 / 2000) - eps)
251     if r0 >= r1 or t0 >= t1 or r0 >= best[0]:
252         return
253     dists = []
254     for r in (5.0, 1500.0):
255         rho = max(r0, min(r1, r * math.cos(t0 + eps)))
256         dists.append(
257             math.sqrt(
258                 max(0.0, r * r + rho * rho - 2 * r * rho * math.cos(t0 + eps))
259             )
260         )
261     lb = djsj(max(dists), r1 * math.sin(t1 - eps)) - 1e-06
262     if lb > target:
263         return
264     serial += 1
265     heapq.heappush(heap, (r0, serial, (r0, r1, t0, t1)))
266
267     add(5.0001, best[0], eps + 1e-08, math.pi / 2 - eps - 1e-08)
268     stoplb = None
269     while heap and nodes < jdmax:
270         lower, _, (r0, r1, t0, t1) = heapq.heappop(heap)
271         if best[0] - lower <= tol:
272             stoplb = lower
273             break
274         if lower >= best[0]:
275             continue
276         nodes += 1
277         rm = (r0 + r1) / 2
278         tm = (t0 + t1) / 2
279         for rho, theta in ((rm, tm), (r1, tm), (rm, min(t1, max(t0, best[1])))):
280             if rho < best[0] and rho < 1000 and (theta <= math.acos(rho / 2000) - eps):
281                 bound = zjsj(rho, theta)[0]
282                 if bound <= target - 1e-05:
283                     best = (rho, theta, bound)
284         if (r1 - r0) / 1000 > (t1 - t0) / (math.pi / 2):
285             add(r0, rm, t0, t1)
286             add(rm, r1, t0, t1)
287         else:
288             add(r0, r1, t0, tm)
289             add(r0, r1, tm, t1)
290     lower = min(
291         [best[0]] + ([stoplb] if stoplb is not None else []) + [x[0] for x in heap]
292     )
293     rho, theta, bound = best
294     a, b = (rho * math.cos(theta), rho * math.sin(theta))
295     return dict(
296         movement_m=rho,
297         a_m=a,
298         b_m=b,
299         diameter_upper_m=bound,
300         target_diameter_m=target,
301         movement_lower_m=lower,
302         movement_gap_m=rho - lower,
303         nodes=nodes,
304         reception_margin_m=1000
305         - max(
306             rho,
307             max(
308                 (
309                     math.dist((a, b), (1000 * math.cos(eps), s * 1000 * math.sin(eps)))
310                     for s in (-1, 1)
311                 )
312             ),
313         ),
314         scope="global_movement_bound_for_the_analytic_strip_criterion_not_exact_physical_diamete"
315         ↪ r",
316     )

```

```

317
318 def main():
319     parser = argparse.ArgumentParser(description=" 前两问的离线几何计算; 不连接任何接口")
320     parser.add_argument(
321         "input", type=lj, help="JSON: observations, 或 first 与 bearing_deg"
322     )
323     parser.add_argument("--problem", type=int, choices=(1, 2), required=True)
324     parser.add_argument("--output", type=lj)
325     args = parser.parse_args()
326     data = json.loads(args.input.read_text(encoding="utf-8-sig"))
327     if args.problem == 1:
328         result = jhqy(data["observations"], data.get("epsilon_deg", 1.005))
329     else:
330         design = (
331             jdyh(data["movement_m"])
332             if "movement_m" in data
333             else zxdj(data.get("target_diameter_m", 163.0))
334         )
335         theta = math.radians(data["bearing_deg"])
336         u = (math.cos(theta), math.sin(theta))
337         v = (-u[1], u[0])
338         options = [
339             tuple(
340                 (
341                     data["first"][i]
342                     + design["a_m"] * u[i]
343                     + sign * design["b_m"] * v[i]
344                     for i in (0, 1)
345                 )
346             )
347             for sign in (1, -1)
348         ]
349         p = (
350             min(options, key=lambda p: math.dist(p, data["current"]))
351             if "current" in data
352             else options[0]
353         )
354         result = dict(
355             second_position=dict(x=p[0], y=p[1]),
356             candidate_region=ecdqy(
357                 tuple(data["first"]), data["bearing_deg"], data.get("query")
358             ),
359             guaranteed_diameter_bound_m=design["diameter_upper_m"],
360             optimization=design,
361             selection_status="Minimum movement for the requested analytic diameter bound;
362             ↪ certified remaining radius gap is reported"
363             if "movement_m" not in data
364             else "Analytic diameter bound optimized over the full feasible angle interval at the
365             ↪ chosen movement radius",
366             rule="movement_m is measured from first; nearest symmetric side if current supplied",
367             scope=" 全向信号; 首次方向有效; 半径 1000 至 1500 米",
368         )
369     text = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False)
370     if args.output:
371         args.output.write_text(text + "\n", encoding="utf-8")
372     print(text)

```

src/jammer_solver/solver.py

文件行数: 1863。

```

1 from fractions import Fraction as fs
2 from itertools import combinations
3 import math
4 import time
5 from .protocol import jyhf
6 from . import exact_geometry as g
7

```

```

8 qcbj = 19.8
9
10
11 def lxcd(start, points, end=None):
12     path = [start] + list(points) + ([[] if end is None else [end]])
13     return sum((math.dist(a, b) for a, b in zip(path, path[1:])))
14
15
16 def lxpx(start, points, end=None):
17     if len(points) < 2:
18         return list(points)
19     rest = list(points)
20     greedy = []
21     p = start
22     while rest:
23         q = min(rest, key=lambda q: (math.dist(p, q), q))
24         greedy.append(q)
25         rest.remove(q)
26         p = q
27     order = min((list(points), greedy), key=lambda qs: lxcd(start, qs, end))
28     for _ in range(8):
29         bestdelta, bestpair = (0.0, None)
30         for i in range(len(order) - 1):
31             a = start if i == 0 else order[i - 1]
32             for j in range(i + 1, len(order)):
33                 b = order[j + 1] if j + 1 < len(order) else end
34                 before = math.dist(a, order[i])
35                 after = math.dist(a, order[j])
36                 if b is not None:
37                     before += math.dist(order[j], b)
38                     after += math.dist(order[i], b)
39                 if after - before < bestdelta - 1e-08:
40                     bestdelta, bestpair = (after - before, (i, j))
41             if bestpair is None:
42                 break
43             i, j = bestpair
44             order[i : j + 1] = reversed(order[i : j + 1])
45     return order
46
47
48 def zjqcd(polygon, current):
49     if not polygon:
50         return None
51     radius = qcbj - 1e-06
52
53     def feasible(p):
54         return all(
55             (math.hypot(p[0] - v[0], p[1] - v[1]) <= radius + 1e-10 for v in polygon)
56         )
57
58     if feasible(current):
59         return current
60     if any(
61         (
62             math.hypot(a[0] - b[0], a[1] - b[1]) > 2 * radius
63             for a, b in combinations(polygon, 2)
64         )
65     ):
66         return None
67     best, zxjl = (None, float("inf"))
68
69     def consider(p):
70         nonlocal best, zxjl
71         travel = math.hypot(current[0] - p[0], current[1] - p[1])
72         if travel < zxjl and feasible(p):
73             best, zxjl = (p, travel)
74
75     for a in polygon:

```

```

76     d = math.hypot(current[0] - a[0], current[1] - a[1])
77     if d > radius:
78         consider(
79             (
80                 a[0] + radius * (current[0] - a[0]) / d,
81                 a[1] + radius * (current[1] - a[1]) / d,
82             )
83         )
84     for a, b in combinations(polygon, 2):
85         d = math.hypot(a[0] - b[0], a[1] - b[1])
86         if d <= 1e-12 or d > 2 * radius:
87             continue
88         mid = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
89         h = math.sqrt(max(0.0, radius * radius - d * d / 4))
90         dx, dy = (-(b[1] - a[1]) / d * h, (b[0] - a[0]) / d * h)
91         consider((mid[0] + dx, mid[1] + dy))
92         consider((mid[0] - dx, mid[1] - dy))
93     return best
94
95
96 def qclxyh(centers, radius, start, current, target, bsmax=24):
97
98     def feasible(p):
99         return all(
100             (math.hypot(p[0] - c[0], p[1] - c[1]) <= radius + 1e-10 for c in centers)
101         )
102
103     if not centers or radius <= 0 or (not feasible(start)):
104         return (start, dict(iterations=0, gap_m=None))
105     corners = []
106     for a, b in combinations(centers, 2):
107         d = math.hypot(a[0] - b[0], a[1] - b[1])
108         if d <= 1e-12 or d > 2 * radius:
109             continue
110         mid = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
111         h = math.sqrt(max(0.0, radius * radius - d * d / 4))
112         dx, dy = (-(b[1] - a[1]) * h / d, (b[0] - a[0]) * h / d)
113         for p in ((mid[0] + dx, mid[1] + dy), (mid[0] - dx, mid[1] - dy)):
114             if feasible(p):
115                 corners.append(p)
116     point, steps, gap = (start, 0, None)
117     for _ in range(bsmax):
118         value = math.hypot(current[0] - point[0], current[1] - point[1]) + math.hypot(
119             point[0] - target[0], point[1] - target[1]
120         )
121         if value - math.hypot(current[0] - target[0], current[1] - target[1]) <= 1e-07:
122             gap = max(
123                 0.0, value - math.hypot(current[0] - target[0], current[1] - target[1])
124             )
125             break
126         d1, d2 = (
127             math.hypot(point[0] - current[0], point[1] - current[1]),
128             math.hypot(point[0] - target[0], point[1] - target[1]),
129         )
130         if min(d1, d2) <= 1e-12:
131             gap = 0.0
132             break
133         g = (
134             (point[0] - current[0]) / d1 + (point[0] - target[0]) / d2,
135             (point[1] - current[1]) / d1 + (point[1] - target[1]) / d2,
136         )
137         norm = math.hypot(*g)
138         if norm <= 1e-12:
139             gap = None
140             break
141         beixuan = corners + [point]
142         for c in centers:
143             cand = (c[0] - radius * g[0] / norm, c[1] - radius * g[1] / norm)

```

```

144         if feasible(cand):
145             beixuan.append(cand)
146         support = min(beixuan, key=lambda p: g[0] * p[0] + g[1] * p[1])
147         gap = max(0.0, g[0] * (point[0] - support[0]) + g[1] * (point[1] - support[1]))
148         if gap <= 1e-05:
149             break
150
151     lo, hi = (0.0, 1.0)
152     for _ in range(36):
153         left, right = ((2 * lo + hi) / 3, (lo + 2 * hi) / 3)
154         lp = (
155             point[0] + left * (support[0] - point[0]),
156             point[1] + left * (support[1] - point[1]),
157         )
158         rp = (
159             point[0] + right * (support[0] - point[0]),
160             point[1] + right * (support[1] - point[1]),
161         )
162         if math.hypot(current[0] - lp[0], current[1] - lp[1]) + math.hypot(
163             lp[0] - target[0], lp[1] - target[1]
164         ) <= math.hypot(current[0] - rp[0], current[1] - rp[1]) + math.hypot(
165             rp[0] - target[0], rp[1] - target[1]
166         ):
167             hi = right
168         else:
169             lo = left
170     cand = min(
171         (
172             point,
173             support,
174             (
175                 point[0] + (lo + hi) / 2 * (support[0] - point[0]),
176                 point[1] + (lo + hi) / 2 * (support[1] - point[1]),
177             ),
178         ),
179         key=lambda p: (
180             math.hypot(current[0] - p[0], current[1] - p[1])
181             + math.hypot(p[0] - target[0], p[1] - target[1])
182         ),
183     )
184     if not feasible(cand):
185         break
186     gain = value - (
187         math.hypot(current[0] - cand[0], current[1] - cand[1])
188         + math.hypot(cand[0] - target[0], cand[1] - target[1])
189     )
190     point = cand
191     steps += 1
192     gap = None
193     if gain <= 1e-09:
194         break
195     return (point, dict(iterations=steps, gap_m=gap))
196
197
198 def zhipai(cost):
199     n = len(cost)
200     u = [0.0] * (n + 1)
201     v = [0.0] * (n + 1)
202     p = [0] * (n + 1)
203     way = [0] * (n + 1)
204     for i in range(1, n + 1):
205         p[0] = i
206         j0 = 0
207         minimum = [float("inf")] * (n + 1)
208         used = [False] * (n + 1)
209         while True:
210             used[j0] = True
211             i0 = p[j0]

```

```

212         delta = float("inf")
213         j1 = 0
214         for j in range(1, n + 1):
215             if not used[j]:
216                 cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
217                 if cur < minimum[j]:
218                     minimum[j] = cur
219                     way[j] = j0
220                 if minimum[j] < delta:
221                     delta = minimum[j]
222                     j1 = j
223         for j in range(n + 1):
224             if used[j]:
225                 u[p[j]] += delta
226                 v[j] -= delta
227             else:
228                 minimum[j] -= delta
229         j0 = j1
230         if p[j0] == 0:
231             break
232         while True:
233             j1 = way[j0]
234             p[j0] = p[j1]
235             j0 = j1
236             if j0 == 0:
237                 break
238         succ = [0] * n
239         for j in range(1, n + 1):
240             succ[p[j] - 1] = j - 1
241         return (sum((cost[i][j] for i, j in enumerate(succ))), succ)
242
243
244 celue = "event-rollout-certified-completion-v8"
245 banben = "20260912-shared-service-r4"
246
247
248 def sscd(problem):
249     if problem == 3:
250         return [(0.0, 0.0)] + [
251             (1125 * math.cos(k * math.pi / 3), 1125 * math.sin(k * math.pi / 3))
252             for k in range(6)
253         ]
254     return (
255         [(0.0, 0.0)]
256         + [
257             (998 * math.cos(k * math.pi / 4), 998 * math.sin(k * math.pi / 4))
258             for k in range(8)
259         ]
260         + [
261             (
262                 1868 * math.cos(k * math.pi / 6 + math.pi / 12),
263                 1868 * math.sin(k * math.pi / 6 + math.pi / 12),
264             )
265             for k in range(12)
266         ]
267     )
268
269
270 def pindao(jilu, status):
271     return {c for c, s in jilu["channels"].items() if s["status"] in status}
272
273
274 def xjjl(problem, dzys=640, *, fgjd=4096, fgds=12, jhbb=1):
275     if jhbb not in (1, 2):
276         raise ValueError("Unknown source geometry version")
277     return dict(
278         problem=problem,
279         extra=dzys,

```

```

280     coverage_nodes=fgjd,
281     coverage_depth=fgsd,
282     geometry_version=jhbb,
283     receipts=[],
284     certificates=[],
285     channels={
286         c: dict(
287             status="unknown",
288             radio=[],
289             optical=[],
290             records=[],
291             pending=set(g.sswd()),
292             source=None,
293             proof=None,
294         )
295         for c in range(1, 21)
296     },
297 )
298
299
300 def xjlx(points, seconds=0.25):
301     return dict(
302         points=list(points),
303         seconds=seconds,
304         signature=None,
305         route=[],
306         proofs=0,
307         deletions=0,
308         replacements=0,
309         pending_stop=None,
310     )
311
312
313 def xjzt(io, problem, jd=None, *, dzys=640, ghsj=0.2, qqsj=0.1, byonly=False):
314     return dict(
315         io=io,
316         problem=problem,
317         progress=jd,
318         ledger=xjzl(problem, dzys, fgjd=20000, fgsd=14, jhbb=2 if problem == 3 else 1),
319         planner=dict(
320             seconds=ghsj,
321             problem=problem,
322             statistics=dict(
323                 decisions=0,
324                 rollout_candidates=0,
325                 rollout_timeouts=0,
326                 continuous_clears=0,
327                 radio_events=0,
328             ),
329         ),
330         request_seconds=qqsj,
331         fallback_only=byonly,
332         position=(0.0, 0.0),
333         channel=1,
334         virtual=0.0,
335         travel=0.0,
336         measures=0,
337         clear_attempts=0,
338         switches=0,
339         entered=False,
340         exit_confirmed=False,
341         exit_attempted=False,
342         certificate=False,
343         reason="not_started",
344         started=None,
345         deadline=math.inf,
346         max_virtual=360000.0,
347         fallback_used=False,

```

```

348         initial_upper=None,
349         search_plan=xjlx(sscd(problem)),
350         guard_checks=0,
351     )
352
353
354 def qjj(poly, other):
355     if len(other) < 3:
356         return poly
357     for a, b in zip(other, other[1:] + other[:1]):
358         edge = (b[0] - a[0], b[1] - a[1])
359         normal = (edge[1], -edge[0])
360         poly = g.clip(poly, normal, normal[0] * a[0] + normal[1] * a[1])
361         if not poly:
362             break
363     return poly
364
365
366 def xjyd(origin, bearing=None, jhbb=1):
367     origin = g.point(origin)
368     yuan = dict(
369         origin=origin,
370         bearing=bearing,
371         points={},
372         cells={},
373         polygon=[],
374         positives=[],
375         negatives=[],
376         anchor_point=origin,
377         anchor_bearing=bearing or 0.0,
378         anchor_radius=1500.0,
379         rounds=0,
380     )
381     domain = g.juxing(-1800, -1800, 1800, 1800)
382     if bearing is None:
383         x, y = origin
384         yuan["polygon"] = qjj(g.juxing(x - 5, y - 5, x + 5, y + 5), domain)
385         yuan["points"] = {0: origin}
386         yuan["cells"] = {0: yuan["polygon"]}
387         yuan["anchor_radius"] = 5.0
388     else:
389         yuan["polygon"] = g.jcfw(domain, origin, fs(bearing))
390         index = 0
391         for row, y in enumerate((-25, 0, 25)):
392             for k in range(61) if row % 2 == 0 else range(60, -1, -1):
393                 x = 25 * k
394                 bounds = (
395                     max(0, x - fs(25, 2)),
396                     max(-30, y - fs(25, 2)),
397                     min(1500, x + fs(25, 2)),
398                     min(30, y + fs(25, 2)),
399                 )
400                 cell = qjj(g.xzqj(origin, bearing, bounds), yuan["polygon"])
401                 q = g.xzd(origin, bearing, x, y)
402                 if cell:
403                     if not g.ypbh(cell, q, fs(20)):
404                         raise RuntimeError("Optical responsibility is not covered")
405                     yuan["points"][index], yuan["cells"][index] = (q, cell)
406                     index += 1
407         yuan["positives"].append(origin)
408     if not yuan["polygon"] or not yuan["cells"]:
409         raise RuntimeError("Known source lost all feasible responsibilities")
410     if jhbb == 2:
411         xdfw(yuan, qjj(yuan["polygon"], g.yfw()))
412         yuan["anchor_radius"] = math.nextafter(
413             max((float(g.jlsx(yuan["anchor_point"], p)) for p in yuan["polygon"])),
414             math.inf,
415         )

```

```

416     return yuan
417
418
419 def xdfw(yuan, poly):
420     if not poly:
421         raise RuntimeError("Accepted evidence contradicts known source")
422     yuan["cells"] = {i: out for i, p in yuan["cells"].items() if (out := qjj(p, poly))}
423     yuan["polygon"] = poly
424     if not yuan["polygon"] or not yuan["cells"]:
425         raise RuntimeError("Known source lost all feasible responsibilities")
426
427
428 def gxyd(yuan, q, kind, bearing=None, problem=4):
429     q = g.point(q)
430     if kind == "direction":
431         xdfw(yuan, g.jcfw(yuan["polygon"], q, fs(bearing)))
432         yuan["positives"].append(q)
433         yuan["anchor_point"], yuan["anchor_bearing"] = (q, bearing)
434         yuan["anchor_radius"] = min(
435             1500.0,
436             math.nextafter(
437                 float(max((g.jlsx(q, v) for v in yuan["polygon"]))), math.inf
438             ),
439         )
440         yuan["rounds"] += 1
441     elif kind == "near":
442         x, y = q
443         xdfw(yuan, qjj(yuan["polygon"], g.juxing(x - 5, y - 5, x + 5, y + 5)))
444         yuan["anchor_point"] = q
445         yuan["anchor_radius"] = math.nextafter(
446             float(max((g.jlsx(q, v) for v in yuan["polygon"]))), math.inf
447         )
448     elif kind == "no_signal":
449         yuan["negatives"].append(q)
450         if problem == 3:
451             poly = yuan["polygon"]
452             for p in yuan["positives"]:
453                 poly = g.clip(
454                     poly,
455                     (q[0] - p[0], q[1] - p[1]),
456                     (q[0] * q[0] + q[1] * q[1] - (p[0] * p[0] + p[1] * p[1])) / 2,
457                 )
458             xdfw(yuan, poly)
459         yuan["cells"] = {
460             i: p
461             for i, p in yuan["cells"].items()
462             if problem != 3 or not g.ypbh(p, q, fs(1000))
463         }
464         if not yuan["polygon"] or not yuan["cells"]:
465             raise RuntimeError("Known source lost all feasible responsibilities")
466     elif kind == "no_target_in_range":
467         yuan["cells"] = {
468             i: p for i, p in yuan["cells"].items() if not g.ypbh(p, q, fs(20))
469         }
470         if not yuan["polygon"] or not yuan["cells"]:
471             raise RuntimeError("Known source lost all feasible responsibilities")
472         yuan["polygon"] = g.hull((v for p in yuan["cells"].values() for v in p))
473
474
475 def syx(yuan, pos, a, b):
476     if (
477         pos not in yuan["positives"]
478         or a not in yuan["negatives"]
479         or b not in yuan["negatives"]
480     ):
481         raise RuntimeError("Pair contraction lacks its observed evidence")
482     poly, changed = g.syx(yuan["polygon"], pos, a, b)
483     if changed:

```

```

484         xdfw(yuan, poly)
485         yuan["anchor_radius"] = min(
486             yuan["anchor_radius"],
487             math.nextafter(
488                 float(max((g.jlsx(yuan["anchor_point"], v) for v in poly))), math.inf
489             ),
490         )
491     yuan["rounds"] += 1
492     return changed
493
494
495 def syrw(jilu):
496     return (
497         sum(
498             (
499                 len(s["pending"])
500                 for s in jilu["channels"].values()
501                 if s["status"] == "unknown"
502             )
503         )
504         + sum(
505             (
506                 len(s["source"]["cells"])
507                 for s in jilu["channels"].values()
508                 if s["status"] == "found"
509             )
510         )
511         + 183 * (16 - len(pindao(jilu, ("found", "cleared"))))
512         + jilu["extra"]
513     )
514
515
516 def bjwy(jilu, c, basis, **details):
517     s = jilu["channels"][c]
518     if s["status"] != "unknown":
519         raise RuntimeError("Cannot mark a discovered source absent")
520     s["status"] = "absent"
521     s["pending"].clear()
522     s["proof"] = dict(channel=c, basis=basis, **details)
523     jilu["certificates"].append(s["proof"])
524
525
526 def zmwy(jilu, c, jdmax=None):
527     s = jilu["channels"][c]
528     if s["status"] != "unknown":
529         return False
530     if not s["pending"]:
531         bjwy(jilu, c, "exact_600m_grid_template", radio_count=len(s["radio"]))
532         return True
533     if len(s["radio"]) < (6 if jilu["problem"] == 3 else 7) and (not s["optical"]):
534         return False
535     if jdmax is None:
536         jdmax = jilu["coverage_nodes"]
537     proved, nodes, box, leaves = g.fgzmm(
538         tuple(sorted(set(s["radio"]))),
539         tuple(sorted(set(s["optical"]))),
540         jilu["problem"],
541         jdmax,
542         jilu["coverage_depth"],
543     )
544     if proved:
545         bjwy(
546             jilu,
547             c,
548             "rational_continuous_coverage",
549             radio_count=len(s["radio"]),
550             optical_count=len(s["optical"]),
551             nodes=nodes,

```

```

552         leaf_count=len(leaves),
553         max_nodes=jdmax,
554         max_depth=jilu["coverage_depth"],
555         radio=[tuple((float(x) for x in q)) for q in s["radio"]],
556         optical=[tuple((float(x) for x in q)) for q in s["optical"]],
557     )
558     return proved
559
560
561 def gxjl(jilu, path, q, c, huifu, prove=True):
562     before = syrw(jilu)
563     q = g.point(q)
564     s = jilu["channels"][c]
565     if s["status"] in ("cleared", "absent"):
566         raise RuntimeError("Action targets a resolved channel")
567     kind = huifu["measure_result" if path == "/measure" else "clear_result"]
568     s["records"].append(
569         dict(
570             path=path,
571             point=tuple((float(x) for x in q)),
572             kind=kind,
573             bearing=huifu.get("svd_deg"),
574         )
575     )
576     if path == "/measure":
577         if s["status"] == "unknown":
578             if kind == "no_signal":
579                 s["radio"].append(q)
580                 s["pending"].discard(q)
581                 if prove:
582                     zmwj(jilu, c)
583             else:
584                 s["status"] = "found"
585                 s["pending"].clear()
586                 s["source"] = xjyd(
587                     q,
588                     huifu["svd_deg"] if kind == "direction" else None,
589                     jilu["geometry_version"],
590                 )
591             elif kind == "direction":
592                 gxyd(s["source"], q, "direction", bearing=huifu["svd_deg"])
593             elif kind == "near":
594                 gxyd(s["source"], q, "near")
595             else:
596                 gxyd(s["source"], q, "no_signal", problem=jilu["problem"])
597     elif kind == "success":
598         s["status"] = "cleared"
599         s["pending"].clear()
600         s["source"] = None
601     elif s["status"] == "unknown":
602         s["optical"].append(q)
603         if prove:
604             zmwj(jilu, c)
605     else:
606         gxyd(s["source"], q, "no_target_in_range")
607     if len(pindao(jilu, ("found", "cleared"))) > 16:
608         raise RuntimeError("More than 16 distinct sources")
609     if len(pindao(jilu, ("found", "cleared"))) == 16:
610         for other in sorted(pindao(jilu, ("unknown",))):
611             bjw(jilu, other, "source_count_upper_bound")
612     if syrw(jilu) >= before:
613         if jilu["extra"] <= 0:
614             raise RuntimeError("An unproductive action had no reserved allowance")
615         jilu["extra"] -= 1
616     after = syrw(jilu)
617     if after >= before or after < 0:
618         raise RuntimeError("Completion rank failed to decrease")
619     receipt = dict(

```

```

620         before=before,
621         after=after,
622         extra_remaining=jilu["extra"],
623         channel=c,
624         path=path,
625         result=kind,
626     )
627     jilu["receipts"].append(receipt)
628     return receipt
629
630
631 def qbqc(jilu):
632     return (
633         not pindao(jilu, ("unknown",))
634         and (not pindao(jilu, ("found",)))
635         and (10 <= len(pindao(jilu, ("cleared",))) <= 16)
636     )
637
638
639 def sysj(jilu, weizhi):
640     p = g.point(weizhi)
641     parts = [
642         lxsj(gpx(jilu["channels"][c]["source"], p), p, p)
643         for c in pindao(jilu, ("found",))
644     ]
645     points = set(
646         (q for c in pindao(jilu, ("unknown",)) for q in jilu["channels"][c]["pending"])
647     )
648     prev = p
649     for q in g.sswd():
650         if q in points:
651             parts.append(math.nextafter(float(g.jlsx(prev, q) / 5), math.inf))
652             prev = q
653     requests = sum(
654         (len(jilu["channels"][c]["pending"]) for c in pindao(jilu, ("unknown",)))
655     )
656     parts.append(6 * requests)
657     parts.append(
658         math.nextafter(
659             1843.1
660             * min(
661                 16 - len(pindao(jilu, ("found", "cleared"))),
662                 len(pindao(jilu, ("unknown",))),
663             ),
664             math.inf,
665         )
666     )
667     return math.nextafter(math.fsum(parts), math.inf)
668
669
670 def jlzy(jilu):
671     return dict(
672         discovered_channels=sorted(pindao(jilu, ("found", "cleared"))),
673         cleared_channels=sorted(pindao(jilu, ("cleared",))),
674         unknown_channels=sorted(pindao(jilu, ("unknown",))),
675         pending_channels=sorted(pindao(jilu, ("found",))),
676         absent_channels=[
677             c for c, s in jilu["channels"].items() if s["status"] == "absent"
678         ],
679         extra_actions_remaining=jilu["extra"],
680         completion_rank=syrw(jilu),
681         rank_receipts=len(jilu["receipts"]),
682         absence_certificates=jilu["certificates"],
683     )
684
685
686 def lxqc(yuan, dq, target=None):
687     polygon = [tuple((float(x) for x in p)) for p in yuan["polygon"]]

```

```

688 cand = zjqcd(polygon, dq)
689 if cand is None:
690     return None
691 options = [cand]
692 if target is not None:
693     q, _ = qclxyh(polygon, 19.8 - 1e-06, cand, dq, target)
694     options.append(q)
695 valid = [
696     g.point(q) for q in options if g.ypbh(yuan["polygon"], g.point(q), fs("19.8"))
697 ]
698 if not valid:
699     return None
700 return min(
701     valid,
702     key=lambda q: (
703         math.dist(dq, tuple((float(x) for x in q)))
704         + (
705             math.dist(tuple((float(x) for x in q)), target)
706             if target is not None
707             else 0
708         )
709     ),
710 )
711
712
713 def gxpx(yuan, dq, mode="snake", cells=None):
714     ids = set(yuan["cells"] if cells is None else cells)
715     if mode == "snake":
716         return [yuan["points"][i] for i in sorted(ids)]
717     order = []
718     p = g.point(dq)
719     while ids:
720         i = min(
721             ids,
722             key=lambda i: (
723                 (p[0] - yuan["points"][i][0]) * (p[0] - yuan["points"][i][0])
724                 + (p[1] - yuan["points"][i][1]) * (p[1] - yuan["points"][i][1]),
725                 i,
726             ),
727         )
728         ids.remove(i)
729         p = yuan["points"][i]
730         order.append(p)
731     return order
732
733
734 def lxsj(points, dq, target=None):
735     p = g.point(dq)
736     spent = fs(0)
737     worst = fs(0)
738     for q in points:
739         spent += g.jlsx(p, q) / 5
740         leave = g.jlsx(q, g.point(target)) / 5 if target is not None else 0
741         worst = max(worst, spent + 5 + leave)
742         spent += 3
743         p = q
744     return math.nextafter(float(worst), math.inf)
745
746
747 def xuduan(yuan, dq, cells=None):
748     return min(
749         (
750             (lxsj(order, dq), mode, order)
751             for mode in ("snake", "nearest")
752             if (order := gxpx(yuan, dq, mode, cells))
753         )
754     )
755

```

```

756
757 def sdc(yuan, dq, ratio=0.5):
758     length = fs(yuan["anchor_radius"])
759     x = 5 + (length - 5) * fs(ratio)
760     side = length / 50
761     if len(yuan["positives"]) >= 2:
762         u, _ = g.fangxiang(fs(yuan["anchor_bearing"]))
763         dist = [
764             (q[0] - yuan["anchor_point"][0]) * u[0]
765             + (q[1] - yuan["anchor_point"][1]) * u[1]
766             for q in yuan["polygon"]
767         ]
768         lo, hi = (min(dist), max(dist))
769         x = (lo + hi) / 2
770         side = max(fs(5), (hi - lo) / 20)
771     points = [
772         g.xzd(yuan["anchor_point"], yuan["anchor_bearing"], x, sign * side)
773         for sign in (-1, 1)
774     ]
775     return tuple(
776         sorted(
777             points,
778             key=lambda q: (
779                 (q[0] - g.point(dq)[0]) * (q[0] - g.point(dq)[0])
780                 + (q[1] - g.point(dq)[1]) * (q[1] - g.point(dq)[1])
781             ),
782         )
783     )
784
785
786 def kdqy(yuan, polygon):
787     if not polygon:
788         return []
789     return [i for i, cell in yuan["cells"].items() if qjj(cell, polygon)]
790
791
792 def sptz(yuan, dq, pair, problem, dl):
793     a, b = pair
794     start = float(g.jlsx(g.point(dq), a) / 5) + 6
795     ceshi = [(a, start)]
796     if problem == 4:
797         ceshi.append((b, start + float(g.jlsx(a, b) / 5) + 5))
798     worst = 0.0
799     paths = 0
800     for q, prefix in ceshi:
801         for lo in range(0, 360, 60):
802             if time.perf_counter() >= dl:
803                 return None
804             region = g.jcfw(yuan["polygon"], q, fs(lo), hi=fs(lo + 60))
805             ids = kdqy(yuan, region)
806             if ids:
807                 upper, _, _ = xuduan(yuan, q, ids)
808                 worst = max(worst, prefix + upper)
809                 paths += 1
810             worst = max(worst, prefix + 5)
811             paths += 1
812     if problem == 3:
813         if not g.ypbh(yuan["polygon"], a, fs(1000)):
814             worst = max(worst, start + xuduan(yuan, a)[0])
815             paths += 1
816     else:
817         prefix = ceshi[-1][1]
818         region, _ = g.syxc(yuan["polygon"], yuan["anchor_point"], a, b)
819         ids = kdqy(yuan, region)
820         if ids:
821             worst = max(worst, prefix + xuduan(yuan, b, ids)[0])
822             paths += 1
823     return (worst, paths)

```

```

824
825
826 def gxjs(source):
827     values = []
828     for poly in source["cells"].values():
829         points = [tuple((float(x) for x in p)) for p in poly]
830         center = tuple((sum((p[k] for p in points)) / len(points) for k in (0, 1)))
831         area = (
832             abs(
833                 sum(
834                     (
835                         a[0] * b[1] - a[1] * b[0]
836                         for a, b in zip(points, points[1:] + points[:1])
837                     )
838                 )
839             )
840             / 2
841         )
842         values.append((center, area))
843     z1 = sum((w for _, w in values))
844     return [(p, w / z1 if z1 > 1e-12 else 1 / len(values)) for p, w in values]
845
846
847 def gxjz(order, wz, worlds, target=None):
848     elapsed = 0.0
849     prev = wz
850     rest = set(range(len(worlds)))
851     mean = 0.0
852     xout = yout = 0.0
853     for q in order:
854         q = tuple((float(x) for x in q))
855         elapsed += math.dist(prev, q) / 5
856         hits = [i for i in rest if math.dist(worlds[i][0], q) <= 20]
857         for i in hits:
858             mean += worlds[i][1] * (
859                 elapsed + 5 + (math.dist(q, target) / 5 if target is not None else 0)
860             )
861             xout += worlds[i][1] * q[0]
862             yout += worlds[i][1] * q[1]
863             rest.remove(i)
864         prev = q
865         elapsed += 3
866     return (math.inf if rest else mean, (xout, yout))
867
868
869 def gxbx(source, wz, target=None):
870     poly = source["polygon"]
871     points = [tuple((float(x) for x in p)) for p in poly]
872     angle = math.radians(source["anchor_bearing"])
873     result = []
874     worlds = None
875     for theta in (angle, angle + math.pi / 2):
876         u = (math.cos(theta), math.sin(theta))
877         w = (-u[1], u[0])
878         xs = [p[0] * u[0] + p[1] * u[1] for p in points]
879         ys = [p[0] * w[0] + p[1] * w[1] for p in points]
880         lo, hi = (min(xs) - 1e-06, max(xs) + 1e-06)
881         mid = (min(ys) + max(ys)) / 2
882         halfwidth = (max(ys) - min(ys)) / 2 + 1e-06
883         if halfwidth >= 19.79:
884             continue
885         half = math.sqrt(19.79**2 - halfwidth**2)
886         n = max(1, math.ceil((hi - lo) / (2 * half)))
887         if not 2 <= n <= 8:
888             continue
889         centers = [lo + half + (hi - lo - 2 * half) * i / (n - 1) for i in range(n)]
890         chain = [
891             g.point((x * u[0] + mid * w[0], x * u[1] + mid * w[1])) for x in centers

```

```

892     ]
893     valid = True
894     for q in chain:
895         part = poly
896         for other in chain:
897             if q != other:
898                 part = g.clip(
899                     part,
900                     (other[0] - q[0], other[1] - q[1]),
901                     (
902                         other[0] * other[0]
903                         + other[1] * other[1]
904                         - (q[0] * q[0] + q[1] * q[1])
905                     )
906                     / 2,
907                 )
908             if part and (not g.ypbh(part, q, fs("19.8"))):
909                 valid = False
910                 break
911     if not valid:
912         continue
913     if worlds is None:
914         worlds = gxjs(source)
915     greedy = []
916     left = list(chain)
917     now = wz
918     rest = set(range(len(worlds)))
919     while left:
920         q = min(
921             left,
922             key=lambda p: (
923                 -sum(
924                     (
925                         worlds[i][1]
926                         for i in rest
927                         if math.dist(worlds[i][0], tuple((float(x) for x in p)))
928                         <= 20
929                     )
930                 )
931                 / (3 + math.dist(now, tuple((float(x) for x in p))) / 5),
932                 (g.point(now)[0] - p[0]) * (g.point(now)[0] - p[0])
933                 + (g.point(now)[1] - p[1]) * (g.point(now)[1] - p[1]),
934             ),
935         )
936         greedy.append(q)
937         left.remove(q)
938         now = tuple((float(x) for x in q))
939         rest = {i for i in rest if math.dist(worlds[i][0], now) > 20}
940     for order in dict.fromkeys(
941         (tuple(chain), tuple(reversed(chain)), tuple(greedy))
942     ):
943         mean, leave = gxjz(order, wz, worlds, target)
944         if math.isfinite(mean):
945             result.append((mean, order, leave, lxsj(order, wz, target)))
946     return result
947
948
949 def gxxz(jihua, yuan, channel, wz, tuned, extra, event, target):
950     if jihua["problem"] != 4 or event["method"] == "verified_continuous" or extra < 8:
951         return event
952     options = gxbx(yuan, wz, target)
953     if not options:
954         return event
955     worlds = gxjs(yuan)
956     q = tuple((float(x) for x in event["points"][0]))
957     ideal = math.dist(wz, q) / 5 + 5 + (channel != tuned) + 5
958     ideal += sum(
959         (

```

```

960         weight
961     * (
962         max(0, math.dist(q, source) - 19.8) / 5
963     + (
964         max(0, math.dist(source, target) - 19.8) / 5
965         if target is not None
966         else 0
967     )
968 )
969     for source, weight in worlds
970 )
971 )
972 mean, order, leave, upper = min(options, key=lambda option: option[0])
973 if mean < ideal:
974     return dict(
975         kind="optical_chain",
976         channel=channel,
977         points=order,
978         estimate=mean,
979         certificate_upper_s=upper,
980         method="complete_cover_expected_first_success",
981     )
982 return event
983
984
985 def yddz(jihua, yuan, channel, dq, tuned, extra, target=None):
986     q = lxqc(yuan, dq, target)
987     if q is not None:
988         jihua["statistics"]["continuous_clears"] += 1
989         return {
990             "kind": "clear",
991             "channel": channel,
992             "points": (q,),
993             "estimate": math.dist(dq, tuple((float(x) for x in q))) / 5 + 5,
994             "method": "verified_continuous",
995         }
996     upper, mode, order = xudian(yuan, dq)
997     best = {
998         "kind": "clear",
999         "channel": channel,
1000         "points": (order[0],),
1001         "estimate": upper,
1002         "method": "complete_optical_" + mode,
1003     }
1004     if extra < 2 or yuan["anchor_radius"] <= 5 or yuan["rounds"] >= 12:
1005         return gxxz(jihua, yuan, channel, dq, tuned, extra, best, target)
1006     dl = time.perf_counter() + jihua["seconds"]
1007     for ratio in (0.5, 0.38, 0.62):
1008         pair = sdc(yuan, dq, ratio)
1009         jieguo = sptz(yuan, dq, pair, jihua["problem"], dl)
1010         if jieguo is None:
1011             jihua["statistics"]["rollout_timeouts"] += 1
1012             break
1013         value, paths = jieguo
1014         jihua["statistics"]["rollout_candidates"] += 1
1015         if value < best["estimate"]:
1016             best = {
1017                 "kind": "radio",
1018                 "channel": channel,
1019                 "points": pair,
1020                 "estimate": value,
1021                 "method": f"interval_rollout_{ratio:g}_{paths}_branches",
1022             }
1023     if best["kind"] == "clear" and len(yuan["cells"]) > 4:
1024         pair = sdc(yuan, dq)
1025         best = {
1026             "kind": "radio",
1027             "channel": channel,

```

```

1028         "points": pair,
1029         "estimate": best["estimate"],
1030         "method": "annular_proposal_with_guard",
1031     }
1032     best = bmcf(jihua, yuan, dq, best)
1033     best = gxxz(jihua, yuan, channel, dq, tuned, extra, best, target)
1034     jihua["statistics"]["radio_events"] += best["kind"] == "radio"
1035     return best
1036
1037
1038 def bmcf(jihua, yuan, wz, event):
1039     if jihua["problem"] != 4 or event["kind"] != "radio":
1040         return event
1041     if not all((q in yuan["negatives"] for q in event["points"])):
1042         return event
1043     stats = jihua["statistics"]
1044     stats["repeated_negative_pairs_skipped"] = (
1045         stats.get("repeated_negative_pairs_skipped", 0) + 1
1046     )
1047     upper, _, order = xudian(yuan, wz)
1048     return dict(
1049         event,
1050         kind="clear",
1051         points=(order[0],),
1052         estimate=upper,
1053         method="avoid_repeated_negative_pair",
1054     )
1055
1056
1057 def pgxsx(tu, order):
1058     tu["evaluations"] += 1
1059     if not order:
1060         return (0.0, [])
1061     costs = {i: (tu["start"][i], [i]) for i in tu["ids"][order[0]]}
1062     for job in order[1:]:
1063         costs = {
1064             j: min(
1065                 (
1066                     (value + tu["edge"][i, j], path + [j])
1067                     for i, (value, path) in costs.items()
1068                 )
1069             )
1070             for j in tu["ids"][job]
1071         }
1072     value, path = min(costs.values())
1073     return (value, [tu["modes"][i] for i in path])
1074
1075
1076 def fpsx(tu):
1077     jobs = list(tu["ids"])
1078     n = len(jobs)
1079     cost = [[1000000000000.0] * (n + 1) for _ in range(n + 1)]
1080     for j, job in enumerate(jobs, 1):
1081         cost[0][j] = min((tu["start"][k] for k in tu["ids"][job]))
1082     for i, a in enumerate(jobs, 1):
1083         cost[i][0] = 0.0
1084         for j, b in enumerate(jobs, 1):
1085             if i != j:
1086                 cost[i][j] = min(
1087                     (tu["edge"][x, y] for x in tu["ids"][a] for y in tu["ids"][b])
1088                 )
1089     _, succ = zhipai(cost)
1090
1091     def cycle(start):
1092         out = [start]
1093         q = succ[start]
1094         while q != start:
1095             out.append(q)

```

```

1096         q = succ[q]
1097     return out
1098
1099     main = cycle(0)
1100     while len(main) < n + 1:
1101         other = cycle(next((i for i in range(n + 1) if i not in main)))
1102         a, b = min(
1103             ((a, b) for a in main for b in other),
1104             key=lambda ab: (
1105                 cost[ab[0]][succ[ab[1]]]
1106                 + cost[ab[1]][succ[ab[0]]]
1107                 - cost[ab[0]][succ[ab[0]]]
1108                 - cost[ab[1]][succ[ab[1]]]
1109             ),
1110         )
1111         succ[a], succ[b] = (succ[b], succ[a])
1112         main = cycle(0)
1113     return [jobs[i - 1] for i in main[1:]]
1114
1115
1116 def aprw(jobs, wz, oldorder=()):
1117     modes = [m for opts in jobs.values() for m in opts]
1118     tu = dict(
1119         modes=modes,
1120         ids={job: [i for i, m in enumerate(modes) if m["job"] == job] for job in jobs},
1121         edge={
1122             (i, j): math.dist(a["exit"], b["entry"]) / 5 + b["service"]
1123             for i, a in enumerate(modes)
1124             for j, b in enumerate(modes)
1125             if a["job"] != b["job"]
1126         },
1127         start={
1128             i: math.dist(wz, m["entry"]) / 5 + m["service"] for i, m in enumerate(modes)
1129         },
1130         evaluations=0,
1131     )
1132     old = [job for job in oldorder if job in jobs]
1133     old += [job for job in jobs if job not in old]
1134     rest = list(jobs)
1135     greedy = []
1136     last = None
1137     while rest:
1138         job = min(
1139             rest,
1140             key=lambda job: min(
1141                 (
1142                     tu["start"][i] if last is None else tu["edge"][last, i]
1143                     for i in tu["ids"][job]
1144                 )
1145             ),
1146         )
1147         last = min(
1148             tu["ids"][job],
1149             key=lambda i: tu["start"][i] if last is None else tu["edge"][last, i],
1150         )
1151         greedy.append(job)
1152         rest.remove(job)
1153     scans = [job for job in jobs if job[0] == "scan"]
1154     sources = [job for job in jobs if job[0] == "source"]
1155     seeds = [old, greedy, sources + scans, scans + sources, fpsx(tu)]
1156     scores = []
1157     for seed in seeds:
1158         value, opts = pgxs(tu, seed)
1159         scores.append((value, seed, opts))
1160     value, order, opts = min(scores, key=lambda e: e[0])
1161     index = {id(m): i for i, m in enumerate(modes)}
1162     for _ in range(2):
1163         before = value

```

```

1164     trials = []
1165     chosen = {m["job"]: index[id(m)] for m in opts}
1166     for i in range(len(order)):
1167         for j in range(len(order)):
1168             if i == j:
1169                 continue
1170             trial = list(order)
1171             item = trial.pop(i)
1172             trial.insert(j, item)
1173             seq = [chosen[job] for job in trial]
1174             score = tu["start"][seq[0]] + sum(
1175                 (tu["edge"][a, b] for a, b in zip(seq, seq[1:]))
1176             )
1177             if score < value - 1e-07:
1178                 trials.append((score, trial))
1179     for _, trial in sorted(trials, key=lambda x: x[0][:16]):
1180         cand, plan = pgxs(tu, trial)
1181         if cand < value - 1e-07:
1182             value, order, opts = (cand, trial, plan)
1183     if value >= before - 1e-07:
1184         break
1185     return dict(estimate=value, order=order, modes=opts, evaluations=tu["evaluations"])
1186
1187
1188 def xzdz(jihua, jilu, dq, tuned, sscd):
1189     jihua["statistics"]["decisions"] += 1
1190     jobs = {}
1191     for c in sorted(pindao(jilu, ("found",))):
1192         yuan = jilu["channels"][c]["source"]
1193         job = ("source", c)
1194         direct = lxqc(yuan, dq)
1195         if direct is not None:
1196             q = tuple((float(x) for x in direct))
1197             event = dict(
1198                 kind="clear",
1199                 channel=c,
1200                 points=(direct,),
1201                 estimate=5.0,
1202                 method="service_direct_clear",
1203             )
1204             modes = [dict(job=job, entry=q, service=5.0, exit=q, event=event)]
1205         else:
1206             center = tuple(
1207                 (
1208                     sum((float(q[k]) for q in yuan["polygon"])) / len(yuan["polygon"])
1209                     for k in (0, 1)
1210                 )
1211             )
1212             modes = []
1213             for ratio in (0.38, 0.5, 0.62):
1214                 pair = sdc(yuan, dq, ratio)
1215                 for order in (pair, pair[::-1]) if jilu["problem"] == 4 else (pair,):
1216                     q = tuple((float(x) for x in order[0]))
1217                     cost = math.dist(q, center) / 5 + 15.0
1218                     event = dict(
1219                         kind="radio",
1220                         channel=c,
1221                         points=order,
1222                         estimate=cost,
1223                         method="service_entry_exit",
1224                     )
1225                     modes.append(
1226                         dict(job=job, entry=q, service=cost, exit=center, event=event)
1227                     )
1228             jobs[job] = modes
1229     for q in sscd:
1230         todo = [
1231             c

```

```

1232         for c in pindao(jilu, ("unknown",))
1233         if g.point(q) not in jilu["channels"][c]["radio"]
1234     ]
1235     if todo:
1236         job = ("scan", tuple(q))
1237         event = dict(
1238             kind="search",
1239             channel=min(todo, key=lambda c: (c != tuned, c)),
1240             points=(g.point(q),),
1241             estimate=6 * len(todo),
1242             method="service_search",
1243         )
1244         jobs[job] = [
1245             dict(job=job, entry=q, service=event["estimate"], exit=q, event=event)
1246         ]
1247     if not jobs:
1248         return None
1249     plan = aprw(jobs, dq, jihua.get("order", ()))
1250     jihua["order"] = plan["order"]
1251     event = plan["modes"][0]["event"]
1252     event["target"] = plan["modes"][1]["entry"] if len(plan["modes"]) > 1 else None
1253     if event["kind"] != "search":
1254         event = bmcf(jihua, jilu["channels"][event["channel"]]["source"], dq, event)
1255     return event
1256
1257
1258 def gxlx(luxian, jilu, dq):
1259     if not pindao(jilu, ("unknown",)):
1260         return []
1261     common = set.intersection(
1262         *(set(jilu["channels"][c]["radio"]) for c in pindao(jilu, ("unknown",)))
1263     )
1264     sycd = [q for q in luxian["points"] if g.point(q) not in common]
1265     sig = (
1266         tuple(sorted(common)),
1267         tuple(sorted(pindao(jilu, ("unknown",)))),
1268         luxian["pending_stop"],
1269     )
1270     route = lxpx(dq, sycd)
1271     if sig == luxian["signature"]:
1272         return route
1273     luxian["signature"] = sig
1274     dl = time.perf_counter() + luxian["seconds"]
1275
1276     def cost(qs):
1277         return lxcd(dq, qs) / 5 + 6 * len(pindao(jilu, ("unknown",))) * len(qs)
1278
1279     for _ in range(3):
1280         old = cost(route)
1281         options = []
1282         for q in route:
1283             rest = [p for p in route if p != q]
1284             trial = lxpx(dq, rest)
1285             options.append((cost(trial), "delete", q, trial))
1286             if (
1287                 luxian["pending_stop"] is not None
1288                 and luxian["pending_stop"] not in rest
1289                 and (g.point(luxian["pending_stop"]) not in common)
1290             ):
1291                 trial = lxpx(dq, rest + [luxian["pending_stop"]])
1292                 options.append((cost(trial), "replace", q, trial))
1293         improved = False
1294         for value, kind, removed, trial in sorted(options):
1295             if value >= old - 1e-06 or time.perf_counter() >= dl:
1296                 break
1297             luxian["proofs"] += 1
1298             points = tuple(sorted(common | {g.point(q) for q in trial}))
1299             proved = g.fgzx(

```

```

1300         points,
1301         (),
1302         jilu["problem"],
1303         jilu["coverage_nodes"],
1304         jilu["coverage_depth"],
1305     )[0]
1306     if proved:
1307         route = trial
1308         improved = True
1309         luxian["deletions"] += kind == "delete"
1310         luxian["replacements"] += kind == "replace"
1311         break
1312     if not improved:
1313         break
1314     luxian["points"] = route
1315     luxian["route"] = route
1316     luxian["pending_stop"] = None
1317     luxian["signature"] = (
1318         tuple(sorted(common)),
1319         tuple(sorted(pindao(jilu, ("unknown",)))),
1320         None,
1321     )
1322     return route
1323
1324
1325 def bcsj(zt, event, **fields):
1326     data = dict(event=event, virtual_time_s=zt["virtual"], **fields)
1327     zt["io"]["record"](data)
1328     if zt["progress"]:
1329         zt["progress"](data)
1330
1331
1332 def syics(zt):
1333     l = zt["ledger"]
1334     return (
1335         sum((len(l["channels"][c]["pending"]) for c in pindao(l, ("unknown",))))
1336         + sum((len(l["channels"][c]["source"]["cells"]) for c in pindao(l, ("found",))))
1337         + 183
1338         * min(16 - len(pindao(l, ("found", "cleared"))), len(pindao(l, ("unknown",))))
1339         + 2
1340     )
1341
1342
1343 def ysjc(zt, points):
1344     jilu = zt["ledger"]
1345     zt["guard_checks"] += 1
1346     upper = syjs(jilu, zt["position"])
1347     p = zt["position"]
1348     dist = 0.0
1349     for q in points:
1350         dist += float(g.jlxs(g.point(p), g.point(q)))
1351         p = tuple((float(x) for x in q))
1352     m = min(16, len(pindao(jilu, ("found",))) + len(points))
1353     virtualok = (
1354         zt["virtual"] + upper + (2 * m + 2) * dist / 5 + 6 * len(points)
1355         < zt["max_virtual"] - 2
1356     )
1357     realok = (
1358         time.monotonic() + (syics(zt) + len(points)) * zt["request_seconds"] + 30
1359         < zt["deadline"]
1360     )
1361     return virtualok and realok
1362
1363
1364 def zhixing(zt, path, q, c, *, prove=True):
1365     jiekou = zt["io"]
1366     jilu = zt["ledger"]
1367     q = tuple((float(x) for x in g.point(q)))

```

```

1368     if jiekou["pending"] is not None:
1369         raise RuntimeError("Earlier request unresolved")
1370     if time.monotonic() >= zt["deadline"] - 2:
1371         raise TimeoutError("Real deadline reserve reached without completion")
1372     moved = math.dist(zt["position"], q)
1373     if zt["virtual"] + moved / 5 + 6 >= zt["max_virtual"] - 1:
1374         raise RuntimeError("Physical action exceeds virtual budget")
1375     sw = int(path == "/measure" and c != zt["channel"])
1376     huifu = jiekou["call"](path, q, c)
1377     jyhfu(path, huifu)
1378     before = zt["virtual"]
1379     zt["position"] = q
1380     zt["virtual"] = huifu["virtual_time_s"]
1381     zt["travel"] += moved
1382     if path == "/measure":
1383         zt["measures"] += 1
1384         zt["switches"] += sw
1385         zt["channel"] = c
1386         cost = 5 + sw
1387     else:
1388         zt["clear_attempts"] += 1
1389         cost = 5 if huifu["clear_result"] == "success" else 3
1390     if abs(zt["virtual"] - before - moved / 5 - cost) > 5e-05:
1391         raise RuntimeError("Accepted response violates documented virtual costs")
1392     found = c in pindao(jilu, ("found", "cleared"))
1393     bcsj(
1394         zt,
1395         "evidence_observation",
1396         path=path,
1397         point=q,
1398         channel=c,
1399         reply=huifu,
1400         prove=prove,
1401     )
1402     receipt = gxjl(jilu, path, q, c, huifu, prove=prove)
1403     bcsj(zt, "rank_receipt", **receipt)
1404     if not found and c in pindao(jilu, ("found", "cleared")):
1405         bcsj(zt, "discovered", channel=c)
1406     if path == "/clear" and huifu["clear_result"] == "success":
1407         zt["search_plan"]["pending_stop"] = tuple(q)
1408         bcsj(zt, "cleared", channel=c, cleared_count=len(pindao(jilu, ("cleared",))))
1409     return huifu
1410
1411
1412 def fxjd(p, q):
1413     if p[0] == q[0] and p[1] == q[1]:
1414         return [(0.0, 360.0)]
1415     angle = math.degrees(math.atan2(q[1] - p[1], q[0] - p[0]))
1416     lo = (angle - 90) % 360
1417     return [(lo, min(360, lo + 180))] + [(0, lo - 180)] if lo > 180 else []
1418
1419
1420 def jdjj(a, b):
1421     return [(max(x, u), min(y, w)) for x, y in a for u, w in b if max(x, u) < min(y, w)]
1422
1423
1424 def jdcj(a, b):
1425     for lo, hi in b:
1426         a = [
1427             piece
1428             for x, y in a
1429             for piece in ((x, min(y, lo)), (max(x, hi), y))
1430             if piece[0] < piece[1]
1431         ]
1432     return a
1433
1434
1435 def jdcd(spans):

```

```

1436     return sum((y - x for x, y in spans))
1437
1438
1439 def spmx(record):
1440     src = record["source"]
1441     pos = []
1442     neg = []
1443     for obs in record["records"]:
1444         if obs["path"] != "/measure":
1445             continue
1446         (neg if obs["kind"] == "no_signal" else pos).append(obs["point"])
1447     zl = 0.0
1448     jiashuo = []
1449     for poly in src["cells"].values():
1450         points = [tuple((float(x) for x in q)) for q in poly]
1451         p = tuple((sum((q[k] for q in points)) / len(points) for k in (0, 1)))
1452         area = (
1453             abs(
1454                 sum(
1455                     (
1456                         a[0] * b[1] - a[1] * b[0]
1457                         for a, b in zip(points, points[1:] + points[:1])
1458                     )
1459                 )
1460             )
1461             / 2
1462         )
1463         lower = max([1000] + [math.dist(p, q) for q in pos])
1464         upper = 1500.0
1465         if lower >= upper or area <= 1e-12:
1466             continue
1467         valid = [(0.0, 360.0)]
1468         for q in pos:
1469             valid = jdjj(valid, fxjd(p, q))
1470         negs = [(math.dist(p, q), q) for q in neg]
1471         radii = sorted({lower, upper} | {d for d, q in negs if lower < d < upper})
1472         for lo, hi in zip(radii, radii[1:]):
1473             allowed = valid
1474             omni = True
1475             for d, q in negs:
1476                 if d < (lo + hi) / 2:
1477                     allowed = jdcj(allowed, fxjd(p, q))
1478                     omni = False
1479             mass = area * (hi - lo) * (0.5 * omni + 0.5 * jdcj(allowed) / 360)
1480             if mass > 0:
1481                 zl += mass
1482             jiashuo.append((p, lo, hi, allowed, omni, area))
1483     return (jiashuo, zl)
1484
1485
1486 def spgl(fitted, q):
1487     jiashuo, zl = fitted
1488     if zl <= 0:
1489         return 0.5
1490     kejian = 0.0
1491     for p, lo, hi, allowed, omni, area in jiashuo:
1492         width = max(0, hi - max(lo, math.dist(p, q)))
1493         if width:
1494             kejian += (
1495                 area
1496                 * width
1497                 * (0.5 * omni + 0.5 * jdcj(jdjj(allowed, fxjd(p, q))) / 360)
1498             )
1499     return min(1, max(0, kejian / zl))
1500
1501
1502 def slcl(zl):
1503     jilu = zl["ledger"]

```

```

1504     wz = zt["position"]
1505     beixuan = []
1506     for c in sorted(pindao(jilu, ("found",))):
1507         yuan = jilu["channels"][c]["source"]
1508         tries = len(yuan["positives"]) + (
1509             len(yuan["negatives"]) if jilu["problem"] == 3 else 0
1510         )
1511         if tries >= 3 or g.point(wz) in yuan["positives"] + yuan["negatives"]:
1512             continue
1513         poly = [tuple((float(x) for x in p)) for p in yuan["polygon"]]
1514         center = tuple((sum((p[k] for p in poly)) / len(poly) for k in (0, 1)))
1515         if jilu["problem"] == 3:
1516             if math.dist(wz, center) > 1000:
1517                 continue
1518             elif not g.ypbh(yuan["polygon"], g.point(wz), fs(999)):
1519                 continue
1520             angle = math.radians(yuan["anchor_bearing"])
1521             u = (math.cos(angle), math.sin(angle))
1522             origin = tuple((float(x) for x in yuan["anchor_point"]))
1523             span = [(p[0] - origin[0]) * u[0] + (p[1] - origin[1]) * u[1] for p in poly]
1524             width = max(span) - min(span)
1525             ray = (center[0] - wz[0], center[1] - wz[1])
1526             dist = math.hypot(*ray)
1527             parallax = abs(u[0] * ray[1] - u[1] * ray[0]) / max(1.0, dist)
1528             after = min(width, 2 * math.radians(1.005) * dist / max(0.01, parallax))
1529             recv = spgl(spmx(jilu["channels"][c]), wz) if jilu["problem"] == 4 else 1
1530             gain = recv * (width - after) / 10 - 6
1531             if parallax >= 0.25 and gain > 0:
1532                 beixuan.append((-gain, c))
1533     for score, c in sorted(beixuan):
1534         if jilu["extra"] < 2 or not ysjc(zt, (wz,)):
1535             return False
1536     bcsj(
1537         zt,
1538         "shared_station_selected",
1539         channel=c,
1540         point=wz,
1541         estimated_gain_s=-score,
1542         scope="heuristic; reception is established only by the actual reply",
1543     )
1544     zhixing(zt, "/measure", wz, c)
1545     return True
1546
1547
1548 def plcl(zt, unknown):
1549     jilu = zt["ledger"]
1550     pd = [
1551         c
1552         for c in sorted(unknown & pindao(jilu, ("found",)))
1553         if len(jilu["channels"][c]["source"]["positives"]) == 1
1554     ]
1555     if len(pd) < 3 or jilu["extra"] < len(pd) + 2:
1556         return True
1557     wz = zt["position"]
1558     sources = []
1559     for c in pd:
1560         yuan = jilu["channels"][c]["source"]
1561         poly = [tuple((float(x) for x in p)) for p in yuan["polygon"]]
1562         center = tuple((sum((p[k] for p in poly)) / len(poly) for k in (0, 1)))
1563         angle = math.radians(yuan["anchor_bearing"])
1564         u = (math.cos(angle), math.sin(angle))
1565         origin = tuple((float(x) for x in yuan["anchor_point"]))
1566         span = [(p[0] - origin[0]) * u[0] + (p[1] - origin[1]) * u[1] for p in poly]
1567         sources.append((center, u, max(span) - min(span)))
1568     targets = [
1569         tuple(
1570             (
1571                 sum((float(p[k]) for p in jilu["channels"][c]["source"]["polygon"]))

```

```

1572         / len(jilu["channels"][c]["source"]["polygon"])
1573         for k in (0, 1)
1574     )
1575 )
1576 for c in pindao(jilu, ("found",))
1577 ]
1578 target = min(targets, key=lambda q: math.dist(wz, q))
1579 fitted = (
1580     [spm(x(jilu["channels"][c]) for c in pd]
1581     if jilu["problem"] == 4
1582     else [None] * len(pd)
1583 )
1584 radius = 350
1585 options = []
1586 for angle in range(0, 360, 30):
1587     q = (
1588         wz[0] + radius * math.cos(math.radians(angle)),
1589         wz[1] + radius * math.sin(math.radians(angle)),
1590     )
1591     gain = 0.0
1592     chosen = []
1593     for c, fit, (center, u, width) in zip(pd, fitted, sources):
1594         ray = (center[0] - q[0], center[1] - q[1])
1595         dist = math.hypot(*ray)
1596         parallax = abs(u[0] * ray[1] - u[1] * ray[0]) / max(1.0, dist)
1597         postwidth = min(width, 2 * math.radians(1.005) * dist / max(0.01, parallax))
1598         benefit = (width - postwidth) / 10.0
1599         if jilu["problem"] == 4:
1600             benefit = spgl(fit, q) * benefit - 6
1601         if jilu["problem"] == 3 or benefit > 0:
1602             gain += benefit
1603             chosen.append(c)
1604         detour = (radius + math.dist(q, target) - math.dist(wz, target)) / 5.0
1605         options.append((detour - gain, angle, q, gain, detour, chosen))
1606     score, angle, q, gain, detour, pd = min(options)
1607     if jilu["problem"] == 4 and (score >= 0 or not pd):
1608         return True
1609     bcsj(
1610         zt,
1611         "batch_observation_selected",
1612         channels=pd,
1613         point=q,
1614         angle=angle,
1615         radius=radius,
1616         estimated_gain_s=gain,
1617         estimated_detour_s=detour,
1618         reception_weighted=jilu["problem"] == 4,
1619     )
1620     for c in pd:
1621         if jilu["extra"] < 2 or not ysjc(zt, (q,)):
1622             return False
1623         zhixing(zt, "/measure", q, c)
1624     return True
1625
1626
1627 def zxsj(zt, event):
1628     jilu = zt["ledger"]
1629     unknown = pindao(jilu, ("unknown",))
1630     if not zx(zt, event):
1631         return False
1632     if event["kind"] == "search":
1633         return plcl(zt, unknown) and slcl(zt)
1634     c = event["channel"]
1635     while c in pindao(jilu, ("found",)):
1636         if jilu["extra"] < 2:
1637             return False
1638         next = yddz(
1639             zt["planner"],

```

```

1640         jilu["channels"][c]["source"],
1641         c,
1642         zt["position"],
1643         zt["channel"],
1644         jilu["extra"],
1645         event.get("target"),
1646     )
1647     if not ysjc(zt, next["points"]) or not zx(zt, next):
1648         return False
1649     return slcl(zt)
1650
1651
1652 def zx(zt, event):
1653     jilu = zt["ledger"]
1654     if event["kind"] == "optical_chain":
1655         if jilu["extra"] < len(event["points"]) or not ysjc(zt, event["points"]):
1656             return False
1657         bcsj(
1658             zt,
1659             "optical_chain_selected",
1660             channel=event["channel"],
1661             points=[tuple((float(x) for x in q)) for q in event["points"]],
1662             upper_s=event["certificate_upper_s"],
1663             mean_s=event["estimate"],
1664             scope="hypothesis-weighted ranking; complete Voronoi disk cover certifies success",
1665         )
1666         for q in event["points"]:
1667             if event["channel"] not in pindao(jilu, ("found",)):
1668                 return True
1669             zhixing(zt, "/clear", q, event["channel"])
1670         if event["channel"] in pindao(jilu, ("found",)):
1671             raise RuntimeError("Certified optical chain exhausted")
1672         return True
1673     bcsj(
1674         zt,
1675         "event_selected",
1676         kind=event["kind"],
1677         channel=event["channel"],
1678         points=[tuple((float(x) for x in q)) for q in event["points"]],
1679         method=event["method"],
1680         candidate_estimate_s=event["estimate"],
1681         estimate_scope="candidate ranking; not a global optimality bound",
1682     )
1683     if event["kind"] == "search":
1684         q = event["points"][0]
1685         for c in sorted(
1686             pindao(jilu, ("unknown",)), key=lambda c: (c != zt["channel"], c)
1687         ):
1688             if c not in pindao(jilu, ("unknown",)) or q in jilu["channels"][c]["radio"]:
1689                 continue
1690             if jilu["extra"] < 1 or not ysjc(zt, (q,)):
1691                 return False
1692             zhixing(zt, "/measure", q, c)
1693         return True
1694     if event["kind"] == "clear":
1695         zhixing(zt, "/clear", event["points"][0], event["channel"])
1696         return True
1697     yuan = jilu["channels"][event["channel"]]["source"]
1698     pos = yuan["anchor_point"]
1699     a, b = event["points"]
1700     first = zhixing(zt, "/measure", a, event["channel"])
1701     if first["measure_result"] == "no_signal" and zt["problem"] == 4:
1702         second = zhixing(zt, "/measure", b, event["channel"])
1703         if second["measure_result"] == "no_signal":
1704             before = syrj(jilu)
1705             changed = syx(yuan, pos, a, b)
1706             bcsj(
1707                 zt,

```

```

1708         "verified_negative_pair",
1709         channel=event["channel"],
1710         changed=changed,
1711         positive=tuple((float(x) for x in pos)),
1712         a=tuple((float(x) for x in a)),
1713         b=tuple((float(x) for x in b)),
1714         rank_before=before,
1715         rank_after=syrw(jilu),
1716     )
1717     return True
1718
1719
1720 def dzqcy(zt, c):
1721     jilu = zt["ledger"]
1722     yuan = jilu["channels"][c]["source"]
1723     order = [(i, yuan["points"][i]) for i in sorted(yuan["cells"])]
1724     for i, q in order:
1725         if c not in pindao(jilu, ("found",)):
1726             return
1727         if i in yuan["cells"]:
1728             zhixing(zt, "/clear", q, c, prove=False)
1729     if c in pindao(jilu, ("found",)):
1730         raise RuntimeError("Constructive optical cover exhausted without success")
1731
1732
1733 def byqc(zt):
1734     jilu = zt["ledger"]
1735     zt["fallback_used"] = True
1736     upper = sysj(jilu, zt["position"])
1737     if zt["virtual"] + upper >= zt["max_virtual"] - 1:
1738         raise RuntimeError("No certified completion fits the remaining virtual budget")
1739     bcsj(
1740         zt,
1741         "fallback_committed",
1742         remaining_upper_s=upper,
1743         requests_upper=syys(zt),
1744         real_time_condition=f"requires sufficient actual latency; assumed
        ↳ {zt['request_seconds']:g} s/request",
1745     )
1746     origin = zt["position"]
1747     known = sorted(
1748         pindao(jilu, ("found",)),
1749         key=lambda c: lxsj(gxpx(jilu["channels"][c]["source"], origin), origin, origin),
1750     )
1751     for c in known:
1752         dzqcy(zt, c)
1753     for q in g.sswd():
1754         for c in sorted(
1755             pindao(jilu, ("unknown",)), key=lambda c: (c != zt["channel"], c)
1756         ):
1757             if (
1758                 c not in pindao(jilu, ("unknown",))
1759                 or q not in jilu["channels"][c]["pending"]
1760             ):
1761                 continue
1762             zhixing(zt, "/measure", q, c, prove=False)
1763             if c in pindao(jilu, ("found",)):
1764                 dzqcy(zt, c)
1765             if qbqc(jilu):
1766                 break
1767     for c in sorted(pindao(jilu, ("unknown",))):
1768         zmwj(jilu, c)
1769     if not qbqc(jilu):
1770         raise RuntimeError(
1771             "Full fallback exhausted without a valid 10-16 source completion"
1772         )
1773
1774

```

```

1775 def yunxing(zt):
1776     jiekou = zt["io"]
1777     jilu = zt["ledger"]
1778     zt["started"] = time.monotonic()
1779     huifu = jiekou["call"]("/enter")
1780     jyh("enter", huifu)
1781     zt["entered"] = True
1782     zt["virtual"] = huifu["virtual_time_s"]
1783     zt["max_virtual"] = huifu["max_virtual_duration_s"]
1784     zt["deadline"] = time.monotonic() + huifu["remaining_real_duration_s"]
1785     jiekou["deadline"] = zt["deadline"]
1786     zt["initial_upper"] = sysj(jilu, zt["position"])
1787     zt["reason"] = "running"
1788     bcsj(
1789         zt,
1790         "v8_started",
1791         problem=zt["problem"],
1792         policy_revision=banben,
1793         extra_actions=jilu["extra"],
1794         coverage_nodes=jilu["coverage_nodes"],
1795         coverage_depth=jilu["coverage_depth"],
1796         geometry_version=jilu["geometry_version"],
1797         max_virtual_s=zt["max_virtual"],
1798     )
1799     while not qbbc(jilu):
1800         if zt["fallback_only"] or jilu["extra"] < 2:
1801             byqc(zt)
1802             break
1803         if time.monotonic() + syscs(zt) * zt["request_seconds"] + 30 >= zt["deadline"]:
1804             byqc(zt)
1805             break
1806         sslx = gxlx(zt["search_plan"], jilu, zt["position"])
1807         event = xzdz(zt["planner"], jilu, zt["position"], zt["channel"], sslx)
1808         if event is None or not ysjc(zt, event["points"]):
1809             byqc(zt)
1810             break
1811         if not zxsj(zt, event):
1812             byqc(zt)
1813             break
1814     zt["certificate"] = qbbc(jilu)
1815     zt["reason"] = "certified_all_cleared" if zt["certificate"] else "incomplete"
1816     bcsj(zt, "completion_checked", certified=zt["certificate"])
1817     tuichu(zt)
1818     return huizong(zt)
1819
1820
1821 def tuichu(zt):
1822     jiekou = zt["io"]
1823     if not zt["entered"] or zt["exit_attempted"]:
1824         return
1825     if jiekou["pending"] is not None:
1826         raise RuntimeError("Cannot exit while earlier request is unresolved")
1827     zt["exit_attempted"] = True
1828     huifu = jiekou["call"]("/exit")
1829     jyh("/exit", huifu)
1830     if abs(huifu["virtual_time_s"] - zt["virtual"]) > 5e-05:
1831         raise RuntimeError("Exit unexpectedly changed virtual time")
1832     zt["exit_confirmed"] = True
1833
1834
1835 def huizong(zt):
1836     jilu = zt["ledger"]
1837     return dict(
1838         strategy=celue,
1839         policy_revision=banben,
1840         problem=zt["problem"],
1841         reason=zt["reason"],
1842         completion_certified=zt["certificate"],

```

```

1843         exit_confirmed=zt["exit_confirmed"],
1844         virtual_time_s=zt["virtual"],
1845         moving_distance_m=zt["travel"],
1846         average_clear_time_s=zt["virtual"] / len(pindao(jilu, ("cleared",)))
1847         if pindao(jilu, ("cleared",))
1848         else None,
1849         measures=zt["measures"],
1850         clear_attempts=zt["clear_attempts"],
1851         switches=zt["switches"],
1852         initial_fallback_upper_s=zt["initial_upper"],
1853         fallback_used=zt["fallback_used"],
1854         budget_guard_checks=zt["guard_checks"],
1855         planner=zt["planner"]["statistics"],
1856         search_plan=dict(
1857             proofs=zt["search_plan"]["proofs"],
1858             deletions=zt["search_plan"]["deletions"],
1859             replacements=zt["search_plan"]["replacements"],
1860         ),
1861         real_time_guarantee="conditional on execution and request latency; not unconditional",
1862         **jlzy(jilu),
1863     )

```

tools/benchmark.py

文件行数: 352。

```

1  """Portable offline comparisons. Does not connect to the official simulator."""
2
3  import argparse
4  from collections import Counter
5  from dataclasses import asdict
6  import hashlib
7  import json
8  import math
9  import random
10 from pathlib import Path
11 import signal
12 import sys
13 import time
14 import traceback
15
16 ROOT = Path(__file__).resolve().parents[1]
17 sys.path.insert(0, str(ROOT / "src"))
18 from simulator import SyntheticSimulator, Source
19 from jammer_solver import solver as v8
20 from verification import replay, calibrate
21
22
23 def offline_io(send):
24     io = dict(
25         pending=None,
26         deadline=math.inf,
27         serial=0,
28         events=[],
29         counts=Counter(),
30         wall_deadline=math.inf,
31     )
32
33 def call(path, point=None, channel=None):
34     if time.monotonic() >= io["wall_deadline"]:
35         raise TimeoutError("Offline wall time limit exceeded")
36     io["serial"] += 1
37     request = dict(
38         arena_id="default",
39         robot_id="offline-v8",
40         request_id=f"offline-{io['serial']}",
41     )
42     if point is not None:

```

```

43         request.update(
44             position=dict(x=float(point[0]), y=float(point[1])), channel=channel
45         )
46     status, reply = send(path, json.dumps(request).encode())
47     io["events"].append(
48         dict(
49             event="offline_response",
50             path=path,
51             request=request,
52             reply=reply,
53             status=status,
54         )
55     )
56     if status != 200:
57         raise RuntimeError(f"Offline HTTP status {status}")
58     return reply
59
60 def record(event):
61     if time.monotonic() >= io["wall_deadline"]:
62         raise TimeoutError("Offline wall time limit exceeded")
63     io["counts"][event["event"]] += 1
64     io["events"].append(event)
65
66     io.update(call=call, record=record)
67     return io
68
69
70 class ExtremeSimulator(SyntheticSimulator):
71     def _detect(self, source, pos):
72         reply = super()._detect(source, pos)
73         if reply["measure_result"] == "direction":
74             angle = math.degrees(
75                 math.atan2(source.position[1] - pos[1], source.position[0] - pos[0])
76             )
77             token = f"{source.channel}:{pos[0]!r}:{pos[1]!r}".encode()
78             error = 1 if hashlib.sha256(token).digest()[0] % 2 else -1
79             reply["svd_deg"] = round((angle + error) % 360, 2) % 360
80         return reply
81
82
83 def case_from_record(record):
84     return dict(
85         name=record["name"],
86         problem=record["problem"],
87         seed=record["seed"],
88         extreme=record["extreme"],
89         sources=[
90             Source(s["channel"], tuple(s["position"]), s["radius"], s["direction"])
91             for s in record["source_truth"]
92         ],
93     )
94
95
96 def cases(suite):
97     if suite == "matched":
98         saved = json.loads((ROOT / "validation" / "expected-fast.json").read_text())
99         return [case_from_record(c) for c in saved["cases"] if c["suite"] == "matched"]
100     if suite == "stress":
101         result = []
102         for problem in (3, 4):
103             for n in (10, 16):
104                 for geometry in ("boundary", "cluster"):
105                     sources = []
106                     for i in range(n):
107                         angle = 360 * i / n + 0.013
108                         r = 1799.99 if geometry == "boundary" else 8 + i * 1.3
109                         p = (
110                             r * math.cos(math.radians(angle)),

```

```

111         r * math.sin(math.radians(angle)),
112     )
113     sources.append(
114         Source(i + 1, p, 1000.0, angle if problem == 4 else None)
115     )
116     result.append(
117         dict(
118             name=f"q{problem}-{geometry}-n{n}",
119             problem=problem,
120             seed=n,
121             sources=sources,
122             extreme=True,
123         )
124     )
125     return result
126 if suite in ("validation", "fresh"):
127     result = []
128     for problem in (3, 4):
129         for n in (10, 12, 14, 16):
130             seed = (
131                 (20261000 if suite == "validation" else 20261100) + problem * 20 + n
132             )
133             rng = random.Random(seed)
134             sources = []
135             for i, c in enumerate(rng.sample(range(1, 21), n)):
136                 r, theta = (
137                     1800 * math.sqrt(rng.random()),
138                     2 * math.pi * rng.random(),
139                 )
140                 radius = rng.uniform(1000, 1500)
141                 direction = rng.uniform(0, 360) if problem == 4 else None
142                 if suite == "fresh" and problem == 4 and i % 3 == 0:
143                     direction = None
144                 sources.append(
145                     Source(
146                         c,
147                         (r * math.cos(theta), r * math.sin(theta)),
148                         radius,
149                         direction,
150                     )
151                 )
152             result.append(
153                 dict(
154                     name=f"q{problem}-{suite}-n{n}",
155                     problem=problem,
156                     seed=seed,
157                     sources=sources,
158                     extreme=False,
159                 )
160             )
161     return result
162 if suite == "fallback":
163     return [
164         dict(
165             name=f"q{p}-fallback-n{n}",
166             problem=p,
167             seed=n,
168             sources=[
169                 Source(
170                     i + 1,
171                     ((i - 5) * 35.0, (i % 3 - 1) * 25.0),
172                     1000.0,
173                     (i * 37) % 360 if p == 4 else None,
174                 )
175                 for i in range(n)
176             ],
177             extreme=True,
178         )

```

```

179         for p, n in ((3, 10), (4, 16))
180     ]
181     raise ValueError(suite)
182
183
184 def run_case(
185     case,
186     algorithm="v8",
187     planning_seconds=0.20,
188     extra_actions=640,
189     fallback_only=False,
190     wall_limit=90,
191     output=None,
192 ):
193     cls = ExtremeSimulator if case.get("extreme") else SyntheticSimulator
194     world = cls(case["sources"], error_seed=case["seed"])
195     io = offline_io(world.transport)
196     if algorithm != "v8":
197         raise ValueError("Only the selected v8 fast policy is included")
198     solver = v8.xjzt(
199         io,
200         case["problem"],
201         ghsj=planning_seconds,
202         dzys=extra_actions,
203         byonly=fallback_only,
204     )
205     result = dict(
206         name=case["name"],
207         algorithm=algorithm,
208         problem=case["problem"],
209         seed=case["seed"],
210         source_count=len(case["sources"]),
211         source_truth=[asdict(s) for s in case["sources"]],
212         noise="fixed extreme +/-1 degree"
213         if case.get("extreme")
214         else "repository fixed coordinate hash",
215         kind="synthetic only; not official simulator evidence",
216         planning_seconds=planning_seconds,
217         extra_actions=extra_actions,
218         fallback_only=fallback_only,
219         wall_limit_s=wall_limit,
220     )
221     if algorithm == "v8":
222         policy_root = Path(v8.__file__).resolve().parent
223         result["policy_revision"] = v8.banben
224         result["policy_source_sha256"] = {
225             path.name: hashlib.sha256(path.read_bytes()).hexdigest()
226             for path in sorted(policy_root.glob("*.py"))
227         }
228     if "saved_v7_s" in case:
229         result["saved_v7_s"] = case["saved_v7_s"]
230
231     def expired(*_):
232         raise TimeoutError("Offline wall time limit exceeded")
233
234     # Windows has no SIGALRM: check cooperatively at action/event boundaries.
235     # POSIX retains the hard alarm, including during long geometry operations.
236     has_alarm = hasattr(signal, "SIGALRM")
237     previous = signal.signal(signal.SIGALRM, expired) if has_alarm else None
238     if has_alarm:
239         signal.alarm(wall_limit)
240     io["wall_deadline"] = time.monotonic() + wall_limit if wall_limit > 0 else math.inf
241     started = time.perf_counter()
242     try:
243         result["summary"] = v8.yunxing(solver)
244         if algorithm == "v8":
245             audit = replay(io["events"])
246             result["replay"] = {

```

```

247         k: audit[k] for k in ("verified", "actions", "virtual_time_s")
248     }
249     receipts = [e for e in io["events"] if e.get("event") == "rank_receipt"]
250     if any(e["after"] >= e["before"] for e in receipts):
251         raise AssertionError("Completion rank did not strictly decrease")
252     complete = solver["certificate"]
253     exited = solver["exit_confirmed"]
254     virtual = solver["virtual"]
255     if len(world.cleared) != len(case["sources"]) or not complete or not exited:
256         raise AssertionError(
257             "Solver failed actual full clearance or declared completion"
258         )
259     if abs(world.virtual_time - virtual) > 5e-5:
260         raise AssertionError("Simulator and solver virtual times differ")
261     result["error"] = None
262 except Exception as exc:
263     result["error"] = f"{type(exc).__name__}: {exc}"
264     result["traceback"] = traceback.format_exc()
265     result["summary"] = v8.huizong(solver)
266 finally:
267     if has_alarm:
268         signal.alarm(0)
269         signal.signal(signal.SIGALRM, previous)
270 result.update(
271     wall_s=time.perf_counter() - started,
272     cleared_count=len(world.cleared),
273     virtual_time_s=world.virtual_time,
274     average_clear_time_s=world.virtual_time / len(case["sources"]),
275     counts=world.counts,
276     event_counts=dict(io["counts"]),
277 )
278 if output is not None:
279     output.mkdir(parents=True, exist_ok=True)
280     name = f"{algorithm}-{case['name']}"
281     raw = "".join(json.dumps(e, ensure_ascii=False) + "\n" for e in io["events"])
282     (output / f"{name}.jsonl").write_text(raw, encoding="utf-8")
283     result["log_sha256"] = hashlib.sha256(raw.encode()).hexdigest()
284     (output / f"{name}.json").write_text(
285         json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
286     )
287 return result, io["events"]
288
289
290 def main():
291     parser = argparse.ArgumentParser(description=__doc__)
292     parser.add_argument(
293         "--suite",
294         choices=("matched", "stress", "fallback", "calibration", "validation", "fresh"),
295         default="matched",
296     )
297     parser.add_argument("--algorithm", choices=("v8",), default="v8")
298     parser.add_argument("--planning-seconds", type=float, default=0.20)
299     parser.add_argument("--extra-actions", type=int, default=640)
300     parser.add_argument("--wall-limit", type=int, default=90)
301     parser.add_argument("--output", type=Path, default=ROOT / "runs" / "offline")
302     args = parser.parse_args()
303     args.output.mkdir(parents=True, exist_ok=True)
304     if args.suite == "calibration":
305         result = [
306             calibrate([(0,), (60,), (120,)], (0, 60, 120)),
307             calibrate([(0, 120), (120, 0)], (0, 60, 120)),
308             calibrate([(0, 180), (60, 120), (180, 0)], (0, 60, 120, 180)),
309         ]
310         (args.output / "calibration.json").write_text(
311             json.dumps(result, indent=2) + "\n"
312         )
313     print(json.dumps(result, indent=2))
314     return 0

```

```

315     results = []
316     for case in cases(args.suite):
317         result, _ = run_case(
318             case,
319             args.algorithm,
320             args.planning_seconds,
321             args.extra_actions,
322             fallback_only=args.suite == "fallback",
323             wall_limit=args.wall_limit,
324             output=args.output,
325         )
326     results.append(result)
327     print(
328         json.dumps(
329             {
330                 k: result[k]
331                 for k in (
332                     "name",
333                     "algorithm",
334                     "source_count",
335                     "cleared_count",
336                     "virtual_time_s",
337                     "wall_s",
338                     "error",
339                 )
340             },
341             ensure_ascii=False,
342         ),
343         flush=True,
344     )
345     (args.output / f"{args.algorithm}-{args.suite}.json").write_text(
346         json.dumps(results, ensure_ascii=False, indent=2) + "\n"
347     )
348     return int(any(r["error"] for r in results))
349
350
351 if __name__ == "__main__":
352     raise SystemExit(main())

```

tools/compare_baseline.py

文件行数：83。

```

1  """Reproduce the historical v7 comparison using synthetic inputs only."""
2
3  from pathlib import Path
4  from types import SimpleNamespace
5  import argparse
6  import hashlib
7  import json
8  import math
9  import sys
10 import time
11
12 ROOT = Path(__file__).resolve().parents[1]
13 sys.path[:0] = [str(ROOT / "src"), str(ROOT / "validation/baseline_v7")]
14 from benchmark import case_from_record, offline_io
15 from simulator import SyntheticSimulator
16 from adaptive_v7 import SolverV7
17
18
19 def main():
20     parser = argparse.ArgumentParser(description=__doc__)
21     parser.add_argument("--case", help="Run one named case; default: all 20 cases")
22     parser.add_argument("--output", type=Path, default=ROOT / "runs/baseline_check")
23     args = parser.parse_args()
24     args.output.mkdir(parents=True, exist_ok=True)
25     reference = json.loads(

```

```

26         (ROOT / "validation/performance-comparison.json").read_text(encoding="utf-8")
27     )
28     inputs = {
29         r["name"]: r
30         for r in json.loads(
31             (ROOT / "validation/expected-fast.json").read_text(encoding="utf-8")
32         )["cases"]
33     }
34     rows = [r for r in reference["cases"] if not args.case or r["name"] == args.case]
35     if not rows:
36         parser.error("Unknown case")
37     checked = []
38     for row in rows:
39         case = case_from_record(inputs[row["name"]])
40         world = SyntheticSimulator(case["sources"], error_seed=case["seed"])
41         io = offline_io(world.transport)
42         io["wall_deadline"] = time.monotonic() + 180
43         adapter = SimpleNamespace(
44             call=io["call"], record=io["record"], pending=None, deadline=math.inf
45         )
46         start = time.monotonic()
47         solver = SolverV7(adapter, case["problem"])
48         solver.run()
49         total = world.virtual_time
50         matches = abs(total - row["v7_s"]) < 1e-6 and len(world.cleared) == len(
51             case["sources"]
52         )
53         result = dict(
54             name=row["name"],
55             source_count=len(case["sources"]),
56             cleared_count=len(world.cleared),
57             virtual_time_s=total,
58             expected_time_s=row["v7_s"],
59             matches=matches,
60             wall_s=time.monotonic() - start,
61             scope="offline synthetic comparison only",
62         )
63         logfile = args.output / ("v7-" + row["name"] + ".jsonl")
64         logfile.write_text(
65             "".join(json.dumps(e, ensure_ascii=False) + "\n" for e in io["events"]),
66             encoding="utf-8",
67         )
68         result["log_sha256"] = hashlib.sha256(logfile.read_bytes()).hexdigest()
69         logfile.with_suffix(".json").write_text(
70             json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
71         )
72         checked.append(result)
73         print(json.dumps(result), flush=True)
74         if not matches:
75             raise RuntimeError("Comparison differs from the frozen result")
76         (args.output / "verification.json").write_text(
77             json.dumps(dict(cases=checked, verified=len(checked)), indent=2) + "\n",
78             encoding="utf-8",
79         )
80
81
82 if __name__ == "__main__":
83     main()

```

tools/lower_bounds.py

文件行数: 38。

```

1 """Certified necessary operation cost to rule out each truly absent channel."""
2
3 from fractions import Fraction as F
4 from pathlib import Path
5 import sys

```

```

6
7 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
8 from jammer_solver.exact_geometry import trig
9
10
11 def per_channel(problem):
12     # Outward angle bounds, checked by the rational Taylor kernel. These
13     # angular measures may overlap; their SUM must nevertheless reach 360.
14     beta = F("1.274")
15     assert trig(beta / 2)[2] >= F(1, 90)
16     if problem == 3:
17         alpha = F("67.499")
18         assert trig(alpha / 2)[2] >= F(5, 9)
19     elif problem == 4:
20         alpha = F("58.110")
21         cl, ch, sl, sh = trig(alpha / 2)
22         assert sl >= F(5, 9) * ch and cl > 0
23     else:
24         raise ValueError("Expected problem 3 or 4")
25     for cost in range(100):
26         if any(
27             m * alpha + k * beta >= 360
28             for m in range(cost // 5 + 1)
29             for k in range((cost - 5 * m) // 3 + 1)
30         ):
31             return cost
32     raise ArithmeticError("Angular lower bound enumeration failed")
33
34
35 def absence_lower(problem, count):
36     if not 10 <= count <= 16:
37         raise ValueError("Expected 10-16 sources")
38     return 0 if count == 16 else (20 - count) * per_channel(problem)

```

tools/simulator.py

文件行数: 142。

```

1 """Synthetic test world, NOT the official simulator or official test results.
2
3 Source placement, orientations and coordinate-hashed noise are our experimental
4 assumptions. The strategy only receives protocol responses, never world objects.
5 """
6
7 from dataclasses import dataclass
8 import hashlib
9 import json
10 import math
11 import random
12 import time
13
14
15 @dataclass(frozen=True)
16 class Source:
17     channel: int
18     position: tuple
19     radius: float
20     direction: float | None = None
21
22
23 class SyntheticSimulator:
24     def __init__(self, sources, error_seed=0):
25         self.sources = {s.channel: s for s in sources}
26         if len(self.sources) != len(sources):
27             raise ValueError("Channels must be unique")
28         self.error_seed = error_seed
29         self.position, self.channel = (0.0, 0.0), 1
30         self.virtual_time = 0.0

```

```

31     self.active = False
32     self.cleared = set()
33     self.cache = {}
34     self.counts = dict(measure=0, clear=0, successful_clear=0, switch=0)
35     self.distance = 0.0
36
37     def transport(self, path, raw):
38         request = json.loads(raw)
39         key = request["request_id"]
40         if key in self.cache:
41             previous_path, previous_raw, previous_reply = self.cache[key]
42             if path == previous_path and raw == previous_raw:
43                 return previous_reply
44             return 409, dict(accepted=False, real_timestamp_ms=0, virtual_time_s=0)
45         if path == "/enter":
46             if self.active:
47                 return 200, dict(accepted=False, real_timestamp_ms=0, virtual_time_s=0)
48             self.active = True
49             fields = dict(
50                 remaining_real_duration_s=1200,
51                 max_virtual_duration_s=360000,
52                 max_real_duration_s=1200,
53             )
54             elif not self.active:
55                 return 200, dict(accepted=False, real_timestamp_ms=0, virtual_time_s=0)
56             elif path == "/exit":
57                 self.active = False
58                 fields = dict(exit_reason="user_exit")
59             elif path in {"/measure", "/clear"}:
60                 pos = (request["position"]["x"], request["position"]["y"])
61                 channel = request["channel"]
62                 distance = math.dist(self.position, pos)
63                 self.distance += distance
64                 self.virtual_time += distance / 5
65                 self.position = pos
66                 source = self.sources.get(channel) if channel not in self.cleared else None
67                 if path == "/measure":
68                     self.counts["measure"] += 1
69                     self.counts["switch"] += channel != self.channel
70                     self.virtual_time += 5 + (channel != self.channel)
71                     self.channel = channel
72                     fields = self._detect(source, pos)
73                 else:
74                     self.counts["clear"] += 1
75                     success = source is not None and math.dist(pos, source.position) <= 20
76                     if success:
77                         self.cleared.add(channel)
78                         self.counts["successful_clear"] += 1
79                         self.virtual_time += 5 if success else 3
80                         fields = dict(
81                             clear_result="success" if success else "no_target_in_range"
82                         )
83                 else:
84                     return 404, dict(accepted=False, real_timestamp_ms=0, virtual_time_s=0)
85             response = (
86                 200,
87                 dict(
88                     accepted=True,
89                     real_timestamp_ms=int(time.time() * 1000),
90                     virtual_time_s=round(self.virtual_time, 6),
91                     **fields,
92                 ),
93             )
94             self.cache[key] = (path, raw, response)
95             return response
96
97     def _detect(self, source, pos):
98         if source is None:

```

```

99         return dict(measure_result="no_signal")
100     distance = math.dist(pos, source.position)
101     covered = distance <= source.radius
102     if source.direction is not None:
103         angle = math.radians(source.direction)
104         outward = (pos[0] - source.position[0]) * math.cos(angle) + (
105             pos[1] - source.position[1]
106         ) * math.sin(angle)
107         covered = covered and outward >= -1e-9
108     if not covered:
109         return dict(measure_result="no_signal")
110     if distance <= 5:
111         return dict(measure_result="near")
112     true_angle = math.degrees(
113         math.atan2(source.position[1] - pos[1], source.position[0] - pos[0])
114     )
115     # Fixed at a given coordinate and channel, no resampling at the same point.
116     token = f"{self.error_seed}:{source.channel}:{pos[0]:.8f}:{pos[1]:.8f}".encode()
117     error = (
118         int.from_bytes(hashlib.sha256(token).digest()[:8], "big") / (2**64 - 1) * 2
119         - 1
120     )
121     return dict(
122         measure_result="direction",
123         svd_deg=round((true_angle + error) % 360, 2) % 360,
124     )
125
126
127 def random_case(seed, mixed=False):
128     rng = random.Random(seed)
129     count = rng.randint(10, 16)
130     sources = []
131     for channel in rng.sample(range(1, 21), count):
132         radius, angle = 1800 * math.sqrt(rng.random()), rng.random() * 2 * math.pi
133         direction = rng.random() * 360 if mixed and rng.random() < 0.6 else None
134         sources.append(
135             Source(
136                 channel,
137                 (radius * math.cos(angle), radius * math.sin(angle)),
138                 rng.uniform(1000, 1500),
139                 direction,
140             )
141         )
142     return sources

```

tools/validate.py

文件行数: 151。

```

1  """Run or replay frozen v8 fast cases and check that layout changes preserve requests."""
2
3  import argparse
4  from datetime import datetime
5  import hashlib
6  import json
7  from pathlib import Path
8
9  from benchmark import ROOT, case_from_record, run_case
10 from jammer_solver import solver
11 from verification import replay
12 from verify_optical import audit_optical_chains
13
14
15 def digest(path):
16     return hashlib.sha256(path.read_bytes()).hexdigest()
17
18
19 def verify(record, result, events, log, policy_hashes):

```

```

20     if result["error"] or result["cleared_count"] != record["source_count"]:
21         raise ValueError(f"Incomplete case: {record['name']}")
22     if result["source_truth"] != record["source_truth"]:
23         raise ValueError(f"Source inputs differ: {record['name']}")
24     if (
25         result["policy_source_sha256"] != policy_hashes
26         or digest(log) != result["log_sha256"]
27     ):
28         raise ValueError(f"Policy or log hash differs: {record['name']}")
29     requests = [
30         (e["path"], e["request"])
31         for e in events
32         if e.get("event") == "offline_response"
33     ]
34     request_hash = hashlib.sha256(
35         json.dumps(requests, sort_keys=True, separators=(",", ":")).encode()
36     ).hexdigest()
37     if request_hash != record["request_sha256"]:
38         raise ValueError(
39             f"Requests changed from the selected v8 fast version: {record['name']}"
40         )
41     if (
42         abs(result["virtual_time_s"] - record["virtual_time_s"]) > 1e-6
43         or result["counts"] != record["counts"]
44     ):
45         raise ValueError(
46             f"Costs changed from the selected v8 fast version: {record['name']}"
47         )
48     replayed = replay(events)
49     optical = audit_optical_chains(events)
50     return dict(
51         name=record["name"],
52         suite=record["suite"],
53         source_count=result["source_count"],
54         cleared_count=result["cleared_count"],
55         virtual_time_s=result["virtual_time_s"],
56         identical_requests=True,
57         request_sha256=request_hash,
58         log_sha256=result["log_sha256"],
59         replay=replayed,
60         optical_audit=optical,
61     )
62
63
64 def main():
65     parser = argparse.ArgumentParser(description=__doc__)
66     parser.add_argument(
67         "--suite",
68         default="all",
69         choices=("all", "matched", "validation", "fresh", "stress", "fallback"),
70     )
71     parser.add_argument("--output", type=Path)
72     parser.add_argument("--verify-only", action="store_true")
73     parser.add_argument("--wall-limit", type=int, default=180)
74     args = parser.parse_args()
75     if args.output is None:
76         args.output = (
77             ROOT / "validation" / "current"
78             if args.verify_only
79             else ROOT / "runs" / f"validation-{datetime.now():%Y%m%d-%H%M%S-%f}"
80         )
81     args.output.mkdir(parents=True, exist_ok=True)
82     frozen = json.loads(
83         (ROOT / "validation/expected-fast.json").read_text(encoding="utf-8")
84     )
85     records = [
86         r for r in frozen["cases"] if args.suite == "all" or r["suite"] == args.suite
87     ]

```

```

88     if not records:
89         raise ValueError("No frozen cases selected")
90     policy_hashes = {
91         p.name: digest(p) for p in sorted(Path(solver.__file__).parent.glob("*.py"))
92     }
93     checked = []
94     for record in records:
95         path = args.output / f"v8-{record['name']}.json"
96         if args.verify_only:
97             result = json.loads(path.read_text(encoding="utf-8"))
98             events = [
99                 json.loads(line)
100                 for line in path.with_suffix(".jsonl")
101                 .read_text(encoding="utf-8")
102                 .splitlines()
103             ]
104         else:
105             result, events = run_case(
106                 case_from_record(record),
107                 planning_seconds=record["planning_seconds"],
108                 extra_actions=record["extra_actions"],
109                 fallback_only=record["fallback_only"],
110                 wall_limit=args.wall_limit,
111                 output=args.output,
112             )
113             # JSON represents positions as arrays on both sides of the comparison.
114             result = json.loads(path.read_text(encoding="utf-8"))
115         checked.append(
116             verify(record, result, events, path.with_suffix(".jsonl"), policy_hashes)
117         )
118     print(
119         json.dumps(
120             {
121                 k: checked[-1][k]
122                 for k in (
123                     "name",
124                     "cleared_count",
125                     "virtual_time_s",
126                     "identical_requests",
127                 )
128             },
129         ),
130         flush=True,
131     )
132     report = dict(
133         scope="Offline synthetic only; no official simulator connected",
134         policy_revision=solver.banben,
135         policy_source_sha256=policy_hashes,
136         expected_cases_sha256=digest(ROOT / "validation/expected-fast.json"),
137         cases=checked,
138         verified_cases=len(checked),
139         optical_chains=sum(len(r["optical_audit"]["chains"]) for r in checked),
140         identical_requests_to_selected_fast=all(
141             r["identical_requests"] for r in checked
142         ),
143     )
144     (args.output / "verification.json").write_text(
145         json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
146     )
147     print(f"Verified {len(checked)} cases, {report['optical_chains']} optical chains.")
148
149
150 if __name__ == "__main__":
151     main()

```

tools/verification.py

文件行数: 277。

```

1 import argparse
2 import json
3 from pathlib import Path
4 import sys
5
6 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
7 from jammer_solver.protocol import jyhf
8 from jammer_solver import exact_geometry as g
9 from jammer_solver.solver import xj1, pindao, gx1, syrw, syx, zmw, qbqc, jlzy
10 from collections import deque
11 from fractions import Fraction as F
12 import math
13
14
15 def build_graph(worlds, positions, radius=1000, clear_radius=20):
16     worlds = tuple((tuple((F(x) for x in w)) for w in worlds))
17     positions = tuple((F(x) for x in positions))
18     if not worlds or not worlds[0] or any((len(w) != len(worlds[0]) for w in worlds)):
19         raise ValueError("Finite calibration requires a fixed, positive source count")
20     if not positions or len(set(positions)) != len(positions):
21         raise ValueError("Positions must be nonempty and distinct")
22     if any(
23         (
24             not any((abs(x - p) <= clear_radius for p in positions))
25             for w in worlds
26             for x in w
27         )
28     ):
29         raise ValueError("Finite action family cannot clear every candidate source")
30     n = len(worlds[0])
31     full = (1 << n) - 1
32     root = (tuple(range(len(worlds))), 0, -1, 0)
33     queue = deque([root])
34     graph = {}
35     terminals = set()
36     while queue:
37         state = queue.popleft()
38         if state in graph:
39             continue
40         belief, cleared, at, tuned = state
41         graph[state] = {}
42         if cleared == full:
43             terminals.add(state)
44             continue
45         dangqian = F(0) if at < 0 else positions[at]
46         for c in range(n):
47             if cleared >> c & 1:
48                 continue
49             for j, q in enumerate(positions):
50                 for kind in ("measure", "clear"):
51                     groups = {}
52                     for w in belief:
53                         x = worlds[w][c]
54                         d = abs(q - x)
55                         if kind == "clear":
56                             observation = "success" if d <= clear_radius else "miss"
57                         elif d > radius:
58                             observation = "no_signal"
59                         elif d <= 5:
60                             observation = "near"
61                         else:
62                             observation = "right" if x > q else "left"
63                         groups.setdefault(observation, []).append(w)
64                     successors = []
65                     for observation, ids in sorted(groups.items()):
66                         new_cleared = (
67                             cleared | 1 << c if observation == "success" else cleared
68                         )

```

```

69         nxt = (
70             tuple(ids),
71             new_cleared,
72             j,
73             c if kind == "measure" else tuned,
74         )
75         fee = (
76             5 + (c != tuned)
77             if kind == "measure"
78             else 5
79             if observation == "success"
80             else 3
81         )
82         cost = abs(dangqian - q) / 5 + fee
83         successors.append((observation, cost, nxt))
84         if nxt not in graph:
85             queue.append(nxt)
86         graph[state][kind, c, j] = tuple(successors)
87     return (root, graph, terminals)
88
89
90 def solve(graph, terminals):
91     values = {s: F(0) if s in terminals else math.inf for s in graph}
92     policy = {}
93     for iteration in range(len(graph) + 1):
94         changed = False
95         updated = dict(values)
96         for s, actions in graph.items():
97             if s in terminals:
98                 continue
99             choices = [
100                 (max((cost + values[t] for _, cost, t in successors)), a)
101                 for a, successors in actions.items()
102             ]
103             if not choices:
104                 continue
105             value, action = min(choices)
106             if value < values[s]:
107                 changed = True
108                 updated[s] = value
109                 policy[s] = action
110         values = updated
111         if not changed:
112             return (values, policy, iteration + 1)
113     raise ArithmeticError("Positive-cost finite policy iteration did not stabilize")
114
115
116 def optical_continuation(graph, terminals, order):
117     values = {s: F(0) if s in terminals else math.inf for s in graph}
118     for action in reversed(order):
119         updated = dict(values)
120         for s, actions in graph.items():
121             if s in terminals or action not in actions:
122                 continue
123             updated[s] = max((cost + values[t] for _, cost, t in actions[action]))
124         values = updated
125     return values
126
127
128 def rollout(graph, terminals, baseline, depth):
129     values = dict(baseline)
130     policy = {}
131     for _ in range(depth):
132         updated = dict(values)
133         for s, actions in graph.items():
134             if s in terminals:
135                 continue
136             value, action = min(

```

```

137         (
138             (max((cost + values[t] for _, cost, t in nexts)), a)
139             for a, nexts in actions.items()
140         )
141     )
142     if value < updated[s]:
143         updated[s] = value
144         policy[s] = action
145     values = updated
146     return (values, policy)
147
148
149 def calibrate(worlds, positions, depths=(0, 1, 2, 3), **kwargs):
150     root, graph, terminals = build_graph(worlds, positions, **kwargs)
151     exact, policy, iterations = solve(graph, terminals)
152     n = len(worlds[0])
153     forward = [("clear", c, j) for c in range(n) for j in range(len(positions))]
154     reverse = [
155         ("clear", c, j)
156         for c in reversed(range(n))
157         for j in reversed(range(len(positions)))
158     ]
159     first, second = [
160         optical_continuation(graph, terminals, order) for order in (forward, reverse)
161     ]
162     baseline = {s: min(first[s], second[s]) for s in graph}
163     results = []
164     for depth in depths:
165         values, _ = rollout(graph, terminals, baseline, depth)
166         results.append(
167             dict(
168                 depth=depth,
169                 upper_s=float(values[root]),
170                 gap_s=float(values[root] - exact[root]),
171             )
172         )
173     return dict(
174         worlds=len(worlds),
175         sources_per_world=n,
176         states=len(graph),
177         exact_time_s=float(exact[root]),
178         exact_fraction=str(exact[root]),
179         first_action=policy[root],
180         bellman_passes=iterations,
181         rollout=results,
182         scope="exact only for these finite worlds and actions; fixed known N",
183     )
184
185
186 def replay(events):
187     jilu = None
188     weizhi = (0.0, 0.0)
189     tuned = 1
190     virtual = 0.0
191     receipt = None
192     completed = False
193     actions = 0
194     for event in events:
195         kind = event.get("event")
196         if kind == "v8_started":
197             if jilu is not None:
198                 raise RuntimeError("Duplicate start")
199             jilu = xj1l(
200                 event["problem"],
201                 event["extra_actions"],
202                 fgjd=event.get("coverage_nodes", 4096),
203                 fgjd=event.get("coverage_depth", 12),
204                 jhbb=event.get("geometry_version", 1),

```

```

205         )
206         virtual = event["virtual_time_s"]
207     elif kind == "evidence_observation":
208         if jilu is None or completed or receipt is not None:
209             raise RuntimeError("Observation outside an active run")
210         path, q, c, huifu = (
211             event["path"],
212             tuple(event["point"]),
213             event["channel"],
214             event["reply"],
215         )
216         jyhfc(path, huifu)
217         cost = (
218             5 + (c != tuned)
219             if path == "/measure"
220             else 5
221             if huifu["clear_result"] == "success"
222             else 3
223         )
224         expected = virtual + math.dist(weizhi, q) / 5 + cost
225         if abs(expected - huifu["virtual_time_s"]) > 5e-05:
226             raise RuntimeError("Replayed virtual cost mismatch")
227         virtual = huifu["virtual_time_s"]
228         weizhi = q
229         if path == "/measure":
230             tuned = c
231         receipt = gxjl(jilu, path, q, c, huifu, prove=event["prove"])
232         actions += 1
233     elif kind == "rank_receipt":
234         if receipt is None or any((event.get(k) != v for k, v in receipt.items())):
235             raise RuntimeError("Logged rank receipt disagrees with evidence")
236         receipt = None
237     elif kind == "verified_negative_pair":
238         yuan = jilu["channels"][event["channel"]]["source"]
239         before = syrw(jilu)
240         changed = syx(
241             yuan,
242             g.point(event["positive"]),
243             g.point(event["a"]),
244             g.point(event["b"]),
245         )
246         if (
247             changed != event["changed"]
248             or before != event["rank_before"]
249             or syrw(jilu) != event["rank_after"]
250         ):
251             raise RuntimeError("Replayed negative pair proof mismatch")
252     elif kind == "completion_checked":
253         for c in sorted(pindao(jilu, ("unknown",))):
254             if not jilu["channels"][c]["pending"]:
255                 zmwj(jilu, c)
256             completed = qbcq(jilu)
257             if not completed or event["certified"] is not True:
258                 raise RuntimeError("Completion claim lacks sufficient evidence")
259 if jilu is None or not completed or receipt is not None:
260     raise RuntimeError("Incomplete evidence stream")
261 return dict(verified=True, actions=actions, virtual_time_s=virtual, **jlzy(jilu))
262
263
264 def main():
265     parser = argparse.ArgumentParser(
266         description="重放 v8 日志，核查动作计费、剩余任务和全清依据"
267     )
268     parser.add_argument("log", type=Path)
269     args = parser.parse_args()
270     result = replay(
271         (json.loads(line) for line in args.log.read_text().splitlines() if line.strip())
272     )

```

```

273     print(json.dumps(result, ensure_ascii=False, indent=2))
274
275
276 if __name__ == "__main__":
277     main()

```

tools/verify_optical.py

文件行数: 126。

```

1 from jammer_solver import solver as v8
2 from jammer_solver import exact_geometry as g
3 from fractions import Fraction as F
4
5
6 def audit_optical_chains(events):
7     if not any(e.get("event") == "optical_chain_selected" for e in events):
8         return dict(verified=True, chains=[])
9     ledger = None
10    position = (0.0, 0.0)
11    virtual = 0.0
12    active = None
13    chains = []
14    for event in events:
15        kind = event.get("event")
16        if kind == "v8_started":
17            ledger = v8.xjxl(
18                event["problem"],
19                event["extra_actions"],
20                fgjd=event.get("coverage_nodes", 4096),
21                fgsd=event.get("coverage_depth", 12),
22                jhbb=event.get("geometry_version", 1),
23            )
24            virtual = event["virtual_time_s"]
25            limit = event["max_virtual_s"]
26        elif kind == "optical_chain_selected":
27            if active is not None:
28                raise ValueError("A new chain interrupted an unfinished chain")
29            points = [g.point(q) for q in event["points"]]
30            if not 2 <= len(points) <= 8 or len(set(points)) != len(points):
31                raise ValueError("Invalid optical chain length or repeated points")
32            polygon = ledger["channels"][event["channel"]]["source"]["polygon"]
33            for q in points:
34                cell = polygon
35                for other in points:
36                    if q != other:
37                        cell = g.clip(
38                            cell,
39                            ((other[0] - q[0], other[1] - q[1])),
40                            (
41                                (other[0] * other[0] + other[1] * other[1])
42                                - (q[0] * q[0] + q[1] * q[1])
43                            )
44                            / 2,
45                        )
46            if cell and not g.ypbh(cell, q, F("19.8")):
47                raise ValueError(
48                    "Optical chain leaves feasible source positions uncovered"
49                )
50            previous = g.point(position)
51            distance = F(0)
52            for q in points:
53                distance += g.jlsx(previous, q)
54            previous = q
55            found = min(16, len(v8.pindao(ledger, ("found",))) + len(points))
56            required = (
57                virtual
58                + v8.sysj(ledger, position)

```

```

59         + float((2 * found + 2) * distance / 5)
60         + 6 * len(points)
61     )
62     if ledger["extra"] < len(points) or required >= limit - 2:
63         raise ValueError(
64             "The complete optical event does not fit its reserved budget"
65         )
66     if v8.lxsj(points, position) > event["upper_s"] + 1e-6:
67         raise ValueError("The recorded optical upper bound is too small")
68     active = dict(
69         channel=event["channel"],
70         start_s=virtual,
71         points=points,
72         count=0,
73         upper_s=event["upper_s"],
74         budget_required_s=required,
75     )
76     elif kind == "evidence_observation":
77         q = tuple(event["point"])
78         if active is not None:
79             index = active["count"]
80             if (
81                 event["path"] != "/clear"
82                 or event["channel"] != active["channel"]
83                 or index >= len(active["points"])
84                 or g.point(q) != active["points"][index]
85             ):
86                 raise ValueError(
87                     "Execution departed from the certified optical chain"
88                 )
89             active["count"] += 1
90         v8.gxjl(
91             ledger,
92             event["path"],
93             q,
94             event["channel"],
95             event["reply"],
96             prove=event["prove"],
97         )
98         position = q
99         virtual = event["reply"]["virtual_time_s"]
100         if active is not None and event["reply"]["clear_result"] == "success":
101             elapsed = virtual - active["start_s"]
102             if elapsed > active["upper_s"] + 1e-5:
103                 raise ValueError(
104                     "The actual optical prefix exceeded its upper bound"
105                 )
106             chains.append(
107                 dict(
108                     channel=active["channel"],
109                     points=len(active["points"]),
110                     actions=active["count"],
111                     actual_s=elapsed,
112                     upper_s=active["upper_s"],
113                     budget_required_s=active["budget_required_s"],
114                 )
115             )
116             active = None
117         elif kind == "verified_negative_pair":
118             v8.syx(
119                 ledger["channels"][event["channel"]]["source"],
120                 g.point(event["positive"]),
121                 g.point(event["a"]),
122                 g.point(event["b"]),
123             )
124         if active is not None or ledger is None or not v8.qbqc(ledger):
125             raise ValueError("Optical audit ended without actual full clearance")
126         return dict(verified=True, chains=chains)

```

tests/test_entrypoint.py

文件行数: 68。

```
1 import contextlib
2 import io
3 import json
4 from pathlib import Path
5 import sys
6 import tempfile
7 import unittest
8 from unittest.mock import patch
9
10 import run_robot as cli
11 from benchmark import offline_io
12 from simulator import Source, SyntheticSimulator
13
14
15 class EntrypointTests(unittest.TestCase):
16     def test_no_connection_without_explicit_flag(self):
17         with patch.object(
18             sys, "argv", ["run_robot.py", "--problem", "4", "--case-code", "offline"]
19         ):
20             with patch.object(cli, "xjkk") as transport:
21                 self.assertEqual(cli.main(), 0)
22                 transport.assert_not_called()
23
24     def test_default_entrypoint_runs_selected_fast_and_records_completion(self):
25         world = SyntheticSimulator(
26             [Source(c, (0.0, 0.0), 1000.0) for c in range(1, 11)]
27         )
28         transport = offline_io(world.transport)
29         with tempfile.TemporaryDirectory() as folder:
30             argv = [
31                 "run_robot.py",
32                 "--problem",
33                 "3",
34                 "--case-code",
35                 "offline",
36                 "--connect",
37                 "--robot-id",
38                 "test-robot",
39                 "--output",
40                 folder,
41             ]
42             with (
43                 patch.object(sys, "argv", argv),
44                 patch.object(cli, "xjkk", return_value=transport),
45             ):
46                 with contextlib.redirect_stdout(io.StringIO()):
47                     self.assertEqual(cli.main(), 0)
48                 output = next(Path(folder).iterdir())
49                 metadata = json.loads((output / "metadata.json").read_text())
50                 summary = json.loads((output / "summary.json").read_text())
51                 self.assertEqual(metadata["policy_revision"], "20260912-shared-service-r4")
52                 self.assertIsNone(summary["error"])
53                 self.assertEqual(len(world.cleared), 10)
54                 self.assertFalse(world.active)
55
56     def test_connect_requires_explicit_identity_before_creating_transport(self):
57         argv = ["run_robot.py", "--problem", "3", "--case-code", "offline", "--connect"]
58         with (
59             patch.object(sys, "argv", argv),
60             patch.object(cli, "xjkk") as transport,
61         ):
62             with (
63                 contextlib.redirect_stderr(io.StringIO()),
64                 self.assertRaises(SystemExit) as error,
```

```

65         ):
66             cli.main()
67             self.assertEqual(error.exception.code, 2)
68             transport.assert_not_called()

```

tests/test_helpers.py

文件行数: 115。

```

1 import json
2 import math
3 from pathlib import Path
4 import tempfile
5 import unittest
6 from unittest.mock import Mock, patch
7 from urllib.error import URLError
8 from jammer_solver.questions import jhqy
9 from jammer_solver.protocol import xjjk
10 from jammer_solver.questions import ecdqy
11 from jammer_solver.solver import zjqcd
12 from jammer_solver.solver import qclxyh
13
14
15 class HelperTests(unittest.TestCase):
16     def test_second_region_strict_expansion(self):
17         result = ecdqy((0, 0), 0, (500, 840))
18         self.assertTrue(result["query_in_region"])
19         e = math.radians(1.005)
20         self.assertGreater(
21             math.dist((500, 840), (1500 * math.cos(e), -1500 * math.sin(e))), 1000
22         )
23
24     def test_route_feasible_and_nonincreasing(self):
25         p, t, start = (0, 3), (3, 0), (0, 1)
26         point, diag = qclxyh([(0, 0)], 1, start, p, t)
27         self.assertEqual(math.hypot(*point), 1 + 1e-10)
28         self.assertLess(
29             (
30                 math.hypot(p[0] - point[0], p[1] - point[1])
31                 + math.hypot(point[0] - t[0], point[1] - t[1])
32             ),
33             (
34                 math.hypot(p[0] - start[0], p[1] - start[1])
35                 + math.hypot(start[0] - t[0], start[1] - t[1])
36             )
37             - 0.1,
38         )
39         self.assertEqual(diag["iterations"], 24)
40
41     def test_clear_projection_two_active_circles(self):
42         polygon = [(-10, 0), (10, 0)]
43         radius = 19.8 - 1e-6
44         point = zjqcd(polygon, (0, 50))
45         self.assertAlmostEqual(point[0], 0, places=9)
46         self.assertAlmostEqual(point[1], math.sqrt(radius * radius - 100), places=8)
47         self.assertIsNone(zjqcd([(-21, 0), (21, 0)], (0, 50)))
48         self.assertEqual(zjqcd(polygon, (0, 0)), (0, 0))
49
50     def test_wedge_unbounded_and_inconsistent(self):
51         self.assertEqual(jhqy([dict(x=0, y=0, bearing_deg=0)])["state"], "UNBOUNDED")
52         result = jhqy([dict(x=0, y=0, bearing_deg=0), dict(x=-1, y=0, bearing_deg=180)])
53         self.assertEqual(result["state"], "EMPTY_OR_NUMERICALLY_UNRESOLVED")
54
55     def test_triangle_diameter_circle(self):
56         vertices = [(0, 0), (1, 0), (0.5, math.sqrt(3) / 2)]
57         observations = []
58         for i, p in enumerate(vertices):
59             q = vertices[(i + 1) % 3]

```

```

60         dx, dy = q[0] - p[0], q[1] - p[1]
61         observations.append(
62             dict(
63                 x=p[0] - 100 * dx,
64                 y=p[1] - 100 * dy,
65                 bearing_deg=math.degrees(math.atan2(dy, dx)) + 1,
66             )
67         )
68         result = jhqy(observations, 1)
69         self.assertEqual(result["state"], "POLYGON")
70         self.assertAlmostEqual(result["diameter"], 1, places=8)
71         self.assertFalse(result["diameter_circle_covers"])
72
73     def test_http_requests_and_evidence_log_without_network(self):
74         with tempfile.TemporaryDirectory() as folder:
75             path = Path(folder) / "actions.jsonl"
76             response = Mock()
77             response.status = 200
78             response.read.return_value = json.dumps(
79                 dict(accepted=True, virtual_time_s=5, measure_result="no_signal")
80             ).encode()
81             response.__enter__ = Mock(return_value=response)
82             response.__exit__ = Mock(return_value=False)
83             opener = Mock()
84             opener.open.return_value = response
85             with path.open("x", encoding="utf-8") as log:
86                 with patch("jammer_solver.protocol.ljq", return_value=opener):
87                     transport = xjmk("test-robot", log)
88                     reply = transport["call"]("/measure", (12.5, -4), 7)
89                     self.assertEqual(reply["measure_result"], "no_signal")
90                     self.assertIsNone(transport["pending"])
91             request = opener.open.call_args.args[0]
92             self.assertEqual(request.full_url, "http://127.0.0.1:2026/measure")
93             payload = json.loads(request.data)
94             self.assertEqual(payload["position"], {"x": 12.5, "y": -4})
95             self.assertEqual(payload["channel"], 7)
96             events = [json.loads(line) for line in path.read_text().splitlines()]
97             self.assertEqual([e["event"] for e in events], ["intent", "response"])
98             self.assertEqual(events[0]["payload"], payload)
99             self.assertEqual(events[1]["request_id"], payload["request_id"])
100
101     def test_network_failure_propagates_and_leaves_request_unconfirmed(self):
102         with tempfile.TemporaryDirectory() as folder:
103             path = Path(folder) / "actions.jsonl"
104             opener = Mock()
105             opener.open.side_effect = URLLError("fixed offline fixture")
106             with path.open("x", encoding="utf-8") as log:
107                 with patch("jammer_solver.protocol.ljq", return_value=opener):
108                     transport = xjmk("test-robot", log)
109                     with self.assertRaises(URLLError):
110                         transport["call"]("/measure", (0, 0), 1)
111                     self.assertIsNotNone(transport["pending"])
112                     with self.assertRaises(RuntimeError):
113                         transport["call"]("/exit")
114             opener.open.assert_called_once()
115             self.assertEqual(len(path.read_text().splitlines()), 1)

```

tests/test_offline_timeout.py

文件行数: 23。

```

1 import time
2 import unittest
3 from benchmark import offline_io
4
5
6 class OfflineTimeoutTests(unittest.TestCase):
7     def test_expired_call_does_not_send(self):

```

```

8         sent = []
9         io = offline_io(lambda *args: sent.append(args))
10        io["wall_deadline"] = time.monotonic() - 1
11        with self.assertRaises(TimeoutError):
12            io["call"]("/enter")
13        self.assertEqual(sent, [])
14
15    def test_expired_record_stops_planning(self):
16        io = offline_io(None)
17        io["wall_deadline"] = time.monotonic() - 1
18        with self.assertRaises(TimeoutError):
19            io["record"]({"event": "planning"})
20
21
22 if __name__ == "__main__":
23     unittest.main()

```

tests/test_solver.py

文件行数: 516。

```

1 from jammer_solver import solver as v8
2 from verification import replay
3 from fractions import Fraction as F
4 import math
5 import copy
6 import itertools
7 import unittest
8 from jammer_solver import exact_geometry as g
9 from verification import build_graph, solve, calibrate
10 from jammer_solver.solver import sscd
11
12
13 def negative():
14     return dict(measure_result="no_signal")
15
16
17 class GeometryTests(unittest.TestCase):
18     def test_rational_distance_rounding(self):
19         p, q = (g.point((0, 0)), g.point((1, 1)))
20         self.assertEqual(g.jlxs(p, q), F("1.415"))
21         self.assertGreaterEqual(g.jlxs(p, q) ** 2, 2)
22         self.assertEqual(g.jlxs(g.point((0, 0)), g.point((3, 4))), F(5))
23
24     def test_trigonometric_enclosures_and_wrap(self):
25         for b in (0, 1.005, 44.9, 45, 90, 179.99, 270, 359.99, -721.005):
26             cl, ch, sl, sh = g.trig(F(b))
27             c, s = (math.cos(math.radians(b)), math.sin(math.radians(b)))
28             self.assertLessEqual(float(cl) - 1e-15, c)
29             self.assertGreaterEqual(float(ch) + 1e-15, c)
30             self.assertLessEqual(float(sl) - 1e-15, s)
31             self.assertGreaterEqual(float(sh) + 1e-15, s)
32             self.assertEqual(g.trig(F("1.005")), g.trig(F("361.005")))
33
34     def test_full_grid_covers_all_directions(self):
35         for p in (3, 4):
36             self.assertTrue(g.fgzm(tuple(g.sswd()), (), p)[0])
37             self.assertEqual(len(g.sswd()), 49)
38
39     def test_seven_omni_stops_do_not_certify_directional_absence(self):
40         points = tuple((g.point(q) for q in sscd(3)))
41         self.assertTrue(g.fgzm(points, (), 3)[0])
42         self.assertFalse(g.fgzm(points, (), 4)[0])
43         self.assertTrue(all((q[0] < 1800 for q in points)))
44
45     def test_budget_exhaustion_is_unknown(self):
46         self.assertFalse(g.fgzm(tuple(g.sswd()), (), 4, 1)[0])
47

```

```

48 def test_current_directional_design_has_continuous_certificate(self):
49     points = tuple(sorted(g.point(q) for q in v8.sscd(4)))
50     self.assertEqual(len(points), 21)
51     self.assertTrue(g.fgzmm(points, (), 4, 20000, 14)[0])
52
53 def test_round_domain_retains_boundary_and_removes_square_corner(self):
54     domain = g.yfw()
55     for angle in range(0, 360, 5):
56         r = 1800 - 1e-9
57         p = g.point(
58             (r * math.cos(math.radians(angle)), r * math.sin(math.radians(angle)))
59         )
60         self.assertTrue(g.contains(domain, p))
61     old = v8.xjyd((1000, 1000), 45, jhbb=1)
62     current = v8.xjyd((1000, 1000), 45, jhbb=2)
63     self.assertTrue(g.contains(old["polygon"], g.point((1700, 1700))))
64     self.assertFalse(g.contains(current["polygon"], g.point((1700, 1700))))
65     self.assertTrue(g.contains(current["polygon"], g.point((1200, 1200))))
66     self.assertTrue(
67         any(
68             g.contains(cell, g.point((1200, 1200)))
69             for cell in current["cells"].values()
70         )
71     )
72
73 def test_unknown_geometry_version_is_rejected(self):
74     with self.assertRaises(ValueError):
75         v8.xjyl(3, jhbb=99)
76
77 def test_second_positive_uses_posterior_lower_range(self):
78     yuan = v8.xjyd((0, 0), 0)
79     v8.gxyd(yuan, (500, 150), "direction", 340, problem=3)
80     origin = yuan["anchor_point"]
81     u, _ = g.fangxiang(F(yuan["anchor_bearing"]))
82     projected = [
83         (
84             (q[0] - origin[0], q[1] - origin[1])[0] * u[0]
85             + (q[0] - origin[0], q[1] - origin[1])[1] * u[1]
86         )
87         for q in yuan["polygon"]
88     ]
89     self.assertGreater(min(projected), F(yuan["anchor_radius"]) / 2)
90     for q in v8.sdc(yuan, (0, 0)):
91         distance = (q[0] - origin[0], q[1] - origin[1])[0] * u[0] + (
92             q[0] - origin[0],
93             q[1] - origin[1],
94         )[1] * u[1]
95         self.assertGreater(distance, min(projected))
96         self.assertLess(distance, max(projected))
97
98 def test_initial_cells_retain_edge_bearings_and_ranges(self):
99     for angle, r, error in (
100         (0, 1500, 1),
101         (359.99, 1499.99, -1),
102         (90, 5.001, 1),
103         (45, 1000, -1),
104     ):
105         p = (r * math.cos(math.radians(angle)), r * math.sin(math.radians(angle)))
106         bearing = round((angle + error) % 360, 2) % 360
107         yuan = v8.xjyd((0, 0), bearing)
108         self.assertTrue(g.contains(yuan["polygon"], g.point(p)))
109         self.assertTrue(
110             any((g.contains(cell, g.point(p)) for cell in yuan["cells"].values()))
111         )
112         self.assertLessEqual(len(yuan["cells"]), 183)
113         for i, cell in yuan["cells"].items():
114             self.assertTrue(g.ypbh(cell, yuan["points"][i], F(20)))
115

```

```

116 def test_direction_interval_uses_whole_interval(self):
117     q = g.point((0, 0))
118     poly = g.juxing(-1800, -1800, 1800, 1800)
119     outer = g.jcfw(poly, q, F(30), hi=F(40))
120     for angle in (29.01, 30, 35, 40, 40.99):
121         p = g.point(
122             (
123                 1000 * math.cos(math.radians(angle)),
124                 1000 * math.sin(math.radians(angle)),
125             )
126         )
127         self.assertTrue(g.contains(outer, p))
128
129 def test_negative_pair_requires_previous_positive(self):
130     yuan = v8.xjyd((0, 0), 0)
131     a, b = v8.sdc(yuan, (0, 0))
132     with self.assertRaises(RuntimeError):
133         v8.syx(yuan, yuan["origin"], a, b)
134     v8.gxyd(yuan, a, "no_signal", problem=4)
135     v8.gxyd(yuan, b, "no_signal", problem=4)
136     self.assertTrue(v8.syx(yuan, yuan["origin"], a, b))
137     self.assertTrue(g.contains(yuan["polygon"], g.point((100, 0))))
138     self.assertFalse(g.contains(yuan["polygon"], g.point((1200, 0))))
139
140 def test_a_single_directional_negative_does_not_delete_truth(self):
141     yuan = v8.xjyd((0, 0), 0)
142     v8.gxyd(yuan, (150, 0), "no_signal", problem=4)
143     self.assertTrue(g.contains(yuan["polygon"], g.point((100, 0))))
144
145 def test_optical_failure_discharges_its_cell(self):
146     yuan = v8.xjyd((0, 0), 0)
147     i = min(yuan["cells"])
148     q = yuan["points"][i]
149     v8.gxyd(yuan, q, "no_target_in_range")
150     self.assertNotIn(i, yuan["cells"])
151     self.assertTrue(g.contains(yuan["polygon"], g.point((1499, 0))))
152
153 def test_near_is_continuously_clearable(self):
154     yuan = v8.xjyd((100, 100))
155     q = v8.lxqc(yuan, (90, 90))
156     self.assertIsNotNone(q)
157     self.assertTrue(g.ypbh(yuan["polygon"], q, F("19.8")))
158
159
160 class EvidenceTests(unittest.TestCase):
161     def test_optional_omni_negative_is_useful_without_assuming_reception(self):
162         class IO:
163             pending = None
164
165             def __init__(self):
166                 self.calls = []
167
168             def call(self, path, q, c):
169                 self.calls.append((path, q, c))
170                 return dict(
171                     accepted=True, virtual_time_s=5.0, measure_result="no_signal"
172                 )
173
174             def record(self, event):
175                 pass
176
177         io = IO()
178         state = v8.xjzt(dict(pending=io.pending, call=io.call, record=io.record), 3)
179         state["position"] = (750.0, 750.0)
180         ledger = state["ledger"]
181         v8.gxjl(
182             ledger, "/measure", (0, 0), 1, dict(measure_result="direction", svd_deg=0)
183         )

```

```

184         source = ledger["channels"][1]["source"]
185         self.assertFalse(g.ypbh(source["polygon"], g.point(state["position"]), F(999)))
186         self.assertTrue(v8.slcl(state))
187         self.assertEqual(io.calls, [("/measure", (750.0, 750.0), 1)])
188         self.assertEqual(state["travel"], 0)
189         self.assertTrue(g.contains(source["polygon"], g.point((10, 0))))
190         self.assertLess(max(float(q[0]) for q in source["polygon"]), 1000)
191         self.assertEqual(ledger["channels"][1]["status"], "found")
192
193     def test_shared_station_has_no_movement_and_keeps_directional_negative_feasible(
194         self,
195     ):
196         class IO:
197             pending = None
198
199             def __init__(self):
200                 self.calls = []
201
202             def call(self, path, q, c):
203                 self.calls.append((path, q, c))
204                 return dict(
205                     accepted=True, virtual_time_s=5.0, measure_result="no_signal"
206                 )
207
208             def record(self, event):
209                 pass
210
211         io = IO()
212         state = v8.xjzt(dict(pending=io.pending, call=io.call, record=io.record), 4)
213         state["position"] = (750.0, 200.0)
214         ledger = state["ledger"]
215         v8.gxjl(
216             ledger, "/measure", (0, 0), 1, dict(measure_result="direction", svd_deg=0)
217         )
218         source = ledger["channels"][1]["source"]
219         v8.xdfw(source, g.juxing(700, -5, 800, 5))
220         self.assertTrue(v8.slcl(state))
221         self.assertEqual(io.calls, [("/measure", (750.0, 200.0), 1)])
222         self.assertEqual(state["travel"], 0.0)
223         self.assertTrue(g.contains(source["polygon"], g.point((750, 0))))
224         self.assertEqual(ledger["channels"][1]["status"], "found")
225         self.assertTrue(v8.slcl(state))
226         self.assertEqual(len(io.calls), 1)
227
228     def test_per_channel_histories_cannot_be_shared(self):
229         jilu = v8.xjzl(3)
230         for q in ssdc(3):
231             v8.gxjl(jilu, "/measure", q, 1, negative())
232             self.assertEqual(jilu["channels"][1]["status"], "absent")
233             self.assertEqual(jilu["channels"][2]["status"], "unknown")
234             self.assertEqual(len(jilu["channels"][2]["pending"]), 49)
235
236     def test_sixteen_discoveries_allow_count_stop_but_ten_do_not(self):
237         for count in (10, 16):
238             jilu = v8.xjzl(4)
239             for c in range(1, count + 1):
240                 v8.gxjl(jilu, "/clear", (0, 0), c, dict(clear_result="success"))
241                 self.assertEqual(v8.qbqc(jilu), count == 16)
242                 self.assertEqual(
243                     len(v8.pindao(jilu, ("unknown",))), 0 if count == 16 else 10
244                 )
245
246     def test_unproductive_credits_never_reset(self):
247         jilu = v8.xjzl(4, 2)
248         for x in (1, 2):
249             receipt = v8.gxjl(jilu, "/measure", (x, 1), 1, negative(), prove=False)
250             self.assertLess(receipt["after"], receipt["before"])
251             self.assertEqual(jilu["extra"], 0)

```

```

252         q = g.sswd()[0]
253         v8.gxjl(jilu, "/measure", q, 1, negative(), prove=False)
254         self.assertEqual(jilu["extra"], 0)
255
256     def test_remaining_bound_includes_undiscovered_sources(self):
257         jilu = v8.xjzl(4, 0)
258         self.assertLess(v8.sysj(jilu, (0, 0)), 41640)
259         self.assertGreater(v8.sysj(jilu, (0, 0)), 40000)
260
261     def test_rejected_request_changes_neither_state_nor_evidence(self):
262
263         class IO:
264             pending = None
265
266             def call(self, *args):
267                 return dict(accepted=False)
268
269         zhuangtai = v8.xjzt(dict(pending=IO.pending, call=IO().call), 4)
270         rank = v8.syrw(zhuangtai["ledger"])
271         with self.assertRaises(RuntimeError):
272             v8.zhixing(zhuangtai, "/measure", (100, 200), 1)
273         self.assertEqual(zhuangtai["position"], (0, 0))
274         self.assertEqual(v8.syrw(zhuangtai["ledger"]), rank)
275         self.assertFalse(zhuangtai["ledger"]["receipts"])
276
277     def test_unresolved_request_prevents_new_action(self):
278
279         class IO:
280             pending = {"request_id": "uncertain"}
281
282             def call(self, *args):
283                 raise AssertionError("No call allowed")
284
285         with self.assertRaises(RuntimeError):
286             v8.zhixing(
287                 v8.xjzt(dict(pending=IO.pending, call=IO().call), 3),
288                 "/measure",
289                 (0, 0),
290                 1,
291             )
292
293     def test_bad_cost_cannot_produce_a_certificate(self):
294
295         class IO:
296             pending = None
297
298             def call(self, *args):
299                 return dict(accepted=True, virtual_time_s=999, measure_result="near")
300
301         zhuangtai = v8.xjzt(dict(pending=IO.pending, call=IO().call), 3)
302         with self.assertRaises(RuntimeError):
303             v8.zhixing(zhuangtai, "/measure", (10, 0), 1)
304         self.assertEqual(zhuangtai["position"], (10, 0))
305         self.assertFalse(v8.pindao(zhuangtai["ledger"], ("found", "cleared")))
306         self.assertFalse(zhuangtai["certificate"])
307
308
309     class PlanningTests(unittest.TestCase):
310         def test_directional_visibility_conditions_on_actual_negative_history(self):
311             poly = g.juxing(-1, -1, 1, 1)
312             record = dict(
313                 source=dict(cells={0: poly}),
314                 records=[
315                     dict(path="/measure", point=(-100, 0), kind="direction"),
316                     dict(path="/measure", point=(100, 0), kind="no_signal"),
317                 ],
318             )
319             before = copy.deepcopy(record)

```

```

320     model = v8.spmx(record)
321     self.assertAlmostEqual(v8.spgl(model, (-200, 0)), 1)
322     self.assertAlmostEqual(v8.spgl(model, (0, 0)), 1)
323     self.assertAlmostEqual(v8.spgl(model, (200, 0)), 0)
324     self.assertAlmostEqual(v8.spgl(model, (-1600, 0)), 0)
325     self.assertEqual(record, before)
326
327     def test_optical_chain_covers_complete_region_not_only_sample_points(self):
328         source = v8.xjyd((0, 0), 0)
329         v8.xdfw(source, g.juxing(500, -8, 600, 8))
330         options = v8.gxbx(source, (0, 0))
331         self.assertTrue(options)
332         for _, order, _, upper in options:
333             self.assertLessEqual(len(order), 8)
334             self.assertGreaterEqual(upper, v8.lxsj(order, (0, 0)))
335             for q in order:
336                 cell = source["polygon"]
337                 for other in order:
338                     if q != other:
339                         cell = g.clip(
340                             cell,
341                             ((other[0] - q[0], other[1] - q[1])),
342                             (
343                                 (other[0] * other[0] + other[1] * other[1])
344                                 - (q[0] * q[0] + q[1] * q[1])
345                             )
346                             / 2,
347                         )
348                 self.assertTrue(not cell or g.ypbh(cell, q, F("19.8")))
349
350     def test_optical_chain_is_rejected_before_action_when_credit_is_insufficient(self):
351         state = v8.xjzt(None, 4, dzys=1)
352         event = dict(
353             kind="optical_chain", channel=1, points=(g.point((10, 0)), g.point((20, 0)))
354         )
355         before = v8.syrw(state["ledger"])
356         self.assertFalse(v8.zx(state, event))
357         self.assertEqual(v8.syrw(state["ledger"]), before)
358         self.assertEqual(state["position"], (0, 0))
359
360     def test_negative_pair_without_contraction_is_not_measured_again(self):
361         state = v8.xjzt(None, 4, ghsj=0)
362         source = v8.xjyd((0, 0), 0)
363         source["positives"].append(g.point((-50, 20)))
364         v8.xdfw(source, g.juxing(500, -8, 600, 8))
365         pair = v8.sdc(source, (0, 0))
366         for q in pair:
367             v8.gxyd(source, q, "no_signal", problem=4)
368             self.assertFalse(v8.syx(source, source["anchor_point"], *pair))
369             before = copy.deepcopy(source)
370             event = v8.yddz(
371                 state["planner"], source, 1, (tuple((float(x) for x in pair[-1])), 1, 640)
372             )
373             self.assertIn(event["kind"], ("clear", "optical_chain"))
374             self.assertEqual(source, before)
375             self.assertEqual(
376                 state["planner"]["statistics"]["repeated_negative_pairs_skipped"], 1
377             )
378
379     def test_small_service_plan_matches_exhaustive_orders_and_modes(self):
380         jobs = {}
381         for channel, points in enumerate(
382             (((9, 0), (0, 8)), ((20, 3), (2, 13)), ((-4, 18), (18, 17))), 1
383         ):
384             job = ("source", channel)
385             jobs[job] = [
386                 dict(
387                     job=job,

```

```

388         entry=p,
389         exit=(p[0] + 2 * channel, p[1] - channel),
390         service=3 + channel + i,
391     )
392     for i, p in enumerate(points)
393 ]
394 position = (1, -2)
395 exact = math.inf
396 for order in itertools.permutations(jobs):
397     for modes in itertools.product(*(jobs[job] for job in order)):
398         cost = math.dist(position, modes[0]["entry"]) / 5 + modes[0]["service"]
399         cost += sum(
400             math.dist(a["exit"], b["entry"]) / 5 + b["service"]
401             for a, b in zip(modes, modes[1:])
402         )
403         exact = min(exact, cost)
404 plan = v8.aprw(jobs, position, (("source", 99), ("source", 2)))
405 self.assertAlmostEqual(plan["estimate"], exact)
406 self.assertCountEqual(plan["order"], jobs)
407 self.assertEqual([mode["job"] for mode in plan["modes"]], plan["order"])
408
409
410 class CalibrationTests(unittest.TestCase):
411     def test_exact_value_and_rollout_gap(self):
412         result = calibrate([(0,), (60,), (120,)], (0, 60, 120))
413         self.assertEqual(result["exact_fraction"], "34")
414         self.assertEqual(result["rollout"][0]["upper_s"], 35)
415         self.assertEqual(result["rollout"][2]["gap_s"], 0)
416
417     def test_bellman_solution_and_nonanticipative_actions(self):
418         root, graph, terminal = build_graph([(0, 120), (120, 0)], (0, 60, 120))
419         values, policy, _ = solve(graph, terminal)
420         for s, actions in graph.items():
421             if s in terminal:
422                 self.assertEqual(values[s], 0)
423                 continue
424             expected = min(
425                 (
426                     max((cost + values[t] for _, cost, t in edges))
427                     for edges in actions.values()
428                 )
429             )
430             self.assertEqual(values[s], expected)
431             for _, cost, t in actions[policy[s]]:
432                 self.assertGreater(cost, 0)
433                 self.assertLess(values[t], values[s])
434             self.assertTrue(math.isfinite(values[root]))
435
436     def test_variable_count_is_rejected(self):
437         with self.assertRaises(ValueError):
438             build_graph([(0,), (0, 60)], (0, 60))
439
440     def test_information_lower_bound(self):
441         from lower_bounds import per_channel, absence_lower
442
443         self.assertEqual((per_channel(3), per_channel(4)), (30, 35))
444         self.assertEqual(absence_lower(4, 10), 350)
445         self.assertEqual(absence_lower(4, 16), 0)
446
447
448 class IntegrationTests(unittest.TestCase):
449     @classmethod
450     def setUpClass(cls):
451         from benchmark import run_case
452         from simulator import Source as TruthSource
453
454         cls.results = []
455         for problem, n, fallback in ((3, 10, False), (4, 16, True)):

```

```

456         case = dict(
457             name="unit-completion",
458             problem=problem,
459             seed=17,
460             extreme=True,
461             sources=[
462                 TruthSource(c, (0.0, 0.0), 1000.0, 0.0 if problem == 4 else None)
463                 for c in range(1, n + 1)
464             ],
465         )
466         result, events = run_case(
467             case,
468             planning_seconds=0,
469             extra_actions=0 if fallback else 640,
470             fallback_only=fallback,
471         )
472         cls.results.append((result, events))
473
474     def test_actual_completion_and_truth_free_replay(self):
475         for result, events in self.results:
476             self.assertIsNone(result["error"], result.get("traceback"))
477             self.assertEqual(result["source_count"], result["cleared_count"])
478             self.assertTrue(replay(events)["verified"])
479
480     def test_zero_credit_fallback_with_failed_optical_attempts(self):
481         result, events = self.results[1]
482         self.assertTrue(result["summary"]["fallback_used"])
483         self.assertEqual(result["summary"]["extra_actions_remaining"], 0)
484         self.assertGreater(result["counts"]["clear"], result["source_count"])
485         self.assertLess(
486             result["virtual_time_s"], result["summary"]["initial_fallback_upper_s"]
487         )
488         self.assertLessEqual(
489             result["counts"]["measure"] + result["counts"]["clear"], 3908
490         )
491
492     def test_tampered_response_is_rejected_by_replay(self):
493         events = copy.deepcopy(self.results[0][1])
494         item = next((e for e in events if e.get("event") == "evidence_observation"))
495         item["reply"]["virtual_time_s"] += 1
496         with self.assertRaises(RuntimeError):
497             replay(events)
498
499     def test_missing_receipt_is_rejected_by_replay(self):
500         events = list(self.results[0][1])
501         i = next((i for i, e in enumerate(events) if e.get("event") == "rank_receipt"))
502         del events[i]
503         with self.assertRaises(RuntimeError):
504             replay(events)
505
506     def test_prospective_search_proofs_do_not_erase_unknowns(self):
507         jilu = v8.xj1l(3)
508         plan = v8.xj1x(sscd(3), seconds=1)
509         v8.gxlx(plan, jilu, (0, 0))
510         self.assertEqual(v8.pindao(jilu, ("unknown",)), set(range(1, 21)))
511         self.assertEqual(v8.syrw(jilu), 980 + 16 * 183 + 640)
512         self.assertFalse(jilu["certificates"])
513
514
515 if __name__ == "__main__":
516     unittest.main()

```

tests/test_submission.py

文件行数: 126。

```

1 import csv
2 import json

```

```

3 from pathlib import Path
4 import tempfile
5 import unittest
6
7 import prepare_submission as package
8
9
10 class SubmissionTests(unittest.TestCase):
11     def setUp(self):
12         self.temporary = tempfile.TemporaryDirectory()
13         self.addCleanup(self.temporary.cleanup)
14         self.root = Path(self.temporary.name)
15         inputs = self.root / "submission_inputs"
16         inputs.mkdir()
17         for directory in (
18             "q2_comparison",
19             "sensitivity",
20             "ablation",
21             "practice_details",
22         ):
23             (inputs / directory).mkdir()
24         (inputs / "human_review.json").write_text("{} ", encoding="utf-8")
25         self.columns = (
26             "problem",
27             "test",
28             "case_code",
29             "cleared_count",
30             "virtual_time_s",
31             "average_time_s",
32             "real_time_s",
33             "log_filename",
34             "upload_confirmed",
35         )
36         self.rows = []
37         for q in (3, 4):
38             for i in (1, 2, 3):
39                 (inputs / f"formal/{q}{i}/test_{i}").mkdir(parents=True)
40                 self.rows.append(
41                     dict.fromkeys(self.columns, "") | dict(problem=str(q), test=str(i))
42                 )
43         self.save()
44
45     def save(self):
46         with (self.root / "submission_inputs/formal_results.csv").open(
47             "w", encoding="utf-8-sig", newline=""
48         ) as f:
49             writer = csv.DictWriter(f, self.columns)
50             writer.writeheader()
51             writer.writerows(self.rows)
52
53     def fixture_log(self, package_type, index=1):
54         # A metadata-only fixture, deliberately not an authentic signed log.
55         path = self.root / "submission_inputs/formal/q3/test_1/fixture.jlog"
56         meta = dict(
57             package_type=package_type,
58             problem_no=3,
59             formal_index=index,
60             case_code="TEST-CASE",
61         )
62         payload = b"JMBPLOG1" + bytes(6) + json.dumps(meta).encode() + bytes(32)
63         path.write_bytes(payload)
64         self.rows[0].update(
65             case_code="TEST-CASE",
66             log_filename=path.name,
67             cleared_count="10",
68             virtual_time_s="3000",
69             average_time_s="300",
70             real_time_s="30",

```

```

71         upload_confirmed="true",
72     )
73     self.save()
74     return path, payload
75
76 def test_empty_slots_remain_missing_and_no_logs_are_created(self):
77     pending, errors, logs = package.check_inputs(self.root)
78     self.assertEqual(errors, [])
79     self.assertEqual(logs, [])
80     self.assertEqual(sum(" 缺少原始正式日志" in item for item in pending), 6)
81     self.assertFalse(list(self.root.rglob("*.jlog")))
82
83 def test_practice_log_cannot_fill_a_formal_slot(self):
84     path, original = self.fixture_log("practice_behavior_log")
85     _, errors, logs = package.check_inputs(self.root)
86     self.assertTrue(any(" 演练日志不能代替" in e for e in errors))
87     self.assertEqual(logs, [])
88     self.assertEqual(path.read_bytes(), original)
89
90 def test_matching_envelope_is_preserved_without_claiming_signature_validation(self):
91     path, original = self.fixture_log("formal_behavior_log")
92     _, errors, logs = package.check_inputs(self.root)
93     self.assertEqual(errors, [])
94     self.assertEqual(logs, [path])
95     self.assertEqual(path.read_bytes(), original)
96
97 def test_wrong_formal_index_is_rejected(self):
98     self.fixture_log("formal_behavior_log", index=2)
99     _, errors, logs = package.check_inputs(self.root)
100    self.assertTrue(any(" 正式次数" in e for e in errors))
101    self.assertEqual(logs, [])
102
103 def test_invalid_average_is_not_accepted(self):
104     self.fixture_log("formal_behavior_log")
105     self.rows[0]["average_time_s"] = "100"
106     self.save()
107     _, errors, _ = package.check_inputs(self.root)
108     self.assertTrue(any(" 平均时间" in e for e in errors))
109
110 def test_identical_inputs_produce_identical_archive_bytes(self):
111     left, right = self.root / "a.zip", self.root / "b.zip"
112     package.write_zip(left, {"b.txt": b"2", "a.txt": b"1"})
113     package.write_zip(right, {"a.txt": b"1", "b.txt": b"2"})
114     self.assertEqual(left.read_bytes(), right.read_bytes())
115
116 def test_identity_patterns_are_reported_without_exposing_values(self):
117     path = self.root / "metadata.json"
118     identity = "123456" + "654321"
119     path.write_text(json.dumps({"robot_id": identity}), encoding="utf-8")
120     report = package.check_anonymity([path])
121     self.assertEqual(report, ["metadata.json"])
122     self.assertNotIn(identity, str(report))
123
124
125 if __name__ == "__main__":
126     unittest.main()

```

validation/baseline_v7/adaptive.py

文件行数: 245。

```

1 """v5: joint evidence, whole-outcome route lookahead and certified optical gate."""
2 from dataclasses import asdict
3 import math
4 from cooperative import CooperativeSolver
5 from solver import ModelMismatch
6 from strategy import Anchor, annular_points, annular_update
7 from posterior import constrain

```

```

8 from planning import open_route
9 from joint_belief import JointBelief, SAFE
10 from decision import terminal_upper, measurement_envelope, optical_decision, optical_cover,
    ↪ chain_cost
11
12 STRATEGY="joint-outcome-lookahead-v5"
13
14
15 class AdaptiveSolver(CooperativeSolver):
16     def __init__(self, transport, problem, progress=None):
17         super().__init__(transport, problem, progress)
18         self.raw_evidence={c:[] for c in range(1,21)}
19         self.beliefs={}
20         self.posterior_stats={}
21         self.reserved_source=None;self.committed_source=None
22         self.forecast_cache={}
23         self.lookahead_decisions=0;self.shared_by_lookahead=0
24         self.optical_accepted=self.optical_overlap_rejected=self.optical_failure_replans=0
25
26     def _sync(self,c):
27         belief=self.beliefs[c]
28         self.regions[c]=belief.hull()
29         if c in self.anchors:
30             self.refresh_history(c)
31         previous=self.posterior_stats.get(c,{})
32         self.posterior_stats[c]=dict(cells=len(belief.cells),
33             max_cells=max(len(belief.cells),previous.get("max_cells",0)),
34             retained_parent_count=belief.retained_parents,
35             raw_observation_count=len(belief.records))
36
37     def _belief(self,c):
38         if c not in self.beliefs:
39             records=self.raw_evidence[c]
40             if not records:
41                 # Supports explicit offline state fixtures; ordinary runs always
42                 # arrive here with the actual discovery observation recorded.
43                 a=self.anchors[c]
44                 records=[dict(kind="direction",point=a.point,bearing=a.bearing)]
45             self.beliefs[c]=JointBelief(self.regions[c],self.problem,records)
46             self._sync(c)
47         return self.beliefs[c]
48
49     def action(self,path,point,channel):
50         reply=super().action(path,point,channel)
51         record=None
52         if path=="/measure":
53             record=dict(kind=reply["measure_result"],point=tuple(point))
54             if record["kind"]=="direction":
55                 record["bearing"]=reply["svd_deg"]
56         elif reply["clear_result"]!="success":
57             record=dict(kind="clear_failure",point=tuple(point))
58         if record is not None:
59             self.raw_evidence[channel].append(record)
60             if channel in self.beliefs and record["kind"]!="near":
61                 self.beliefs[channel].observe(record)
62                 self._sync(channel)
63             self.forecast_cache.clear()
64         return reply
65
66     def record_positive(self,channel,point):
67         super().record_positive(channel,point)
68         belief=self._belief(channel)
69         belief.intersect(self.regions[channel])
70         self._sync(channel)
71
72     def finish_clear(self,channel):
73         super().finish_clear(channel)
74         self.beliefs.pop(channel,None)

```

```

75         if self.committed_source==channel:
76             self.committed_source=None
77         if self.reserved_source==channel:
78             self.reserved_source=None
79         self.forecast_cache.clear()
80
81     def _forecast(self,c,q,target,cost=6):
82         if self.optional_counts[c]>=2 or self.anchors[c].rounds!=0:
83             return None
84         if tuple(q) in self.positives[c]+self.optional_positions[c]:
85             return None
86         belief=self._belief(c)
87         key=(c,len(self.raw_evidence[c]),self.anchors[c],tuple(q),target,cost)
88         if key not in self.forecast_cache:
89             self.forecast_cache[key]=measurement_envelope(belief,self.anchors[c],q,self.problem,
90                 ↪ target,cost)
91         return self.forecast_cache[key]
92
93     def choose_candidates(self,route):
94         self.reserved_source=None
95         if self.committed_source in self.anchors:
96             c=self.committed_source
97             ready=[c] if self.clear_plan(c)[0] is not None else []
98             return [(0.,c)],ready
99         target=route[0] if route else None
100         direct=math.dist(self.position,target)/5 if target is not None else 0.
101         now={}
102         for c in self.anchors:
103             now[c]=terminal_upper(self._belief(c),self.anchors[c],self.position,self.problem,tar
104                 ↪ get)[0]-direct
105         if not route:
106             return [(value,c) for c,value in now.items()], [c for c in now if
107                 ↪ self.clear_plan(c)[0] is not None]
108         future=route[1] if len(route)>1 else None
109         edge=math.dist(target,future)/5 if future is not None else 0.
110         # All sources receive route-aware bounds; at most three receive the
111         # more expensive whole-response branching in one decision.
112         shortlist=set(sorted(now,key=lambda c:now[c])[:3])
113         immediate=[];ready=[];deferred=[]
114         for c,current in now.items():
115             later=terminal_upper(self._belief(c),self.anchors[c],target,self.problem,future)[0]-
116                 ↪ edge
117             read=False;envelope=None
118             if c in shortlist and self.clear_plan(c)[0] is None:
119                 envelope=self._forecast(c,target,future)
120                 if envelope is not None and envelope["upper_s"]-edge<later-1:
121                     later=envelope["upper_s"]-edge;read=True
122             if current<=later+1:
123                 immediate.append((current,c))
124                 if self.clear_plan(c)[0] is not None:
125                     ready.append(c)
126             else:
127                 deferred.append((current-later,c,read))
128         self.lookahead_decisions+=1
129         self.emit("route_information_comparison",channel=c,now_extra_upper_s=current,
130             next_stop_extra_upper_s=later,next_stop=target,after_stop=future,
131             proposed_shared_measure=read,
132             outcome_count=0 if envelope is None else len(envelope["branches"]),
133             interpretation="conditional_upper_bounds_not_realized_savings")
134         if not immediate:
135             choices=[entry for entry in deferred if entry[2]]
136             if choices:
137                 self.reserved_source=max(choices)[1]
138         return immediate,ready
139
140     def shared_measurements(self,point):
141         route=open_route(point,self.remaining) if self.unknown and len(self.discovered)<16 else []

```

```

138     target=route[0] if route else None
139     chosen=self.reserved_source if self.reserved_source in self.anchors else None
140     self.reserved_source=None
141     if chosen is None:
142         options=[]
143         for c in sorted(self.anchors,key=lambda c:math.dist(self.anchors[c].point,point))[:3]:
144             if self.clear_plan(c)[0] is not None:
145                 continue
146             before,_=terminal_upper(self._belief(c),self.anchors[c],point,self.problem,target)
147             plan=self._forecast(c,point,target,5+int(c!=self.channel))
148             if plan is not None and plan["upper_s"]<before-1:
149                 options.append((before-plan["upper_s"],c))
150     if options:
151         chosen=max(options)[1]
152     if chosen is None:
153         self.stage="search";return
154     c=chosen
155     if self.clear_plan(c)[0] is not None:
156         self.committed_source=c;return
157     plan=self._forecast(c,point,target,5+int(c!=self.channel))
158     if plan is None:
159         return
160     self.stage="shared_measure"
161     reply=self.action("/measure",point,c)
162     self.optional_counts[c]+=1;self.optional_positions[c].append(tuple(point))
163     self.shared_by_lookahead+=1
164     self.emit("shared_measure_by_outcomes",channel=c,result=reply["measure_result"],
165             predicted_completion_upper_s=plan["upper_s"],outcome_count=len(plan["branches"]
166             ↪ ))
167     if reply["measure_result"]=="near":
168         self.clear(point,c)
169     elif reply["measure_result"]=="direction":
170         if self.anchors[c].rounds!=0:
171             raise ModelMismatch("Shared observation attempted after local contraction began")
172         bound=max(math.dist(point,v) for v in self.regions[c])+SAFE
173         self.anchors[c]=Anchor(tuple(point),reply["svd_deg"],min(1000,bound),0)
174         self.regions[c]=constrain(self.regions[c],self.anchors[c],annular=True)
175         self.record_positive(c,point)
176     elif self.problem==3:
177         raise ModelMismatch("In-range shared observation lost omnidirectional signal")
178     else:
179         self.directions[c].negative(point,self.positives[c])
180     if c in self.anchors:
181         # Execute the continuation after the chosen observation; do not keep
182         # spending shared reads merely because new forecasts look attractive.
183         self.committed_source=c
184     self.stage="search"
185
186 def approach(self,channel):
187     self._belief(channel)
188     self.stage="localize"
189     while channel not in self.cleared:
190         anchor=self.anchors[channel]
191         point,method=self.improved_clear(channel,self.position,self.route_target)
192         if point is not None:
193             self.emit("clear_planned",channel=channel,point=point,method=method)
194             self.clear(point,channel);return
195         plan,decision=optical_decision(self.beliefs[channel],anchor,self.position,
196                                     self.route_target,int(channel!=self.channel))
197         self.emit("optical_bound_comparison",channel=channel,**decision)
198         self.optical_overlap_rejected+=int(decision["reason"]=="bounds_overlap_keep_rf")
199         if plan is not None:
200             self.optical_accepted+=1
201             self.emit("optical_cover_planned",channel=channel,**plan)
202             self.stage="clear";remaining=list(plan["points"]);attempts=0
203             while remaining and attempts<3:
204                 q=remaining.pop(0);attempts+=1;self.optical_probes+=1
205                 reply=self.action("/clear",q,channel)

```

```

205         if reply["clear_result"]=="success":
206             self.finish_clear(channel);return
207         self.failed_clears+=1
208         self.emit("optical_probe_negative",channel=channel,point=q)
209         if remaining:
210             revised=optical_cover(self.beliefs[channel],self.anchors[channel],
211                                   self.position,self.route_target,3-attempts)
212             if revised is not None and revised["worst_time_s"]<chain_cost(remaining,
213                                   ↪ self.position,self.route_target)-1e-5:
214                 remaining=list(revised["points"]);self.optical_failure_replans+=1
215                 self.emit("optical_failure_replanned",channel=channel,**revised)
216             raise ModelMismatch("Certified optical cover exhausted without success")
217         if anchor.rounds>=(5 if self.problem==3 else 6):
218             raise ModelMismatch("Annular round bound exceeded")
219         for q in annular_points(anchor,self.position):
220             reply=self.action("/measure",q,channel)
221             self.source_measures[channel]=self.source_measures.get(channel,0)+1
222             if reply["measure_result"]=="near":
223                 self.clear(q,channel);return
224             if reply["measure_result"]=="direction":
225                 self.anchors[channel]=annular_update(anchor,q,reply["svd_deg"])
226                 self.regions[channel]=constrain(self.regions[channel],self.anchors[channel],
227                                   ↪ annular=True)
228                 self.record_positive(channel,q)
229                 break
230             if self.problem==3:
231                 raise ModelMismatch("Guaranteed omnidirectional measurement lost signal")
232             self.directions[channel].negative(q,self.positives[channel])
233         else:
234             self.anchors[channel]=annular_update(anchor)
235             self.negative_pairs[channel]=self.negative_pairs.get(channel,0)+1
236             self.regions[channel]=constrain(self.regions[channel],self.anchors[channel],annu
237                                   ↪ lar=True)
238             self.beliefs[channel].intersect(self.regions[channel]);self._sync(channel)
239             self.emit("anchor_updated",channel=channel,anchor=asdict(self.anchors[channel]))
240
241     def summary(self):
242         result=super().summary()
243         result.update(strategy=STRATEGY,posterior_cells=self.posterior_stats,
244                       lookahead_decisions=self.lookahead_decisions,shared_by_lookahead=self.shar
245                                   ↪ ed_by_lookahead,
246                       optical_dominance_accepts=self.optical_accepted,
247                       optical_overlap_rejections=self.optical_overlap_rejected,
248                       optical_failure_replans=self.optical_failure_replans)
249
250     return result

```

validation/baseline_v7/adaptive_v6.py

文件行数: 261。

```

1  """v6: residual coverage edits and executable common-endpoint episodes."""
2  from dataclasses import asdict
3  import math
4  import time
5  from adaptive import AdaptiveSolver
6  from solver import ModelMismatch, BudgetStop
7  from strategy import Anchor, annular_points, annular_update
8  from posterior import constrain
9  from planning import open_route
10 from joint_belief import SAFE
11 from refinement import DecisionBelief
12 from decision import terminal_upper, chain_cost
13 from decision_v6 import optical_decision, optical_cover
14 from tail_search import revise_route
15
16 STRATEGY="residual-committed-policy-v6"
17

```

```

18
19 class SolverV6(AdaptiveSolver):
20     def __init__(self,transport,problem,progress=None):
21         super().__init__(transport,problem,progress)
22         self.episode=None;self.episodes=[];self.tail_edits=[]
23         self.last_tail_revision=None;self.tail_proof_calls=0
24         self.refinement_decisions=[]
25
26     def _belief(self,c):
27         if c not in self.beliefs:
28             records=self.raw_evidence[c] or [dict(kind="direction",point=self.anchors[c].point,b
29             ↪ earing=self.anchors[c].bearing)]
30             self.beliefs[c]=DecisionBelief(self.regions[c],self.problem,records)
31             self._sync(c)
32         return self.beliefs[c]
33
34     def refine_actions(self,c,points,target):
35         diagnostic=self._belief(c).refine_for_actions(points,self.position,target)
36         if diagnostic["splits"]:
37             self._sync(c);self.forecast_cache.clear()
38             self.refinement_decisions.append(dict(channel=c,**diagnostic))
39             self.emit("action_refinement",channel=c,**diagnostic)
40
41     def shared_measurements(self,point):
42         # The episode explicitly selects the only optional reading. A scan must
43         # not secretly insert another source or change the committed endpoint.
44         self.stage="search"
45
46     def action(self,path,point,channel):
47         before=self.virtual;start=self.position
48         result=super().action(path,point,channel)
49         if self.episode is not None and self.episode.get("awaiting_endpoint") and
50         ↪ tuple(point)==self.episode["endpoint"]:
51             arrival=before+math.dist(start,point)/5
52             self.close_episode("arrived",arrival-self.episode["start_virtual_s"])
53         return result
54
55     def close_episode(self,status,cost=None):
56         if self.episode is None:
57             return
58         record=dict(self.episode,status=status,actual_to_endpoint_s=cost)
59         if cost is not None:
60             record["upper_minus_actual_s"]=record["selected_upper_s"]-cost
61         self.episodes.append(record);self.emit("episode_finished",**record)
62         self.episode=None
63
64     def search_needed(self):
65         return bool(self.remaining and self.unknown and len(self.discovered)<16)
66
67     def next_route(self):
68         route=open_route(self.position,self.remaining)
69         revision=(len(self.visited),len(self.cleared))
70         # Proof work is bounded independently of scan quota and never creates an
71         # extra request. A new real scan/clear is needed before proposing again.
72         if self.visited and self.unknown and revision!=self.last_tail_revision and
73         ↪ self.tail_proof_calls<16:
74             self.last_tail_revision=revision
75             route,diagnostic=revise_route(self.position,self.visited,route,self.problem,
76             ↪ min(2,16-self.tail_proof_calls))
77             self.tail_proof_calls+=diagnostic["proofs_attempted"]
78             self.tail_edits.append(diagnostic);self.emit("residual_route_revision",**diagnostic)
79             self.remaining=list(route)
80         return route
81
82     def select_episode(self,route):
83         q=route[0];t=route[1] if len(route)>1 else None
84         if not self.anchors:
85             return None

```

```

83     # With only one scan left, finish a source then scan it. This removes
84     # the otherwise ambiguous free endpoint from the comparison entirely.
85     short=sorted(self.anchors,key=lambda c:terminal_upper(self._belief(c),self.anchors[c],se
    ↪ lf.position,self.problem,q)[0])[0:3]
86     plans=[]
87     for c in short:
88         self.refine_actions(c,[q,t],t)
89         b=self._belief(c);a=self.anchors[c]
90         now=terminal_upper(b,a,self.position,self.problem,q)[0]
91         if t is None:
92             plans.append(dict(channel=c,order="finish_then_scan",first_scan=q,
93                               ↪ endpoint=q,selected_upper_s=now+11*len(self.unknown),read=False
    ↪ e))
94         continue
95         # Both policies include the same global batch at q and end at t.
96         # <= 6 per unknown RF + <= 5 for each possible same-point near clear.
97         scan_upper=11*len(self.unknown)
98         now+=math.dist(q,t)/5+scan_upper
99         later=terminal_upper(b,a,q,self.problem,t)[0]
100         forecast=self._forecast(c,q,t)
101         read=forecast is not None and forecast["upper_s"]<later-1e-6
102         if read:
103             later=forecast["upper_s"]
104             later+=math.dist(self.position,q)/5+scan_upper
105             selected=now<=later
106             plan=dict(channel=c,order="finish_then_scan" if selected else "scan_then_finish",
107                       first_scan=q,endpoint=t,now_upper_s=now,later_upper_s=later,
108                       selected_upper_s=min(now,later),read=read and not selected,
109                       scan_upper_s=scan_upper,comparison="same_endpoint_policy_upper_bounds")
110             plans.append(plan);self.lookahead_decisions+=1
111     return min(plans,key=lambda p:(p["selected_upper_s"],p["channel"]))
112
113 def episode_read(self,c,q,target):
114     self.stage="shared_measure"
115     reply=self.action("/measure",q,c)
116     self.optional_counts[c]+=1;self.optional_positions[c].append(tuple(q))
117     self.shared_by_lookahead+=1
118     if reply["measure_result"]=="near":
119         self.clear(q,c)
120     elif reply["measure_result"]=="direction":
121         if self.anchors[c].rounds!=0:
122             raise ModelMismatch("Shared measurement after contraction began")
123         bound=max(math.dist(q,v) for v in self.regions[c])+SAFE
124         self.anchors[c]=Anchor(tuple(q),reply["svd_deg"],min(1000,bound),0)
125         self.regions[c]=constrain(self.regions[c],self.anchors[c],annular=True)
126         self.record_positive(c,q)
127     elif self.problem==3:
128         raise ModelMismatch("Guaranteed shared reception failed")
129     else:
130         self.directions[c].negative(q,self.positives[c])
131     self.stage="search"
132
133 def execute_episode(self,plan):
134     c=plan["channel"];q=plan["first_scan"];t=plan["endpoint"]
135     self.episode=dict(plan,start_virtual_s=self.virtual,awaiting_endpoint=False)
136     self.emit("episode_committed",**self.episode)
137     if plan["order"]=="finish_then_scan":
138         self.route_target=q;self.approach(c)
139         if self.search_needed():
140             self.scan(q)
141     else:
142         self.scan(q)
143         self.route_target=t
144         if plan["read"] and c in self.anchors:
145             self.episode_read(c,q,t)
146         if c in self.anchors:
147             self.approach(c)
148     if t==q:

```

```

149         # Last-stop policy includes the complete batch, not just its arrival.
150         if q in self.visited and tuple(self.position)==tuple(q):
151             self.close_episode("arrived",self.virtual-self.episode["start_virtual_s"])
152         else:
153             self.close_episode("endpoint_cancelled_search_no_longer_needed")
154     elif self.search_needed() and t in self.remaining:
155         self.episode["awaiting_endpoint"]=True
156         self.scan(t) # No source insertion or route rewrite before t.
157     else:
158         self.close_episode("endpoint_cancelled_search_no_longer_needed")
159
160 def improved_clear(self,c,position,target):
161     # Execute the exact feasible terminal alternative used by the bound.
162     bound,q=terminal_upper(self._belief(c),self.anchors[c],position,self.problem,target)
163     return (q,"forecast_feasible_terminal") if q is not None else (None,None)
164
165 def approach(self,channel):
166     self._belief(channel)
167     self.stage="localize"
168     while channel not in self.cleared:
169         self.refine_actions(channel,list(annular_points(self.anchors[channel],self.position)
170         ↪ )+[self.route_target],self.route_target)
171         anchor=self.anchors[channel]
172         point,method=self.improved_clear(channel,self.position,self.route_target)
173         if point is not None:
174             self.emit("clear_planned",channel=channel,point=point,method=method)
175             self.clear(point,channel);return
176         plan,decision=optical_decision(self.beliefs[channel],anchor,self.position,
177         ↪ self.route_target,int(channel!=self.channel))
178         self.emit("optical_bound_comparison",channel=channel,**decision)
179         self.optical_overlap_rejected+=int(decision["reason"]=="bounds_overlap_keep_rf")
180         if plan is not None:
181             self.optical_accepted+=1
182             self.emit("optical_cover_planned",channel=channel,**plan)
183             self.stage="clear";remaining=list(plan["points"]);attempts=0
184             while remaining and attempts<3:
185                 q=remaining.pop(0);attempts+=1;self.optical_probes+=1
186                 reply=self.action("/clear",q,channel)
187                 if reply["clear_result"]=="success":
188                     self.finish_clear(channel);return
189                 self.failed_clears+=1
190                 self.emit("optical_probe_negative",channel=channel,point=q)
191                 if remaining:
192                     revised=optical_cover(self.beliefs[channel],self.anchors[channel],
193                     ↪ self.position,self.route_target,3-attempts)
194                     if revised is not None and revised["worst_time_s"]<chain_cost(remaining,
195                     ↪ self.position,self.route_target)-1e-5:
196                         remaining=list(revised["points"]);self.optical_failure_replans+=1
197                         self.emit("optical_failure_replanned",channel=channel,**revised)
198                     raise ModelMismatch("Certified optical cover exhausted without success")
199             if anchor.rounds>=(5 if self.problem==3 else 6):
200                 raise ModelMismatch("Annular round bound exceeded")
201             for q in annular_points(anchor,self.position):
202                 reply=self.action("/measure",q,channel)
203                 self.source_measures[channel]=self.source_measures.get(channel,0)+1
204                 if reply["measure_result"]=="near":
205                     self.clear(q,channel);return
206                 if reply["measure_result"]=="direction":
207                     self.anchors[channel]=annular_update(anchor,q,reply["svd_deg"])
208                     self.regions[channel]=constrain(self.regions[channel],self.anchors[channel],
209                     ↪ annular=True)
210                     self.record_positive(channel,q)
211                     break
212             if self.problem==3:
213                 raise ModelMismatch("Guaranteed omnidirectional measurement lost signal")
214             self.directions[channel].negative(q,self.positives[channel])
215     else:

```

```

213         self.anchors[channel]=annular_update(anchor)
214         self.negative_pairs[channel]=self.negative_pairs.get(channel,0)+1
215         self.regions[channel]=constrain(self.regions[channel],self.anchors[channel],annu
        ↪ lar=True)
216         self.beliefs[channel].intersect(self.regions[channel]);self._sync(channel)
217         self.emit("anchor_updated",channel=channel,anchor=asdict(self.anchors[channel]))
218
219     def run(self):
220         self.started=time.monotonic();reply=self.io.call("/enter");self.entered=True
221         self.virtual=reply["virtual_time_s"];self.max_virtual=reply["max_virtual_duration_s"]
222         self.deadline=self.started+reply["remaining_real_duration_s"];self.io.deadline=self.deadl
        ↪ ne
223         self.reason="running"
224         try:
225             while self.anchors or self.search_needed():
226                 route=self.next_route() if self.search_needed() else []
227                 if not route:
228                     if self.anchors:
229                         self.route_target=None
230                         self.approach(min(self.anchors,key=lambda c:terminal_upper(self._belief(
        ↪ c),self.anchors[c],self.position,self.problem)[0]))
231                         continue
232                     break
233                 plan=self.select_episode(route)
234                 if plan is None:
235                     self.scan(route[0])
236                 else:
237                     self.execute_episode(plan)
238                 if self.search_needed():
239                     self.certify_unknown()
240             if self.anchors or self.discovered!=self.cleared:
241                 raise ModelMismatch("Uncleared discovered sources remain")
242             if not 10<=len(self.discovered)<=16:
243                 raise ModelMismatch("Completion contradicts source-count interval")
244             if len(self.discovered)==16:
245                 self.completion_basis="source_count_upper_bound"
246             elif not self.remaining:
247                 self.completion_basis="proved_coverage_schedule"
248             elif self.unknown:
249                 raise ModelMismatch("Unresolved unknown channels remain")
250             self.absent.update(self.unknown);self.unknown.clear()
251             self.certificate=True;self.reason="complete_with_certificate"
252         except BudgetStop as exc:
253             self.reason=str(exc);self.close_episode("budget_interrupted")
254         self.exit();return self.summary()
255
256     def summary(self):
257         result=super().summary()
258         result.update(strategy=STRATEGY,route_episodes=self.episodes,tail_route_edits=self.tail_
        ↪ edits,
259                 tail_proof_calls=self.tail_proof_calls,action_refinements=self.refinement_
        ↪ decisions,
260                 pending_route_episode=self.episode)
261         return result

```

validation/baseline_v7/adaptive_v7.py

文件行数: 243。

```

1 """v7: complete-obligation planning, active observations and service accounting."""
2 from dataclasses import asdict
3 import math
4 import time
5 from adaptive_v6 import SolverV6
6 from solver import ModelMismatch,BudgetStop
7 from strategy import Anchor,annular_points,annular_update,annular_clear_point
8 from posterior import constrain,nearest_clear_point

```

```

9 from planning import fallback_bound, open_route
10 from joint_belief import SAFE
11 from service_graph import build_graph, source_modes
12 from interception import choose_interception
13 from coverage_v7 import revise_route, proof_allowance
14 from optical_v7 import optical_decision, optical_cover, conditional_chain
15 from probe_frontier import regional_upper
16
17 STRATEGY='obligation-service-interception-v7'
18
19
20 class SolverV7(SolverV6):
21     def __init__(self, transport, problem, progress=None):
22         super().__init__(transport, problem, progress)
23         self.incumbent=[]; self.incumbent_modes=[]
24         self.global_plans=[]; self.service_receipts=[]; self.information_decisions=[]
25         self.active_information_reads=0; self.stationary_information_reads=0
26         self.max_pending_sources=0; self.first_discovery_times={}
27         self.source_waits={}; self.source_service_start={}
28         self.active_service=None
29         self.regional_probe_count=0
30
31     def discover(self, channel):
32         super().discover(channel)
33         self.first_discovery_times[channel]=self.virtual
34
35     def finish_clear(self, channel):
36         if channel in self.first_discovery_times:
37             start=self.source_service_start.get(channel, self.virtual)
38             self.source_waits[channel]=dict(wait_until_service_s=start-self.first_discovery_time
39             ↪ s[channel],
39             discovery_to_clear_s=self.virtual-self.first_discovery_times[channel])
40         super().finish_clear(channel)
41
42     def next_route(self):
43         route=open_route(self.position, self.remaining)
44         revision=(len(self.visited), len(self.cleared))
45         allowance=proof_allowance(len(self.visited), self.scan_limit, self.tail_proof_calls)
46         if self.visited and self.unknown and revision!=self.last_tail_revision and allowance:
47             self.last_tail_revision=revision
48             route,d=revise_route(self.position, self.visited, route, self.problem, allowance)
49             self.tail_proof_calls+=d['proofs_attempted']; self.tail_edits.append(d)
50             self.emit('residual_route_revision', **d); self.remaining=list(route)
51         return route
52
53     def radio_round(self, c, anchor=None, first=None):
54         a=self.anchors[c] if anchor is None else anchor
55         if a.rounds>=(5 if self.problem==3 else 6) or a.bound<=44.5:
56             raise ModelMismatch('Radio round violates finite contraction conditions')
57         pair=list(annular_points(a, self.position))
58         if first is not None:
59             if min(math.dist(first, p) for p in pair)>1e-7:
60                 raise ModelMismatch('Planned probe does not belong to its certified anchor')
61             pair.sort(key=lambda p: math.dist(p, first))
62         # An older but still valid outer anchor may be retained by the incumbent.
63         # It has the same consumed round count and does not reset any work.
64         self.anchors[c]=a; self.stage='localize'
65         for q in pair:
66             reply=self.action('/measure', q, c)
67             self.source_measures[c]=self.source_measures.get(c, 0)+1
68             if reply['measure_result']=='near':
69                 self.clear(q, c); return
70             if reply['measure_result']=='direction':
71                 self.anchors[c]=annular_update(a, q, reply['svd_deg'])
72                 self.regions[c]=constrain(self.regions[c], self.anchors[c], annular=True)
73                 self.record_positive(c, q); break
74         if self.problem==3:
75             raise ModelMismatch('Guaranteed omnidirectional reception failed')

```

```

76         self.directions[c].negative(q,self.positives[c])
77     else:
78         self.anchors[c]=annular_update(a)
79         self.negative_pairs[c]=self.negative_pairs.get(c,0)+1
80         self.regions[c]=constrain(self.regions[c],self.anchors[c],annular=True)
81         self.beliefs[c].intersect(self.regions[c]);self._sync(c)
82     self.emit('anchor_updated',channel=c,anchor=asdict(self.anchors[c]))
83
84 def optical(self,c,target):
85     plan,d=optical_decision(self._belief(c),self.anchors[c],self.position,target,int(c!=self
86     ↪ .channel))
87     self.emit('optical_bound_comparison',channel=c,**d)
88     self.optical_overlap_rejected+=int(d['reason']=='bounds_overlap_keep_rf')
89     if plan is None:
90         return False
91     self.optical_accepted+=1;self.emit('optical_cover_planned',channel=c,**plan)
92     start=self.virtual;limit=plan['worst_time_s']
93     alone=conditional_chain(self.beliefs[c].polygons(),plan['points'],self.position)['worst_
94     ↪ time_s']
95     self.stage='clear';remaining=list(plan['points']);attempts=0
96     while remaining and attempts<3:
97         q=remaining.pop(0);attempts+=1;self.optical_probes+=1
98         reply=self.action('/clear',q,c)
99         if reply['clear_result']=='success':
100             self.finish_clear(c);return True
101         self.failed_clears+=1;self.emit('optical_probe_negative',channel=c,point=q)
102         if remaining:
103             revised=optical_cover(self.beliefs[c],self.anchors[c],self.position,target,3-att
104             ↪ empts)
105             spent=self.virtual-start
106             if revised is not None:
107                 other=conditional_chain(self.beliefs[c].polygons(),revised['points'],self.po
108                 ↪ sition)['worst_time_s']
109                 if revised['worst_time_s']<=limit-spent-1e-6 and other<=alone-spent-1e-6:
110                     remaining=list(revised['points']);self.optical_failure_replans+=1
111                     self.emit('optical_failure_replanned',channel=c,**revised)
112         raise ModelMismatch('Certified optical chain exhausted')
113
114 def complete_reference(self,c,target):
115     """Execute a continuation bounded by the ordinary annular fallback."""
116     while c in self.anchors:
117         a=self.anchors[c]
118         self.refine_actions(c,list(annular_points(a,self.position))+[target],target)
119         a=self.anchors[c];p=self.position;poly=self._belief(c).hull()
120         switch=int(c!=self.channel)
121         alone=fallback_bound(a,p,self.problem)-1+switch
122         bound=fallback_bound(a,p,self.problem,target,poly)-1+switch if target is not None
123         ↪ else alone
124         points=[nearest_clear_point(poly,p),annular_clear_point(a,p)]
125         if target is not None:points.append(nearest_clear_point(poly,target))
126         candidates=[]
127         for q in points:
128             if q is None:continue
129             local=math.dist(p,q)/5+5
130             total=local+(math.dist(q,target))/5 if target is not None else 0.)
131             if local<=alone+1e-6 and total<=bound+1e-6:
132                 candidates.append((total,q))
133         if candidates:
134             self.clear(min(candidates)[1],c);return
135         if self.optical(c,target):return
136         regional=regional_upper(self._belief(c),a,p,target)
137         if regional is not None and regional[0]<=bound-1e-6 and regional[1]<=alone-1e-6:
138             self.regional_round(c,a,regional[2]['point'],regional[2]['bound'])
139             continue
140         self.radio_round(c)
141
142 def regional_round(self,c,anchor,point,bound):

```

```

138         if bound>.502*anchor.bound-2.4+1e-7 or anchor.rounds>=(5 if self.problem==3 else 6):
139             raise ModelMismatch('Regional probe does not preserve the contraction bound')
140         self.stage='localize';reply=self.action('/measure',point,c)
141         self.source_measures[c]=self.source_measures.get(c,0)+1;self.regional_probe_count+=1
142         self.emit('regional_probe_executed',channel=c,point=point,bound=bound)
143         if reply['measure_result']=='near':self.clear(point,c);return
144         if reply['measure_result']!='direction':raise ModelMismatch('Certified visible regional
        ↪ probe lost signal')
145         self.anchors[c]=Anchor(tuple(point),reply['svd_deg'],bound,anchor.rounds+1)
146         self.regions[c]=constrain(self.regions[c],self.anchors[c],annular=True)
147         self.record_positive(c,point)
148
149     def information_read(self,c,plan,target):
150         point=plan['point'];before=self.position;self.stage='shared_measure'
151         reply=self.action('/measure',point,c)
152         self.optional_counts[c]+=1;self.optional_positions[c].append(tuple(point))
153         stationary=math.dist(before,point)<1e-6
154         self.stationary_information_reads+=int(stationary)
155         self.active_information_reads+=int(not stationary);self.shared_by_lookahead+=1
156         self.emit('information_intercept_executed',channel=c,stationary=stationary,
157                 response=reply['measure_result'],**plan)
158         if reply['measure_result']=='near':self.clear(point,c);return
159         if reply['measure_result']=='direction':
160             if self.anchors[c].rounds!=0:raise ModelMismatch('Information step reset consumed
        ↪ rounds')
161             bound=max(math.dist(point,v) for v in self.regions[c])+SAFE
162             self.anchors[c]=Anchor(tuple(point),reply['svd_deg'],min(1500.,bound),0)
163             self.regions[c]=constrain(self.regions[c],self.anchors[c],annular=True)
164             self.record_positive(c,point)
165         elif self.problem==3:raise ModelMismatch('Guaranteed information point lost reception')
166         else:self.directions[c].negative(point,self.positives[c])
167         self.complete_reference(c,target)
168
169     def execute_source(self,c,mode,target):
170         self.source_service_start[c]=self.virtual
171         self.route_target=target;self.stage='localize'
172         if mode.kind=='clear':
173             self.clear(mode.entry,c);return
174         if self.optical(c,target):return
175         alone=math.dist(self.position,mode.entry)/5+mode.service_s
176         total=alone+(mode.departure(target) if target is not None else 0.)
177         plan,d=choose_interception(self._belief(c),self.anchors[c],self.position,self.problem,ta
        ↪ rget,
178             total,alone,self.optional_counts[c],self.optional_positions[c],int(c!=self.channel))
179         self.information_decisions.append(dict(channel=c,**d,selected=plan))
180         self.emit('information_intercept_decision',channel=c,**d,selected=plan)
181         if plan is not None:
182             self.information_read(c,plan,target);return
183         if mode.kind=='regional_probe':self.regional_round(c,mode.anchor,mode.entry,mode.probe_b
        ↪ ound)
184         else:self.radio_round(c,mode.anchor,mode.first_probe)
185         if c in self.anchors:self.complete_reference(c,target)
186
187     def approach(self,c):
188         modes=source_modes(c,self.anchors[c],self._belief(c).hull(),self.problem,self.position,b
        ↪ elief=self._belief(c))
189         mode=min(modes,key=lambda
        ↪ m:math.dist(self.position,m.entry)/5+m.service_s+(m.departure(self.route_target) if
        ↪ self.route_target is not None else 0.))
190         self.execute_source(c,mode,self.route_target)
191
192     def run(self):
193         self.started=time.monotonic();reply=self.io.call('/enter');self.entered=True
194         self.virtual=reply['virtual_time_s'];self.max_virtual=reply['max_virtual_duration_s']
195         self.deadline=self.started+reply['remaining_real_duration_s'];self.io.deadline=self.dead
        ↪ line
196         self.reason='running'

```

```

197     try:
198         while self.anchors or self.search_needed():
199             route=self.next_route() if self.search_needed() else []
200             if not route and not self.anchors:break
201             self.max_pending_sources=max(self.max_pending_sources,len(self.anchors))
202             graph=build_graph(self,route,self.incumbent_modes)
203             plan=graph.optimize(self.incumbent)
204             self.incumbent=list(plan['order']);self.incumbent_modes=list(plan['modes'])
205             first=plan['modes'][0];target=plan['modes'][1].entry if len(plan['modes'])>1 else
                ↪ None
206             diagnostic={k:v for k,v in plan.items() if k!='modes'}
207             diagnostic.update(pending_sources=len(self.anchors),pending_scans=len(route),
208                             first_kind=first.kind,first_entry=first.entry,
209                             interpretation='bounds_on_current_known_obligations_not_future_hidden_source'
                ↪ s')
210             self.global_plans.append(diagnostic);self.emit('complete_obligation_plan',**diag
                ↪ nostic)
211             start=self.virtual;position=self.position
212             local_bound=math.dist(position,first.entry)/5+first.service_s
213             with_target=local_bound+(first.departure(target) if target is not None else 0.)
214             self.active_service=dict(job=first.job,start_virtual_s=start,local_upper_s=local
                ↪ _bound,
215                                     completion_and_successor_upper_s=with_target,target=tar
                ↪ get)
216             if first.kind=='scan':self.scan(first.entry)
217             else:self.execute_source(first.job[1],first,target)
218             actual=self.virtual-start;remaining_leg=math.dist(self.position,target)/5 if
                ↪ target is not None else 0.
219             receipt=dict(self.active_service,actual_service_s=actual,
220                         feasible_successor_leg_s=remaining_leg,
221                         local_upper_slack_s=local_bound-actual,
222                         completion_and_successor_slack_s=with_target-actual-remaining_leg)
223             self.service_receipts.append(receipt);self.emit('service_completed',**receipt);s
                ↪ elf.active_service=None
224             if min(receipt['local_upper_slack_s'],receipt['completion_and_successor_slack_s']
                ↪ )<-1e-3:
225                 raise ModelMismatch('Executed service exceeded its committed upper bound')
226             if self.search_needed():self.certify_unknown()
227             if self.anchors or self.discovered!=self.cleared:raise ModelMismatch('Uncleared
                ↪ discovered sources remain')
228             if not 10<len(self.discovered)<=16:raise ModelMismatch('Completion contradicts
                ↪ source-count interval')
229             if len(self.discovered)==16:self.completion_basis='source_count_upper_bound'
230             elif not self.remaining:self.completion_basis='proved_coverage_schedule'
231             elif self.unknown:raise ModelMismatch('Unresolved unknown channels remain')
232             self.absent.update(self.unknown);self.unknown.clear();self.certificate=True;self.rea
                ↪ son='complete_with_certificate'
233         except BudgetStop as exc:self.reason=str(exc)
234         self.exit();return self.summary()
235
236     def summary(self):
237         result=super().summary()
238         result.update(strategy=STRATEGY,global_obligation_plans=self.global_plans,
239                     service_receipts=self.service_receipts,information_decisions=self.information_decisi
                ↪ ons,
240                     active_information_reads=self.active_information_reads,stationary_information_reads=
                ↪ self.stationary_information_reads,
241                     max_pending_sources=self.max_pending_sources,source_waits=self.source_waits,pending_
                ↪ service=self.active_service,
242                     regional_probe_count=self.regional_probe_count)
243         return result

```

validation/baseline_v7/cooperative.py

文件行数: 350。

1 """Certified finite v4: compact coverage, joint scheduling, shared observations.

```

2
3 Each source has at most two stationary optional readings, then the original
4 5/6-round completion method or a  $\leq 3$ -disk optical cover. No model simulator.
5 """
6 from dataclasses import asdict
7 import math
8 import time
9 from coverage import design, certify
10 from evidence import disk_outer, outside_disk_hull, DirectionEvidence
11 from planning import open_route, route_length, attachment, optical_plan, fallback_bound
12 from posterior import initial_region, constrain, clear_choice, nearest_clear_point
13 from routing import improve_clear_route, two_leg
14 from solver import Solver, ModelMismatch, BudgetStop
15 from strategy import Anchor, annular_points, annular_update, annular_clear_disk,
   ↪ annular_clear_point, CLEAR_RADIUS
16
17 STRATEGY="certified-cooperative-v4"
18
19
20 class CooperativeSolver(Solver):
21     def __init__(self, transport, problem, progress=None):
22         super().__init__(transport, problem, progress, use_history=True)
23         self.remaining=design(problem)
24         self.scan_limit=len(self.remaining)
25         self.positives={c:[] for c in range(1,21)}
26         self.directions={c:DirectionEvidence() for c in range(1,21)}
27         self.optional_counts={c:0 for c in range(1,21)}
28         self.optional_positions={c:[] for c in range(1,21)}
29         self.optical_probes=self.failed_clears=0
30         self.replacements=0
31         self.absent=set()
32         self.coverage_diagnostics=[]
33         self.completion_basis=None
34         self.route_target=None
35         self.deferred_decisions=0
36         self.joint_clear_batches=0
37         self.stage="search"
38         self.stage_costs={key:dict(travel_m=0.,virtual_time_s=0.,measure_count=0,clear_count=0)
39                             for key in ("search","shared_measure","localize","clear")}
40
41     def budget(self, point):
42         if time.monotonic()>=self.deadline-25:
43             raise BudgetStop("real_time_reserve")
44         if self.virtual+math.dist(self.position,point)/5+6>=self.max_virtual-1:
45             raise BudgetStop("virtual_time_reserve")
46         if self.measures+self.clear_attempts >= (300 if self.problem==3 else 772):
47             raise ModelMismatch("v4 finite action bound exceeded")
48
49     def action(self, path, point, channel):
50         before=self.virtual;travel=self.travel
51         result=super().action(path,point,channel)
52         entry=self.stage_costs[self.stage]
53         entry["virtual_time_s"]+=self.virtual-before
54         entry["travel_m"]+=self.travel-travel
55         entry["measure_count"]+=int(path=="measure")
56         entry["clear_count"]+=int(path=="clear")
57         return result
58
59     def finish_clear(self, channel):
60         self.cleared.add(channel)
61         self.anchors.pop(channel, None);self.regions.pop(channel, None)
62         self.emit("cleared", channel=channel, cleared_count=len(self.cleared))
63
64     def clear(self, point, channel):
65         previous=self.stage;self.stage="clear"
66         reply=self.action("/clear", point, channel)
67         self.stage=previous
68         if reply["clear_result"]!="success":

```

```

69         raise ModelMismatch(f"Certified clear failed for channel {channel}")
70     self.finish_clear(channel)
71
72     def record_positive(self, channel, point):
73         if tuple(point) not in self.positives[channel]:
74             self.positives[channel].append(tuple(point))
75         self.regions[channel]=disk_outer(self.regions[channel], point, 1500)
76         self.apply_negative_history(channel, point)
77         if self.problem==3:
78             for negative in self.search_negatives[channel]:
79                 self.regions[channel]=outside_disk_hull(self.regions[channel], negative)
80         self.refresh_history(channel)
81
82     def scan(self, point):
83         self.stage="search"
84         if len(self.visited)>=self.scan_limit:
85             raise ModelMismatch("Global scan quota exceeded")
86         # A replacement point is appended only after proving its schedule covers D.
87         for channel in sorted(self.unknown, key=lambda c: (c!=self.channel, c)):
88             if len(self.discovered)==16:
89                 break
90             reply=self.action("/measure", point, channel)
91             if reply["measure_result"]=="no_signal":
92                 self.search_negatives[channel].append(tuple(point))
93                 continue
94             self.discover(channel)
95             if reply["measure_result"]=="near":
96                 self.clear(point, channel)
97             else:
98                 self.anchors[channel]=Anchor(tuple(point), reply["svd_deg"])
99                 region=initial_region(self.anchors[channel], annular=True)
100                 self.regions[channel]=disk_outer(region, (0., 0.), 1800, 128)
101                 self.record_positive(channel, point)
102                 self.emit("discovered", channel=channel, bearing=reply["svd_deg"])
103             self.remaining.remove(point); self.visited.append(tuple(point))
104             self.emit("search_point_done", visited=len(self.visited), remaining=len(self.remaining))
105             self.shared_measurements(point)
106
107     def shared_measurements(self, point):
108         self.stage="shared_measure"
109         for channel in sorted(list(self.anchors), key=lambda c: (c!=self.channel, c)):
110             if self.optional_counts[channel]>=2 or tuple(point) in
111                 ↪ self.positives[channel]+self.optional_positions[channel]:
112                 continue
113             polygon=self.regions[channel]
114             bound=max(math.dist(point, v) for v in polygon)+1e-6
115             if bound>=1000-1e-5 or self.clear_plan(channel)[0] is not None:
116                 continue
117             center=tuple(sum(v[i] for v in polygon)/len(polygon) for i in (0, 1))
118             a=(point[0]-center[0], point[1]-center[1]); norm=math.hypot(*a)
119             if norm<1e-6:
120                 continue
121             parallax=0.
122             for old in self.positives[channel]:
123                 b=(old[0]-center[0], old[1]-center[1]); den=norm*math.hypot(*b)
124                 if den>1e-8:
125                     parallax=max(parallax, abs(a[0]*b[1]-a[1]*b[0])/den)
126             if parallax<math.sin(math.radians(15)):
127                 continue
128             evidence=self.directions[channel]
129             if evidence.directional and evidence.minimum_dot(point, polygon)<=1e-5:
130                 continue
131             reply=self.action("/measure", point, channel)
132             self.optional_counts[channel]+=1; self.optional_positions[channel].append(tuple(point))
133             self.emit("shared_measure", channel=channel, result=reply["measure_result"], parallax_s_
134                 ↪ core=parallax)
135             if reply["measure_result"]=="near":
136                 self.clear(point, channel)

```

```

135         elif reply["measure_result"]=="direction":
136             temporary=Anchor(tuple(point),reply["svd_deg"],bound)
137             self.regions[channel]=constrain(polygon,temporary,annular=True)
138             # Keep the old contraction anchor/counter. Extra evidence only tightens it.
139             self.record_positive(channel,point)
140         elif self.problem==3:
141             raise ModelMismatch("In-range shared measurement lost omnidirectional signal")
142         else:
143             evidence.negative(point,self.positives[channel])
144         self.stage="search"
145
146     def improved_clear(self,channel,position,target):
147         point,method=clear_choice(self.anchors[channel],self.regions[channel],position,annular=True)
148         if point is None or target is None:
149             return point,method
150         options=[(point,method)]
151         start=nearest_clear_point(self.regions[channel],position)
152         if start is not None:
153             q,_=improve_clear_route(self.regions[channel],CLEAR_RADIUS-1e-6,start,position,target)
154             options.append((q,"history_route"))
155         disk=annular_clear_disk(self.anchors[channel])
156         if disk is not None:
157             start=annular_clear_point(self.anchors[channel],position)
158             q,_=improve_clear_route([disk[0]],disk[1],start,position,target)
159             options.append((q,"anchor_route"))
160         return min(options,key=lambda item:two_leg(item[0],position,target))
161
162     def approach(self,channel):
163         self.stage="localize"
164         while channel not in self.cleared:
165             anchor=self.anchors[channel]
166             point,method=self.improved_clear(channel,self.position,self.route_target)
167             if point is not None:
168                 self.emit("clear_planned",channel=channel,point=point,method=method)
169                 self.clear(point,channel)
170                 self.history_clears+=int(method.startswith("history"))
171                 return
172             plan=optical_plan(anchor,self.regions[channel],self.position,self.problem,self.route_target)
173             if plan is not None:
174                 self.emit("optical_cover_planned",channel=channel,**plan)
175                 self.stage="clear"
176                 for q in plan["points"]:
177                     self.optical_probes+=1
178                     reply=self.action("/clear",q,channel)
179                     if reply["clear_result"]=="success":
180                         self.finish_clear(channel)
181                         return
182                     self.failed_clears+=1
183                     self.emit("optical_probe_negative",channel=channel,point=q)
184                     raise ModelMismatch("Certified optical cover exhausted without success")
185                 if anchor.rounds>=(5 if self.problem==3 else 6):
186                     raise ModelMismatch("Annular round bound exceeded")
187                 for q in annular_points(anchor,self.position):
188                     reply=self.action("/measure",q,channel)
189                     self.source_measures[channel]=self.source_measures.get(channel,0)+1
190                     if reply["measure_result"]=="near":
191                         self.clear(q,channel);return
192                     if reply["measure_result"]=="direction":
193                         self.anchors[channel]=annular_update(anchor,q,reply["svd_deg"])
194                         self.regions[channel]=constrain(self.regions[channel],self.anchors[channel],
195                                                         annular=True)
196                         self.record_positive(channel,q)
197                         break
198                 if self.problem==3:
199                     raise ModelMismatch("Guaranteed omnidirectional measurement lost signal")

```

```

199         self.directions[channel].negative(q,self.positives[channel])
200     else:
201         self.anchors[channel]=annular_update(anchor)
202         self.negative_pairs[channel]=self.negative_pairs.get(channel,0)+1
203         self.regions[channel]=constrain(self.regions[channel],self.anchors[channel],annu
        ↪ lar=True)
204         self.refresh_history(channel)
205         self.emit("anchor_updated",channel=channel,anchor=asdict(self.anchors[channel]))
206
207     def certify_unknown(self):
208         if not self.unknown:
209             return True
210         # All remaining unknown channels received exactly the same global scans.
211         channel=min(self.unknown)
212         points=self.search_negatives[channel]
213         if any(self.search_negatives[c]!=points for c in self.unknown):
214             raise ModelMismatch("Unknown-channel evidence diverged")
215         result=certify(points,self.problem)
216         self.coverage_diagnostics.append(result)
217         if result["proved"]:
218             self.absent.update(self.unknown);self.unknown.clear()
219             self.completion_basis="continuous_box_certificate"
220             self.emit("unknown_channels_excluded",channels=sorted(self.absent),**result)
221             return True
222         return False
223
224     def replacement_scan(self):
225         if self.replacements>=2 or not self.remaining or not self.unknown or
        ↪ len(self.discovered)>=16:
226             return
227         p=self.position
228         if any(math.dist(p,q)<1e-6 for q in self.visited):
229             return
230         route=open_route(p,self.remaining)
231         for q in sorted(route,key=lambda q:math.dist(p,q))[:2]:
232             without=[v for v in route if v!=q]
233             saving=route_length(p,route)-route_length(p,without)
234             if saving<=30*len(self.unknown):
235                 continue
236             result=certify(self.visited+without+[p],self.problem,max_nodes=4096)
237             if not result["proved"]:
238                 continue
239             self.remaining.remove(q);self.remaining.append(p)
240             self.replacements+=1
241             self.emit("coverage_vertex_replaced",old=q,new=p,route_saving_m=saving,proof=result)
242             self.scan(p)
243             return
244
245     def joint_clear(self,channels,target):
246         """Bounded 2-opt order plus coordinate improvement; no global optimality claim."""
247         candidates={c:self.clear_plan(c)[0] for c in channels}
248         # Pair identity retained even if two channels share the same safe point.
249         order=sorted(channels,key=lambda c:(math.dist(self.position,candidates[c]),c))
250         points=[candidates[c] for c in order]
251         for _ in range(4):
252             old=route_length(self.position,points,target);best=None
253             for i in range(len(order)-1):
254                 for j in range(i+1,len(order)):
255                     trial=points[:i]+list(reversed(points[i:j+1]))+points[j+1:]
256                     value=route_length(self.position,trial,target)
257                     if value<old-1e-7:
258                         old=value;best=(i,j)
259             if best is None:
260                 break
261             i,j=best;order[i:j+1]=reversed(order[i:j+1]);points[i:j+1]=reversed(points[i:j+1])
262         for _ in range(2):
263             for i,c in enumerate(order):
264                 previous=self.position if i==0 else points[i-1]

```

```

265         after=points[i+1] if i+1<len(points) else target
266         q,_=self.improved_clear(c,previous,after)
267         if q is not None:
268             before=math.dist(previous,points[i])+(math.dist(points[i],after) if after is
                ↪ not None else 0)
269             cost=math.dist(previous,q)+(math.dist(q,after) if after is not None else 0)
270             if cost<=before:
271                 points[i]=q
272         self.joint_clear_batches+=1
273         self.emit("joint_clear_planned",channels=order,points=points,target=target)
274         for c,q in zip(order,points):
275             self.clear(q,c)
276
277     def choose_candidates(self,route):
278         """v4 routing policy; v5 overrides this bounded scheduling hook."""
279         candidates=[];ready=[]
280         for c in self.anchors:
281             q,_=self.clear_plan(c)
282             polygon=self.regions[c]
283             estimate=q if q is not None else tuple(sum(v[i] for v in polygon)/len(polygon) for i
                ↪ in (0,1))
284             extra,leg=attachment(self.position,route,estimate)
285             if leg==0:
286                 cost=fallback_bound(self.anchors[c],self.position,self.problem) if q is None else
                ↪ math.dist(self.position,q)/5+5
287                 candidates.append((cost,c))
288                 if q is not None:
289                     ready.append(c)
290         return candidates,ready
291
292     def run(self):
293         self.started=time.monotonic()
294         reply=self.io.call("/enter");self.entered=True
295         self.virtual=reply["virtual_time_s"];self.max_virtual=reply["max_virtual_duration_s"]
296         self.deadline=self.started+reply["remaining_real_duration_s"];self.io.deadline=self.dead
                ↪ line
297         self.reason="running"
298         try:
299             while self.remaining or self.anchors:
300                 search_needed=bool(self.remaining and self.unknown and len(self.discovered)<16)
301                 route=open_route(self.position,self.remaining) if search_needed else []
302                 self.route_target=route[0] if route else None
303                 candidates,ready=self.choose_candidates(route)
304                 if ready:
305                     self.joint_clear(ready,self.route_target)
306                     self.replacement_scan()
307                     continue
308                 if candidates:
309                     self.approach(min(candidates)[1])
310                     self.replacement_scan()
311                     continue
312                 if not search_needed:
313                     break
314                 if self.anchors:
315                     self.deferred_decisions+=1
316                     self.emit("sources_deferred",channels=sorted(self.anchors),next_search=route
                ↪ [0])
317                     self.scan(route[0])
318                     if self.remaining and len(self.discovered)<16:
319                         self.certify_unknown()
320             if self.anchors or self.discovered!=self.cleared:
321                 raise ModelMismatch("Uncleared discovered sources remain")
322             if not 10<=len(self.discovered)<=16:
323                 raise ModelMismatch("Completion contradicts source-count interval")
324             if len(self.discovered)==16:
325                 self.completion_basis="source_count_upper_bound"
326                 self.absent.update(self.unknown);self.unknown.clear()

```

```

327         elif not self.remaining:
328             self.completion_basis="proved_coverage_schedule"
329             self.absent.update(self.unknown);self.unknown.clear()
330         elif self.unknown:
331             raise ModelMismatch("Unresolved unknown channels remain")
332             self.certificate=True;self.reason="complete_with_certificate"
333     except BudgetStop as exc:
334         self.reason=str(exc)
335     self.exit()
336     return self.summary()
337
338 def summary(self):
339     result=super().summary()
340     # These v3 diagnostics do not describe v4's joint schedule.
341     for key in ("route_planned_reduction_m","next_search_committed","history_clear_count"):
342         result.pop(key,None)
343     result.update(strategy=STRATEGY,completion_basis=self.completion_basis,
344                  absent_channels=sorted(self.absent),optional_measure_counts=self.optional_
345                  ↪ counts,
346                  optical_probe_count=self.optical_probes,failed_clear_count=self.failed_cle
347                  ↪ ars,
348                  coverage_replacements=self.replacements,coverage_diagnostics=self.coverage
349                  ↪ _diagnostics,
350                  deferred_decisions=self.deferred_decisions,joint_clear_batches=self.joint_
351                  ↪ clear_batches,
352                  directional_channels=[c for c,e in self.directions.items() if
353                  ↪ e.directional],
354                  stage_costs=self.stage_costs,action_bound=300 if self.problem==3 else 772)
355     return result

```

validation/baseline_v7/coverage.py

文件行数：72。

```

1  """Finite coverage designs and conservative continuous-domain certificates.
2
3  Subdivision proves whole boxes, never treats a point sample as domain coverage.
4  An unfinished proof returns unknown and leaves the guaranteed schedule intact.
5  """
6  import math
7  from geometry import hull, cross
8
9
10 def design(problem):
11     if problem == 3:
12         return [(0.,0.)]+[(1125*math.cos(k*math.pi/3),1125*math.sin(k*math.pi/3)) for k in
13         ↪ range(6)]
14     if problem != 4:
15         raise ValueError("problem must be 3 or 4")
16     inner=[(990*math.cos(k*math.pi/4),990*math.sin(k*math.pi/4)) for k in range(8)]
17     outer=[(1836*math.cos(k*math.pi/8),1836*math.sin(k*math.pi/8)) for k in range(16)]
18     return [(0.,0.)]+inner+[outer[(14+k)%16] for k in range(16)]
19
20 def mesh():
21     points=design(4)
22     a=points[1:9]
23     b=[(1836*math.cos(k*math.pi/8),1836*math.sin(k*math.pi/8)) for k in range(16)]
24     triangles=[]
25     for i in range(8):
26         x,y=a[i],a[(i+1)%8]
27         e,m,f=b[2*i],b[(2*i+1)%16],b[(2*i+2)%16]
28         triangles.extend([(0.,0.),x,y],[x,e,m],[x,m,y],[y,m,f]])
29     return triangles
30
31
32 def contains_strict(polygon,point):

```

```

33     if len(polygon)<3:
34         return False
35     for a,b in zip(polygon,polygon[1:]+polygon[:1]):
36         edge=(b[0]-a[0],b[1]-a[1])
37         if cross(edge,(point[0]-a[0],point[1]-a[1]))<=1e-5*math.hypot(*edge):
38             return False
39     return True
40
41
42 def certify(points,problem,max_nodes=2048,max_depth=13,domain_radius=1800.):
43     """Return proved/unknown, work count and a region still unresolved.
44
45     For q4, common nearby vertices surround all four box corners; convexity
46     then covers the entire box with every source orientation. Positive margins
47     are conservative engineering tolerances, not interval-arithmetic claims.
48     """
49     points=list(dict.fromkeys(tuple(p) for p in points))
50     stack=[(-domain_radius,-domain_radius,domain_radius,domain_radius,0)]
51     count=0
52     while stack and count<max_nodes:
53         x0,y0,x1,y1,depth=stack.pop()
54         count+=1
55         dx=max(x0,0.,-x1);dy=max(y0,0.,-y1)
56         if math.hypot(dx,dy)>domain_radius+1e-5:
57             continue
58         corners=[(x0,y0),(x1,y0),(x1,y1),(x0,y1)]
59         nearby=[q for q in points if max(math.dist(q,v) for v in corners)<1000-1e-5]
60         if problem==3 and nearby:
61             continue
62         if problem==4 and len(nearby)>=3:
63             polygon=hull(nearby)
64             if all(contains_strict(polygon,v) for v in corners):
65                 continue
66         if depth>=max_depth:
67             return dict(proved=False,nodes=count,unresolved_box=[x0,y0,x1,y1])
68         mx,my=(x0+x1)/2,(y0+y1)/2
69         stack.extend([(x0,y0,mx,my,depth+1),(mx,y0,x1,my,depth+1),
70                     (x0,my,mx,y1,depth+1),(mx,my,x1,y1,depth+1)])
71     return dict(proved=not stack,nodes=count,
72                 unresolved_box=list(stack[-1][:4]) if stack else None)

```

validation/baseline_v7/coverage_v7.py

文件行数: 50.

```

1  """Fair deletion/replacement candidates; reserve proof work for later evidence."""
2  import math
3  from coverage import certify
4  from tail_search import residual_regions
5  from planning import open_route,route_length
6
7
8  def proof_allowance(visited,quota,spent):
9      released=min(16,4*(1+4*visited//max(1,quota)))
10     return max(0,min(2,released-spent))
11
12
13 def revise_route(position,visited,remaining,problem,proof_limit=2):
14     route=open_route(position,remaining);old=route_length(position,route)
15     if not route or proof_limit<=0:
16         return route,dict(changed=False,proofs_attempted=0,reason='reserved_for_later')
17     residual=residual_regions(visited,problem)
18     boxes=residual['boxes'];extra=[]
19     if boxes:
20         b=max(boxes,key=lambda b:(b[2]-b[0])*(b[3]-b[1]))
21         extra.append(((b[0]+b[2])/2,(b[1]+b[3])/2))
22     deletions=[];replacements=[]
23     radius=1820. if problem==3 else 1836.

```

```

24     for q in route:
25         without=[p for p in route if p!=q]
26         deletions.append((route_length(position,without),without,q,None))
27         for raw in [position,tuple((position[k]+q[k])/2 for k in (0,1))]+extra:
28             new=tuple(x*min(1.,radius/max(radius,math.hypot(*raw))) for x in raw)
29             if new in route or new in visited:
30                 continue
31             trial=min((without[:i]+[new]+without[i:] for i in range(len(without)+1)),key=lambda
32                 ↪ r:route_length(position,r))
33             replacements.append((route_length(position,trial),trial,q,new))
34     # A guaranteed slot for each operation family prevents the cheapest failed
35     # deletions from starving geometrically feasible relocations indefinitely.
36     slots=[]
37     for kind,family in (('replacement',replacements),('deletion',deletions)):
38         useful=[p for p in family if p[0]<old-1.]
39         if useful:
40             slots.append((kind,min(useful,key=lambda p:p[0])))
41     attempts=[]
42     for kind,(length,trial,q,new) in slots[:proof_limit]:
43         proof=certify(list(visited)+trial,problem,max_nodes=4096)
44         attempts.append(dict(kind=kind,proved=proof['proved'],nodes=proof['nodes'],
45             unresolved_box=proof.get('unresolved_box'),removed=q,inserted=new))
46         if proof['proved']:
47             return trial,dict(changed=True,operation=kind,removed=q,inserted=new,
48                 planned_travel_reduction_m=old-length,proof=proof,
49                 proofs_attempted=len(attempts),attempts=attempts)
50     return route,dict(changed=False,proofs_attempted=len(attempts),attempts=attempts,
51         unresolved_boxes=len(boxes),residual_nodes=residual['nodes'])

```

validation/baseline_v7/decision.py

文件行数：214。

```

1  """Whole-outcome lookahead and optical plans gated by a valid RF lower bound."""
2  from dataclasses import replace
3  import math
4  from geometry import hull
5  from posterior import nearest_clear_point, clip
6  from strategy import annular_clear_point, CLEAR_RADIUS
7  from planning import fallback_bound, rectangle_chain, route_length
8  from joint_belief import EPS, SAFE, TAU, minimum_distance
9
10
11 def terminal_upper(belief,anchor,position,problem,target=None,allow_anchor_disk=True):
12     poly=belief.hull()
13     bound=min(anchor.bound,max(math.dist(anchor.point,v) for v in poly)+SAFE)
14     tightened=replace(anchor,bound=max(5+SAFE,bound))
15     best=fallback_bound(tightened,position,problem,target,poly)
16     choices=[nearest_clear_point(poly,position)]
17     if allow_anchor_disk:
18         choices.append(annular_clear_point(tightened,position))
19     if target is not None:
20         choices.append(nearest_clear_point(poly,target))
21     point=None
22     for q in choices:
23         if q is not None:
24             value=math.dist(position,q)/5+5+(math.dist(q,target)/5 if target is not None else 0)
25             if value<best:
26                 best,value,q
27     return best,point
28
29
30 def output_intervals(poly,q,max_bins=12):
31     if minimum_distance(poly,q)<SAFE:
32         start,width=0.,TAU
33     else:
34         angles=sorted((math.atan2(v[1]-q[1],v[0]-q[0])%TAU) for v in poly)
35         gaps=[(angles[(i+1)%len(angles)]-a)%TAU for i,a in enumerate(angles)]

```

```

36         i=max(range(len(gaps)),key=lambda i:gaps[i])
37         start=angles[(i+1)%len(angles)]-EPS
38         width=TAU-gaps[i]+2*EPS
39         count=min(max_bins,max(1,math.ceil(width/math.radians(4))))
40         return [(start+i*width/count,start+(i+1)*width/count) for i in range(count)]
41
42
43 def measurement_envelope(belief,anchor,q,problem,target=None,measure_cost=6):
44     """A finite upper envelope over ALL near/negative/bearing outcomes.
45
46     Measurements must be in the minimum receive radius of every cell. Output
47     intervals cover the entire possible bearing range; no probability or
48     sampled hidden environment is used. Before local contraction has started,
49     an actual positive reading may install a new <=1000 m anchor. No already
50     consumed contraction round is reset.
51     """
52     poly=belief.hull()
53     if max(math.dist(q,v) for v in poly)>=1000-SAFE:
54         return None
55     branches=[]
56     if belief.outcome(q,"near") is not None:
57         branches.append(dict(kind="near",upper_s=5+(math.dist(q,target)/5 if target is not None
58         ↪ else 0)))
59     negative=belief.outcome(q,"no_signal") if problem==4 else None
60     if negative is not None:
61         value,_=terminal_upper(negative,anchor,q,problem,target)
62         branches.append(dict(kind="no_signal",upper_s=value))
63     for lo,hi in output_intervals(poly,q):
64         posterior=belief.outcome(q,"direction",(lo,hi))
65         if posterior is not None:
66             if anchor.rounds==0:
67                 bound=max(5+SAFE,max(math.dist(q,v) for v in posterior.hull())+SAFE)
68                 temporary=replace(anchor,point=tuple(q),bearing=math.degrees((lo+hi)/2)%360,
69                 bound=min(1500,bound))
70                 # Mid-bin angle is NOT an observed bearing. Only the
71                 # orientation-independent fallback bound may use this anchor.
72                 value,_=terminal_upper(posterior,temporary,q,problem,target,False)
73             else:
74                 value,_=terminal_upper(posterior,anchor,q,problem,target)
75                 branches.append(dict(kind="direction",interval_rad=[lo,hi],upper_s=value))
76     if not branches:
77         raise ArithmeticError("Measurement has no geometrically possible response")
78     return dict(upper_s=measure_cost+max(b["upper_s"] for b in branches),
79               branches=branches,scope="whole_outcome_intervals")
80
81 def rf_lower_bound(belief,position,target=None,switch_cost=0):
82     """Lower bound for EVERY policy that measures this source before clearing.
83
84     Each accepted witness is a complete world satisfying all real observations.
85     Even perfect information cannot avoid reaching a 20 m neighbourhood of it.
86     Witnesses only improve a lower bound; they are never used as a coverage proof.
87     """
88     direct=0. if target is None else math.dist(position,target)
89     travel=direct;witnesses=belief.witnesses()
90     for witness in witnesses:
91         x=witness["point"]
92         lower=max(0.,math.dist(position,x)-20)
93         if target is not None:
94             lower+=max(0.,math.dist(target,x)-20)
95         travel=max(travel,lower)
96     return dict(lower_s=10+switch_cost+travel/5,witness_count=len(witnesses),
97               scope="all_measure_first_single_source_policies_to_same_target")
98
99
100 def chain_cost(points,position,target=None):
101     elapsed=0.;previous=position;worst=0.
102     for i,q in enumerate(points):

```

```

103         elapsed+=math.dist(previous,q)/5
104         worst=max(worst,elapsed+3*i+5+(math.dist(q,target)/5 if target is not None else 0))
105         previous=q
106     return worst
107
108
109 def optical_cover(belief,anchor,position,target=None,max_attempts=3):
110     """Best retained certificate among rectangles and a finite disk-cover search.
111
112     Convex atoms preserve separated posterior components. Every atom must be
113     wholly covered by at least one chosen disk; thus the finite search certifies
114     the entire continuous union, not merely its vertices as unrelated samples.
115     """
116     if max_attempts<1:
117         return None
118     for component in belief.polygons():
119         if any(math.dist(a,b)>2*CLEAR_RADIUS*max_attempts
120                for i,a in enumerate(component) for b in component[i+1:]):
121             # A convex component contains its longest segment. k disks cover
122             # at most 2*k*r length along that segment, irrespective of centres.
123             return None
124     poly=belief.hull();angle=math.radians(anchor.bearing)
125     mean=tuple(sum(v[i] for v in poly)/len(poly) for i in (0,1))
126     xx=sum((v[0]-mean[0])**2 for v in poly);yy=sum((v[1]-mean[1])**2 for v in poly)
127     xy=sum((v[0]-mean[0])*(v[1]-mean[1]) for v in poly)
128     principal=.5*math.atan2(2*xy,xx-yy)
129     best=None;points=[]
130     def consider(chain,certificate):
131         nonlocal best
132         cost=chain_cost(chain,position,target)
133         if best is None or cost<best["worst_time_s"]:
134             best=dict(points=chain,worst_time_s=cost,certificate=certificate)
135     for theta in (angle,angle+math.pi/2,principal):
136         chain=rectangle_chain(poly,theta)
137         if chain is not None and len(chain)<=max_attempts:
138             consider(chain,"rectangle_cover");consider(list(reversed(chain)),"rectangle_cover")
139         points.extend(chain)
140     # Six spatial slabs permit the finite cover search to use different disks
141     # for different portions of a long connected polygon, while retaining holes.
142     nx,ny=math.cos(principal),math.sin(principal)
143     lo=min(nx*v[0]+ny*v[1] for v in poly);hi=max(nx*v[0]+ny*v[1] for v in poly)
144     atoms=[]
145     for component in belief.polygons():
146         pieces=[]
147         for i in range(6):
148             a=lo+(hi-lo)*i/6;b=lo+(hi-lo)*(i+1)/6
149             piece=clip(component,nx,ny,b,-nx,-ny,-a)
150             if piece:
151                 pieces.append(piece)
152         if len(atoms)+len(pieces)<=288:
153             atoms.extend(pieces)
154         else:
155             # Resource cap keeps the unsplit component; no component is dropped.
156             atoms.append(component)
157     for atom in atoms:
158         for p in (position,target):
159             if p is not None:
160                 q=nearest_clear_point(atom,p)
161                 if q is not None:
162                     points.append(q)
163     points=list(dict.fromkeys(tuple(q) for q in points))
164     masks=[]
165     for q in points:
166         mask=sum(1<<i for i,a in enumerate(atoms) if max(math.dist(q,v) for v in
167                    ↪ a)<CLEAR_RADIUS-1e-7)
168         if mask:
169             masks.append((q,mask))
170     # Fixed finite candidate family, explicitly distinct from a global optimum.

```

```

170 masks=sorted(masks,key=lambda
    ↪ qm:(-qm[1].bit_count()/(1+math.dist(position,qm[0])/100),qm[0]))[:14]
171 full=(1<<len(atoms))-1;nodes=0
172 def search(chain,covered,elapsed,worst):
173     nonlocal nodes
174     nodes+=1
175     if covered==full:
176         consider(chain,"continuous_atom_cover")
177         return
178     if len(chain)>=max_attempts:
179         return
180     previous=chain[-1] if chain else position
181     for q,mask in masks:
182         if mask|covered==covered:
183             continue
184             travel=elapsed+math.dist(previous,q)/5
185             value=max(worst,travel+3*len(chain)+5+(math.dist(q,target)/5 if target is not None
    ↪ else 0))
186             if best is not None and value>=best["worst_time_s"]-1e-8:
187                 continue
188             search(chain+[q],covered|mask,travel,value)
189 if masks and full:
190     search([],0,0.,0.)
191 if best is not None:
192     best.update(atom_count=len(atoms),candidate_count=len(masks),search_nodes=nodes)
193 return best
194
195
196 def optical_decision(belief,anchor,position,target=None,switch_cost=0):
197     candidate=optical_cover(belief,anchor,position,target)
198     if candidate is None:
199         return None,dict(accepted=False,reason="no_three_disk_cover",optical_upper_s=None)
200     lower=rf_lower_bound(belief,position,target,switch_cost)
201     diagnostics=dict(*lower,optical_upper_s=None if candidate is None else
    ↪ candidate["worst_time_s"],
202                     accepted=False,reason="no_three_disk_cover")
203     gap=lower["lower_s"]-candidate["worst_time_s"]
204     # Also require dominance for clearing alone: later route changes cannot
205     # invalidate the decision solely by removing the planned outgoing leg.
206     standalone=rf_lower_bound(belief,position,None,switch_cost)
207     standalone_gap=standalone["lower_s"]-chain_cost(candidate["points"],position)
208     gap=min(gap,standalone_gap)
209     accepted=gap>1e-4
210     diagnostics.update(accepted=accepted,dominance_gap_s=gap,
211                       standalone_rf_lower_s=standalone["lower_s"],
212                       standalone_optical_upper_s=chain_cost(candidate["points"],position),
213                       reason="upper_below_rf_lower" if accepted else "bounds_overlap_keep_rf")
214     return (candidate if accepted else None),diagnostics

```

validation/baseline_v7/decision_v6.py

文件行数: 184。

```

1 """Stronger witness bounds and continuous refinement of certified disk covers."""
2 import math
3 from geometry import hull
4 from posterior import clip, nearest_clear_point
5 from routing import improve_clear_route
6 from strategy import CLEAR_RADIUS
7 from decision import optical_cover as finite_cover, chain_cost
8
9
10 def disk_visit_lower(position,x,target=None,radius=20.):
11     """Convex supporting-plane dual bounds, never a numerical primal minimum.
12
13     Every iterate z gives  $f(z)+g.(x-z)-r*||g|| \leq \min_{\{B(x,r)\}} f.$ 
14     Retaining the largest valid lower bound makes early termination harmless.
15     """

```

```

16     if target is None:
17         return max(0.,math.dist(position,x)-radius)
18     lower=max(math.dist(position,target),math.dist(position,x)+math.dist(target,x)-2*radius)
19     z=x
20     for iteration in range(24):
21         d1=math.dist(z,position);d2=math.dist(z,target)
22         g=tuple((z[i]-position[i])/d1 if d1>1e-12 else 0. for i in (0,1))
23         g=tuple(g[i]+((z[i]-target[i])/d2 if d2>1e-12 else 0.) for i in (0,1))
24         norm=math.hypot(*g)
25         dual=d1+d2+sum(g[i]*(x[i]-z[i]) for i in (0,1))-radius*norm
26         lower=max(lower,dual-1e-6)
27         if norm<1e-12:
28             break
29         vertex=tuple(x[i]-radius*g[i]/norm for i in (0,1))
30         # Feasible Frank-Wolfe iterates; convergence is not needed for validity.
31         step=2/(iteration+2)
32         z=tuple((1-step)*z[i]+step*vertex[i] for i in (0,1))
33     return max(0.,lower)
34
35
36 def cost_witnesses(belief,position,target=None,limit=12):
37     seeds=belief.witnesses(limit)
38     found={tuple(w["point"]):w for w in seeds}
39     def objective(p):
40         return disk_visit_lower(position,p,target)
41     # Cost-directed candidates are spread across all components. Each proposed
42     # world is rechecked against raw observations; a polygon alone is not proof.
43     for poly in belief.polygons():
44         center=tuple(sum(v[i] for v in poly)/len(poly) for i in (0,1))
45         extreme=max(poly,key=objective)
46         for alpha in (1.,.999,.9,.5):
47             p=tuple(alpha*extreme[i]+(1-alpha)*center[i] for i in (0,1))
48             if (w:=belief.witness(p)) is not None:
49                 found[p]=w
50     ranked=sorted(found.values(),key=lambda w:objective(w["point"]),reverse=True)[:4]
51     vertices=[v for p in belief.polygons() for v in p]
52     for w in ranked:
53         p=w["point"];direction=max(vertices,key=objective)
54         for alpha in (.5,.75,.875,.9375,.96875,.984375,.9921875):
55             candidate=tuple((1-alpha)*p[i]+alpha*direction[i] for i in (0,1))
56             if objective(candidate)>objective(p) and (new:=belief.witness(candidate)) is not None:
57                 found[candidate]=new
58     return sorted(found.values(),key=lambda w:objective(w["point"]),reverse=True)[:limit]
59
60
61 def rf_lower_bound(belief,position,target=None,switch_cost=0):
62     witnesses=cost_witnesses(belief,position,target)
63     lower=0. if target is None else math.dist(position,target)
64     lower=max([lower]+[disk_visit_lower(position,w["point"],target) for w in witnesses])
65     return dict(lower_s=10+switch_cost+lower/5,witness_count=len(witnesses),
66                 scope="all_measure_first_single_source_policies_to_same_target",
67                 lower_method="validated_cost_witnesses_and_disk_supporting_planes")
68
69
70 def voronoi_atoms(polygons,centers):
71     groups=[] for _ in centers]
72     for poly in polygons:
73         for i,q in enumerate(centers):
74             part=poly
75             for j,r in enumerate(centers):
76                 if i==j:
77                     continue
78                 nx,ny=r[0]-q[0],r[1]-q[1]
79                 part=clip(part,nx,ny,(r[0]**2+r[1]**2-q[0]**2-q[1]**2)/2)
80             if part:
81                 groups[i].extend(part)
82     return groups
83

```

```

84
85 def refine_centers(polygons,centers,position,target=None):
86     """Continuous feasible-coordinate improvement of the max-prefix cost.
87
88     Voronoi pieces partition every continuous polygon. Each retained piece is
89     entirely in its assigned disk, before and after each centre modification.
90     """
91     centers=list(centers);groups=voronoi_atoms(polygons,centers)
92     safety=CLEAR_RADIUS-1e-6
93     if any(not points or max(math.dist(q,v) for v in points)>safety
94         for q,points in zip(centers,groups)):
95         return centers,0
96     improvement=0.
97     for _ in range(3):
98         changed=False
99         for i,points in enumerate(groups):
100             poly=hull(points);old=centers[i]
101             previous=position if i==0 else centers[i-1]
102             after=target if i+1==len(centers) else centers[i+1]
103             candidates=[nearest_clear_point(poly,p) for p in (position,previous,target,after) if p
104                 ↪ is not None]
105             if after is not None:
106                 q,_=improve_clear_route(poly,safety,old,previous,after)
107                 candidates.append(q)
108             before=chain_cost(centers,position,target)
109             best=(before,old)
110             for q in candidates:
111                 if q is None or max(math.dist(q,v) for v in points)>safety:
112                     continue
113                 # Full max-prefix objective, not just adjacent-edge length.
114                 for alpha in (1.,.5,.25):
115                     trial=tuple(old[k]+alpha*(q[k]-old[k]) for k in (0,1))
116                     seq=centers[:i]+[trial]+centers[i+1:]
117                     cost=chain_cost(seq,position,target)
118                     if cost<best[0]-1e-7:
119                         best=(cost,trial)
120             if best[0]<before:
121                 centers[i]=best[1];improvement+=before-best[0];changed=True
122         if not changed:
123             break
124     return centers,improvement
125
126 def optical_cover(belief,anchor,position,target=None,max_attempts=3):
127     if max_attempts<1 or any(any(math.dist(a,b)>2*CLEAR_RADIUS*max_attempts
128         for i,a in enumerate(p) for b in p[i+1:]) for p in belief.polygons()):
129         return None
130     baseline=finitive_cover(belief,anchor,position,target,max_attempts)
131     candidates=[] if baseline is None else [baseline]
132     polygons=belief.polygons();poly=belief.hull()
133     # Additional analytic partitions change the cover itself, rather than only
134     # moving one of the old finite candidate points. All pieces are preserved.
135     mean=tuple(sum(v[i] for v in poly)/len(poly) for i in (0,1))
136     xx=sum((v[0]-mean[0])**2 for v in poly);yy=sum((v[1]-mean[1])**2 for v in poly)
137     xy=sum((v[0]-mean[0])*(v[1]-mean[1]) for v in poly)
138     principal=.5*math.atan2(2*xy,xx-yy)
139     for angle in (principal,math.radians(anchor.bearing)):
140         nx,ny=math.cos(angle),math.sin(angle)
141         lo=min(nx*v[0]+ny*v[1] for v in poly);hi=max(nx*v[0]+ny*v[1] for v in poly)
142         for count in range(2,max_attempts+1):
143             chain=[]
144             for i in range(count):
145                 a=lo+(hi-lo)*i/count;b=lo+(hi-lo)*(i+1)/count
146                 vertices=[]
147                 for p in polygons:
148                     vertices.extend(clip(clip(p,nx,ny,b),-nx,-ny,-a))
149                 if not vertices:
150                     continue

```

```

151         q=nearest_clear_point(hull(vertices),position if not chain else chain[-1])
152         if q is None:
153             break
154         chain.append(q)
155     else:
156         for seq in (chain,list(reversed(chain))):
157             if seq:
158                 candidates.append(dict(points=seq,worst_time_s=chain_cost(seq,position,t)
159                                     ↪ target),certificate="continuous_slab_cover"))
159     if not candidates:
160         return None
161     candidates.sort(key=lambda c:c["worst_time_s"])
162     best=candidates[0]
163     for candidate in candidates[:4]:
164         centers,gain=refine_centers(polygons,candidate["points"],position,target)
165         cost=chain_cost(centers,position,target)
166         if cost<best["worst_time_s"]-1e-7:
167             best=dict(points=centers,worst_time_s=cost,certificate="continuous_voronoi_piece_cov
168                 ↪ er",
169                 continuous_improvement_s=gain)
170     return best
171
172 def optical_decision(belief,anchor,position,target=None,switch_cost=0):
173     candidate=optical_cover(belief,anchor,position,target)
174     if candidate is None:
175         return None,dict(accepted=False,reason="no_three_disk_cover",optical_upper_s=None)
176     lower=rf_lower_bound(belief,position,target,switch_cost)
177     standalone=lower if target is None else rf_lower_bound(belief,position,None,switch_cost)
178     standalone_upper=chain_cost(candidate["points"],position)
179     gap=min(lower["lower_s"]-candidate["worst_time_s"],standalone["lower_s"]-standalone_upper)
180     accepted=gap>1e-4
181     diagnostic=dict(**lower,accepted=accepted,optical_upper_s=candidate["worst_time_s"],
182                   standalone_rf_lower_s=standalone["lower_s"],standalone_optical_upper_s=stand
183                   ↪ alone_upper,
184                   dominance_gap_s=gap,reason="upper_below_rf_lower" if accepted else
185                   ↪ "bounds_overlap_keep_rf")
186     return (candidate if accepted else None),diagnostic

```

validation/baseline_v7/evidence.py

文件行数：83。

```

1  """Conservative circular refinement and independent orientation constraints."""
2  import math
3  from geometry import hull
4  from posterior import clip
5
6
7  def disk_outer(polygon,center,radius,sides=32):
8      for i in range(sides):
9          t=2*math.pi*i/sides;nx,ny=math.cos(t),math.sin(t)
10         polygon=clip(polygon,nx,ny,nx*center[0]+ny*center[1]+radius)
11     if not polygon:
12         raise ArithmeticError("Circular evidence is inconsistent")
13     return polygon
14
15
16 def outside_disk_hull(polygon,center,radius=1000-1e-5):
17     """Outer convex hull after removing an open disk.
18
19     Extreme points are surviving polygon vertices or edge/circle intersections;
20     an interior concave circular boundary cannot expose a new hull extremum.
21     The shrunk exclusion radius leaves a margin around strict negative evidence.
22     """
23     points=[p for p in polygon if math.dist(p,center)>=radius]
24     if len(polygon)==1:

```

```

25         if not points:
26             raise ArithmeticError("Negative distance contradicts source region")
27         return points
28     for a,b in zip(polygon,polygon[1:]+polygon[:1]):
29         dx,dy=b[0]-a[0],b[1]-a[1]
30         ux,uy=a[0]-center[0],a[1]-center[1]
31         aa=dx*dx+dy*dy
32         if aa<=1e-24:
33             continue
34         bb=2*(ux*dx+uy*dy);cc=ux*ux+uy*uy-radius*radius
35         disc=bb*bb-4*aa*cc
36         if disc<0:
37             continue
38         root=math.sqrt(disc)
39         for t in ((-bb-root)/(2*aa),(-bb+root)/(2*aa)):
40             if 0<=t<=1:
41                 points.append((a[0]+t*dx,a[1]+t*dy))
42     if not points:
43         raise ArithmeticError("Negative distance contradicts source region")
44     return hull(points)
45
46
47 class DirectionEvidence:
48     def __init__(self):
49         self.arcs=[(0.,2*math.pi)]
50         self.directional=False
51
52     def negative(self,q,positives):
53         self.directional=True
54         for p in positives:
55             x,y=p[0]-q[0],p[1]-q[1]
56             if math.hypot(x,y)<1e-8:
57                 raise ArithmeticError("Same-point positive and in-range negative")
58             center=math.atan2(y,x)
59             allowed=[]
60             # Relax strict dot>0 outward in angle. Never exclude a true direction.
61             for k in range(-2,3):
62                 lo=center-math.pi/2-1e-10+k*2*math.pi
63                 hi=center+math.pi/2+1e-10+k*2*math.pi
64                 for a,b in self.arcs:
65                     if max(a,lo)<=min(b,hi):
66                         allowed.append((max(a,lo),min(b,hi)))
67             self.arcs=allowed
68             if not self.arcs:
69                 raise ArithmeticError("Directional evidence is inconsistent")
70
71     def minimum_dot(self,point,polygon):
72         values=[]
73         for vertex in polygon:
74             x,y=point[0]-vertex[0],point[1]-vertex[1]
75             minimum=math.atan2(y,x)+math.pi
76             for lo,hi in self.arcs:
77                 candidates=[lo,hi]
78                 for k in range(-2,3):
79                     t=minimum+k*2*math.pi
80                     if lo<=t<=hi:
81                         candidates.append(t)
82             values.extend(x*math.cos(t)+y*math.sin(t) for t in candidates)
83     return min(values)

```

validation/baseline_v7/geometry.py

文件行数: 100。

```

1 """Bearing-wedge intersection without an artificial bounding box."""
2 import math
3 from itertools import combinations
4

```

```

5
6 def cross(a, b):
7     return a[0]*b[1] - a[1]*b[0]
8
9
10 def hull(points):
11     points = sorted(set(points))
12     if len(points) <= 1:
13         return points
14     def chain(seq):
15         out = []
16         for p in seq:
17             while len(out) >= 2 and cross((out[-1][0]-out[-2][0], out[-1][1]-out[-2][1]),
18                 (p[0]-out[-1][0], p[1]-out[-1][1])) <= 0:
19                 out.pop()
20             out.append(p)
21         return out
22     return chain(points)[-1] + chain(points[::-1])[:-1]
23
24
25 def farthest_pair(vertices):
26     """Rotating calipers on a counterclockwise convex hull."""
27     n = len(vertices)
28     if n == 1:
29         return vertices[0], vertices[0]
30     if n == 2:
31         return tuple(vertices)
32     best = (vertices[0], vertices[1])
33     best_d = math.dist(*best)
34     j = 1
35     for i in range(n):
36         nxt = (i+1) % n
37         edge = (vertices[nxt][0]-vertices[i][0], vertices[nxt][1]-vertices[i][1])
38         def area(k):
39             return abs(cross(edge, (vertices[k][0]-vertices[i][0],
40                 ↪ vertices[k][1]-vertices[i][1])))
41         steps = 0
42         while area((j+1) % n) > area(j) and steps < n:
43             j = (j+1) % n
44             steps += 1
45         for a in (i, nxt):
46             for b in (j, (j+1) % n):
47                 d = math.dist(vertices[a], vertices[b])
48                 if d > best_d:
49                     best, best_d = (vertices[a], vertices[b]), d
50     return best
51
52 def solve_wedges(observations, epsilon_deg=1.005):
53     """Each observation is {x,y,bearing_deg}; all units are metres/degrees.
54
55     Boundary intersections plus feasibility filtering cost  $O(n^3)$ . This small-n
56     implementation prioritizes explicit degeneracy states over hidden clipping.
57     Floating-point results are numerical estimates, not exact certificates.
58     """
59     if not math.isfinite(epsilon_deg) or not 0 < epsilon_deg < 90:
60         raise ValueError("epsilon_deg must lie in (0, 90)")
61     planes = []
62     for obs in observations:
63         x, y, angle = (float(obs[k]) for k in ("x", "y", "bearing_deg"))
64         if not all(math.isfinite(v) for v in (x, y, angle)):
65             raise ValueError("Observations must be finite")
66         lo, hi = math.radians(angle-epsilon_deg), math.radians(angle+epsilon_deg)
67         for nx, ny in ((math.sin(lo), -math.cos(lo)), (-math.sin(hi), math.cos(hi))):
68             planes.append((nx, ny, nx*x+ny*y))
69     base = dict(epsilon_deg=epsilon_deg, diameter=None, vertices=[],
70         numerical_method="float64; relative feasibility tolerance 1e-10")
71     if not planes:

```

```

72     return dict(base, state="UNBOUNDED")
73 def feasible(x, y):
74     return all(a*x+b*y-c <= 1e-10*(1+abs(a*x)+abs(b*y)+abs(c)) for a,b,c in planes)
75 candidates = []
76 for (a,b,c), (d,e,f) in combinations(planes, 2):
77     det = a*e-b*d
78     if abs(det) <= 1e-14:
79         continue
80     x, y = (c*e-b*f)/det, (a*f-c*d)/det
81     if not math.isfinite(x) or not math.isfinite(y):
82         return dict(base, state="NUMERICALLY_UNRESOLVED")
83     if feasible(x, y):
84         candidates.append((x,y))
85 if not candidates:
86     return dict(base, state="EMPTY_OR_NUMERICALLY_UNRESOLVED")
87 # A nonzero recession ray makes the intersection unbounded.
88 for a,b,_ in planes:
89     for dx,dy in ((b,-a),(-b,a)):
90         if all(u*dx+v*dy <= 1e-14 for u,v,_ in planes):
91             return dict(base, state="UNBOUNDED", feasible_point=candidates[0])
92 vertices = hull(candidates)
93 p,q = farthest_pair(vertices)
94 diameter = math.dist(p,q)
95 center = ((p[0]+q[0])/2, (p[1]+q[1])/2)
96 radius = diameter/2
97 return dict(base, state={1:"POINT",2:"SEGMENT"}.get(len(vertices),"POLYGON"),
98             vertices=vertices, diameter=diameter, farthest_pair=(p,q),
99             diameter_circle_center=center, diameter_circle_radius=radius,
100             diameter_circle_covers=all(math.dist(v,center) <= radius+1e-9*(1+radius) for v in
    ↪ vertices))

```

validation/baseline_v7/interception.py

文件行数: 107。

```

1  """Actively choose a useful observation location with a whole-response bound."""
2  from dataclasses import replace
3  import math
4  from posterior import nearest_clear_point
5  from strategy import offset
6  from joint_belief import definitely_in_range, SAFE
7  from decision import output_intervals
8  from planning import fallback_bound
9
10
11 def reception_guaranteed(belief,point):
12     return all(definitely_in_range(poly,point,belief.positives()) for poly in belief.polygons())
13
14
15 def observation_candidates(belief,anchor,position,target=None,domain_radius=1836.):
16     poly=belief.hull();points=[tuple(position)]
17     # Scale a general receive-disk intersection into the existing exact finite
18     # disk-projection routine. This finds the closest guaranteed reception point.
19     scale=19.8/999.9
20     q=nearest_clear_point([(x*scale,y*scale) for x,y in poly],tuple(x*scale for x in position))
21     if q is not None:
22         points.append(tuple(x/scale for x in q))
23     points.extend(offset(anchor,.5,side) for side in (-.15,.15))
24     wide=[offset(anchor,.443,.405),offset(anchor,.443,-.405)]
25     points.append(min(wide,key=lambda q:math.dist(position,q)+(math.dist(q,target) if target else
    ↪ 0)))
26     accepted=[]
27     for q in dict.fromkeys(points):
28         if math.hypot(*q)>domain_radius or tuple(q) in [tuple(r['point']) for r in belief.records
    ↪ if r['kind']=='direction']:
29             continue
30         if reception_guaranteed(belief,q):
31             accepted.append(q)

```

```

32     return accepted
33
34
35 def observation_envelope(belief, anchor, point, problem, target=None, max_bins=24, measure_cost=6):
36     if anchor.rounds!=0 or not reception_guaranteed(belief, point):
37         return None
38     def completion(posterior, kind, interval=None):
39         if kind=='near':
40             return 5+(math.dist(point, target)/5 if target else 0), 5.
41         temporary=anchor
42         if kind=='direction':
43             bound=min(1500., max(5+SAFE, max(math.dist(point, v) for v in posterior.hull())+SAFE))
44             temporary=replace(anchor, point=tuple(point), bound=bound, bearing=0.)
45             # One executable continuation must satisfy BOTH budgets. Separately
46             # minimizing target and standalone policies would not establish this.
47             # Actual complete_reference only makes two-budget-safe substitutions.
48             alone=fallback_bound(temporary, point, problem)-1
49             value=fallback_bound(temporary, point, problem, target, posterior.hull())-1 if target is not
               ↪ None else alone
50             return value, alone
51     branches=[]
52     for kind in ('near', 'no_signal'):
53         if kind=='no_signal' and problem==3:
54             continue
55         posterior=belief.outcome(point, kind)
56         if posterior is not None:
57             value, alone=completion(posterior, kind)
58             branches.append(dict(kind=kind, upper_s=value, standalone_s=alone))
59     def normal(lo, hi):
60         posterior=belief.outcome(point, 'direction', (lo, hi))
61         if posterior is None:
62             return None
63         value, alone=completion(posterior, 'direction', (lo, hi))
64         return dict(kind='direction', interval=(lo, hi), upper_s=value, standalone_s=alone)
65     normals=[b for lo, hi in output_intervals(belief.hull(), point, max_bins=4) if
               ↪ (b:=normal(lo, hi)) is not None]
66     leaves=len(normals); splits=0
67     while normals and leaves<max_bins:
68         largest=max(range(len(normals)), key=lambda
               ↪ i: max(normals[i]['upper_s'], normals[i]['standalone_s']))
69         b=normals[largest]; lo, hi=b['interval']
70         if hi-lo<math.radians(.1):
71             break
72         other=max((max(v['upper_s'], v['standalone_s']) for v in branches), default=0.)
73         if max(b['upper_s'], b['standalone_s'])<=other:
74             break
75         mid=(lo+hi)/2
76         children=[child for a, z in ((lo, mid), (mid, hi)) if (child:=normal(a, z)) is not None]
77         normals[largest:largest+1]=children; leaves+=1; splits+=1
78     all_branches=branches+normals
79     if not all_branches:
80         raise ArithmeticError('No possible response at proposed observation')
81     return dict(upper_s=measure_cost+max(b['upper_s'] for b in all_branches),
82               standalone_s=measure_cost+max(b['standalone_s'] for b in all_branches),
83               normal_bins=len(normals), interval_splits=splits, branches=all_branches,
84               range_basis='minimum_radius_or_same_source_positive_distance')
85
86
87 def choose_interception(belief, anchor, position, problem, target, baseline_upper, baseline_standalone,
88                         used=0, previous=(), channel_switch=1):
89     if used>=2 or anchor.rounds!=0:
90         return None, dict(reason='optional_budget_or_local_round', candidates=0)
91     best=None; count=0
92     for q in observation_candidates(belief, anchor, position, target, 1820. if problem==3 else 1836.):
93         if tuple(q) in previous:
94             continue
95         count+=1

```

```

96     envelope=observation_envelope(belief,anchor,q,problem,target,measure_cost=5+channel_swit
↪ ch)
97     if envelope is None:
98         continue
99     move=math.dist(position,q)/5
100     upper=move+envelope['upper_s'];alone=move+envelope['standalone_s']
101     if upper<baseline_upper-1. and alone<baseline_standalone-1.:
102         if best is None or upper<best['upper_s']:
103             best=dict(point=q,upper_s=upper,standalone_s=alone,
104                 baseline_upper_s=baseline_upper,baseline_standalone_s=baseline_standal
↪ one,
105                 normal_bins=envelope['normal_bins'],interval_splits=envelope['interval
↪ _splits'],
106                 range_basis=envelope['range_basis'])
107     return best,dict(reason='accepted' if best else
↪ 'no_certified_upper_improvement',candidates=count)

```

validation/baseline_v7/joint_belief.py

文件行数: 342。

```

1  """Finite outer unions of position polygons coupled to direction intervals.
2
3  All real observations are retained. A refinement that exceeds the cell budget
4  keeps its parent; it never discards a possible branch to meet a resource limit.
5  """
6  from dataclasses import dataclass, replace
7  import math
8  from geometry import hull, cross
9  from posterior import clip
10
11  TAU=2*math.pi
12  EPS=math.radians(1.005)
13  SAFE=2e-5
14
15
16  def safe_disk_outer(polygon,q,radius):
17      for i in range(32):
18          t=i*TAU/32;nx,ny=math.cos(t),math.sin(t)
19          polygon=clip(polygon,nx,ny,nx*q[0]+ny*q[1]+radius)
20          if not polygon:
21              break
22      return polygon
23
24
25  def polygon_intersection(a,b):
26      if len(b)<3:
27          # Degenerate outer regions are rare; retaining a is conservative.
28          return a
29      for p,q in zip(b,b[1:]+b[:1]):
30          dx,dy=q[0]-p[0],q[1]-p[1]
31          length=math.hypot(dx,dy)
32          if length>1e-12:
33              a=clip(a,dy/length,-dx/length,(dy*p[0]-dx*p[1])/length)
34      return a
35
36
37  def minimum_distance(polygon,point):
38      if not polygon:
39          return float("inf")
40      # Clipping can leave almost identical consecutive vertices. A tiny edge
41      # must not turn an interior point into a false "outside" classification.
42      if len(polygon)>=3 and all(math.dist(a,b)<1e-10 or
43          cross((b[0]-a[0],b[1]-a[1]),(point[0]-a[0],point[1]-a[1]))
44          >=-1e-8*math.dist(a,b) for a,b in zip(polygon,polygon[1:]+polygon[:1])):
45          return 0.
46      best=min(math.dist(point,p) for p in polygon)
47      for a,b in zip(polygon,polygon[1:]+polygon[:1]):

```

```

48     dx,dy=b[0]-a[0],b[1]-a[1];den=dx*dx+dy*dy
49     if den>1e-24:
50         t=max(0.,min(1.,((point[0]-a[0])*dx+(point[1]-a[1])*dy)/den))
51         best=min(best,math.dist(point,(a[0]+t*dx,a[1]+t*dy)))
52     return best
53
54
55 def subtract_disk(polygon,point,radius):
56     """Outer union of P minus a disk, using an inscribed 32-gon.
57
58     The removed polygon lies strictly inside the physical disk. Keeping its
59     thin circular rim is conservative; unlike a convex hull, no entire hole
60     is filled back in. Each returned piece is convex and ordered.
61     """
62     if not polygon:
63         return []
64     if max(math.dist(v,point) for v in polygon)<radius-SAFE:
65         return []
66     if minimum_distance(polygon,point)>=radius:
67         return [polygon]
68     bound=(radius-SAFE)*math.cos(math.pi/32)
69     inside=polygon;pieces=[]
70     for i in range(32):
71         t=(2*i+1)*math.pi/32;nx,ny=math.cos(t),math.sin(t)
72         b=nx*point[0]+ny*point[1]+bound
73         outside=clip(inside,-nx,-ny,-b)
74         if outside:
75             pieces.append(outside)
76             inside=clip(inside,nx,ny,b)
77         if not inside:
78             break
79     return pieces
80
81
82 def bearing_clip(polygon,q,lo,hi):
83     """A whole output-angle interval, expanded by the measurement error."""
84     lo-=EPS;hi+=EPS
85     if hi-lo>=math.pi:
86         return polygon
87     for nx,ny in ((math.sin(lo),-math.cos(lo)),(-math.sin(hi),math.cos(hi))):
88         polygon=clip(polygon,nx,ny,nx*q[0]+ny*q[1])
89     mid=(lo+hi)/2;nx,ny=-math.cos(mid),-math.sin(mid)
90     return clip(polygon,nx,ny,nx*q[0]+ny*q[1]-5*math.cos((hi-lo)/2))
91
92
93 @dataclass
94 class Cell:
95     polygon: list
96     # None denotes an omnidirectional hypothesis, not an unknown direction.
97     arc: tuple | None=None
98
99
100 def angle_plane(cell,q,visible):
101     if cell.arc is None:
102         return cell.polygon if visible else []
103     lo,hi=cell.arc;mid=(lo+hi)/2
104     nx,ny=math.cos(mid),math.sin(mid)
105     # |n(theta)-n(mid)| <= 2 sin((hi-lo)/4), uniformly on this interval.
106     radius=max(math.dist(q,v) for v in cell.polygon)
107     margin=2*math.sin((hi-lo)/4)*radius+SAFE
108     if visible:
109         return clip(cell.polygon,nx,ny,nx*q[0]+ny*q[1]+margin)
110     return clip(cell.polygon,-nx,-ny,-nx*q[0]-ny*q[1]+margin)
111
112
113 def definitely_in_range(polygon,q,positives):
114     if max(math.dist(q,v) for v in polygon)<1000-SAFE:
115         return True

```

```

116     # R is also at least each previously observed positive distance.
117     return any((min(math.dist(v,p)**2-math.dist(v,q)**2 for v in polygon)>SAFE
118                 for p in positives)
119
120
121 def range_branch(polygon,q,positives):
122     for p in positives:
123         d=math.dist(p,q)
124         if d<1e-10:
125             return []
126         nx,ny=(q[0]-p[0])/d,(q[1]-p[1])/d
127         mid=((p[0]+q[0])/2,(p[1]+q[1])/2)
128         polygon=clip(polygon,nx,ny,nx*mid[0]+ny*mid[1])
129         if not polygon:
130             return []
131     return subtract_disk(polygon,q,1000)
132
133
134 class JointBelief:
135     def __init__(self,polygon,problem,records=(),max_cells=48):
136         self.problem=problem;self.max_cells=max_cells
137         self.cells=[Cell(polygon)]
138         if problem==4:
139             self.cells += [Cell(polygon,(i*TAU/8,(i+1)*TAU/8)) for i in range(8)]
140         self.records=[];self.retained_parents=0
141         for record in records:
142             self.observe(record,refine=False)
143         self.refine_angles()
144
145     @classmethod
146     def from_cells(cls,cells,problem,max_cells=48):
147         obj=cls.__new__(cls);obj.problem=problem;obj.max_cells=max_cells
148         obj.cells=cells;obj.records=[];obj.retained_parents=0
149         return obj
150
151     def positives(self):
152         return [r["point"] for r in self.records if r["kind"] in ("direction","near")]
153
154     def _replace_cells(self,operation):
155         old=self.cells;out=[]
156         for i,cell in enumerate(old):
157             children=operation(cell)
158             if len(out)+len(children)+len(old)-i-1>self.max_cells:
159                 out.append(cell);self.retained_parents+=1
160             else:
161                 out.extend(children)
162         self.cells=out
163         if not out:
164             raise ArithmeticError("Joint position-direction evidence is inconsistent")
165
166     def observe(self,record,refine=True):
167         self.records.append(record)
168         q=record["point"];kind=record["kind"];positives=self.positives()
169         def update(cell):
170             poly=cell.polygon
171             if kind=="direction":
172                 theta=math.radians(record["bearing"])
173                 poly=bearing_clip(poly,q,theta,theta)
174                 if not poly:
175                     return []
176                 poly=safe_disk_outer(poly,q,1500)
177                 if not poly:
178                     return []
179                 poly=angle_plane(replace(cell,polygon=poly),q,True)
180                 return [replace(cell,polygon=poly)] if poly else []
181             if kind=="clear_failure":
182                 return [replace(cell,polygon=p) for p in subtract_disk(poly,q,20)]
183             if kind=="no_signal":

```

```

184         if definitely_in_range(poly,q,positives):
185             hidden=angle_plane(cell,q,False)
186             return [replace(cell,polygon=hidden)] if hidden else []
187         pieces=range_branch(poly,q,positives)
188         if cell.arc is not None:
189             hidden=angle_plane(cell,q,False)
190             if hidden:
191                 pieces.append(hidden)
192             return [replace(cell,polygon=p) for p in pieces]
193         return [cell]
194     self._replace_cells(update)
195     if kind=="direction":
196         # New positives raise the same source's unknown but fixed radius
197         # lower bound. Revisit OLD negatives with all positive distances.
198         for previous in self.records[:-1]:
199             if previous["kind"]!="no_signal":
200                 continue
201             negative=previous["point"]
202             def revisit(cell):
203                 if definitely_in_range(cell.polygon,negative,positives):
204                     p=angle_plane(cell,negative,False)
205                     return [replace(cell,polygon=p)] if p else []
206                 pieces=range_branch(cell.polygon,negative,positives)
207                 if cell.arc is not None:
208                     p=angle_plane(cell,negative,False)
209                     if p:
210                         pieces.append(p)
211                 return [replace(cell,polygon=p) for p in pieces]
212             self._replace_cells(revisit)
213     if refine:
214         self.refine_angles()
215
216 def _contract_angles(self,cell):
217     for r in self.records:
218         if not cell.polygon:
219             return None
220         if r["kind"]=="direction":
221             cell=replace(cell,polygon=angle_plane(cell,r["point"],True))
222         elif r["kind"]=="no_signal" and
223             ↪ definitely_in_range(cell.polygon,r["point"],self.positives()):
224             cell=replace(cell,polygon=angle_plane(cell,r["point"],False))
225     return cell if cell.polygon else None
226
227 def refine_angles(self,max_splits=8):
228     contracted=[self._contract_angles(c) for c in self.cells]
229     self.cells=[c for c in contracted if c is not None]
230     if not self.cells:
231         raise ArithmeticError("No joint hypothesis remains")
232     if not any(r["kind"]=="no_signal" for r in self.records):
233         return
234     for _ in range(max_splits):
235         choices=[(c.arc[1]-c.arc[0],i) for i,c in enumerate(self.cells)
236                 if c.arc is not None and c.arc[1]-c.arc[0]>math.pi/512]
237         if not choices or len(self.cells)>=self.max_cells:
238             break
239         _,i=max(choices);c=self.cells[i];lo,hi=c.arc;mid=(lo+hi)/2
240         children=[self._contract_angles(replace(c,arc=a)) for a in ((lo,mid),(mid,hi))]
241         self.cells[i:i+1]=[c for c in children if c is not None]
242         if not self.cells:
243             raise ArithmeticError("Angular refinement removed every hypothesis")
244
245 def intersect(self,polygon):
246     self._replace_cells(lambda c:[replace(c,polygon=p)] if
247         ↪ (p:=polygon_intersection(c.polygon,polygon)) else [])
248     self.refine_angles()
249
250 def polygons(self):
251     unique={}

```

```

250     for c in self.cells:
251         unique.setdefault(tuple(c.polygon),c.polygon)
252     return list(unique.values())
253
254 def hull(self):
255     return hull([v for c in self.cells for v in c.polygon])
256
257 def outcome(self,q,kind,interval=None):
258     """Outer posterior for every response in a whole interval, not a sample."""
259     cells=[]
260     for c in self.cells:
261         poly=c.polygon
262         if kind=="direction":
263             poly=bearing_clip(poly,q,*interval)
264             if poly:
265                 poly=angle_plane(replace(c,polygon=poly),q,True)
266         elif kind=="near":
267             if minimum_distance(poly,q)>5+SAFE:
268                 continue
269             # Exact near semantics allow immediate clear; no polygon needed.
270         else:
271             poly=angle_plane(c,q,False)
272         if poly:
273             cells.append(replace(c,polygon=poly))
274     return JointBelief.from_cells(cells,self.problem,self.max_cells) if cells else None
275
276 def witness(self,x):
277     """Validate one complete physical world before using it in a LOWER bound."""
278     if math.hypot(*x)>1800:
279         return None
280     positives=[];negatives=[];seen={}
281     for r in self.records:
282         q=r["point"];d=math.dist(x,q)
283         if r["kind"]=="direction":
284             if not 5<d<=1500:
285                 return None
286             bearing=math.radians(r["bearing"])
287             error=(math.atan2(x[1]-q[1],x[0]-q[0])-bearing+math.pi)%TAU-math.pi
288             if abs(error)>EPS-1e-10:
289                 return None
290             if tuple(q) in seen and seen[tuple(q)]!=r["bearing"]:
291                 return None
292             seen[tuple(q)]=r["bearing"];positives.append(q)
293         elif r["kind"]=="no_signal":
294             negatives.append(q)
295         elif r["kind"]=="clear_failure" and d<=20:
296             return None
297     if not positives:
298         return None
299     radius=max([1000.]+[math.dist(x,q) for q in positives])
300     in_range=[q for q in negatives if math.dist(x,q)<=radius]
301     if not in_range:
302         return dict(point=x,radius=radius,type="omni")
303     if self.problem==3:
304         return None
305     constraints=[(q,True) for q in positives]+[(q,False) for q in in_range]
306     endpoints=[0.,TAU]
307     for q,_ in constraints:
308         t=math.atan2(q[1]-x[1],q[0]-x[0])
309         endpoints.extend(((t-math.pi/2)%TAU,(t+math.pi/2)%TAU))
310     endpoints=sorted(set(endpoints))
311     angles=endpoints+[(a+b)/2 for a,b in zip(endpoints,endpoints[1:])]
312     for t in angles:
313         nx,ny=math.cos(t),math.sin(t)
314         if all(((q[0]-x[0])*nx+(q[1]-x[1])*ny>SAFE if visible else
315                (q[0]-x[0])*nx+(q[1]-x[1])*ny<-SAFE)
316                for q,visible in constraints):
317             return dict(point=x,radius=radius,type="directional",angle=t)

```

```

318         return None
319
320     def witnesses(self, limit=12):
321         points=[]
322         # Inner convex combinations are candidates only. witness() checks all
323         # circular, angular and fixed-radius constraints before accepting them.
324         for poly in self.polygons():
325             center=tuple(sum(v[i] for v in poly)/len(poly) for i in (0,1))
326             points.append(center)
327             for v in poly:
328                 points.extend((v,tuple(.999*v[i]+.001*center[i] for i in (0,1))))
329         found={}
330         for p in points[:800]:
331             w=self.witness(p)
332             if w is not None:
333                 found.setdefault(tuple(p),w)
334         values=list(found.values())
335         if len(values)<=limit:
336             return values
337         # Keep spatial extremes, not just the first small component.
338         selected=[]
339         for i in range(limit):
340             t=i*TAU/limit;u=(math.cos(t),math.sin(t))
341             selected.append(max(values,key=lambda w:sum(w["point"][j]*u[j] for j in (0,1))))
342         return list({tuple(w["point"]):w for w in selected}.values())

```

validation/baseline_v7/optical_v7.py

文件行数：69。

```

1  """First-success branches condition on all earlier optical failures."""
2  from itertools import permutations
3  import math
4  from joint_belief import subtract_disk,minimum_distance,SAFE
5  from decision_v6 import optical_cover as geometric_cover,rf_lower_bound
6
7
8  def failure_residual(polygons,point,limit=384):
9      out=[]
10     for i,poly in enumerate(polygons):
11         children=subtract_disk(poly,point,20.)
12         if len(out)+len(children)+len(polygons)-i-1>limit:
13             out.append(poly) # Keep possible worlds if the representation fills.
14         else:
15             out.extend(children)
16     return out
17
18
19 def conditional_chain(polygons,points,position,target=None):
20     residual=list(polygons);chosen=[];elapsed=0.;worst=0.;branches=[]
21     previous=position
22     for point in points:
23         if not residual:
24             break
25         if not any(minimum_distance(poly,point)<=20.+SAFE for poly in residual):
26             continue # Proven impossible success: omit this useless attempt.
27         elapsed+=math.dist(previous,point)/5
28         value=elapsed+3*len(chosen)+5+(math.dist(point,target)/5 if target else 0.)
29         branches.append(dict(index=len(chosen)+1,upper_s=value,residual_components=len(residual)))
30         worst=max(worst,value);chosen.append(point);previous=point
31         residual=failure_residual(residual,point)
32     return dict(points=chosen,worst_time_s=worst,first_success_branches=branches,
33               omitted_attempts=len(points)-len(chosen))
34
35
36 def optical_cover(belief,anchor,position,target=None,max_attempts=3):
37     plan=geometric_cover(belief,anchor,position,target,max_attempts)
38     if plan is None:

```

```

39         return None
40     choices=[conditional_chain(belief.polygons(),list(seq),position,target) for seq in
    ↪ permutations(plan['points'])]
41     best=min(choices,key=lambda p:p['worst_time_s'])
42     if not best['points']:
43         return None
44     best['certificate']=plan['certificate']+'+conditioned_first_success'
45     best['unconditioned_geometric_upper_s']=plan['worst_time_s']
46     return best
47
48
49 def optical_decision(belief,anchor,position,target=None,switch_cost=0):
50     geometric=geometric_cover(belief,anchor,position,target)
51     if geometric is None:
52         return None,dict(accepted=False,reason='no_three_disk_cover',optical_upper_s=None)
53     lower=rf_lower_bound(belief,position,target,switch_cost)
54     alone=lower if target is None else rf_lower_bound(belief,position,None,switch_cost)
55     accepted=[];all_plans=[]
56     for seq in permutations(geometric['points']):
57         plan=conditional_chain(belief.polygons(),list(seq),position,target)
58         standalone=conditional_chain(belief.polygons(),plan['points'],position)['worst_time_s']
59         gap=min(lower['lower_s']-plan['worst_time_s'],alone['lower_s']-standalone)
60         item=(plan,gap,standalone);all_plans.append(item)
61         if plan['points'] and gap>1e-4:
62             accepted.append(item)
63     plan,gap,standalone=min(accepted or all_plans,key=lambda x:x[0]['worst_time_s'])
64     plan['certificate']=geometric['certificate']+'+conditioned_first_success'
65     ok=bool(accepted)
66     return (plan if ok else None),dict(**lower,accepted=ok,optical_upper_s=plan['worst_time_s'],
67         standalone_rf_lower_s=alone['lower_s'],standalone_optical_upper_s=standalone,
68         dominance_gap_s=gap,omitted_attempts=plan['omitted_attempts'],
69         reason='upper_below_rf_lower' if ok else 'bounds_overlap_keep_rf')

```

validation/baseline_v7/planning.py

文件行数: 109。

```

1  """Bounded route improvement and certified optical endgame candidates."""
2  import math
3  from strategy import CLEAR_RADIUS, EPSILON_RAD
4
5
6  def route_length(start,points,end=None):
7      path=[start]+list(points)+([end] if end is not None else [])
8      return sum(math.dist(a,b) for a,b in zip(path,path[1:]))
9
10
11 def open_route(start,points,end=None):
12     """Keep original and nearest-neighbour seeds, then bounded open 2-opt."""
13     if len(points)<2:
14         return list(points)
15     remaining=list(points);greedy=[];p=start
16     while remaining:
17         q=min(remaining,key=lambda q:(math.dist(p,q),q))
18         greedy.append(q);remaining.remove(q);p=q
19     order=min((list(points),greedy),key=lambda qs:route_length(start,qs,end))
20     for _ in range(8):
21         best_delta,best_pair=0.,None
22         for i in range(len(order)-1):
23             a=start if i==0 else order[i-1]
24             for j in range(i+1,len(order)):
25                 b=order[j+1] if j+1<len(order) else end
26                 before=math.dist(a,order[i]);after=math.dist(a,order[j])
27                 if b is not None:
28                     before+=math.dist(order[j],b);after+=math.dist(order[i],b)
29                 if after-before<best_delta-1e-8:
30                     best_delta,best_pair=after-before,(i,j)
31     if best_pair is None:

```

```

32         break
33         i,j=best_pair;order[i:j+1]=reversed(order[i:j+1])
34     return order
35
36
37 def attachment(start,route,point):
38     """Cheapest insertion leg including a free final endpoint; heuristic."""
39     costs=[];previous=start
40     for i,next_point in enumerate(route):
41         costs.append((math.dist(previous,point)+math.dist(point,next_point)-math.dist(previous,n
42             ↪ ext_point),i))
43         previous=next_point
44     costs.append((math.dist(previous,point),len(route)))
45     return min(costs)
46
47 def fallback_bound(anchor,position,problem,target=None,polygon=None):
48     """Conservative time bound for uninterrupted v3 annular completion."""
49     L=anchor.bound;travel=math.dist(position,anchor.point);rounds=0
50     while L>44.5:
51         if problem==3:
52             travel+=.501*L+2.5;L=.502*L-2.4
53         else:
54             travel+=1.042*L+5;L=.502*L+2.505
55         rounds+=1
56     travel+=L+CLEAR_RADIUS
57     cost=travel/5+5*rounds*(1 if problem==3 else 2)+6
58     if target is not None:
59         cost+=(max(math.dist(v,target) for v in polygon)+CLEAR_RADIUS)/5
60     return cost
61
62
63 def rectangle_chain(polygon,angle):
64     """At most three equal disks covering an enclosing oriented rectangle."""
65     u=(math.cos(angle),math.sin(angle));v=(-u[1],u[0])
66     xs=[p[0]*u[0]+p[1]*u[1] for p in polygon]
67     ys=[p[0]*v[0]+p[1]*v[1] for p in polygon]
68     lo,hi=min(xs)-1e-6,max(xs)+1e-6
69     mid=(min(ys)+max(ys))/2;w=(max(ys)-min(ys))/2+1e-6
70     radius=CLEAR_RADIUS-1e-5
71     if w>=radius:
72         return None
73     half=math.sqrt(radius*radius-w*w)
74     count=max(1,math.ceil((hi-lo)/(2*half)))
75     if count>3:
76         return None
77     coords=[(lo+hi)/2] if count==1 else [lo+half+(hi-lo-2*half)*i/(count-1) for i in range(count)]
78     return [(x*u[0]+mid*v[0],x*u[1]+mid*v[1]) for x in coords]
79
80
81 def optical_plan(anchor,polygon,position,problem,target=None):
82     """Compare certified disk covers against a fallback upper bound, not its realized cost."""
83     angle=math.radians(anchor.bearing)
84     # Either enclosure suffices independently. No unproved rectangle intersection.
85     a=anchor.point;u=(math.cos(angle),math.sin(angle));v=(-u[1],u[0])
86     sector=[]
87     for x in (5*math.cos(EPSILON_RAD),anchor.bound):
88         for y in (-anchor.bound*math.sin(EPSILON_RAD),anchor.bound*math.sin(EPSILON_RAD)):
89             sector.append((a[0]+x*u[0]+y*v[0],a[1]+x*u[1]+y*v[1]))
90     candidates=[]
91     for enclosure in (polygon,sector):
92         for theta in (angle,angle+math.pi/2):
93             chain=rectangle_chain(enclosure,theta)
94             if chain is None:
95                 continue
96             for points in (chain,list(reversed(chain))):
97                 elapsed=0.;previous=position;costs=[]
98                 for i,q in enumerate(points):

```

```

99         elapsed+=math.dist(previous,q)/5
100         costs.append(elapsed+3*i+5+(math.dist(q,target)/5 if target is not None else
    ↪ 0))
101         previous=q
102         candidates.append((max(costs),points))
103     if not candidates:
104         return None
105     value,points=min(candidates,key=lambda item:item[0])
106     upper=fallback_bound(anchor,position,problem,target,polygon)
107     if value>=upper-1e-6:
108         return None
109     return dict(points=points,worst_time_s=value,fallback_upper_s=upper)

```

validation/baseline_v7/posterior.py

文件行数: 126。

```

1  """History-aware outer regions and minimum-travel robust clearance.
2
3  Independent implementation inspired by the supplied repository's region idea.
4  No simulator, third-party runtime, probability model, or network dependency.
5  """
6  from dataclasses import replace
7  from itertools import combinations
8  import math
9  from strategy import CLEAR_RADIUS, distance, terminal_clear_point, annular_clear_point
10
11  MARGIN = 1e-7 # Metres; relax halfplanes outward before clipping.
12  EPSILON = math.radians(1.005)
13
14
15  def clip(polygon, nx, ny, bound):
16      """Intersect an ordered convex polygon with a relaxed unit-normal plane."""
17      if not polygon:
18          return []
19      bound += MARGIN
20      out = []
21      previous = polygon[-1]
22      dp = nx*previous[0]+ny*previous[1]-bound
23      for point in polygon:
24          dq = nx*point[0]+ny*point[1]-bound
25          if (dp <= 0) != (dq <= 0):
26              ratio = dp/(dp-dq)
27              out.append((previous[0]+ratio*(point[0]-previous[0]),
28                          previous[1]+ratio*(point[1]-previous[1])))
29          if dq <= 0:
30              out.append(point)
31          previous, dp = point, dq
32      return out
33
34
35  def constrain(polygon, anchor, annular=False):
36      angle = math.radians(anchor.bearing)
37      lo, hi = angle-EPSILON, angle+EPSILON
38      # A truncated forward wedge is an outer approximation of the range sector.
39      for nx,ny,offset in ((math.sin(lo),-math.cos(lo),0),
40                           (-math.sin(hi),math.cos(hi),0),
41                           (math.cos(angle),math.sin(angle),anchor.bound)):
42          polygon = clip(polygon,nx,ny,nx*anchor.point[0]+ny*anchor.point[1]+offset)
43      if annular:
44          nx,ny = -math.cos(angle),-math.sin(angle)
45          polygon = clip(polygon,nx,ny,nx*anchor.point[0]+ny*anchor.point[1]-5*math.cos(EPSILON))
46      if not polygon or not all(math.isfinite(x) for p in polygon for x in p):
47          raise ArithmeticError("History region is empty or numerically unresolved")
48      return polygon
49
50
51  def initial_region(anchor, annular=False):

```

```

52 # This box comes from the specified 1800 m source disk, not arbitrary clipping.
53 r = 1800+MARGIN
54 polygon = [(-r,-r),(r,-r),(r,r),(-r,r)]
55 for k in range(16):
56     t = 2*math.pi*k/16
57     polygon = clip(polygon,math.cos(t),math.sin(t),1800)
58 return constrain(polygon,anchor,annular)
59
60
61 def omni_negative_cuts(polygon, positive, negatives):
62     """Use only pre-clear, same-channel, omnidirectional observations."""
63     for negative in negatives:
64         d = distance(positive,negative)
65         if d == 0:
66             raise ArithmeticError("Same position has incompatible positive and negative
67                                     ↪ responses")
68         nx,ny = (negative[0]-positive[0])/d,(negative[1]-positive[1])/d
69         midpoint = ((positive[0]+negative[0])/2,(positive[1]+negative[1])/2)
70         polygon = clip(polygon,nx,ny,nx*midpoint[0]+ny*midpoint[1])
71     if not polygon:
72         raise ArithmeticError("Omnidirectional positive/negative history is inconsistent")
73     return polygon
74
75 def tighten_anchor(anchor, polygon):
76     # Norm is convex: maximum over a polygon occurs at a vertex.
77     history_bound = max(distance(anchor.point,v) for v in polygon)+MARGIN
78     return replace(anchor,bound=min(anchor.bound,history_bound))
79
80
81 def nearest_clear_point(polygon, current):
82     """Project current onto intersection_v B(v, r); 0(m^3), small convex regions.
83
84     A projection is current itself, a projection onto one active circle, or an
85     intersection of two active circles. An inward radius margin handles roundoff.
86     """
87     if not polygon:
88         return None
89     radius = CLEAR_RADIUS-1e-6
90     def feasible(p):
91         return all(distance(p,v) <= radius+1e-10 for v in polygon)
92     if feasible(current):
93         return current
94     if any(distance(a,b) > 2*radius for a,b in combinations(polygon,2)):
95         return None
96     best, best_distance = None, float("inf")
97     def consider(p):
98         nonlocal best, best_distance
99         travel = distance(current,p)
100         if travel < best_distance and feasible(p):
101             best,best_distance = p,travel
102     for a in polygon:
103         d = distance(current,a)
104         if d > radius:
105             consider((a[0]+radius*(current[0]-a[0])/d,
106                     a[1]+radius*(current[1]-a[1])/d))
107     for a,b in combinations(polygon,2):
108         d = distance(a,b)
109         if d <= 1e-12 or d > 2*radius:
110             continue
111         midpoint = ((a[0]+b[0])/2,(a[1]+b[1])/2)
112         h = math.sqrt(max(0.0,radius*radius-d*d/4))
113         dx,dy = -(b[1]-a[1])/d*h,(b[0]-a[0])/d*h
114         consider((midpoint[0]+dx,midpoint[1]+dy))
115         consider((midpoint[0]-dx,midpoint[1]-dy))
116     return best
117
118

```

```

119 def clear_choice(anchor, polygon, current, annular=False):
120     # The independent anchor certificate remains available even if a relaxed
121     # outer polygon is slightly too wide to admit a history-based clear.
122     baseline = annular_clear_point(anchor,current) if annular else
        ↪ terminal_clear_point(anchor,current)
123     history = nearest_clear_point(polygon,current)
124     if history is not None and (baseline is None or distance(current,history) <
        ↪ distance(current,baseline)):
125         return history,"history_projection"
126     return baseline,"anchor_certificate"

```

validation/baseline_v7/probe_frontier.py

文件行数: 44。

```

1 """Nearest probes that certify contraction of the WHOLE compatible region."""
2 from dataclasses import replace
3 import math
4 from posterior import nearest_clear_point
5 from joint_belief import angle_plane,SAFE
6 from planning import fallback_bound
7
8
9 def disk_intersection_projection(polygon,position,radius):
10     if radius<=0:return None
11     scale=19.8/radius
12     point=nearest_clear_point([(v[0]*scale,v[1]*scale) for v in polygon],tuple(x*scale for x in
        ↪ position))
13     return None if point is None else tuple(x/scale for x in point)
14
15
16 def visibility_guaranteed(belief,point):
17     return belief.problem==3 or all(not angle_plane(cell,point,False) for cell in belief.cells)
18
19
20 def guaranteed_probes(belief,anchor,position):
21     if anchor.bound<=44.5:return []
22     limit=.502*anchor.bound-2.4
23     radii=sorted(set(r for r in (44.5,89.9,179.9,limit) if 5<r<=limit))
24     poly=belief.hull();probes=[]
25     for radius in radii:
26         point=disk_intersection_projection(poly,position,radius-1e-3)
27         if point is None or not visibility_guaranteed(belief,point):continue
28         bound=max(5+SAFE,max(math.dist(point,v) for v in poly)+SAFE)
29         if bound>limit or bound>=1000-SAFE:continue
30         temporary=replace(anchor,point=point,bearing=0.,bound=bound,rounds=anchor.rounds+1)
31         service=6+max(5.,fallback_bound(temporary,point,belief.problem)-1.)
32         probes.append(dict(point=point,bound=bound,service_s=service,limit=limit,
33                             proof='whole_region_receive_radius_and_visible_halfplane'))
34     return probes
35
36
37 def regional_upper(belief,anchor,position,target=None):
38     """One certified regional measurement followed by the original fallback."""
39     choices=[];poly=belief.hull()
40     outgoing=(max(math.dist(v,target) for v in poly)+20)/5 if target is not None else 0.
41     for probe in guaranteed_probes(belief,anchor,position):
42         alone=math.dist(position,probe['point'])/5+probe['service_s']
43         choices.append((alone+outgoing,alone,probe))
44     return min(choices,key=lambda c:c[0]) if choices else None

```

validation/baseline_v7/refinement.py

文件行数: 70。

```

1 """Allocate angular resolution to current actions and their cost bottlenecks."""
2 from dataclasses import replace

```

```

3 import math
4 from joint_belief import JointBelief, angle_plane, definitely_in_range, SAFE
5
6
7 class DecisionBelief(JointBelief):
8     def refine_angles(self,max_splits=8):
9         # Automatic updates contract evidence only. Splitting waits until the
10         # controller supplies real candidate actions rather than generic angles.
11         self.cells=[c for old in self.cells if (c:=self._contract_angles(old)) is not None]
12         if not self.cells:
13             raise ArithmeticError("No joint hypothesis remains")
14         self.action_splits=getattr(self,"action_splits",0)
15         self.split_revision=getattr(self,"split_revision",(-1,0))
16
17     def refine_for_actions(self,points,position,target=None,max_splits=8):
18         points=list(dict.fromkeys(tuple(p) for p in points if p is not None))[:4]
19         revision=len(self.records)
20         old_revision,used=self.split_revision
21         if old_revision!=revision:
22             used=0
23         remaining=min(max_splits,8-used)
24         splits=0;reduction=0.
25         if not points or remaining<=0:
26             return dict(splits=0,reason="no_new_action_budget")
27         def support(c):
28             return max(math.dist(position,v)+(math.dist(v,target) if target is not None else 0)
29                        for v in c.polygon)
30         def ambiguity(c):
31             if c.arc is None:
32                 return 0.
33             uncertainty=2*math.sin((c.arc[1]-c.arc[0])/4)
34             score=0.
35             for q in points:
36                 if definitely_in_range(c.polygon,q,self.positives()) and angle_plane(c,q,True)
37                 ↪ and angle_plane(c,q,False):
38                     score=max(score,uncertainty*max(math.dist(q,v) for v in c.polygon))
39             return score
40         for _ in range(remaining):
41             if len(self.cells)>=self.max_cells:
42                 break
43             global_support=max(support(c) for c in self.cells)
44             choices=[]
45             for i,c in enumerate(self.cells):
46                 if c.arc is None or c.arc[1]-c.arc[0]<=math.pi/512:
47                     continue
48                 # A cell is relevant when it can change signal/no-signal at a
49                 # real candidate action, or attain the worst travel support.
50                 a=ambiguity(c);s=support(c)
51                 if a<SAFE and s<global_support-1e-4:
52                     continue
53                 choices.append((a+max(0.,s-global_support+1.),i))
54             trials=[]
55             for _,i in sorted(choices,reverse=True)[:4]:
56                 c=self.cells[i];lo,hi=c.arc;mid=(lo+hi)/2
57                 children=[child for arc in ((lo,mid),(mid,hi))
58                           if (child:=self._contract_angles(replace(c,arc=arc))) is not None]
59                 old_score=support(c)+ambiguity(c)
60                 new_score=max((support(k)+ambiguity(k) for k in children),default=0.)
61                 if old_score-new_score>1e-7:
62                     trials.append((old_score-new_score,i,children))
63             if not trials:
64                 break
65             gain,i,children=max(trials,key=lambda t:t[0])
66             self.cells[i:i+1]=children;splits+=1;reduction+=gain
67             if not self.cells:
68                 raise ArithmeticError("Action refinement removed every hypothesis")
69         self.action_splits+=splits;self.split_revision=(revision,used+splits)
70         return dict(splits=splits,action_support_reduction_m=reduction,

```

validation/baseline_v7/routing.py

文件行数: 76。

```

1 """Bounded convex descent for one clear followed by one committed search point.
2
3 No solver dependency or exploration. Each returned point is feasible and never
4 increases the fixed-target two-leg cost of its supplied feasible starting point.
5 The Frank-Wolfe gap is a numerical diagnostic, not an interval certificate.
6 """
7 import math
8 from itertools import combinations
9 from strategy import distance
10
11
12 def two_leg(point,current,target):
13     return distance(current,point)+distance(point,target)
14
15
16 def improve_clear_route(centers,radius,start,current,target,max_steps=24):
17     def feasible(p):
18         return all(distance(p,c)<=radius+1e-10 for c in centers)
19     if not centers or radius<=0 or not feasible(start):
20         return start,dict(iterations=0,gap_m=None)
21     corners = []
22     for a,b in combinations(centers,2):
23         d = distance(a,b)
24         if d <= 1e-12 or d > 2*radius:
25             continue
26         mid = (a[0]+b[0])/2,(a[1]+b[1])/2
27         h = math.sqrt(max(0.0,radius*radius-d*d/4))
28         dx,dy = -(b[1]-a[1])*h/d,(b[0]-a[0])*h/d
29         for p in ((mid[0]+dx,mid[1]+dy),(mid[0]-dx,mid[1]-dy)):
30             if feasible(p):
31                 corners.append(p)
32     point,steps,gap = start,0,None
33     for _ in range(max_steps):
34         value = two_leg(point,current,target)
35         if value-distance(current,target)<=1e-7:
36             gap = max(0.0,value-distance(current,target))
37             break
38         d1,d2 = distance(point,current),distance(point,target)
39         if min(d1,d2)<=1e-12:
40             gap=0.0
41             break
42         g = ((point[0]-current[0])/d1+(point[0]-target[0])/d2,
43              (point[1]-current[1])/d1+(point[1]-target[1])/d2)
44         norm = math.hypot(*g)
45         if norm<=1e-12:
46             gap=None
47             break
48         candidates = corners+[point]
49         for c in centers:
50             candidate = c[0]-radius*g[0]/norm,c[1]-radius*g[1]/norm
51             if feasible(candidate):
52                 candidates.append(candidate)
53         support = min(candidates,key=lambda p:g[0]*p[0]+g[1]*p[1])
54         gap = max(0.0,g[0]*(point[0]-support[0])+g[1]*(point[1]-support[1]))
55         if gap<=1e-5:
56             break
57         # Along a feasible chord the objective is convex. Fixed work, no sweep.
58         def interpolate(t):
59             return point[0]+t*(support[0]-point[0]),point[1]+t*(support[1]-point[1])
60         lo,hi=0.0,1.0
61         for _ in range(36):
62             left,right=(2*lo+hi)/3,(lo+2*hi)/3

```

```

63         if two_leg(interpolate(left),current,target)<=two_leg(interpolate(right),current,target)
        ↪ get):
64             hi=right
65         else:
66             lo=left
67         candidate=min((point,support,interpolate((lo+hi)/2)),key=lambda
        ↪ p:two_leg(p,current,target))
68         if not feasible(candidate):
69             break
70         improvement=value-two_leg(candidate,current,target)
71         point=candidate
72         steps+=1
73         gap=None # Previous gradient gap does not describe the new iterate.
74         if improvement<=1e-9:
75             break
76     return point,dict(iterations=steps,gap_m=gap)

```

validation/baseline_v7/service_graph.py

文件行数: 203。

```

1  """All outstanding obligations share one directed, bounded service graph.
2
3  Every source is serviced once in every compared plan. A radio mode enters at
4  its first actual probe, not at a fictional return to the discovery anchor.
5  Exit polygons bound the true completion position, rather than point estimates.
6  """
7  from dataclasses import dataclass
8  import math
9  from strategy import annular_points, annular_update, annular_clear_point
10 from posterior import nearest_clear_point
11 from planning import fallback_bound, open_route
12 from probe_frontier import guaranteed_probes
13
14
15 @dataclass
16 class Mode:
17     job: tuple
18     kind: str
19     entry: tuple
20     service_s: float
21     exit_vertices: list
22     exit_radius: float=0.
23     first_probe: tuple | None=None
24     anchor: object=None
25
26     def departure(self,target):
27         return (max(math.dist(v,target) for v in self.exit_vertices)+self.exit_radius)/5
28
29
30 def radio_modes(channel,anchor,polygon,problem):
31     if anchor.bound<=44.5:
32         return []
33     pair=annular_points(anchor,anchor.point);modes=[]
34     for q,other in (pair,pair[::-1]):
35         # Bearing zero here is only a placeholder: fallback_bound uses no angle.
36         positive=annular_update(anchor,q,0.)
37         first=max(5.,fallback_bound(positive,q,problem)-1.)
38         if problem==4:
39             second=annular_update(anchor,other,0.)
40             hidden=annular_update(anchor)
41             after_second=max(5.,fallback_bound(second,other,problem)-1.,
42                             fallback_bound(hidden,other,problem)-1.)
43             first=max(first,math.dist(q,other)/5+5+after_second)
44         modes.append(Mode(('source',channel),'radio',q,6+first,polygon,20.,q,anchor))
45     return modes
46
47

```

```

48 def assignment(cost):
49     """Square minimum assignment via primal-dual potentials,  $O(n^3)$ ."""
50     n=len(cost);u=[0.]*(n+1);v=[0.]*(n+1);p=[0]*(n+1);way=[0]*(n+1)
51     for i in range(1,n+1):
52         p[0]=i;j0=0;minimum=float('inf')*(n+1);used=[False]*(n+1)
53         while True:
54             used[j0]=True;i0=p[j0];delta=float('inf');j1=0
55             for j in range(1,n+1):
56                 if not used[j]:
57                     cur=cost[i0-1][j-1]-u[i0]-v[j]
58                     if cur<minimum[j]:minimum[j]=cur;way[j]=j0
59                     if minimum[j]<delta:delta=minimum[j];j1=j
60             for j in range(n+1):
61                 if used[j]:u[p[j]]+=delta;v[j]-=delta
62                 else:minimum[j]-=delta
63             j0=j1
64             if p[j0]==0:break
65         while True:
66             j1=way[j0];p[j0]=p[j1];j0=j1
67             if j0==0:break
68     successor=[0]*n
69     for j in range(1,n+1):successor[p[j]-1]=j-1
70     return sum(cost[i][j] for i,j in enumerate(successor)),successor
71
72
73 def source_modes(channel,anchor,polygon,problem,position,target=(),belief=None):
74     points=[]
75     for target in [position,anchor.point]+list(target)[:2]:
76         q=nearest_clear_point(polygon,target)
77         if q is not None:
78             points.append(q)
79         q=annular_clear_point(anchor,target)
80         if q is not None:
81             points.append(q)
82     points=list(dict.fromkeys(tuple(q) for q in points))
83     if points:
84         return [Mode(('source',channel),'clear',q,5.,[q]) for q in points[:4]]
85     modes=radio_modes(channel,anchor,polygon,problem)
86     if belief is not None:
87         for probe in guaranteed_probes(belief,anchor,position):
88             mode=Mode(('source',channel),'regional_probe',probe['point'],probe['service_s'],poly_
89                 ↳ gon,20.,anchor=anchor)
89             mode.probe_bound=probe['bound'];modes.append(mode)
90     return modes
91
92
93 class ServiceGraph:
94     def __init__(self,position,jobs):
95         self.position=tuple(position);self.jobs=jobs
96         self.modes=[m for variants in jobs.values() for m in variants]
97         self.ids={job:[i for i,m in enumerate(self.modes) if m.job==job] for job in jobs}
98         self.index={id(m):i for i,m in enumerate(self.modes)}
99         self.edge={(i,j):a.departure(b.entry)+b.service_s
100             ↳ for i,a in enumerate(self.modes) for j,b in enumerate(self.modes) if
101                 ↳ a.job!=b.job}
101         self.start={i:math.dist(position,m.entry)/5+m.service_s for i,m in enumerate(self.modes)}
102         self.evaluations=0
103
104     def evaluate(self,order):
105         """Exact mode DP for one fixed job order, with a free final endpoint."""
106         self.evaluations+=1
107         if not order:
108             return 0.,[]
109         costs={i:(self.start[i],[i]) for i in self.ids[order[0]]}
110         for job in order[1:]:
111             costs={j:min((v+self.edge[i,j],path+[j]) for i,(v,path) in costs.items()) for j in
112                 ↳ self.ids[job]}
112         value,path=min(costs.values())

```

```

113         return value,[self.modes[i] for i in path]
114
115     def optimize(self,incumbent=(),passes=3):
116         if not self.jobs:
117             return dict(upper_s=0.,modes=[],order=[],evaluations=0,baseline_upper_s=0.)
118         # Complete any old plan before comparing it: new obligations are never
119         # dropped merely because they were absent from the previous prediction.
120         old=[j for j in incumbent if j in self.jobs]
121         old+= [j for j in self.jobs if j not in old]
122         seeds=[old]
123         remaining=list(self.jobs);greedy=[];last=None
124         while remaining:
125             job=min(remaining,key=lambda j:min(self.start[i] if last is None else
126             ↪ self.edge[last,i] for i in self.ids[j]))
127             last=min(self.ids[j],key=lambda i:self.start[i] if last is None else
128             ↪ self.edge[last,i])
129             greedy.append(job);remaining.remove(job)
130         seeds.append(greedy)
131         scans=[j for j in self.jobs if j[0]=='scan'];sources=[j for j in self.jobs if
132         ↪ j[0]=='source']
133         seeds.extend((sources+scans,scans+sources))
134         lower,patched=self.assignment_seed()
135         seeds.append(patched)
136         baseline=self.evaluate(old)[0]
137         evaluated=[]
138         for seed in seeds:
139             v,m=self.evaluate(seed);evaluated.append((v,seed,m))
140         value,order,modes=min(evaluated,key=lambda e:e[0])
141         for _ in range(passes):
142             before=value;trials=[];n=len(order)
143             # Mode choices are reoptimized for every candidate order. No local
144             # improvement relies on treating uncertain source exits as centres.
145             for i in range(n):
146                 for j in range(n):
147                     if i==j:
148                         continue
149                     trial=list(order);item=trial.pop(i);trial.insert(j,item)
150                     # Cheap current-mode edge score only shortlists candidates;
151                     # the complete directed mode DP decides actual acceptance.
152                     chosen={m.job:self.index[id(m)] for m in modes};seq=[chosen[k] for k in trial]
153                     score=self.start[seq[0]]+sum(self.edge[a,b] for a,b in zip(seq,seq[1:]))
154                     if score<value-1e-7:
155                         trials.append((score,trial))
156             for _,trial in sorted(trials,key=lambda x:x[0])[:16]:
157                 candidate,variants=self.evaluate(trial)
158                 if candidate<value-1e-7:
159                     value,order,modes=candidate,trial,variants
160             if value>=before-1e-7:
161                 break
162         return dict(upper_s=value,modes=modes,order=order,evaluations=self.evaluations,
163         baseline_upper_s=baseline,assignment_lower_s=lower,
164         finite_graph_gap_s=max(0.,value-lower),
165         scope='all_current_known_jobs_same_obligations')
166
167     def assignment_seed(self):
168         jobs=list(self.jobs);n=len(jobs);cost=[[1e12]*(n+1) for _ in range(n+1)]
169         for j,job in enumerate(jobs,1):cost[0][j]=min(self.start[k] for k in self.ids[j])
170         for i,a in enumerate(jobs,1):
171             cost[i][0]=0.
172             for j,b in enumerate(jobs,1):
173                 if i!=j:cost[i][j]=min(self.edge[x,y] for x in self.ids[a] for y in self.ids[b])
174         lower,successor=assignment(cost)
175         def cycle(start):
176             out=[start];q=successor[start]
177             while q!=start:out.append(q);q=successor[q]
178             return out
179         main=cycle(0)
180         while len(main)<n+1:

```

```

178         other=cycle(next(i for i in range(n+1) if i not in main))
179         a,b=min(((a,b) for a in main for b in other),
180                 key=lambda ab:cost[ab[0]][successor[ab[1]]]+cost[ab[1]][successor[ab[0]]]
181                 -cost[ab[0]][successor[ab[0]]]-cost[ab[1]][successor[ab[1]]])
182         successor[a],successor[b]=successor[b],successor[a];main=cycle(0)
183         return max(0.,lower-1e-6),[jobs[i-1] for i in main[1:]]
184
185
186 def build_graph(solver,route,incumbent_modes=()):
187     jobs={}
188     for point in route:
189         job=('scan',tuple(point))
190         jobs[job]=[Mode(job,'scan',tuple(point),11.*len(solver.unknown),[tuple(point)])]
191     targets=list(route[:2])
192     for c in solver.anchors:
193         belief=solver._belief(c)
194         jobs['source',c]=source_modes(c,solver.anchors[c],belief.hull(),solver.problem,solver.poj
195         ↪ sition,targets,belief)
196     for old in incumbent_modes:
197         if old.job!=('source',c):continue
198         if old.kind=='clear' or (old.anchor is not None and
199         ↪ old.anchor.rounds==solver.anchors[c].rounds):
200             # Historical modes remain valid under additional constraints;
201             # retaining them keeps the old complete plan available.
202             jobs['source',c].append(old)
203     if not jobs['source',c]:
204         raise ArithmeticError('Source has neither a clear mode nor a finite radio mode')
205     return ServiceGraph(solver.position,jobs)

```

validation/baseline_v7/solver.py

文件行数：268。

```

1  """Search/batch/approach state machine implementing the final mathematical design."""
2  from dataclasses import asdict
3  import time
4  from strategy import (Anchor, distance, paired_points, positive_update, negative_pair_update,
5                        terminal_clear_point, search_points, next_search_point,
6                        channel_order, preview_source_action)
7  from posterior import initial_region, constrain, tighten_anchor, clear_choice,
8  ↪ omni_negative_cuts, nearest_clear_point
9  from strategy import annular_points, annular_update, annular_clear_disk, annular_clear_point,
10 ↪ CLEAR_RADIUS
11 from routing import improve_clear_route, two_leg
12
13 class ModelMismatch(RuntimeError): pass
14 class BudgetStop(RuntimeError): pass
15
16 class Solver:
17     def __init__(self, transport, problem, progress=None, use_history=True):
18         self.io, self.problem, self.progress = transport, problem, progress
19         self.remaining = search_points(problem)
20         self.visited = []
21         self.position, self.channel, self.virtual = (0.0, 0.0), 1, 0.0
22         self.unknown = set(range(1, 21))
23         self.discovered, self.cleared = set(), set()
24         self.anchors, self.source_measures, self.negative_pairs = {}, {}, {}
25         self.use_history = use_history
26         self.regions = {}
27         self.history_clears = self.history_tightenings = 0
28         self.search_negatives = {c:[] for c in range(1,21)}
29         self.negative_cut_count = 0
30         self.next_search_committed = None
31         self.route_planned_reduction_m = 0.0
32         self.started = None
33         self.deadline, self.max_virtual = float("inf"), 360000.0

```

```

34     self.entered = self.exit_attempted = self.exit_confirmed = False
35     self.certificate = False
36     self.reason = "not_started"
37     self.measures = self.clear_attempts = self.switches = 0
38     self.travel = 0.0
39
40     def emit(self, event, **fields):
41         record = dict(event=event, virtual_time_s=self.virtual, **fields)
42         self.io.record(record)
43         if self.progress:
44             self.progress(record)
45
46     def budget(self, point):
47         if time.monotonic() >= self.deadline - 25:
48             raise BudgetStop("real_time_reserve")
49         if self.virtual + distance(self.position, point)/5 + 6 >= self.max_virtual - 1:
50             raise BudgetStop("virtual_time_reserve")
51         limit = (236 if self.use_history else 252) if self.problem == 3 else 828
52         if self.measures + self.clear_attempts >= limit:
53             raise ModelMismatch("Action count exceeds the frozen strategy bound")
54
55     def action(self, path, point, channel):
56         self.budget(point)
57         moved = distance(self.position, point)
58         switching = int(path == "/measure" and channel != self.channel)
59         response = self.io.call(path, point, channel)
60         before = self.virtual
61         self.position = tuple(point)
62         self.travel += moved
63         self.virtual = response["virtual_time_s"]
64         if path == "/measure":
65             self.measures += 1
66             self.switches += switching
67             self.channel = channel
68             cost = 5 + switching
69         else:
70             self.clear_attempts += 1
71             cost = 5 if response["clear_result"] == "success" else 3
72         if abs(self.virtual - (before + moved/5 + cost)) > 5e-5:
73             raise ModelMismatch("Returned virtual time disagrees with the documented action cost")
74         return response
75
76     def discover(self, channel):
77         self.discovered.add(channel)
78         self.unknown.discard(channel)
79         if len(self.discovered) > 16:
80             raise ModelMismatch("More than 16 distinct source channels")
81
82     def clear(self, point, channel):
83         response = self.action("/clear", point, channel)
84         if response["clear_result"] != "success":
85             raise ModelMismatch(f"Distance-certified clear failed for channel {channel}")
86         self.cleared.add(channel)
87         self.anchors.pop(channel, None)
88         self.regions.pop(channel, None)
89         self.emit("cleared", channel=channel, cleared_count=len(self.cleared))
90
91     def scan(self, point):
92         for channel in channel_order(self.channel, self.unknown):
93             if len(self.discovered) == 16:
94                 break
95             reply = self.action("/measure", point, channel)
96             kind = reply["measure_result"]
97             if kind == "no_signal":
98                 if self.use_history and self.problem == 3:
99                     self.search_negatives[channel].append(tuple(point))
100                 continue
101             self.discover(channel)

```

```

102         if kind == "near":
103             self.clear(point, channel)
104         else:
105             self.anchors[channel] = Anchor(tuple(point), reply["svd_deg"])
106             if self.use_history:
107                 self.regions[channel] = initial_region(self.anchors[channel], annular=True)
108                 self.apply_negative_history(channel, point)
109                 self.refresh_history(channel)
110                 self.emit("discovered", channel=channel, bearing=reply["svd_deg"])
111             self.remaining.remove(point)
112             self.visited.append(point)
113             self.emit("search_point_done", visited=len(self.visited), remaining=len(self.remaining))
114
115     def approach(self, channel):
116         while channel not in self.cleared:
117             anchor = self.anchors[channel]
118             point, method = self.clear_plan(channel)
119             if point is not None:
120                 if self.use_history:
121                     point, method = self.route_clear(channel, point, method)
122                     self.emit("clear_planned", channel=channel, method=method, point=point)
123                     self.clear(point, channel)
124                     self.history_clears += int(method.startswith("history_"))
125                     return
126             round_limit = 5 if self.use_history and self.problem == 3 else 6
127             if anchor.rounds >= round_limit:
128                 raise ModelMismatch("Contraction round bound did not reach the clearance
129                                     ↳ condition")
130             ordered = annular_points(anchor, self.position) if self.use_history else
131                     ↳ paired_points(anchor, self.position)
132             for point in ordered:
133                 reply = self.action("/measure", point, channel)
134                 self.source_measures[channel] = self.source_measures.get(channel, 0) + 1
135                 kind = reply["measure_result"]
136                 if kind == "near":
137                     self.clear(point, channel)
138                     return
139                 if kind == "direction":
140                     self.anchors[channel] = (annular_update(anchor, point, reply["svd_deg"]) if
141                                             ↳ self.use_history
142                                             else positive_update(anchor, point, reply["svd_deg"]))
143                     if self.use_history:
144                         self.apply_negative_history(channel, point)
145                     break
146                 if self.problem == 3:
147                     raise ModelMismatch("Guaranteed in-range omnidirectional source returned
148                                         ↳ no_signal")
149             else:
150                 self.anchors[channel] = annular_update(anchor) if self.use_history else
151                                     ↳ negative_pair_update(anchor)
152                 self.negative_pairs[channel] = self.negative_pairs.get(channel, 0) + 1
153             if self.use_history:
154                 # A lone no_signal never clips the region. Only the proven
155                 # paired-negative contraction adds a new radial constraint.
156                 self.regions[channel] =
157                     ↳ constrain(self.regions[channel], self.anchors[channel], annular=True)
158                 self.refresh_history(channel)
159             self.emit("anchor_updated", channel=channel, anchor=asdict(self.anchors[channel]))
160
161     def refresh_history(self, channel):
162         before = self.anchors[channel]
163         after = tighten_anchor(before, self.regions[channel])
164         self.anchors[channel] = after
165         if after.bound < before.bound:
166             self.history_tightenings += 1
167             self.emit("history_tightened", channel=channel, before=before.bound, after=after.bound)
168
169     def apply_negative_history(self, channel, positive):

```

```

164         if self.problem == 3:
165             negatives=self.search_negatives[channel]
166             self.regions[channel]=omni_negative_cuts(self.regions[channel],positive,negatives)
167             self.negative_cut_count+=len(negatives)
168
169     def route_clear(self,channel,original,method):
170         # Only the last pending source in a batch. Commit the next search point,
171         # so the second leg is actually executed, not an unfulfilled proxy.
172         if len(self.anchors)!=1 or len(self.discovered)>=16 or not self.remaining:
173             return original,method
174         target=next_search_point(original,self.remaining)
175         options=[(original,method)]
176         diagnostics=[]
177         polygon=self.regions[channel]
178         start=nearest_clear_point(polygon,self.position)
179         if start is not None:
180             candidate,diag=improve_clear_route(polygon,CLEAR_RADIUS-1e-6,start,self.position,target)
181             ↪ get)
182             options.append((candidate,"history_route"));diagnostics.append(diag)
183             disk=annular_clear_disk(self.anchors[channel])
184             if disk is not None:
185                 start=annular_clear_point(self.anchors[channel],self.position)
186                 candidate,diag=improve_clear_route([disk[0]],disk[1],start,self.position,target)
187                 options.append((candidate,"anchor_route"));diagnostics.append(diag)
188             chosen,method=min(options,key=lambda item:two_leg(item[0],self.position,target))
189             reduction=two_leg(original,self.position,target)-two_leg(chosen,self.position,target)
190             self.next_search_committed=target
191             self.route_planned_reduction_m+=max(0.0,reduction)
192             self.emit("route_clear_planned",channel=channel,target=target,
193                     ↪ two_leg_reduction_m=max(0.0,reduction),diagnostics=diagnostics)
194             return chosen,method
195
196     def clear_plan(self, channel):
197         anchor = self.anchors[channel]
198         if self.use_history:
199             return clear_choice(anchor,self.regions[channel],self.position,annular=True)
200         return terminal_clear_point(anchor,self.position),"anchor_certificate"
201
202     def choose_source(self):
203         def cost(c):
204             point,_ = self.clear_plan(c)
205             if point is not None:
206                 return distance(self.position,point)/5+5,c
207             if self.use_history:
208                 point=annular_points(self.anchors[c],self.position)[0]
209                 return distance(self.position,point)/5+5+int(self.channel!=c),c
210             return preview_source_action(self.anchors[c],self.position,self.channel,c)[2],c
211         return min(self.anchors,key=cost)
212
213     def run(self):
214         self.started = time.monotonic()
215         reply = self.io.call("/enter")
216         self.entered = True
217         self.virtual = reply["virtual_time_s"]
218         self.max_virtual = reply["max_virtual_duration_s"]
219         # Counting from before /enter is conservative when its response is delayed/retried.
220         self.deadline = self.started + reply["remaining_real_duration_s"]
221         self.io.deadline = self.deadline
222         self.reason = "running"
223         try:
224             while self.remaining and len(self.discovered) < 16:
225                 search=self.next_search_committed
226                 self.next_search_committed=None
227                 self.scan(search if search is not None else
228                     ↪ next_search_point(self.position,self.remaining))
229                 while self.anchors:
230                     self.approach(self.choose_source())

```

```

229         if self.discovered != self.cleared or not (len(self.discovered) == 16 or not
↪ self.remaining):
230             raise ModelMismatch("Completion conditions not satisfied")
231         if not 10 <= len(self.discovered) <= 16:
232             raise ModelMismatch("Complete search contradicts the specified source-count
↪ interval")
233         self.certificate = True
234         self.reason = "complete_with_certificate"
235     except BudgetStop as exc:
236         self.reason = str(exc)
237     self.exit()
238     return self.summary()
239
240 def exit(self):
241     if (not self.entered or self.exit_attempted or self.io.pending is not None
242         or time.monotonic() >= self.deadline):
243         return
244     self.exit_attempted = True
245     reply = self.io.call("/exit")
246     self.virtual = reply["virtual_time_s"]
247     self.exit_confirmed = True
248
249 def summary(self):
250     return dict(problem=self.problem, reason=self.reason,
↪ completion_certificate=self.certificate,
251                 strategy="annular-history-route-v3" if self.use_history else "midpoint-v1",
252                 negative_halfplane_count=self.negative_cut_count,
253                 route_planned_reduction_m=self.route_planned_reduction_m,
254                 next_search_committed=self.next_search_committed,
255                 history_clear_count=self.history_clears, history_tightening_count=self.history
↪ y_tightenings,
256                 pending_regions=self.regions,
257                 exit_confirmed=self.exit_confirmed, entered=self.entered,
258                 discovered_channels=sorted(self.discovered),
↪ cleared_channels=sorted(self.cleared),
259                 cleared_count=len(self.cleared), unknown_channels=sorted(self.unknown),
260                 virtual_time_s=self.virtual,
261                 average_clear_time_s=self.virtual/len(self.cleared) if self.cleared else None,
262                 local_elapsed_s=time.monotonic()-self.started if self.started is not None
↪ else None,
263                 official_program_runtime_s=None, source_total_from_simulator=None,
264                 travel_m=self.travel, measure_count=self.measures,
↪ clear_attempts=self.clear_attempts,
265                 switch_count=self.switches, source_measure_counts=self.source_measures,
266                 negative_pair_counts=self.negative_pairs,
267                 visited_search_points=self.visited, remaining_search_points=self.remaining,
268                 pending_anchors={c:asdict(a) for c,a in self.anchors.items()})

```

validation/baseline_v7/strategy.py

文件行数: 154。

```

1  """Frozen midpoint strategy. Pure functions: no network, simulator, or randomness."""
2  from dataclasses import dataclass, replace
3  import math
4
5  Point = tuple[float, float]
6  INITIAL_BOUND = 1500.0
7  CLEAR_RADIUS = 19.8
8  POSITIVE_FACTOR = 0.502
9  NEGATIVE_FACTOR = 0.501
10 EPSILON_RAD = math.radians(1.005)
11 ANNULAR_STOP = 44.5
12
13
14 def distance(a: Point, b: Point) -> float:
15     return math.hypot(a[0] - b[0], a[1] - b[1])
16

```

```

17
18 @dataclass(frozen=True)
19 class Anchor:
20     point: Point
21     bearing: float
22     bound: float = INITIAL_BOUND
23     rounds: int = 0
24
25     def __post_init__(self):
26         if (not all(math.isfinite(v) for v in (*self.point, self.bearing, self.bound))
27             or not 0 <= self.bearing < 360 or not 0 < self.bound <= INITIAL_BOUND
28             or self.rounds < 0):
29             raise ValueError("Invalid anchor")
30
31
32 def offset(anchor: Anchor, forward: float, sideways: float) -> Point:
33     angle = math.radians(anchor.bearing)
34     ux, uy = math.cos(angle), math.sin(angle)
35     return (anchor.point[0] + anchor.bound * (forward * ux - sideways * uy),
36           anchor.point[1] + anchor.bound * (forward * uy + sideways * ux))
37
38
39 def paired_points(anchor: Anchor, current: Point) -> tuple[Point, Point]:
40     plus, minus = offset(anchor, .5, .02), offset(anchor, .5, -.02)
41     return (plus, minus) if distance(current, plus) <= distance(current, minus) else (minus, plus)
42
43
44 def terminal_clear_point(anchor: Anchor, current: Point) -> Point | None:
45     slack = CLEAR_RADIUS - NEGATIVE_FACTOR * anchor.bound
46     if slack < 0:
47         return None
48     midpoint = offset(anchor, .5, 0)
49     d = distance(current, midpoint)
50     if d <= slack:
51         return current
52     # A tiny inward margin avoids rounding outside the certified clearance disk.
53     ratio = max(0.0, slack - 1e-8) / d
54     return (midpoint[0] + ratio * (current[0] - midpoint[0]),
55           midpoint[1] + ratio * (current[1] - midpoint[1]))
56
57
58 def positive_update(anchor: Anchor, point: Point, bearing: float) -> Anchor:
59     return Anchor(point, bearing, POSITIVE_FACTOR * anchor.bound, anchor.rounds + 1)
60
61
62 def negative_pair_update(anchor: Anchor) -> Anchor:
63     return replace(anchor, bound=NEGATIVE_FACTOR * anchor.bound, rounds=anchor.rounds + 1)
64
65
66 def search_points(problem: int) -> list[Point]:
67     if problem == 3:
68         return [(0.0, 0.0)] + [
69             (1200 * math.cos(math.radians(60*k)), 1200 * math.sin(math.radians(60*k)))
70             for k in range(6)]
71     if problem == 4:
72         return [(990.0 * (i + j/2), 990 * math.sqrt(3) * j/2)
73                 for i in range(-4, 5) for j in range(-4, 5) if i*i + i*j + j*j <= 7]
74     raise ValueError("problem must be 3 or 4")
75
76
77 def next_search_point(current: Point, remaining: list[Point]) -> Point:
78     return min(remaining, key=lambda q: (distance(current, q), q[0], q[1]))
79
80
81 def channel_order(current_channel: int, unknown: set[int]) -> list[int]:
82     return sorted(unknown, key=lambda c: (c != current_channel, c))
83
84

```

```

85 def preview_source_action(anchor: Anchor, current: Point, tuned: int, channel: int):
86     clear_point = terminal_clear_point(anchor, current)
87     if clear_point is not None:
88         return "/clear", clear_point, distance(current, clear_point)/5 + 5
89     point = paired_points(anchor, current)[0]
90     return "/measure", point, distance(current, point)/5 + 5 + int(tuned != channel)
91
92
93 def second_point(first: Point, bearing: float, current: Point | None = None) -> Point:
94     anchor = Anchor(first, bearing)
95     plus, minus = offset(anchor, .5, .4), offset(anchor, .5, -.4)
96     return plus if current is None or distance(current, plus) <= distance(current, minus) else
97     ↪ minus
98
99 def annular_points(anchor: Anchor, current: Point) -> tuple[Point, Point]:
100     """Only for a normal direction response, hence source distance > 5 m."""
101     forward = (anchor.bound+5)/(2*anchor.bound)
102     plus, minus = offset(anchor, forward, .02), offset(anchor, forward, -.02)
103     return (plus, minus) if distance(current, plus) <= distance(current, minus) else (minus, plus)
104
105
106 def annular_update(anchor: Anchor, point=None, bearing=None) -> Anchor:
107     if anchor.bound <= ANNULAR_STOP:
108         raise ValueError("The source should already be cleared; do not start another round")
109     if point is None:
110         return replace(anchor, bound=.501*anchor.bound+2.505, rounds=anchor.rounds+1)
111     return Anchor(tuple(point), bearing, .502*anchor.bound-2.4, anchor.rounds+1)
112
113
114 def annular_clear_disk(anchor: Anchor):
115     if anchor.bound <= 5:
116         raise ValueError("A normal bearing must have source distance greater than 5 m")
117     if anchor.bound > ANNULAR_STOP:
118         return None
119     L = anchor.bound
120     cover = math.sqrt((L-5)**2/4+L*(L+5)*(1-math.cos(EPSILON_RAD)))
121     return offset(anchor, (L+5)/(2*L), 0), max(0.0, CLEAR_RADIUS-cover-1e-7)
122
123
124 def point_in_disk_nearest(center, radius, current):
125     d = distance(center, current)
126     if d <= radius:
127         return current
128     t = radius/d
129     return center[0]+t*(current[0]-center[0]), center[1]+t*(current[1]-center[1])
130
131
132 def annular_clear_point(anchor: Anchor, current: Point):
133     disk = annular_clear_disk(anchor)
134     return None if disk is None else point_in_disk_nearest(*disk, current)
135
136
137 def second_point_region(first: Point, bearing: float, query=None):
138     """Three-disk description after eliminating R >= max(1000, r)."""
139     anchor = Anchor(first, bearing)
140     centers = [tuple(first)]
141     for sign in (1, -1):
142         angle = math.radians(bearing)+sign*EPSILON_RAD
143         centers.append((first[0]+1000*math.cos(angle), first[1]+1000*math.sin(angle)))
144     normal = math.cos(math.radians(bearing)), math.sin(math.radians(bearing))
145     result = dict(disk_centers=centers, disk_radii=[1000]*3,
146                 forward_halfplane=dict(normal=normal, origin=first, relation="dot(normal,
147                 ↪ q-origin) >= 0"),
148                 epsilon_deg=1.005,
149                 scope="Exact for relaxed radial prior [0,1500] within forward halfplane;
150                 ↪ conservative for actual r>5 and source domain")
151     if query is not None:

```

```

150         if len(query)!=2 or not all(math.isfinite(v) for v in query):
151             raise ValueError("Query must contain two finite coordinates")
152         result["query_in_region"] = (all(distance(query,c)<=1000 for c in centers)
153             and sum(normal[k]*(query[k]-first[k]) for k in (0,1))>=0)
154     return result

```

validation/baseline_v7/tail_search.py

文件行数: 79。

```

1  """Residual outer regions and quota-preserving continuous coverage rewrites."""
2  import math
3  from geometry import hull
4  from coverage import contains_strict, certify
5  from planning import open_route, route_length
6
7
8  def residual_regions(points, problem, max_nodes=512, max_depth=8, radius=1800.):
9      """All unresolved boxes are retained, including the unprocessed stack.
10
11      These boxes can contain already excluded space; neither their centres nor
12      their area is evidence of a hidden source. Empty means a whole-domain proof.
13      """
14      stack=[(-radius,-radius,radius,radius,0)];regions=[];nodes=0
15      while stack and nodes<max_nodes:
16          x0,y0,x1,y1,depth=stack.pop();nodes+=1
17          if math.hypot(max(x0,0.,-x1),max(y0,0.,-y1))>radius+1e-5:
18              continue
19          corners=[(x0,y0),(x1,y0),(x1,y1),(x0,y1)]
20          nearby=[q for q in points if max(math.dist(q,v) for v in corners)<1000-1e-5]
21          if problem==3 and nearby:
22              continue
23          if problem==4 and len(nearby)>=3:
24              polygon=hull(nearby)
25              if all(contains_strict(polygon,v) for v in corners):
26                  continue
27          if depth>=max_depth:
28              regions.append((x0,y0,x1,y1));continue
29          mx,my=(x0+x1)/2,(y0+y1)/2
30          stack.extend([(x0,y0,mx,my,depth+1),(mx,y0,x1,my,depth+1),
31              (x0,my,mx,y1,depth+1),(mx,my,x1,y1,depth+1)])
32      regions.extend(tuple(box[:4]) for box in stack)
33      return dict(boxes=regions,nodes=nodes,proved=not regions)
34
35
36  def revise_route(position, visited, remaining, problem, proof_limit=2):
37      """Rank finitely many edits by travel; adopt only a full coverage proof.
38
39      No extra scan is added. Search ordering still has its ordinary finite route
40      heuristic; residual geometry proposes new vertices rather than claiming an
41      area score proves discovery. An inconclusive proof leaves the plan intact.
42      """
43      route=open_route(position,remaining)
44      residual=residual_regions(visited,problem)
45      if not route:
46          return route,dict(changed=False,residual=residual)
47      boxes=residual["boxes"]
48      candidates=[tuple(position)]
49      if boxes:
50          area=sum((b[2]-b[0])*(b[3]-b[1]) for b in boxes)
51          candidates.append(tuple(sum((b[i]+b[i+2])/2*(b[2]-b[0])*(b[3]-b[1]) for b in boxes)/area
52              ↪ for i in (0,1)))
53          b=max(boxes,key=lambda b:(b[2]-b[0])*(b[3]-b[1]))
54          candidates.append(((b[0]+b[2])/2,(b[1]+b[3])/2))
55      radius=1820. if problem==3 else 1836.
56      candidates=[tuple(x*min(1.,radius/max(radius,math.hypot(*p))) for x in p) for p in candidates]
57      old_length=route_length(position,route);proposals=[]
58      for old in route:

```

```

58     without=[q for q in route if q!=old]
59     proposals.append((route_length(position,without),without,old,None))
60     for new in candidates:
61         if new in route or any(math.dist(new,q)<1e-5 for q in visited):
62             continue
63         # Insertion position is itself optimized in this finite family.
64         trial=min((without[:i]+[new]+without[i:] for i in range(len(without)+1)),
65                 key=lambda r:route_length(position,r))
66         proposals.append((route_length(position,trial),trial,old,new))
67     checked=0
68     for length,trial,old,new in sorted(proposals,key=lambda e:e[0]):
69         if length>=old_length-1. or checked>=proof_limit:
70             break
71         checked+=1
72         proof=certify(list(visited)+trial,problem,max_nodes=4096)
73         if proof["proved"]:
74             return trial,dict(changed=True,removed=old,inserted=new,
75                             planned_travel_reduction_m=old_length-length,proof=proof,
76                             proofs_attempted=checked,residual_nodes=residual["nodes"],
77                             unresolved_boxes=len(boxes))
78     return route,dict(changed=False,proofs_attempted=checked,
79                     residual_nodes=residual["nodes"],unresolved_boxes=len(boxes))

```

paper/build.py

文件行数: 125。

```

1  """Compile the manuscript and full listings without using a simulator or network."""
2
3  from pathlib import Path
4  import argparse
5  import hashlib
6  import json
7  import re
8  import shutil
9  import subprocess
10
11  from generate_appendix import generate, source_files
12  from make_formal_tables import generate as formal_tables
13
14
15  def input_hashes(root):
16      paths = set(source_files(root))
17      paths.update((root / "paper").glob("*.tex"))
18      paths.update((root / "paper/figures").glob("*.pdf"))
19      paths.update(
20          root / name
21          for name in (
22              "paper/results.json",
23              "paper/figure-data.json",
24              "submission_inputs/formal_results.csv",
25              "submission_inputs/human_review.json",
26          )
27      )
28      return {
29          p.relative_to(root).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
30          for p in sorted(paths)
31      }
32
33
34  def main():
35      parser = argparse.ArgumentParser()
36      parser.add_argument("--engine", default="xelatex")
37      parser.add_argument("--format", help="Optional prebuilt XeLaTeX format for xetex")
38      args = parser.parse_args()
39      paper = Path(__file__).resolve().parent
40      root = paper.parent
41      build = paper / ".build"

```

```

42     build.mkdir(exist_ok=True)
43     (root / "submission").mkdir(exist_ok=True)
44     engine = shutil.which(args.engine)
45     if engine is None:
46         raise SystemExit(
47             "Install TeX Live or MiKTeX with XeLaTeX, ctex, fvextra and Fandol fonts."
48         )
49     formal_tables(root)
50     records = generate(root)
51     command = [engine]
52     if args.format:
53         command += [f"-fmt={Path(args.format).resolve()}", "-progrname=xelatex"]
54     command += [
55         "-interaction=nonstopmode",
56         "-halt-on-error",
57         "-output-directory=.build",
58     ]
59     report = {
60         "documents": [],
61         "source_files": len(records),
62         "source_lines": sum(r["lines"] for r in records),
63     }
64     for source, destination in [
65         ("main", "submission/论文.pdf"),
66         ("ai-usage", "AI 工具使用详情.pdf"),
67     ]:
68         for _ in range(2):
69             with (build / f"{source}-compile.log").open(
70                 "w", encoding="utf-8"
71             ) as output:
72                 completed = subprocess.run(
73                     command + [source + ".tex"],
74                     cwd=paper,
75                     stdout=output,
76                     stderr=subprocess.STDOUT,
77                 )
78                 if completed.returncode:
79                     raise SystemExit(
80                         f"Compile failed; inspect paper/.build/{source}-compile.log"
81                     )
82                 log = (build / f"{source}.log").read_text(encoding="utf-8", errors="replace")
83                 defects = re.findall(
84                     r".*(?:Overfull|Missing character|undefined references|undefined on input).*",
85                     log,
86                 )
87                 if defects:
88                     raise SystemExit("\n".join(defects))
89                 pages = int(
90                     re.search(r"Output written on .*\((\d+) pages?", log, re.S).group(1)
91                 )
92                 body_pages = None
93                 if source == "main":
94                     aux = (build / "main.aux").read_text(encoding="utf-8")
95                     body_pages = int(
96                         re.search(r"\newlabel\{main_end\}\{[\~]*\}\{(\d+)\}", aux).group(1)
97                     )
98                 if body_pages > 30 or pages <= body_pages:
99                     raise SystemExit(
100                         f"Main text is {body_pages} pages; total is {pages}. Check the page limit and
101                             ↪ appendix."
102                     )
103                 target = root / destination
104                 shutil.copy2(build / f"{source}.pdf", target)
105                 report["documents"].append(
106                     dict(
107                         file=destination,
108                         pages=pages,
109                         body_and_references_pages=body_pages,

```

```

109         sha256=hashlib.sha256(target.read_bytes()).hexdigest(),
110         missing_glyphs=False,
111         overfull_boxes=False,
112         undefined_references=False,
113     )
114 )
115 print(
116     f"Built {target.name}: {pages} pages; main text and references: {body_pages}"
117 )
118 report["input_sha256"] = input_hashes(root)
119 (paper / "build-report.json").write_text(
120     json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
121 )
122
123
124 if __name__ == "__main__":
125     main()

```

paper/generate_appendix.py

文件行数：78。

```

1  """Generate complete listings and hashes from the actual submitted source files."""
2
3  import hashlib
4  import json
5  from pathlib import Path
6
7  ROOT = Path(__file__).resolve().parents[1]
8
9
10 def source_files(root=ROOT):
11     paths = [
12         root / name
13         for name in (
14             "run_robot.py",
15             "solve_questions.py",
16             "start_robot.ps1",
17             "check.py",
18             "prepare_submission.py",
19         )
20     ]
21     for directory in (
22         "src/jammer_solver",
23         "tools",
24         "tests",
25         "validation/baseline_v7",
26         "paper",
27     ):
28         paths.extend(sorted((root / directory).glob("*.py")))
29     return paths
30
31
32 def manifest(root=ROOT):
33     return [
34         dict(
35             path=p.relative_to(root).as_posix(),
36             sha256=hashlib.sha256(p.read_bytes()).hexdigest(),
37             lines=len(p.read_text(encoding="utf-8").splitlines()),
38         )
39         for p in source_files(root)
40     ]
41
42
43 def generate(root=ROOT):
44     records = manifest(root)
45     lines = [
46         r"\section*{附录：完整程序清单}",

```

```

47     " 下列程序按提交时的实际文件逐字列出，未省略函数或实现。"
48     " 主运行策略为 v8 快速版 r4; v7 仅用于复算正文保留的历史对照，不能从比赛入口切换。"
49     " 支撑压缩包保留相同目录，文件散列见程序清单索引。",
50     f" 共 {len(records)} 个源文件、{sum(r['lines'] for r in records)} 行。",
51     r"\begingroup\linespread{1.0}\selectfont",
52 ]
53 for record in records:
54     name = record["path"]
55     lines += [
56         r"\subsection*{\texttt{\detokenize{" + name + "}}}",
57         f" 文件行数: {record['lines']}。",
58     ]
59     if record["lines"]:
60         lines.append(
61             r"\VerbatimInput[fontsize=\fontsize{8.5}{10.5}\selectfont,"
62             r"breaklines=true,breakanywhere=true,numbers=left,numbersep=4pt,"
63             r"xleftmargin=17pt,tabsize=4]{\RepoRoot/" + name + "}"
64         )
65     else:
66         lines.append(" 该文件为空，用于标记 Python 包。")
67     lines.append(r"\endgroup")
68     (root / "paper/appendix_sources.tex").write_text(
69         "\n".join(lines) + "\n", encoding="utf-8"
70     )
71     (root / "paper/source_manifest.json").write_text(
72         json.dumps(records, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
73     )
74     return records
75
76
77 if __name__ == "__main__":
78     generate()

```

paper/make_figures.py

文件行数: 386。

```

1 from pathlib import Path
2 import json
3 import math
4 import sys
5 import os
6
7 os.environ.setdefault("MPLCONFIGDIR", "/tmp/mcm-paper-matplotlib")
8 import numpy as np
9 import matplotlib
10
11 matplotlib.use("Agg")
12 import matplotlib.pyplot as plt
13 from matplotlib import font_manager
14 from matplotlib.patches import Circle, Polygon, FancyBboxPatch
15
16 ROOT = Path(__file__).resolve().parents[1]
17 sys.path[:0] = [str(ROOT / "src"), str(ROOT / "tools")]
18 from jammer_solver import solver as v8, exact_geometry as g
19 from jammer_solver.questions import jdyh, jhgy
20
21 PAPER = ROOT / "paper"
22 DATA = json.loads((PAPER / "results.json").read_text(encoding="utf-8"))
23 font_candidates = [
24     os.environ.get("MCM_PAPER_FONT"),
25     "/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf",
26     "C:/Windows/Fonts/simhei.ttf",
27     "/System/Library/Fonts/PingFang.ttc",
28 ]
29 font_family = None
30 for candidate in font_candidates:
31     if candidate and Path(candidate).is_file():

```

```

32     font_manager.fontManager.addfont(candidate)
33     font_family = font_manager.FontProperties(fname=candidate).get_name()
34     break
35 if font_family is None:
36     raise SystemExit(
37         "Set MCM_PAPER_FONT to an installed Chinese TrueType font before regenerating figures."
38     )
39 plt.rcParams.update(
40     {
41         "font.family": [font_family, "DejaVu Sans"],
42         "mathtext.fontset": "dejavuserif",
43         "font.size": 10,
44         "axes.labelsize": 10,
45         "legend.fontsize": 9,
46         "axes.titlesize": 11,
47         "axes.unicode_minus": False,
48         "lines.linewidth": 2,
49         "axes.spines.top": False,
50         "axes.spines.right": False,
51         "savefig.dpi": 320,
52         "pdf.fonttype": 42,
53     }
54 )
55 BLUE = "#0072B2"
56 RED = "#D55E00"
57 GREEN = "#009E73"
58 PURPLE = "#7B61A8"
59 GRAY = "#555555"
60
61
62 def save(fig, name):
63     fig.savefig(PAPER / "figures" / f"{name}.pdf", bbox_inches="tight", pad_inches=0.04)
64     plt.close(fig)
65
66
67 def equal(ax, xlabel=" 东向坐标 / m", ylabel=" 北向坐标 / m"):
68     ax.set_aspect("equal")
69     ax.set_xlabel(xlabel)
70     ax.set_ylabel(ylabel)
71     ax.grid(alpha=0.18)
72
73
74 fig, ax = plt.subplots(figsize=(6.3, 3.2))
75 ax.set_xlim(0, 10)
76 ax.set_ylim(0, 5)
77 ax.axis("off")
78 boxes = [
79     (0.1, 3.2, 2.65, 1.0, " 实际观测\n 方向 / 近距 / 阴性", BLUE),
80     (3.6, 3.2, 2.65, 1.0, " 证据账本\n 频道状态与可行域", BLUE),
81     (7.1, 3.2, 2.65, 1.0, " 完成证书\n 实际清除 + 排除未知", GREEN),
82     (0.1, 0.55, 2.65, 1.15, " 覆盖搜索\n Q3: 7 站 / Q4: 21 站", GRAY),
83     (3.6, 0.55, 2.65, 1.15, " 联合任务排序\n 移动 + 测向 + 清除", BLUE),
84     (7.1, 0.55, 2.65, 1.15, " 执行下一动作\n 射频 / 完整光学链", RED),
85 ]
86 for x, y, w, h, t, c in boxes:
87     ax.add_patch(
88         FancyBboxPatch(
89             (x, y),
90             w,
91             h,
92             boxstyle="round,pad=.06",
93             linewidth=1.5,
94             edgecolor=c,
95             facecolor="white",
96         )
97     )
98     ax.text(x + w / 2, y + h / 2, t, ha="center", va="center", fontsize=10)
99 for a, b in [

```

```

100     ((2.8, 3.7), (3.5, 3.7)),
101     ((6.3, 3.7), (7, 3.7)),
102     ((2.8, 1.1), (3.5, 1.1)),
103     ((6.3, 1.1), (7, 1.1)),
104     ((4.9, 3.1), (4.9, 1.8)),
105     ((8.5, 1.8), (8.5, 2.4)),
106     ((8.5, 2.4), (1.4, 2.4)),
107     ((1.4, 2.4), (1.4, 3.1)),
108 ]:
109     ax.annotate("", b, a, arrowprops=dict(arrowstyle="->", lw=1.5, color=GRAY))
110 ax.text(5, 4.65, " 评分决定 “先做什么”；证书决定 “能否完成” ", ha="center", fontsize=11)
111 save(fig, "flow")
112
113 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.8), layout="constrained")
114 obs = [dict(x=0, y=0, bearing_deg=45), dict(x=100, y=0, bearing_deg=135)]
115 region = jhqy(obs)
116 poly = np.array(region["vertices"])
117 ax = axs[0]
118 ax.add_patch(Polygon(poly, facecolor="#DCECF5", edgecolor=BLUE, lw=2))
119 a, b = np.array(region["farthest_pair"])
120 ax.plot([a[0], b[0]], [a[1], b[1]], "o--", color=RED, ms=4)
121 ax.set_xlim(47.5, 52.5)
122 ax.set_ylim(47.5, 52.5)
123 equal(ax)
124 ax.set_title("(a) 两次观测的交会区域")
125 ax.text(50, 47.8, "D = 3.5096 m", ha="center", fontsize=9)
126 ax = axs[1]
127 tri = np.array([[ -0.5, 0], [0.5, 0], [0, np.sqrt(3) / 2]])
128 ax.add_patch(Polygon(tri, fill=False, edgecolor="black", lw=2))
129 ax.add_patch(Circle((0, 0), 0.5, fill=False, ls="--", color=RED, lw=2))
130 ax.add_patch(
131     Circle((0, np.sqrt(3) / 6), 1 / np.sqrt(3), fill=False, ls="-. ", color=BLUE, lw=2)
132 )
133 ax.scatter(*tri.T, color="black", s=20)
134 ax.set_xlim(-0.8, 0.8)
135 ax.set_ylim(-0.65, 1)
136 equal(ax, "x / d", "y / d")
137 ax.set_title("(b) 同直径圆的反例")
138 ax.plot([], [], "--", color=RED, label=" 半径 d/2")
139 ax.plot([], [], "-.", color=BLUE, label=r" 半径 $d/\sqrt{3}$")
140 ax.legend(loc="lower center", fontsize=8)
141 save(fig, "q1_geometry")
142
143 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.85), layout="constrained")
144 a = np.linspace(0, 1100, 500)
145 b = np.linspace(-1100, 1100, 600)
146 A, B = np.meshgrid(a, b)
147 eps = math.radians(1.005)
148 mask = A * A + B * B <= 1000**2
149 for s in [-1, 1]:
150     mask &= (A - 1000 * math.cos(eps)) ** 2 + (
151         B - s * 1000 * math.sin(eps)
152     ) ** 2 <= 1000**2
153 axs[0].contourf(A, B, mask.astype(float), levels=[0.5, 1.5], colors=["#DCECF5"])
154 axs[0].contour(A, B, mask.astype(float), levels=[0.5], colors=[BLUE], linewidths=1.5)
155 p = DATA["q2_minimum"]
156 axs[0].plot(p["a_m"], p["b_m"], "*", ms=10, color=RED)
157 axs[0].plot(500, 840, "s", ms=4, color=GREEN)
158 axs[0].set_title("(a) 保证接收的候选区域")
159 equal(axs[0], " 前向位移 a / m", " 侧向位移 b / m")
160 rs = np.linspace(500, 999, 50)
161 vals = [jdyh(float(r))["diameter_upper_m"] for r in rs]
162 axs[1].plot(rs, vals, color=BLUE)
163 axs[1].axhline(163, ls="--", color=RED, lw=1.5)
164 axs[1].plot(p["movement_m"], p["diameter_upper_m"], "o", color=RED)
165 axs[1].set_xlabel(" 移动预算 \u03c1 / m")
166 axs[1].set_ylabel(" 解析直径上界 / m")
167 axs[1].set_title("(b) 移动与定位精度的折中")

```

```

168 axs[1].grid(alpha=0.18)
169 save(fig, "q2_design")
170
171 fig, axs = plt.subplots(1, 2, figsize=(6.3, 3.1), layout="constrained")
172 for ax, problem in zip(axs, [3, 4]):
173     q = np.array(v8.sscd(problem))
174     ax.add_patch(Circle((0, 0), 1800, fill=False, color="black", lw=1.6))
175     if problem == 3:
176         for p in q:
177             ax.add_patch(Circle(p, 1000, fill=False, color=BLUE, lw=0.65, alpha=0.45))
178     else:
179         ax.plot(*q[1:9].T, "o", color=BLUE, ms=4)
180         ax.plot(*q[9:].T, "s", color=RED, ms=4)
181     ax.plot(q[:, 0], q[:, 1], "o", color=BLUE, ms=3.5)
182     ax.plot(0, 0, "*", color="black", ms=9)
183     ax.set_xlim(-2400, 2400)
184     ax.set_ylim(-2400, 2400)
185     equal(ax)
186     ax.set_xticks([-1800, 0, 1800])
187     ax.set_yticks([-1800, 0, 1800])
188     ax.set_title(f"问题 {problem}: {len(q)} 个搜索站")
189 save(fig, "coverage_layouts")
190
191 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.95), layout="constrained")
192 ax = axs[0]
193 source = np.array([0.0, 0.0])
194 witness = np.array([[-450, -300], [500, -200], [150, 550]])
195 ax.add_patch(Polygon(witness, facecolor="#DCECF5", edgecolor=BLUE, lw=2))
196 ax.scatter(*witness.T, color=BLUE, s=35)
197 ax.plot(0, 0, "*", ms=12, color=RED)
198 ax.fill_between([-650, 700], -600, 600, color=GREEN, alpha=0.05)
199 ax.axvline(0, color=GRAY, ls="--", lw=1.5)
200 ax.annotate(
201     "任意闭半平面\mathbf{n} 至少包含一个证据点",
202     (20, 10),
203     (70, -530),
204     arrowprops=dict(arrowstyle="->", lw=1.3),
205     fontsize=9,
206 )
207 ax.set_xlim(-650, 700)
208 ax.set_ylim(-650, 750)
209 equal(ax)
210 ax.set_title("(a) 源位于近邻测点凸包内")
211 ax = axs[1]
212 o = (0, 0)
213 a = (750, -200)
214 b = (750, 200)
215 ax.add_patch(
216     Polygon(
217         [a, (1250, -1000 / 3), (1250, 1000 / 3), b],
218         facecolor="#F5DFD4",
219         edgecolor=RED,
220         lw=2,
221     )
222 )
223 ax.plot([0, 1300], [0, 0], ":", color=GRAY)
224 ax.plot(0, 0, "*", ms=11, color=GREEN)
225 ax.plot([750, 750], [-200, 200], "s", color=RED)
226 ax.text(60, 60, "实际阳性锚点", fontsize=9)
227 ax.text(1020, -80, "可排除部分", ha="center", fontsize=9)
228 ax.text(780, 360, "双阴性测点", fontsize=9)
229 ax.set_xlim(-100, 1350)
230 ax.set_ylim(-500, 500)
231 equal(ax)
232 ax.set_title("(b) 双阴性收缩的几何条件")
233 save(fig, "directional_proof")
234
235 from fractions import Fraction

```

```

236
237 snapshot = json.loads((PAPER / "figure-data.json").read_text(encoding="utf-8"))
238 chain = snapshot["event"]
239 poly = tuple(tuple(Fraction(x) for x in p) for p in snapshot["polygon_rational"])
240 points = [g.point(p) for p in chain["points"]]
241 origin = np.array((tuple((float(x) for x in points[0])))
242 axis = np.array((tuple((float(x) for x in points[-1])))) - origin
243 axis /= np.linalg.norm(axis)
244 normal = np.array([-axis[1], axis[0]])
245
246
247 def local(point):
248     delta = np.array((tuple((float(x) for x in point)))) - origin
249     return np.array([delta @ axis, delta @ normal])
250
251
252 fig, ax = plt.subplots(figsize=(6.3, 2.35), layout="constrained")
253 colors = [BLUE, RED, GREEN, PURPLE]
254 for i, q in enumerate(points):
255     part = poly
256     for other in points:
257         if q != other:
258             part = g.clip(
259                 part,
260                 ((other[0] - q[0], other[1] - q[1])),
261                 (
262                     (other[0] * other[0] + other[1] * other[1])
263                     - (q[0] * q[0] + q[1] * q[1])
264                 )
265                 / 2,
266             )
267     color = colors[i % len(colors)]
268     qf = local(q)
269     ax.add_patch(Circle(qf, 19.8, fill=False, ls="--", color=color, lw=1.6))
270     if part:
271         ax.add_patch(
272             Polygon(
273                 [local(v) for v in part], facecolor=color, edgecolor=color, alpha=0.23
274             )
275         )
276     ax.plot(*qf, "o", color=color, ms=4)
277     ax.annotate(str(i + 1), qf, xytext=(5, 4), textcoords="offset points", fontsize=9)
278 qs = np.array([local(q) for q in points])
279 ax.plot(qs[:, 0], qs[:, 1], "-", color=GRAY, lw=1.2)
280 ax.add_patch(Polygon([local(v) for v in poly], fill=False, color="black", lw=2))
281 ax.autoscale_view()
282 equal(ax, " 沿链轴坐标 / m", " 横向坐标 / m")
283 ax.set_title(" 实际决策时的完整可行域与光学覆盖链")
284 save(fig, "optical_chain")
285
286 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.85), layout="constrained")
287 for p, c, m in [(3, BLUE, "o"), (4, RED, "s")]:
288     rows = [r for r in DATA["comparison"] if r["problem"] == p]
289     axs[0].scatter(
290         [r["v7_s"] for r in rows],
291         [r["v8_s"] for r in rows],
292         c=c,
293         marker=m,
294         s=28,
295         label=f" 问题 {p}",
296     )
297 axs[0].plot([2500, 8200], [2500, 8200], "--", color=GRAY, lw=1.5)
298 axs[0].set_xlim(2500, 8200)
299 axs[0].set_ylim(2500, 8200)
300 axs[0].set_xlabel("v7 总虚拟时间 / s")
301 axs[0].set_ylabel("v8 总虚拟时间 / s")
302 axs[0].legend()
303 axs[0].grid(alpha=0.18)

```

```

304 for j, p in enumerate([3, 4]):
305     rows = [
306         r
307         for r in DATA["cases"]
308         if r["name"].startswith(f"q{p}-")
309         and r["suite"] in ("matched", "validation", "fresh")
310     ]
311     costs = [
312         np.mean([r["moving_distance_m"] / 5 for r in rows]),
313         np.mean([r["counts"]["measure"] * 5 for r in rows]),
314         np.mean([r["counts"]["switch"] for r in rows]),
315         np.mean(
316             [
317                 r["counts"]["successful_clear"] * 5
318                 + (r["counts"]["clear"] - r["counts"]["successful_clear"]) * 3
319                 for r in rows
320             ]
321         ),
322     ]
323     base = 0
324     for i, (value, color, label) in enumerate(
325         zip(costs, [BLUE, RED, GREEN, PURPLE], [" 移动", " 测向", " 切频", " 光学与清除"]))
326     ):
327         axs[i].bar(
328             j,
329             value,
330             bottom=base,
331             width=0.58,
332             color=color,
333             label=label if j == 0 else None,
334             hatch=["", "//", "xx", ".."][i],
335         )
336         base += value
337     axs[i].set_xticks([0, 1], [" 问题 3", " 问题 4"])
338     axs[i].set_ylabel(" 平均虚拟时间 / s")
339     axs[i].legend(fontsize=8, ncol=2, loc="upper left")
340     axs[i].set_ylim(0, 8200)
341     save(fig, "performance")
342
343 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.7), layout="constrained")
344 for i, c in enumerate(DATA["calibration"]):
345     axs[0].plot(
346         [v["depth"] for v in c["rollout"]],
347         [v["gap_s"] for v in c["rollout"]],
348         ["o-", "s--", "^-."][i],
349         color=[BLUE, RED, GREEN][i],
350         label=f" 有限问题 {i + 1}",
351     )
352     axs[0].set_xlabel(" 前瞻深度")
353     axs[0].set_ylabel(" 距有限问题最优值 / s")
354     axs[0].set_xticks(range(4))
355     axs[0].grid(alpha=0.18)
356     axs[0].legend(fontsize=8)
357     ns = list(range(10, 17))
358     for p, c, m in [(3, BLUE, "o-"), (4, RED, "s--")]:
359         vals = [0 if n == 16 else (20 - n) * DATA["absence_cost"][str(p)] for n in ns]
360         axs[1].plot(ns, vals, m, color=c, label=f" 问题 {p}")
361     axs[1].set_xlabel(" 实际源总数 N")
362     axs[1].set_ylabel(" 无源频道的必要操作成本 / s")
363     axs[1].set_xticks(ns)
364     axs[1].grid(alpha=0.18)
365     axs[1].legend(fontsize=8)
366     save(fig, "calibration_bounds")
367
368 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.65), layout="constrained")
369 for ax, p in zip(axs, [3, 4]):
370     rows = [r for r in DATA["practice"] if r["problem"] == p]
371     x = [

```

```

372         r["reported_source_count"] if p == 3 else r["reported_directional"]
373     for r in rows
374 ]
375 y = [r["reported_virtual_time_s"] for r in rows]
376 ax.scatter(x, y, color=BLUE if p == 3 else RED, s=32)
377 for i, (xx, yy) in enumerate(zip(x, y)):
378     ax.annotate(
379         str(i + 1), (xx, yy), xytext=(5, 4), textcoords="offset points", fontsize=9
380     )
381 ax.set_xlabel(" 截图报告源个数" if p == 3 else " 截图报告定向源个数")
382 ax.set_ylabel(" 截图报告总时间 / s")
383 ax.set_title(f" 问题 {p}: 五次不同案例")
384 ax.grid(alpha=0.18)
385 save(fig, "practice")
386 print("Generated 9 figures in PDF format")

```

paper/make_formal_tables.py

文件行数: 57。

```

1  """Use measured CSV values when present; preserve genuinely empty result cells."""
2
3  import csv
4  from pathlib import Path
5
6  ROOT = Path(__file__).resolve().parents[1]
7
8
9  def escape(value):
10     replacements = {
11         "\\": r"\textbackslash{}",
12         "&": r"\&",
13         "%": r"\%",
14         "$": r"\$",
15         "#": r"\#",
16         "_": r"\_",
17         "{": r"\{",
18         "}": r"\}",
19         "~": r"\textasciitilde{}",
20         "^": r"\textasciicircum{}",
21     }
22     return "".join(replacements.get(c, c) for c in str(value))
23
24
25  def generate(root=ROOT):
26     with (root / "submission_inputs/formal_results.csv").open(
27         encoding="utf-8-sig", newline=""
28     ) as f:
29         rows = list(csv.DictReader(f))
30         for problem in (3, 4):
31             lines = []
32             for index in (1, 2, 3):
33                 row = next(
34                     r
35                     for r in rows
36                     if r["problem"] == str(problem) and r["test"] == str(index)
37                 )
38                 cells = [
39                     row[key].strip()
40                     for key in (
41                         "case_code",
42                         "cleared_count",
43                         "average_time_s",
44                         "real_time_s",
45                     )
46                 ]
47                 lines.append(
48                     " & ".join(escape(c) if c else r"\blank" for c in cells) + r"\\[10pt]"

```

```

49         )
50         lines[-1] += r"\bottomrule"
51         (root / f"paper/formal_q{problem}.tex").write_text(
52             "\n".join(lines) + "\n", encoding="utf-8"
53         )
54
55
56 if __name__ == "__main__":
57     generate()

```

paper/make_tables.py

文件行数：72。

```

1 from pathlib import Path
2 import json
3
4 PAPER = Path(__file__).resolve().parent
5 data = json.loads((PAPER / "results.json").read_text(encoding="utf-8"))
6
7
8 def case_label(case):
9     name = case["name"]
10    count = case["source_count"]
11    suite = case["suite"]
12    if suite == "matched":
13        return "M" + ("1" if "20260911" in name else "2")
14    if suite == "validation":
15        return f"V{count}"
16    if suite == "fresh":
17        return f"F{count}"
18    if suite == "fallback":
19        return f"R{count}"
20    return ("B" if "boundary" in name else "C") + str(count)
21
22
23 for problem in (3, 4):
24     rows = [r for r in data["cases"] if r["name"].startswith(f"q{problem}-")]
25     if problem == 3:
26         header = r" 案例 & 源数 & 总时间/s & 平均时间/(s/源) & $M$ & $K_f$"
27         specs = "crrrrr"
28     else:
29         header = r" 案例 & 全向/定向 & 总时间/s & 平均时间/(s/源) & $K_f$"
30         specs = "ccrrrr"
31     lines = [
32         r"\begin{table}[H]\centering\caption{问题"
33         + str(problem)
34         + r" 的 15 场离线固定输入结果}\label{tab:q"
35         + str(problem)
36         + "}",
37         r"\small\setlength{\tabcolsep}{7pt}",
38         r"\begin{tabular}{ " + specs + r"}\toprule",
39         header + r"\\ \midrule",
40     ]
41     for case in rows:
42         n, t, counts = case["source_count"], case["virtual_time_s"], case["counts"]
43         failure = counts["clear"] - counts["successful_clear"]
44         if problem == 3:
45             line = f"{case_label(case)} & {n} & {t:.2f} & {t / n:.2f} & {counts['measure']} & \\
46                 ↳ {failure}"
47         else:
48             d = case["directional_count"]
49             line = (
50                 f"{case_label(case)} & {n - d}/{d} & {t:.2f} & {t / n:.2f} & {failure}"
51             )
52         lines.append(line + r"\\")
53     lines += [r"\bottomrule\end{tabular}", r"\end{table}"]
54     (PAPER / f"table_q{problem}.tex").write_text(

```

```

54         "\n".join(lines) + "\n", encoding="utf-8"
55     )
56
57 lines = [
58     r"\begin{table}[H]\centering\caption{十次演练截图的成绩汇总}\label{tab:practice}",
59     r"\small\setlength{\tabcolsep}{4pt}\begin{tabular}{cccr}\toprule",
60     r" 题号/次序 & 案例编码 & 报告全向/定向 & 报告个数 & 总时间 (约) /s\\\midrule",
61 ]
62 for problem in (3, 4):
63     for i, row in enumerate(
64         [r for r in data["practice"] if r["problem"] == problem], 1
65     ):
66         lines.append(
67             f"{problem}/{i} & {row['case_code']} & "
68             f"{row['reported_omnidirectional']}/{row['reported_directional']} & "
69             f"{row['reported_source_count']} & {row['reported_virtual_time_s']}" + r"\\"
70         )
71 lines += [r"\bottomrule\end{tabular}", r"\end{table}"]
72 (PAPER / "table_practice.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")

```